

Библия геймдизайна Gidede

> **Проект:** Gidede — Game Design AI System > **Версия:** Компиляция 12 разделов > **Описание:** Систематизированный свод знаний по геймдизайну, основанный на анализе 17 книг и практическом опыте

Оглавление

1. [Фундамент геймдизайна](#bible_2_1_fundament) 2. [Элементы игры] (#bible_2_2_elements) 3. [MDA Framework](#bible_2_3_mda_framework) 4. [Core Loop](#bible_2_4_core_loop) 5. [Баланс](#bible_2_5_balance) 6. [Экономика и прогрессия](#bible_2_6_economy_progression) 7. [Левел-дизайн] (#bible_2_7_level_design) 8. [Нарратив и эмоциональный дизайн] (#bible_2_8_narrative_emotional_design) 9. [Монетизация и удержание] (#bible_2_9_monetization_retention) 10. [Плейтестинг и итерация] (#bible_2_10_playtesting_iteration) 11. [Шаблоны GDD и чек-листы] (#bible_2_11_gdd_templates_checklists) 12. [Компиляция] (#bible_2_12_compilation)

Глава 1. Фундамент геймдизайна

> **Раздел Библии геймдизайна:** 2.1 — Фундамент > **Источники:** KB-01 (Определения и принципы), KB-02 (Элементы игры) > **Книги:** Шелл (Кн. 1), Искусство геймдизайна (Кн. 2), Фуллертон (Кн. 5), Зубек (Кн. 6), Селлерс (Кн. 13), Бонд (Кн. 17)

1.1. Что такое игра: спектр определений

Прежде чем проектировать инструмент для создания игр, необходимо ответить на фундаментальный вопрос: что именно мы создаём? Определение игры — не просто академическое упражнение; для AI-системы оно должно быть **операциональным**, то есть содержать проверяемые критерии, которые можно алгоритмически оценить. Без чёткого определения невозможно

построить валидатор, невозможно определить границы системы и невозможно отличить «игру» от «не-игры».

Проблема определения игры имеет долгую философскую традицию. От Витгенштейна, указавшего на «семейное сходство» игр (настольные, спортивные, карточные, видеоигры — что их объединяет?), до современных формальных подходов — каждый автор подчёркивает разные аспекты. Мы проанализировали 17 книг по геймдизайну и выявили спектр определений от концептуальных до строго формальных.

1.1.1. Спектр формализаций

Каждое определение фокусируется на определённом аспекте игры, и ни одно не является исчерпывающим само по себе. Вместе они образуют **спектр**, в котором каждый следующий уровень добавляет формализацию и проверяемость:

Концептуальный полюс: «Игра — это средство создания опыта» (Джесси Шелл, Кн. 1, 2). Шелл подчёркивает, что игра — это прежде всего опыт, проживаемый игроком. Механики, история, эстетика и технология — лишь инструменты для создания этого опыта. Это определение идеально как целевая функция (мы создаём опыт), но трудно проверяемо алгоритмически — как измерить «опыт»? Как определить, что данный набор механик создаст задуманный опыт?

Структурный полюс: «Игра — это структурированная игра с правилами и целями» (Трейси Фуллертон, Кн. 5). Фуллертон добавляет к опыту структурные компоненты: правила задают рамки, цели задают направление. Это определение уже частично операционально — можно проверить наличие правил и целей. Фуллертон также развёртывает модель в восемь формальных элементов (игроки, цели, процедуры, правила, ресурсы, конфликт, границы, результат), каждый из которых можно проверить на существование в конкретной игре.

Конфликтный полюс: «Игра — это структурированный конфликт» (Кевин Сьюте, Кн. 17). Это определение фокусируется на центральности конфликта — без вызова, без препятствия нет игры. Конфликт проверяем: существует ли противодействие целям игрока? Конфликт может быть с другими игроками (PvP), с системой (PvE), с самим собой (головоломка) или с случаем (азартные игры). Это определение полезно как диагностический инструмент: если в игре нет конфликта — что-то не так.

Формально-системный полюс: «Игра — это система правил» (Сид Мейер, Кн. 17). Мейер сводит игру к формальной системе — набору правил, определяющих допустимые состояния и переходы между ними. Это максимально проверяемое определение: правила можно перечислить, состояния можно пронумеровать, переходы можно смоделировать. Однако это определение слишком узко — оно упускает опыт игрока, который делает правила «игрой», а не просто «системой».

Решающий полюс: «Игра — это система эндогенно значимых решений» (Кейт Бургун, Кн. 17). Это самое операциональное определение из всех. «Эндогенно значимых» означает, что значимость решений возникает внутри самой системы игры, а не привносится извне. Решения проверяемы: есть ли у игрока выбор? Влияет ли выбор на исход? Имеет ли выбор значение в контексте правил игры? Это определение объединяет формальную систему с опытом, поскольку «значимость» решений — это и структурное, и психологическое понятие.

1.1.2. Рабочее определение Gidede

Для целей AI-системы Gidede мы синтезируем все пять подходов в одно **операциональное определение**, каждый элемент которого проверяем алгоритмически:

> Игра — это замкнутая формальная система с неопределённостью исхода, в которой игрок принимает эндогенно значимые решения, создавая эстетический опыт.

Разберём каждый элемент определения и его операциональную проверку:

Элемент	Значение	Проверка в AI-системе	----- ----- -----
Замкнутая	Игра отделена от реальности границами — «магическим кругом» (Хёйзинга)	Существование явных границ: начало, конец, пространство игры	
Формальная система	Правила определяют допустимые состояния и переходы	Перечислимость правил, моделируемость состояний	
Неопределённость исхода	Исход не предопределён заранее	Отсутствие доминантной стратегии; фактор ветвления > 1	
Эндогенно значимые решения	Значимость возникает внутри системы	Влияние выбора на состояние; осмысленность альтернатив	
Эстетический опыт	Игра создаёт переживание у игрока	Соответствие механик целевым эстетикам (MDA-валидация)	

Это определение работает как **фильтр**: если проектируемая система не проходит хотя бы одну проверку, она не является игрой в полном смысле. Это

особенно важно для AI-генерации — система должна гарантировать, что сгенерированный результат удовлетворяет всем критериям.

1.1.3. Магический круг и принцип перехода

Концепция «магического круга» (Йохан Хёйзинга, развита Саленом и Циммерманом) описывает фундаментальное свойство игры: она создаёт отдельное пространство-время со своими правилами, отличными от реальности. Игрок входит в магический круг, принимая правила игры, и выходит из него, когда игра заканчивается. Это не просто метафора — это ключевое понятие для понимания того, почему игры могут иметь «эндогенно значимые» решения, которые бессмысленны за пределами круга.

Принцип перехода собственности (Шелл, Кн. 1) дополняет эту концепцию: когда игрок входит в магический круг, реальные объекты обретают новое, игровое значение. Монополийные деньги ничего не стоят в реальности, но внутри игры — это валюта. Меч на экране — просто пиксели, но в контексте игры — оружие. Этот принцип критически важен для AI-генерации: система должна понимать, что значимость элементов определяется контекстом игры, а не их реальными свойствами.

1.2. Принципы геймдизайна

Определение игры задаёт границы, но не говорит, как создать хорошую игру. Для этого нужны принципы — фундаментальные правила, которые отличают работающий геймдизайн от неработающего. Эти принципы выявлены через кросс-анализ 17 книг и подтверждены тем, что присутствуют в 12+ из 17 источников.

1.2.1. Пять универсальных принципов

Анализ 17 книг выявил **пять концепций**, присутствующих в 12+ книгах из 17. Они образуют ядро единой модели и могут быть сформулированы как универсальные принципы геймдизайна:

Принцип 1: Итеративный цикл (15/17 книг). Дизайн → Прототип → Тест → Итерация — универсальный процесс, который признаётся практически всеми авторами. Шелл формулирует его как «Правило цикла»: чем больше итераций тест→улучшение, тем лучше игра. Фуллертон называет это «Playcentric Design Process» — цикл от цели опыта через прототип и плейтест к итерации. Гэри описывает «базовый цикл разработки» из 6 шагов. Селлерс выделяет «4

главные петли» дизайнера. Независимо от терминологии, суть одна: геймдизайн — это не линейный процесс создания, а спиральный процесс уточнения. Для AI-системы это означает, что генерация не должна быть однократной — система должна поддерживать итеративный цикл генерации, валидации и улучшения.

Принцип 2: Баланс — нет доминантной стратегии (14/17 книг).

Осмысленный выбор возможен только тогда, когда нет очевидно лучшего варианта. Это признаётся и как принцип баланса (Шрайбер, Шелл), и как условие потока (Чиксентмихайи), и как условие веселья (Селлерс).

Доминантная стратегия убивает неопределённость, а без неопределённости нет интереса. Для AI-системы это означает: каждый сгенерированный набор механик должен проверяться на наличие доминантных стратегий, и если они обнаружены — система должна предлагать коррекции.

Принцип 3: Core Loop — петля опыта (13/17 книг). Минимальный цикл FUN-активности, мотивирующий продолжать. Зубек описывает геймплейные циклы разной периодичности (от секунд до часов). Гэри формулирует «5 вопросов кор-геймплея». Шрайбер приводит концепцию «30 секунд веселья» от Halo. Core Loop — это «сердцебиение» игры; если оно не работает, не работает ничего. Для AI-системы это означает: генерация должна начинаться с Core Loop, и все остальные элементы должны строиться вокруг него.

Принцип 4: Поток (12/17 книг). Баланс вызова и мастерства, оптимальное напряжение. Чиксентмихайи описывает «канал потока»: слишком сложно — фрустрация, слишком легко — скука. Селлерс добавляет нейрохимическую основу (дофамин, серотонин, окситоцин, эндорфины). Фуллертон представляет матрицу вызов/навык с шестью состояниями (тревога, поток, контроль, скука, расслабление, апатия). Для AI-системы это означает: система должна отслеживать кривую сложности и обеспечивать нахождение игрока в канале потока, адаптируя вызовы под растущее мастерство.

Принцип 5: Эмерджентность (12/17 книг). Возникновение непредсказуемого опыта из простых правил. Адамс и Дорманс формализуют это через дихотомию Юла: игры эмерджентности (мало правил, много взаимодействий) vs игры прогрессии (много правил, заданный путь). Селлерс развивает концепцию «второпорядочного дизайна»: дизайнер создаёт пространство состояний, а не один путь. Для AI-системы ключевой инсайт: система должна не генерировать «правильный» путь, а создавать пространство возможностей, из которых эмерджентно возникает опыт.

1.2.2. Четыре атрибута веселья

Майкл Селлерс (Кн. 13) выделяет четыре атрибута веселья, которые образуют операциональную модель для проверки качества геймдизайна:

1. **Вызов** (Challenge). Веселье возникает на границе возможностей. Без вызова — скука, при невозможном вызове — фрустрация. Вызов может быть физическим (рефлексы), ментальным (стратегия), социальным (соревнование) или эмоциональным (эмпатия). Для AI-системы: кривая вызова должна быть настраиваемой и адаптивной.

2. **Новизна** (Novelty). Мозг вознаграждает обнаружение нового. Новизна может быть содержательной (новый уровень, новый враг), механической (новая способность, новый тип взаимодействия) или системной (новая комбинация известных элементов). Новизна — конечный ресурс: когда игрок изучил все паттерны, новизна исчерпывается. Для AI-системы: система должна обеспечивать постоянное поступление новизны через прогрессию механик и генерацию вариантов.

3. **Обучение и мастерство** (Learning/Mastery). Удовольствие от роста компетентности — один из глубочайших источников веселья. Костер формулирует это как «веселье — это обучение» (fun is learning). Когда игрок овладевает навыком, возникает чувство мастерства. Когда мастерство подтверждается сложным вызовом — возникает поток. Для AI-системы: система должна моделировать кривую обучения и обеспечивать постоянное, но не подавляющее, возрастание мастерства.

4. **Эмоциональная связь** (Emotional Connection). Веселье усиливается, когда оно разделяемо или когда оно связано с нарративом, персонажем или сообществом. Серотонин (статус), окситоцин (связь), эндорфины (преодоление) — нейрохимические корреляты эмоциональной связи. Для AI-системы: генерируемый геймдизайн должен включать возможности для эмоциональной связи — через нарратив, через социальные механики, через чувство принадлежности.

1.2.3. Принцип приоритета исключений

Заимствованный из формализованных систем правил (D&D, Кн. 16), этот принцип гласит: **если конкретное правило противоречит общему правилу, применяется конкретное**. Это кажется тривиальным, но имеет глубокие последствия для геймдизайна. Исключения — это то, что делает правила интересными. Без исключений правила были бы простыми и скучными; исключения создают «характер» системы, её уникальность.

Для AI-системы этот принцип означает: генерация правил должна включать не только базовые правила, но и осмысленные исключения, которые создают интересные стратегии. При этом система должна отслеживать иерархию правил и обеспечивать непротиворечивость: исключение не должно ломать фундаментальные правила, на которых строится вся система.

1.2.4. Убийцы удовольствия

Треиси Фуллертон (Кн. 5) формулирует пять «убийц удовольствия» — паттернов геймдизайна, которые систематически разрушают опыт игрока:

1. **Микроменеджмент** — когда игра заставляет игрока принимать тривиальные решения, не влияющие на исход. Это нарушение принципа эндогенной значимости: решения есть, но они не значимы. 2. **Застой** — когда игра перестаёт предлагать новые вызовы. Это нарушение принципа новизны и обучения. 3. **Непреодолимые препятствия** — когда вызов настолько превышает мастерство, что игрок попадает в фрустрацию, а не в поток. 4. **Произвольные события** — когда исход определяется чистой случайностью, а не решениями игрока. Это нарушение принципа осмысленного выбора. 5. **Предсказуемые пути** — когда существует очевидно лучший вариант. Это нарушение принципа баланса (доминантная стратегия).

Для AI-системы эти пять паттернов — базовый набор анти-паттернов, которые система должна обнаруживать и предлагать исправления.

1.3. Элементная модель игры

Если определение и принципы задают «что» и «почему», то элементная модель отвечает на вопрос «из чего» состоит игра. Существует несколько конкурирующих моделей, и каждая подчёркивает разные аспекты. Мы рассмотрим четыре модели — от базовой до расширенной — и покажем, как они дополняют друг друга в единую систему.

1.3.1. Базовая тетрада Шелла

Джесси Шелл (Кн. 1, 2) предлагает самую интуитивную и широко признанную модель: любая игра состоит из четырёх равноправных элементов:

- **Механика** — правила и процессы игры. Цели, допустимые и запрещённые действия, последствия выбора. Механика делает игру игрой, определяя, что можно и чего нельзя делать. Примеры: система боя в Dark Souls, система ходов

в шахматах, экономическая модель в Civilization. - **История** — последовательность событий, линейная или разветвлённая. История даёт смысл действиям, превращая абстрактные механические взаимодействия в нарративный опыт. Примеры: сюжетная линия The Last of Us, лор Dark Souls, предыстория мира Warcraft. - **Эстетика** — как игра выглядит, звучит, ощущается. Эстетика воздействует на опыт игрока наиболее напрямую, через органы чувств. Примеры: пиксель-арт Celeste, оркестровый саундтрек NieR, кинестетика управления в Super Mario. - **Технология** — материалы и взаимодействия, позволяющие игре функционировать. Не обязательно «хай-тек» — любая основа: картон и пластик для настольной игры, игровая консоль для видеоигры, веб-браузер для браузерной игры. Технология определяет, что возможно, а что нет.

Ключевой принцип: ни один элемент не важнее другого. Все четыре одинаково влияют на опыт игрока и должны учитываться в каждом дизайнерском решении. Игры часто проваливаются не потому, что один элемент плох, а потому, что элементы не согласованы между собой. Великие игры — это игры, в которых все четыре элемента усиливают друг друга, создавая синергию.

Призма голографического дизайна (Шелл, Призма 10) углубляет этот принцип: каждый элемент должен содержать «голографическое» отражение целого. Механика должна воплощать историю, история — раскрывать механику, эстетика — поддерживать технологию, и так далее. Когда элемент работает против остальных, возникает диссонанс — нарушение единства, которое игрок чувствует, даже если не может сформулировать.

1.3.2. Расширенная тетрада Бонда (матрица 4×3)

Джереми Бонд (Кн. 17) расширяет тетраду Шелла, добавляя **три уровня владения** для каждого элемента. Это ключевое расширение, потому что оно учитывает, что элементы игры существуют не только в том виде, в каком их создал разработчик, но и в том, как они «живут» в руках игроков и в культуре:

Механика История Эстетика Технология
--- ----- ----- ----- ----- Фиксированный уровень (создано разработчиком) Правила, формулы, алгоритмы Сценарий, лор, диалоги
Визуал, звук, анимации Движок, платформа, API Динамический уровень (возникает при взаимодействии) Эмерджентные стратегии, метагейм
Эмергентный нарратив, фан-теории Игровые моменты, скриншоты Моды, настройки, макросы Культурный уровень (вне контроля разработчика)

Мета-игра, спидран, турнирные правила | Фанфики, теории, мемы | Фан-арт, косплей, ремиксы | Моддинг, инструменты, порты |

Значение каждого уровня:

Фиксированный уровень — это то, что полностью контролирует разработчик. Правила игры, написанный сценарий, созданные ассеты, выбранный движок. Это «замороженное» состояние элементов, заданное на момент релиза. Для AI-системы это входные данные генерации: система создаёт именно фиксированный уровень, определяя правила, сюжет, визуальный стиль и технологическую базу.

Динамический уровень — это то, что возникает, когда игрок взаимодействует с фиксированным уровнем. Эмерджентные стратегии (например, «вилка» в шахматах — не прописанное правилами явление, а возникающее из взаимодействия фигур), эмергентный нарратив (истории, которые игроки рассказывают о своих игровых сессиях), игровые моменты (скриншоты, клипы, стримы). Для AI-системы это означает: генерируя фиксированный уровень, нужно предвидеть динамический — какие стратегии могут возникнуть, какие нарративы могут сложиться, какие моменты могут стать вирусными.

Культурный уровень — это то, что выходит за пределы игры и живёт в культуре. Мемы, фан-арт, косплей, моддинг, спидран-комьюнити, турнирные форматы. Разработчик не контролирует этот уровень, но может создавать условия для его возникновения. Для AI-системы это означает: система должна проектировать «пространство возможностей» для культурного уровня — давать игрокам инструменты для творчества, оставлять пространство для интерпретаций, создавать условия для формирования сообщества.

Матрица 4×3 — это **основной аналитический инструмент** Gidede, потому что она позволяет: (1) анализировать каждый элемент игры на трёх уровнях; (2) выявлять диссонансы между уровнями (фиксированный уровень подразумевает одну динамику, а возникает другая); (3) проектировать с учётом всех трёх уровней, а не только фиксированного.

1.3.3. Восемь формальных элементов Фуллертон

Трейси Фуллертон (Кн. 5) предлагает альтернативную декомпозицию игры на восемь формальных элементов. В отличие от тетрады Шелла, которая описывает «типы контента», модель Фуллертон описывает **структурные компоненты** — из чего функционально состоит любая игра:

1. **Игроки** — кто участвует в игре? Сколько игроков? Какова их роль? Каковы отношения между ними (кооперация, соревнование, соло)? Количество и тип игроков определяют всю структуру игры: однопользовательская игра принципиально отличается от многопользовательской.

2. **Цели** — чего пытаются достичь игроки? Цели задают направление всему геймплею. Без чётких целей нет мотивации. Типы целей: победа/поражение, накопление, решение, погоня, спасение, избегание и другие.

3. **Процедуры** — что игроки могут делать? Процедуры — это «глаголы» игры, доступные действия. Они определяют пространство действий игрока и должны быть ясными, отзывчивыми и осмысленными.

4. **Правила** — что игроки могут и чего не могут делать? Правила определяют ограничения и возможности. Три уровня правил: операционные (как играть), фундаментальные (математические системы) и поведенческие (социальные нормы).

5. **Ресурсы** — что игроки используют для достижения целей? Ресурсы — это «существительные» игры: здоровье, мана, золото, время, информация, территория. Тип и распределение ресурсов определяют экономику игры.

6. **Конфликт** — что препятствует достижению целей? Конфликт — двигатель игры. Без конфликта нет вызова, без вызова нет веселья. Конфликт может быть с системой, с другими игроками или с самим собой.

7. **Границы** — где происходит игра? Границы определяют пространство игры, как физическое (игровое поле), так и концептуальное (правила, определяющие, что «внутри» игры, а что «снаружи»). Границы создают магический круг.

8. **Результат** — чем заканчивается игра? Результат должен быть неопределённым до конца — если исход известен заранее, мотивация исчезает. Результат также должен быть измеримым, чтобы игроки могли оценивать свой успех.

Значение для Gidede: восемь формальных элементов — это **модель данных** для AI-генерации. Каждый элемент — это слот, который должен быть заполнен при генерации игры. Если хотя бы один элемент не определён, генерация неполна. Система должна проверять: есть ли игроки? есть ли цели? определены ли процедуры? сформулированы ли правила? выделены ли ресурсы? встроен ли конфликт? заданы ли границы? неопределён ли результат?

1.3.4. Трёхуровневая модель Зубека

Роберт Зубек (Кн. 6), соавтор оригинальной статьи MDA, предлагает альтернативную трёхуровневую модель, сознательно отказываясь от терминов «динамика» и «эстетика», которые, по его мнению, вызывают путаницу в практическом гейм-дизайне:

| Уровень | Описание | Кто инициирует | |-----|-----|-----| | **Механики и системы** | Игровые объекты, действия и правила; системы из механик | Дизайнер создаёт, игрок взаимодействует | | **Геймплей** | Динамический процесс взаимодействия игрока с механиками и системами | Игрок и игра совместно | | **Опыт игрока** | Субъективные ощущения и переживания от геймплея | Игрок испытывает |

Критический вывод Зубека: дизайнер не может напрямую управлять опытом игрока — только через манипуляцию механиками и системами. Это «проблема второго порядка»: мы создаём динамический механизм и надеемся, что игрок получит задуманный опыт. Это фундаментальное ограничение, которое определяет весь процесс геймдизайна: мы не можем «спроектировать веселье», мы можем лишь создать условия, при которых веселье возникает.

Два подхода к дизайну: (1) **Сверху вниз** — начинаем с задуманного опыта → планируем геймплей → определяем механики; (2) **Снизу вверх** — начинаем с механик → тестируем геймплей → наблюдаем опыт. Практика — гибридный итеративный подход: планируем сверху вниз, экспериментируем снизу вверх, создаём прототипы с ранних этапов. Для AI-системы это означает поддержку обоих направлений: генерация от опыта к механикам (Reverse MDA) и валидация от механик к опыту (классический MDA).

1.3.5. Интеграция моделей: система Gidede

Четыре модели не противоречат друг другу, а описывают игру с разных ракурсов. В системе Gidede они интегрируются следующим образом:

- **Тетрада Шелла** задаёт **типы контента** — четыре «столпа», на которых стоит игра. Используется как высокоуровневый фреймворк при генерации концепции. - **Матрица 4x3 Бонда** задаёт **уровни анализа** — от замороженного замысла до культурного феномена. Используется как аналитический инструмент для проверки согласованности на всех уровнях. - **Формальные элементы Фуллертона** задаёт **модель данных** — структурные слоты, которые должны быть заполнены. Используется как чек-лист полноты генерации. - **Трёхуровневая модель Зубека** задаёт **направление**

проектирования — от механик к опыту или от опыта к механикам. Используется как архитектурный каркас генеративного процесса.

1.4. Типологии игроков

Понимание того, для кого мы создаём игру, не менее важно, чем понимание того, что мы создаём. Разные игроки получают удовольствие от разных аспектов игры, и геймдизайн, который игнорирует типологию игроков, обречён на узкую аудиторию. Мы рассмотрим четыре модели — от интуитивных до научно обоснованных.

1.4.1. Модель Бартла (4 типа)

Ричард Бартл (1996) создал первую типологию игроков MUD (многопользовательских текстовых миров), которая стала классической:

- **Достижитель** (Achiever) — получает удовольствие от накопления и достижения целей. Собирает предметы, выполняет задания, набирает очки. Достижители — основная аудитория RPG и ММО: система уровней, добычи и достижений создана для них. - **Исследователь** (Explorer) — получает удовольствие от открытия нового. Изучает мир, ищет секреты, тестирует границы системы. Исследователи — аудитория open-world игр и метроидваний: им нужен мир, полный тайн. - **Социализатор** (Socializer) — получает удовольствие от общения и взаимодействия. Строит отношения, создаёт сообщества, общается. Социализаторы — аудитория ММО и социальных игр: им нужны инструменты для взаимодействия. - **Убийца** (Killer) — получает удовольствие от доминирования над другими. Соревнуется, побеждает, навязывает свою волю. Убийцы — аудитория PvP-игр и соревновательных шутеров: им нужны арены и таблицы лидеров.

Ограничения модели: Бартл создавал типологию для MUD, и она плохо обобщается на другие жанры. Она интуитивна, но не эмпирически обоснована — нет строгих данных, подтверждающих именно эти четыре типа. Тем не менее, модель полезна как **быстрый классификатор** для первоначальной оценки аудитории.

1.4.2. Восемь типов удовольствия ЛеБланка

Марк ЛеБланк (соавтор MDA) предлагает типологию не игроков, а **типов удовольствия**, которые игры могут доставлять. Это более гранулярная модель, которая фокусируется на «что», а не на «кто»:

1. **Чувственное удовольствие** (Sensation) — радость от чувственного восприятия: красота графики, музыка, кинестетика управления. Примеры: Journey (визуальная красота), Beat Saber (ритм и движение), Rez (синестезия звука и визуала). 2. **Фантазия** (Fantasy) — удовольствие от пребывания в другом мире, от роли, которую играешь. Примеры: Skyrim (фэнтезийный мир), The Witcher (отыгрыш роли), Animal Crossing (идиллическая жизнь). 3. **Нарратив** (Narrative) — удовольствие от истории, от драматического развёртывания событий. Примеры: The Last of Us, Disco Elysium, 80 Days. 4. **Вызов** (Challenge) — удовольствие от преодоления трудностей, от роста мастерства. Примеры: Dark Souls, Super Meat Boy, Celeste. 5. **Товарищество** (Fellowship) — удовольствие от связи с другими игроками, от сотрудничества и общения. Примеры: Overcooked, Journey (анонимная кооперация), ММО-рейды. 6. **Открытие** (Discovery) — удовольствие от нахождения нового, от понимания того, как устроен мир. Примеры: Outer Wilds, Return of the Obra Dinn, Hollow Knight. 7. **Выражение** (Expression) — удовольствие от самовыражения, от создания и персонализации. Примеры: Minecraft, The Sims, customize-системы в RPG. 8. **Подчинение** (Submission) — удовольствие от подчинения правилам игры, от отдыха в структурированной деятельности. Примеры: фермы в ММО, гринд в Diablo, рутинные занятия в Animal Crossing.

Значение для Gidede: восемь типов удовольствия — это **целевая функция** генерации. Когда AI определяет, какую игру создать, он сначала определяет целевые типы удовольствия, а затем подбирает механики, которые их обеспечивают. Это основа Reverse MDA-процесса: от эстетики (типы удовольствия) к динамике к механике.

1.4.3. Модель мотивации Йи (12 типов в 3 кластерах)

Ник Йи (Nick Yee) создал наиболее научно обоснованную модель игровой мотивации на основе масштабных эмпирических исследований (Quantic Foundry, 250 000+ респондентов). Модель выявляет 12 типов мотивации, сгруппированных в 3 кластера:

Кластер «Экшен-социальный» (корреляция с экстраверсией по Big Five): - **Разрушение** (Destruction) — удовольствие от хаоса, разрушения, применения грубой силы - **Возбуждение** (Excitement) — поиск острых ощущений, адреналина, быстрых темпов - **Состязание** (Competition) — стремление превзойти других, измеримое сравнение навыков - **Общение** (Socializing) — взаимодействие с другими, беседы, совместная деятельность

Кластер «Мастерство-достижения» (корреляция с добросовестностью): - **Вызов** (Challenge) — стремление к трудным задачам, требующим мастерства и

практики - **Стратегия** (Strategy) — удовольствие от планирования, оптимизации, принятия решений - **Завершение** (Completion) — стремление собрать всё, завершить все задания, достижение 100% - **Влияние** (Power) — стремление к контролю, доминированию, управлению ресурсами и юнитами

Кластер «Погружение-творчество» (корреляция с открытостью): -

Фантазия (Fantasy) — быть кем-то другим, проживать другую жизнь в другом мире - **Сюжет** (Story) — интерес к нарративу, персонажам, развитию событий - **Дизайн** (Design) — удовольствие от создания, кастомизации, эстетического оформления - **Открытие** (Discovery) — исследование, поиск секретов, понимание скрытых механик

Ключевой инсайт Йи: игры не позволяют «убежать от себя» — наоборот, они **усиливают выражение реальных черт личности**. Игры, в которые мы играем, отражают то, что мы находим привлекательным. Это означает, что мотивационный профиль аудитории — это не абстрактная категория, а проекция реальных психологических черт.

Значение для Gidede: модель Йи — **основная типология** для AI-системы по трём причинам: (1) научно обоснована и эмпирически верифицирована; (2) 12 типов дают достаточную гранулярность для точного профилирования; (3) корреляция с Big Five позволяет связывать игровые предпочтения с психологическими чертами. Система должна использовать модель Йи как входные данные: дизайнер указывает целевой мотивационный профиль, и AI подбирает механики, эстетику и нарратив под этот профиль.

1.4.4. Модель Big Five (психометрическое профилирование)

Модель «Большая пятёрка» (OCEAN) — это золотой стандарт академической психологии личности, описывающий пять фундаментальных измерений:

- **Открытость опыту** (Openness) — склонность к новому, креативности, исследованию - **Добросовестность** (Conscientiousness) — организованность, дисциплина, стремление к достижению целей - **Экстраверсия** (Extraversion) — общительность, энергичность, стремление к стимуляции - **Приятность** (Agreeableness) — кооперативность, доверчивость, альтруизм - **Нейротизм** (Neuroticism) — эмоциональная нестабильность, тревожность, чувствительность к стрессу

Селлерс (Кн. 13) связывает Big Five с игровыми предпочтениями: экстраверсия коррелирует с социальными и соревновательными механиками, добросовестность — с прогрессией и завершением, открытость — с исследованием и нарративом. Это позволяет использовать Big Five как

глубинный аналитический слой: если мы знаем Big Five-профиль аудитории, мы можем предсказать не только «какие механики им нравятся», но и «почему они им нравятся».

Интеграция трёх моделей в Gidede:

| Слой | Модель | Роль в системе | |-----|-----|-----| | Быстрая классификация | Bartle (4 типа) | Первоначальная оценка аудитории, маппинг на жанры | | Целевая функция | ЛеБланк (8 типов) | Определение эстетических целей генерации | | Основной профиль | Йи (12 типов) | Гранулярное профилирование, подбор механик | | Глубинный анализ | Big Five (5 измерений) | Психологическое обоснование, предсказание поведения |

Маппинг между моделями: каждый тип Йи может быть маппирован на 1-2 типа Бартла и 1-2 типа удовольствия ЛеБланка. Например, «Вызов» (Йи) → Достижитель/Убийца (Бартл) → Вызов (ЛеБланк). «Фантазия» (Йи) → Исследователь (Бартл) → Фантазия + Нарратив (ЛеБланк). Это обеспечивает совместимость моделей на всех уровнях анализа.

1.5. Типологии механик

Механики — строительные блоки игры. Их классификация — фундамент для генеративного модуля AI-системы: если мы знаем, какие типы механик существуют и как они сочетаются, мы можем алгоритмически подбирать и комбинировать их. Как и в случае с типологиями игроков, существует несколько уровней классификации, от абстрактных до конкретных.

1.5.1. Фундаментальный уровень: 7 механик Шелла

Джесси Шелл (Кн. 1, 2) выделяет 7 базовых механик, из которых строится любая игра. Это самый абстрактный уровень классификации, применимый к любому жанру и платформе:

1. **Пространство** — дискретное или непрерывное; число измерений; границы; вложенные пространства. Убрать эстетику — и получится абстрактная функциональная модель. Например, Монополия — это одномерный цикл из 40 точек, а шахматы — двумерная сетка 8×8. Пространство определяет, где происходит игра и как игроки перемещаются.

2. **Время** — реальное время vs пошаговый; переменные интервалы; время как ресурс; лимиты; асимметрия времени. Время определяет темп игры и

давление на игрока. Реальное время создаёт срочность, пошаговый режим — стратегическую глубину, асимметрия — разные типы взаимодействия.

3. **Объекты и состояния** — сущности с атрибутами (числовые, булевы, перечислимые) и состояниями. Скрытая vs открытая информация. Изменение состояний — двигатель игры: без изменения состояний нет прогресса. Объекты — это «существительные» игры.

4. **Действия** — оперативные (базовые глаголы: двигаться, атаковать, использовать) vs результирующие (эмерджентные стратегии: «фланкировать», «кемперить», «зонить»). Разные режимы дают разные наборы действий. Действия — это «глаголы» игры.

5. **Правила** — операционные (как играть), фундаментальные (математические системы), поведенческие (социальные нормы). Правила определяют идентичность игры: изменение даже одного правила может превратить шахматы в совершенно другую игру.

6. **Навык** — физические (ловкость, тайминг, реакция) vs ментальные (стратегия, память, головоломки, наблюдательность). Навык определяет кривую обучения и тип вызова. Игры с физическим навыком — это игры «рук», игры с ментальным навыком — игры «головы».

7. **Шанс** — случайность и вероятность; ожидаемое значение; риск vs неопределённость. Шанс добавляет непредсказуемость и реиграбельность, но должен балансироваться с навыком. Слишком много шанса — и решения теряют значимость; слишком мало — и игра становится детерминированной.

Значение для Gidede: семь механик Шелла — это **верхний уровень таксономии** MechanicsDB (Level 0). При генерации игры система сначала определяет, какие из семи фундаментальных механик будут центральными (например, шутер = пространство + время + действия + навык + шанс; стратегия = пространство + объекты + действия + правила).

1.5.2. Структурный уровень: 5 типов Адамса/Дорманса

Адамс и Дорманс (Кн. 4) предлагают классификацию на уровне **типов взаимодействия**, а не базовых элементов. Это средний уровень абстракции, который определяет, как механики работают в системе:

1. **Физика** — механики, основанные на физическом моделировании: гравитация, столкновения, траектории. Игры на физике: Angry Birds, Portal, BeamNG.drive. Физика создаёт интуитивно понятное пространство действий:

игроки понимают, что произойдёт, потому что законы физики знакомы из реальности.

2. **Внутренняя экономика** — механики, основанные на производстве, потреблении и обмене ресурсов. Игры на экономике: Civilization, Factorio, Idle-игры. Экономика создаёт стратегическую глубину: игроки должны оптимизировать распределение ограниченных ресурсов.

3. **Прогрессия** — механики, основанные на продвижении по заданному пути: уровни, главы, сюжетные точки. Игры на прогрессии: Half-Life, Uncharted, линейные RPG. Прогрессия создаёт структуру опыта: дизайнер контролирует последовательность впечатлений.

4. **Тактическое маневрирование** — механики, основанные на позиционировании и принятии решений в конкретных ситуациях. Игры на тактике: XCOM, Into the Breach, шахматы. Тактическое маневрирование создаёт осмысленный выбор: каждое решение о позиционировании имеет последствия.

5. **Социальное взаимодействие** — механики, основанные на взаимодействии между игроками: торговля, дипломатия, предательство, сотрудничество. Игры на социальном взаимодействии: Diplomacy, Among Us, Eve Online. Социальные механики создают эмерджентность: непредсказуемость человеческого поведения порождает уникальные ситуации.

Значение для Gidede: пять типов Адамса/Дорманса — это **средний уровень таксономии** MechanicsDB (Level 1). При генерации игры система определяет, какой тип взаимодействия будет доминирующим (например, RPG = экономика + прогрессия + тактическое маневрирование; спортивная игра = физика + тактическое маневрирование).

1.5.3. Семейства механик: 4 семейства Зубека

Зубек (Кн. 6) группирует механики по **функциональному назначению**, а не по типу взаимодействия:

1. **Механики контроля (Three C's)** — Character, Camera, Control: персонаж, камера, управление. Это фундамент любой игры с прямым управлением: как игрок воспринимает мир (камера), как действует в нём (персонаж), как отдаёт команды (управление). Стили управления: Donkey Kong (прыжок + бег), Super Mario (добавлен набор скорости), Asteroids (вращение + тяга). Правильные Three C's — это разница между «игра ощущается хорошо» и «управление неуклюже».

2. **Механики прогрессии** — прямые (очки, уровни, достижения, списки лидеров) и не прямые (изменение среды, расширение пространства действий). Прямая прогрессия видима и измерима, не прямая — ощущается через изменение геймплея. Геймификация = механики прогрессии, перенесённые вне игр, но она не создаёт мотивацию сама по себе — только структурирует уже существующую.

3. **Механики неопределённости** — случайность (стационарная: кости; нестационарная: тасование карт), скрытая информация, непредсказуемость других игроков. Неопределённость — это то, что делает игру интересной; без неё решения тривиальны. Но неопределённость должна быть справедливой: игрок должен понимать, почему он проиграл, и чувствовать, что мог решить иначе.

4. **Механики управления ресурсами** — юниты (с параметрами и способностями), модификаторы (баффы/нерфы), валюта, объекты, расходимые материалы, сырьё, пространство как ресурс. Управление ресурсами создаёт экономический слой игры: ограниченность ресурсов порождает осмысленный выбор — что потратить, что сохранить, на что обменять.

1.5.4. Дискретный уровень: 127 механик SW.BAND

На самом гранулярном уровне — **127 дискретных механик** из колоды карт геймдизайнера (Кн. 15), сгруппированных в 15 категорий. Это «генеративный алфавит» для AI-системы: каждая механика — это конкретный строительный блок, который можно выбрать и скомбинировать с другими.

Ключевое свойство этого уровня — **модель совместимости**: - **Синергии**: механики, усиливающие друг друга (Стелс + Бесшумное оружие + Ночное видение = эффективная стелс-система) - **Конфликты**: взаимоисключающие механики (Без интерфейса + Показ цели = противоречие в дизайне) - **Нейтральные**: механики, сочетающиеся с большинством без эффекта

Также существует **матрица «Механика → Жанр»** (127 механик × 6 жанров), где каждая клетка содержит трёхуровневую оценку релевантности. Это позволяет AI-системе при генерации игры для конкретного жанра автоматически фильтровать механики по релевантности.

Значение для Gidede: 127 механик — это **нижний (генеративный) уровень** MechanicsDB (Level 3). При генерации система: (1) определяет жанр и целевые эстетики; (2) фильтрует механики по матрице «Механика → Жанр»; (3)

проверяет совместимость выбранных механик; (4) формирует набор механик с максимальной синергией и без конфликтов.

1.5.5. Многоуровневая таксономия MechanicsDB

Объединяя все четыре уровня классификации, мы получаем **многоуровневую таксономию механик** — основу генеративного модуля Gidede:

```
``` MechanicsDB |— Level 0: Фундаментальные типы (7 от Шелла) | |—  
Пространство | |— Время | |— Объекты/Состояния | |— Действия | |—
Правила | |— Навык | |— Шанс |— Level 1: Структурные типы (5 от Адамса/
Дорманса) | |— Физика | |— Внутренняя экономика | |— Прогрессия | |—
Тактическое маневрирование | |— Социальное взаимодействие |— Level 2:
Паттерны дизайна (16+ от Адамса/Дорманса) | |— Двигатели: Static, Dynamic,
Converter, Engine Building | |— Трения: Static Friction, Dynamic Friction, Stopping,
Attrition | |— Эскалации: Escalating Challenge, Complexity, Arms Race | |—
Прочие: Play-Style Reinforcement, Multiple Feedback, | Trade, Worker Placement,
Slow Cycle |— Level 3: Дискретные механики (127 от SW.BAND) | |— Группа 1:
Основные (10) | |— Группа 2: Симуляция/Сложность (9) | |— Группа 3:
Пространство/Экономика (9) | |— Группа 4: Враги/Бой (8) | |— Группа 5:
Время/Пространство (8) | |— Группа 6: Персонаж/Смерть (8) | |— Группа 7:
Социальность/Информация (8) | |— Группа 8: Оружие/Обучение (8) | |—
Группа 9: Развитие персонажа (8) | |— Группа 10: Управление/Бой (8) | |—
Группа 11: Уровни (8) | |— Группа 12: Особенности игры (8) | |— Группа 13:
Особое (6) | |— Группа 14: Управление (расш.) (7) | |— Группа 15: Уровни
(расш.) (5) |— Level 4: Жанровые реализации (8 от Бонда + шаблоны) |—
Apple Catcher |— Physics Launcher |— Shoot-em-up |— Card Game |— Card MP
Game |— Word Game |— RPG ```
```

**Принцип работы таксономии:** генерация идёт сверху вниз. Сначала система определяет, какие фундаментальные типы (Level 0) релевантны для данного жанра. Затем — какие структурные типы (Level 1) будут доминировать. Затем — какие паттерны дизайна (Level 2) обеспечат нужную динамику. Затем — какие конкретные механики (Level 3) будут реализованы. Наконец — какие жанровые шаблоны (Level 4) зададут общую структуру.

### 1.5.6. Языковая метафора механик

Зубек (Кн. 6) и Гэзэуэй (Кн. 9) предлагают продуктивную метафору для понимания и генерации механик — **языковую**:

- **Объекты** = **Существительные** — сущности в игре (меч, зелье, враг, дверь) -  
**Действия** = **Глаголы** — что можно делать с объектами (атаковать, пить,

сражаться, открывать) - **Правила = Грамматика** — как глаголы и существительные сочетаются (можно атаковать врага мечом, но не зельем) - **Модификаторы = Прилагательные/наречия** — уточнения (огненный меч, быстро бежать, скрытно атаковать)

Гэзэуэй развивает эту метафору в **грамматику игры**: существительные = атрибуты (данные), глаголы = механики (правила/код), предложения = микросистемы, главы = макросистемы, книги = игры. Это иерархическая модель, в которой каждый уровень строится на предыдущем.

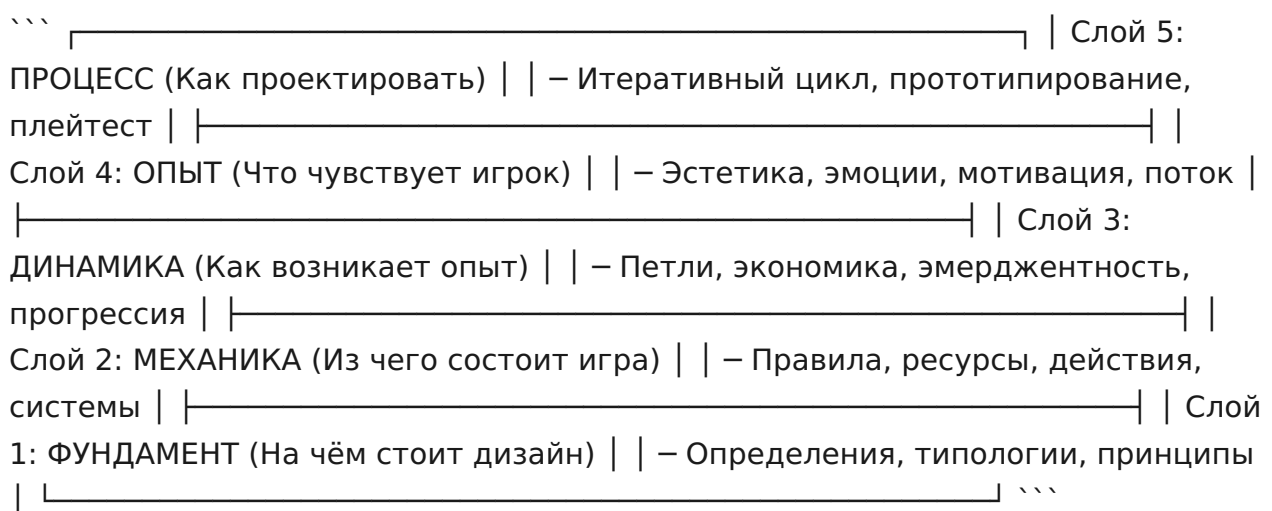
**Значение для Gidede**: языковая метафора — основа для NLP-генерации механик. Когда AI получает описание концепции на естественном языке («игра про исследование подземелий»), он может разобрать его на «существительные» (подземелье, сокровище, ловушка, монстр) и «глаголы» (исследовать, находить, избегать, сражаться), а затем сгенерировать механики, комбинируя их по правилам «грамматики» игры.

---

## 1.6. Пятислойная модель Gidede

---

Объединяя все рассмотренные выше модели — определения, принципы, элементы, типологии игроков и механик — мы приходим к **пятислойной модели геймдизайна**, которая является архитектурным каркасом всей системы Gidede. Каждый слой соответствует уровню абстракции, от фундаментальных принципов до конкретных реализуемых алгоритмов:



### 1.6.1. Слой 1: Фундамент

Фундамент — это предмет данной главы. Он включает: - **Определения игры** — что мы создаём, и как это проверить - **Принципы геймдизайна** — что

отличает хороший дизайн от плохого - **Элементные модели** — из чего состоит игра (тетрада, формальные элементы, трёхуровневая модель) - **Типологии игроков** — для кого мы создаём - **Типологии механик** — из чего строим

Фундамент не генерируется — он задаётся как константа системы. Это «физика» AI-системы: законы, в рамках которых работает генерация.

### 1.6.2. Слой 2: Механика

Механика — это конкретные правила, объекты, действия и системы, из которых состоит игра. Сюда входят MechanicsDB (5-уровневая таксономия), формальные модели (пространство состояний, Machinations), и паттерны дизайна (16+ паттернов Адамса/Дорманса). Механика — то, что AI генерирует из базы данных и что алгоритмы проверяют на корректность. Подробно рассматривается в Главе 2 «Элементы игры».

### 1.6.3. Слой 3: Динамика

Динамика — это то, что происходит, когда механика встречается с игроком. Сюда входят петли обратной связи, экономика игры, эмерджентность, прогрессия и типология систем (Engines/Economies/Ecologies). Динамика — самое сложное для генерации и самое важное для валидации. Подробно рассматривается в Главах 3-6.

### 1.6.4. Слой 4: Опыт

Опыт — это то, что чувствует игрок: эстетика, эмоции, мотивация, поток. Сюда входят 8 типов эстетики ЛеБланка, теория потока Чиксентмихайи, нейрехимия вовлечения, типы мотивации, нарратив и racing. Опыт — целевая функция генерации: мы начинаем с желаемого опыта (Reverse MDA) и проверяем результат (классический MDA). Подробно рассматривается в Главах 7-8.

### 1.6.5. Слой 5: Процесс

Процесс — это то, как мы работаем: итеративный дизайн, прототипирование, плейтестирование, документация. Процесс — мета-слой, который определяет, как все остальные слои взаимодействуют в реальной практике разработки. Подробно рассматривается в Главах 10-11.

### 1.6.6. Двойной MDA-процесс Gidede

Пятислойная модель определяет два направления работы AI-системы:

**Генеративное направление (Reverse MDA, от Бонда, Кн. 17):** от Слоя 4 (Опыт) через Слой 3 (Динамика) к Слою 2 (Механика). Дизайнер определяет, какой опыт он хочет создать (какие эстетики, какие эмоции, какая мотивация). AI выводит, какая динамика нужна для этого опыта. Затем AI подбирает механики из MechanicsDB, которые порождают нужную динамику. Это «сверху вниз» — от цели к средству.

**Аналитическое направление (классический MDA, от Hunicke/LeBlanc/Zubek):** от Слоя 2 (Механика) через Слой 3 (Динамика) к Слою 4 (Опыт). AI анализирует, какие эстетики порождают данные механики через динамику. Это проверка — соответствует ли сгенерированный результат задуманному опыту. Это «снизу вверх» — от средства к результату.

Оба направления работают итеративно в рамках Слоя 5 (Процесс): генерируем → анализируем → корректируем → повторяем.

---

## Резюме главы

---

Фундамент геймдизайна — это основа, на которой строится всё остальное. В этой главе мы рассмотрели:

1. **Определение игры** как операциональное понятие для AI-системы: «Игра — это замкнутая формальная система с неопределённостью исхода, в которой игрок принимает эндогенно значимые решения, создавая эстетический опыт». Каждый элемент определения проверяем алгоритмически.
2. **Пять универсальных принципов** геймдизайна: итеративный цикл, баланс (нет доминантной стратегии), Core Loop, поток и эмерджентность. Эти принципы подтверждены 12+ книгами из 17 и образуют ядро системы валидации.
3. **Элементные модели** — четыре взгляда на то, из чего состоит игра: тетрада Шелла (типы контента), матрица 4×3 Бонда (уровни анализа), формальные элементы Фуллerton (модель данных), трёхуровневая модель Зубека (направление проектирования).
4. **Типологии игроков** — четыре модели: Bartle (быстрая классификация), ЛеБланк (целевая функция), Йи (основной профиль), Big Five (глубинный анализ). Интеграция моделей через маппинг обеспечивает совместимость.
5. **Типологии механик** — многоуровневая таксономия MechanicsDB: от 7 фундаментальных типов Шелла (Level 0) через 5 структурных типов Адамса/

Дорманса (Level 1), 16+ паттернов дизайна (Level 2) и 127 дискретных механик SW.BAND (Level 3) до жанровых реализаций (Level 4).

6. **Пятислойная модель** Gidede — архитектурный каркас системы: Фундамент → Механика → Динамика → Опыт → Процесс, с двойным MDA-процессом (генеративный + аналитический) как механизмом работы.

Этот фундамент будет развёрнут в последующих главах: Глава 2 углубится в элементы игры и MechanicsDB, Глава 3 — в MDA Framework, Глава 4 — в Core Loop, и так далее по всем 12 разделам Библии геймдизайна.

---

## Глава 2. Элементы игры

---

> **Раздел Библии геймдизайна:** 2.2 — Элементы игры > **Источники:** KB-02 (Элементы игры), KB-04 (MechanicsDB), KB-05 (Типология систем) > **Книги:** Шелл (Кн. 1, 2), Адамс/Дорманс (Кн. 4), Фуллертон (Кн. 5), Зубек (Кн. 6), Селлерс (Кн. 13), SW.BAND (Кн. 15), Бонд (Кн. 17)

---

### 2.1. От фундамента к элементам: архитектура игры

---

В первой главе мы определили, что такое игра, сформулировали принципы геймдизайна и рассмотрели типологии игроков и механик. Теперь мы переходим от «что и почему» к «из чего» — к конкретным элементам, из которых строится игровая система. Если фундамент — это законы физики, то элементы — это атомы и молекулы, из которых состоит материальный мир игры.

Любая игра, от простейшей настольной игры до самой сложной ММО, состоит из четырёх макроэлементов: **механики, динамики, эстетики и истории** (тетрада Шелла). Но каждый из этих макроэлементов, в свою очередь, состоит из множества микроэлементов — конкретных правил, ресурсов, действий, систем обратной связи, визуальных стилей и нарративных структур. В этой главе мы детально разберём каждый макроэлемент и его внутреннее устройство, а также покажем, как они взаимодействуют, создавая единый игровой опыт.

---

## 2.2. Механика: правила, ресурсы и действия

---

Механика — это «скелет» игры, её формальная система. Механика определяет, что возможно, что запрещено и какие последствия имеют действия игрока. Без механики нет игры — есть только неинтерактивный контент (фильм, книга, картина). Механика делает игру игрой: она создаёт пространство состояний, в котором игрок принимает решения.

### 2.2.1. Три уровня механики

Механика игры существует на трёх уровнях, каждый из которых играет свою роль в проектировании и анализе:

**Операционный уровень** — правила, которые игрок непосредственно применяет. Как ходить? Как атаковать? Сколько стоит построить юнита? Это «инструкция по эксплуатации» игры, которую игрок изучает и осваивает. Операционные правила могут быть явными (описаны в инструкции) или неявными (скрыты в коде и открываются через игру).

**Фундаментальный уровень** — математические системы, лежащие в основе операционных правил. Формула урона, таблица вероятностей, кривая прогрессии, экономическая модель. Игрок может не знать этих формул, но они определяют его опыт. Фундаментальный уровень — это «движок» игры, и именно он поддаётся формализации и алгоритмизации.

**Поведенческий уровень** — социальные нормы и негласные правила, возникающие вокруг игры. Это могут быть правила этикета в ММО, негласные договорённости в спортивных играх, или традиции сообщества (например, «не атаковать новичков» в EVE Online). Поведенческий уровень не контролируется дизайнером напрямую, но дизайн может создавать условия для его формирования.

Для AI-системы Gidede критически важно разделение операционного и фундаментального уровней: AI генерирует фундаментальный уровень (формулы, модели, структуры), а операционный уровень формулируется как понятные игроку правила поверх фундаментального.

### 2.2.2. Семь базовых механик Шелла

Джесси Шелл (Кн. 1, 2) выделяет семь базовых механик, из которых строится любая игра. Эти механики — не типы контента, а **фундаментальные измерения** игрового пространства, каждое из которых можно настраивать:



**1. Пространство** — абстрактная функциональная модель среды игры. Важно не визуальное оформление, а структурные свойства: дискретное или непрерывное, сколько измерений, какие границы, есть ли вложенные пространства. Шелл приводит пример: уберите эстетику из Монополии — и останется одномерный цикл из 40 точек. Шахматы — двумерная сетка 8×8 с особыми свойствами клеток. Пространство определяет, где игрок может действовать и как перемещается.

Типы пространств: дискретные (клетки, гексы, узлы графа) и непрерывные (открытый мир, 2D/3D-пространство). Вложенные пространства (комната внутри здания внутри города) создают иерархию масштабов. Границы пространства (стены, края карты, зоны доступа) определяют, что доступно, а что нет.

**2. Время** — темп и ритм игры. Реальное время создаёт срочность и давление: игрок должен действовать быстро или проиграть. Пошаговый режим создаёт стратегическую глубину: игрок может обдумывать каждый ход. Но время — не только режим; это и ресурс (таймеры, лимиты, кулдауны), и структура (раунды, фазы, волны), и асимметрия (один игрок действует в реальном времени, другой — пошагово).

Переменные интервалы времени — мощный инструмент дизайна. Ускорение времени создаёт напряжение (таймер обратного отсчёта), замедление — драматический эффект (bullet time). Время как ресурс заставляет игрока выбирать: потратить время на одно действие или на другое.

**3. Объекты и состояния** — сущности, из которых состоит игровой мир. Каждый объект имеет атрибуты (числовые, булевы, перечислимые) и состояния. Изменение состояний — двигатель игры: без изменения состояний нет прогресса, нет опыта. Скрытая vs открытая информация определяет, что игрок знает о состояниях объектов — и это ключевой источник неопределённости.

Объекты — это «существительные» игры (меч, зелье, враг, дверь, юнит, ресурс). Их состояния — это значения атрибутов (HP = 50, открытость = true, уровень = 3). Переходы между состояниями — это события, которые и составляют геймплей.

**4. Действия** — «глаголы» игры, то, что игрок может делать. Шелл различает два типа действий: оперативные (базовые глаголы — двигаться, атаковать, использовать, строить) и результирующие (эмерджентные стратегии — «фланкировать», «кемперить», «зонить», «гриндить»). Чем больше

оперативных действий и чем свободнее их комбинации, тем больше результирующих действий — и тем глубже игра.

Разные режимы игры дают разные наборы действий: в режиме исследования доступны одни действия, в режиме боя — другие, в режиме крафтинга — третьи. Это создаёт контраст и разнообразие, но требует аккуратного дизайна: переключение режимов не должно ломать поток.

**5. Правила** — определяют идентичность игры. Изменение даже одного правила может превратить шахматы в совершенно другую игру. Правила задают ограничения и возможности, определяют допустимые действия и их последствия. Без правил нет структуры — есть только хаос.

**6. Навык** — способность игрока выполнять действия эффективно. Физические навыки (ловкость, тайминг, реакция) и ментальные навыки (стратегия, память, наблюдательность, логика) определяют кривую обучения и тип вызова. Баланс навыка и шанса — один из 12 типов баланса Шелла: игры чистого навыка (шахматы), игры чистого шанса (рулетка), и большинство игр — где навык и шанс сосуществуют.

**7. Шанс** — источник неопределённости и реиграбельности. Шанс добавляет вариативность: каждый розыгрыш уникален, каждое прохождение отличается от предыдущего. Но шанс должен балансироваться с навыком: слишком много шанса — и решения теряют значимость (рулетка); слишком мало — и игра становится детерминированной (решённой).

Ожидаемое значение ( $EV = \text{вероятность} \times \text{ценность}$ ) — фундаментальный инструмент анализа баланса между навыком и шансом. Если для двух вариантов  $EV$  одинаков, но один требует мастерства, а другой — удачи, игрок воспринимает их по-разному: мастерство ощущается «заслуженным», удача — «случайной».

### 2.2.3. Пять структурных типов механики

Адамс и Дорманс (Кн. 4) предлагают классификацию на уровне **типов взаимодействия**, определяющих, как механики работают в системе:

**1. Физика** — механики, основанные на физическом моделировании: гравитация, столкновения, траектории, силы. Физика создаёт интуитивно понятное пространство действий: игроки понимают, что произойдёт, потому что законы физики знакомы из реальности. Игры на физике (Angry Birds, Portal, BeamNG.drive) обладают естественной эмерджентностью: даже простые правила порождают сложное поведение. Физика — непрерывная механика, моделируемая в реальном времени через уравнения.

**2. Внутренняя экономика** — механики, основанные на производстве, потреблении и обмене дискретных ресурсов. «Экономика» понимается широко: не только деньги, но и здоровье, опыт, боеприпасы, юниты, очки действий — любой ресурс, который можно посчитать. Экономика — дискретная механика, поддающаяся строгому математическому анализу. Это единственный тип механики, который можно полностью формализовать и смоделировать без написания кода (через Machinations).

Пять элементов внутренней экономики (по Адамсу/Дормансу): Источник (генерирует ресурс), Сток (уничтожает ресурс), Пул (хранит ресурс), Конвертер (превращает один ресурс в другой), Торговец (обменивает ресурсы между пулами). Пять структур: Петля (обратная связь), Блокировка (взаимозависимость), Равновесие (производство = потребление), Дефицит (потребление > производство), Профицит (производство > потребление).

**3. Прогрессия** — механики, основанные на продвижении по заданному пути. Уровни, главы, сюжетные точки, механизм «замка и ключа» — всё это прогрессия. Прогрессия создаёт структуру опыта: дизайнер контролирует последовательность впечатлений, обеспечивая, что игрок встречает вызовы в правильном порядке и с правильной сложностью. Прогрессия — доминирующий тип механики в линейных играх (Half-Life, Uncharted, Portal), но присутствует и в нелинейных (основная сюжетная линия в Skyrim).

**4. Тактическое маневрирование** — механики, основанные на позиционировании и принятии решений в конкретных ситуациях. Контроль территории, линия видимости, зоны влияния, покрытие — всё это тактическое маневрирование. Оно создаёт осмысленный выбор: каждое решение о позиционировании имеет последствия, и «правильная позиция» зависит от контекста. Тактическое маневрирование — доминирующий тип механики в стратегиях (XCOM, Into the Breach, шахматы).

**5. Социальное взаимодействие** — механики, основанные на взаимодействии между игроками: торговля, дипломатия, предательство, сотрудничество, блеф. Социальные механики создают самый высокий уровень эмерджентности: непредсказуемость человеческого поведения порождает уникальные ситуации, которые невозможно запланировать. Социальное взаимодействие — доминирующий тип механики в многопользовательских играх (Diplomacy, Among Us, EVE Online).

## 2.2.4. Типология игровых систем: Engines, Economies, Ecologies

Майкл Селлерс (Кн. 13) предлагает формальную классификацию игровых систем по двум осям: тип петли (усиливающая или балансирующая) и тип ресурсного потока (один ресурс или конвертация):

**Двигатели (Engines)** — усиливающие или балансирующие петли с одним ресурсом. Усиливающие (boosting) двигатели добавляют ресурсы в игру, создавая рост, но рискуя runaway-эффектом («богатые богатеют»). Пример: Монополия — деньги → недвижимость → больше денег. Тормозящие (braking) двигатели удаляют ресурсы, предотвращая runaway. Пример: налоги, износ, расход маны.

Диагностика: усиливающий двигатель без тормозящего = runaway. Решение: добавить тормозящий двигатель. Тупик (deadlock) = ресурс цикла нужен для запуска. Остановка (stall) = недостаточно ресурсов для продолжения.

**Экономики (Economies)** — усиливающие петли с обменом/конвертацией ресурсов. Игрок обменивает один ресурс на другой с нелинейным приростом ценности. Экономики обладают свойством «развёртывания сложности»: начинают просто (хлеб + дерево) и постепенно добавляют новые ресурсы и цепочки (руда → железо → сталь → оружие). Каждая новая валюта увеличивает глубину решений. Валюты — каталитические ресурсы, которые не потребляются при обмене (золото, серебро), но подвержены инфляции и стагнации.

Пример RPG: убить монстра → получить опыт и золото → повысить уровень → стать сильнее → убить более сильного монстра → получить больше опыта и золота. Это экономика с встроенным двигателем: XP → уровень → способности → больше XP.

**Экологии (Ecologies)** — балансирующие петли с обменом/конвертацией ресурсов. Целое стремится к динамическому равновесию (метастабильности), хотя части постоянно меняются. Экологии обладают свойством устойчивости (resilience) или хрупкости (brittleness): здоровая экология восстанавливается после возмущения, хрупкая — разрушается.

Типы экологий в играх: инвентарь (больше предметов → меньше места → баланс), бой (два противника уравнивают друг друга), социальные экологии (три фракции: если одна усиливается, две другие объединяются — Dark Age of Camelot).

**Комбинирование систем:** реальная игра = иерархия вложенных систем. RPG: экономика (ядро прокачки) + экология (бой) + двигатель (ресурсы) — все вложены друг в друга. Стратегия: несколько двигателей (добыча + производство) + экономика (торговля) + экология (баланс сил).

### 2.2.5. MechanicsDB: многоуровневая таксономия

Объединяя все уровни классификации, MechanicsDB формирует пятиуровневую таксономию — основу генеративного модуля Gidede:

**Level 0: Фундаментальные типы (7 от Шелла)** — пространство, время, объекты, действия, правила, навык, шанс. Это самые абстрактные измерения, применимые к любой игре.

**Level 1: Структурные типы (5 от Адамса/Дорманса)** — физика, внутренняя экономика, прогрессия, тактическое маневрирование, социальное взаимодействие. Это типы взаимодействия, определяющие, как механики работают.

**Level 2: Паттерны дизайна (16+ от Адамса/Дорманса)** — конкретные структурные паттерны, каждый из которых формализован в Machinations-нотации. Четыре группы: двигатели (Static Engine, Dynamic Engine, Converter Engine, Engine Building), трения (Static Friction, Dynamic Friction, Stopping Mechanism, Attrition), эскалации (Escalating Challenge, Escalating Complexity, Arms Race), и прочие (Play-Style Reinforcement, Multiple Feedback, Trade, Worker Placement, Slow Cycle).

**Level 3: Дискретные механики (127 от SW.BAND)** — конкретные «строительные блоки» в 15 группах. Каждая механика — это именованный, описанный и оценённый элемент, который можно выбрать и скомбинировать. Модель совместимости: синергии (механики, усиливающие друг друга), конфликты (взаимоисключающие механики), нейтральные (без эффекта). Матрица «Механика → Жанр»: 127 механик × 6 жанров с трёхуровневой оценкой релевантности.

**Level 4: Жанровые реализации** — конкретные шаблоны реализации для определённых жанров: Apple Catcher, Physics Launcher, SHMUP, Card Game, Card MP Game, Word Game, RPG. Каждый шаблон описывает типичную структуру механик для данного жанра.

**Принцип генерации:** система работает сверху вниз — от фундаментальных типов к дискретным механикам. Сначала определяет, какие измерения Шелла релевантны. Затем — какие структурные типы доминируют. Затем — какие паттерны дизайна обеспечат нужную динамику. Затем — какие конкретные

механики из 127 будут реализованы. Наконец — какой жанровый шаблон задаст общую структуру.

---

## 2.3. Динамика: петли, обратная связь и эмерджентность

---

Если механика — это то, что дизайнер создаёт, то динамика — это то, что происходит, когда игрок взаимодействует с механикой. Динамика — это «дыхание» игры: механика статична, динамика жива. Петли обратной связи, экономические циклы, эмерджентные стратегии — всё это динамика.

### 2.3.1. Петли обратной связи

Петля обратной связи — фундаментальный элемент динамики игры. Адамс и Дорманс (Кн. 4) формализуют каждую петлю через 8 характеристик:

1. **Тип** — положительная (усиливающая) или отрицательная (балансирующая). Положительные петли усиливают эффект (богатые богатеют), отрицательные — ослабляют (резиновая лента). Оба типа необходимы: без положительных нет роста и прогрессии, без отрицательных нет баланса и устойчивости.

2. **Эффект** — конструктивный (помогает игроку) или деструктивный (вредит). Положительная петля может быть конструктивной (прокачка → больше силы → больше побед) или деструктивной (ранний отставание → ещё большее отставание). Отрицательная петля тоже может быть обоих типов.

3. **Инвестиция** — сколько ресурсов нужно вложить для активации петли. Высокая инвестиция делает петлю стратегическим выбором (построить фабрику vs купить армию), низкая — автоматической (каждый ход приносит доход).

4. **Отдача** — сколько ресурсов возвращается. Отношение отдачи к инвестиции — это «рентабельность» петли. Слишком высокая отдача = runaway, слишком низкая = бессмысленность.

5. **Скорость** — как быстро срабатывает петля. Мгновенная петля (убил врага → сразу получил опыт) ощущается иначе, чем петля с задержкой (инвестировал в здание → прибыль через 5 ходов).

6. **Длительность** — одноразовая или постоянная. Одноразовая петля (бонус за первый вход) создаёт первоначальный импульс, постоянная (ежедневный доход) — долгосрочную структуру.

7. **Косвенность** — прямая или опосредованная. Прямая петля (больше юнитов → больше боевой мощи) понятна игроку. Опосредованная (больше юнитов → противник строит больше защиты → истощение противника) требует глубокой ментальной модели.

8. **Определённость** — детерминированная или вероятностная.

Детерминированная петля (каждый ход +10 золота) предсказуема и стратегична. Вероятностная (20% шанс критического удара) добавляет напряжённость и вариативность.

### 2.3.2. Шесть комбинаций петель

Шрайбер (Кн. 7) выявляет шесть комбинаций, определяющих тип динамики, по двум осям: сумма (положительная, нулевая, отрицательная) и тип обратной связи (положительная, отрицательная):

| | Положительная ОС | Отрицательная ОС | |---|-----|-----| |  
**Положительная сумма** | Экспоненциальный разрыв (богатые богатеют) |  
«Качели» (попеременное лидерство) | | **Нулевая сумма** | «Косичка»  
(поочерёдное доминирование) | Порочный круг (оба теряют) | |  
**Отрицательная сумма** | Саморазрушение (все теряют, лидер медленнее) |  
Стабильное равновесие (оба находят баланс) |

Каждая комбинация создаёт определённый тип динамики, и дизайнер должен сознательно выбирать, какая комбинация нужна для данного аспекта игры. Экспоненциальный разрыв уместен в PvE (ощущение роста), но разрушителен в PvP. Стабильное равновесие хорошо для долгих игр, но может быть скучным. «Качели» создают напряжение, но могут фрустрировать.

### 2.3.3. Четыре главные петли игры

Селлерс (Кн. 13) выделяет четыре главные петли, описывающие всю динамику игры:

**1. Петля модели игры** — внутренняя динамическая модель мира игры. Если модель статична — нет интерактивности, нет геймплея. Динамическая модель создаёт пространство для геймплея через второпорядочный дизайн: дизайнер создаёт не один путь, а пространство возможностей.

**2. Петля ментальной модели игрока** — динамическая модель, которую игрок строит в своём сознании. Когда ментальная модель совпадает с моделью игры, игрок «понимает» игру. Когда ментальная модель неверна, возникает удивление (если в хорошем смысле) или фрустрация (если в

плохом). Цель дизайна — создавать ситуации, где ментальная модель игрока постоянно уточняется и углубляется.

**3. Интерактивная петля** — цикл «действие → обратная связь → обновление ментальной модели → новое действие». «Заккрытие петли» — первый магический момент разработки: когда игрок впервые может действовать, видеть результат и учиться. Шесть временных масштабов этой петли: Action/Feedback (мс-с) → Short-term Cognitive (с-мин) → Long-term Cognitive (мин-ч) → Social (ч-дни) → Emotional (ч-недели) → Cultural (недели-месяцы).

**4. Петля дизайнера** — мета-петля, в которой дизайнер создаёт/изменяет игру на основе наблюдений за игроком. Это самая долгая петля, и её качество определяет качество всей игры. Правило цикла Шелла: чем больше итераций дизайнерской петли, тем лучше игра.

### 2.3.4. Эмерджентность

Эмерджентность — возникновение непредсказуемого поведения из простых правил — это «святой грааль» геймдизайна. Адамс и Дорманс (Кн. 4) вводят дихотомию Юла: игры эмерджентности (мало правил, много взаимодействий — шахматы, Go, RTS) vs игры прогрессии (много правил, заданный путь — Adventure, Portal). Большинство успешных игр комбинируют обе парадигмы.

Типология эмерджентности Фромма (4 уровня): номинальная (предсказуемая, нет петель ОС) → слабая (однонаправленная ОС) → множественная (несколько петель ОС) → сильная (комплекс взаимозависимых петель, принципиально непредсказуемая). Для AI-системы это шкала: генерируя механики, нужно стремиться к множественной или сильной эмерджентности, но избегать номинальной (скучно) и чрезмерно сильной (хаотично).

Второпорядочный дизайн (Селлерс, Кн. 13) — ключевая концепция: дизайнер создаёт пространство состояний, а не один путь. Динамика эмерджентна из механики. AI-система должна не генерировать «правильный» путь, а создавать пространство возможностей, из которых эмерджентно возникает опыт.

---

## 2.4. Эстетика: ощущение, восприятие, впечатление

---

Если механика — это скелет, а динамика — дыхание, то эстетика — это лицо игры, то, что игрок видит, слышит и чувствует непосредственно. Эстетика



воздействует на органы чувств и формирует первое впечатление — а первое впечатление часто определяет, продолжит ли игрок играть.

### 2.4.1. Восемь типов эстетики (ЛеБланк)

Марк ЛеБланк (соавтор MDA) выделяет восемь типов эстетики — видов удовольствия, которые игры могут доставлять. Эти типы — не жанры, а измерения опыта: одна игра может доставлять несколько типов одновременно.

1. **Чувственное удовольствие (Sensation)** — радость от чувственного восприятия: красота графики, музыка, кинестетика управления, тактильная обратная связь. Это самый прямой путь к опыту игрока — через органы чувств, минуя когнитивную обработку. Примеры: Journey (визуальная красота), Beat Saber (ритм и движение), Rez (синестезия звука и визуала), Gran Turismo (реалистичная физика вождения).

2. **Фантазия (Fantasy)** — удовольствие от пребывания в другом мире, от роли, которую играешь. Фантазия переносит игрока в пространство, невозможное в реальности: стать пиратом, волшебником, космонавтом, богом. Фантазия работает через идентификацию: игрок проецирует себя в аватар и через него проживает невозможный опыт. Примеры: Skyrim (фэнтезийный мир), The Witcher (отыгрыш роли ведьмака), Animal Crossing (идиллическая жизнь).

3. **Нарратив (Narrative)** — удовольствие от истории, от драматического развёртывания событий. Нарратив в играх отличается от нарратива в кино и литературе: игрок — не просто зритель, а участник. Это создаёт уникальное напряжение между свободой игрока и контролем автора, которое мы подробно рассмотрим в главе о нарративе. Примеры: The Last of Us, Disco Elysium, 80 Days.

4. **Вызов (Challenge)** — удовольствие от преодоления трудностей, от роста мастерства. Вызов — самый прямой путь к состоянию потока: когда вызов соответствует мастерству, возникает оптимальное напряжение. Вызов может быть физическим (рефлекс), ментальным (стратегия), эмоциональным (эмпатия) или социальным (соревнование). Примеры: Dark Souls, Super Meat Boy, Celeste, Sekiro.

5. **Товарищество (Fellowship)** — удовольствие от связи с другими игроками, от сотрудничества и общения. Товарищество — это не просто «мультиплеер», а осознание связи: чувство, что ты не один, что тебя понимают, что ты часть чего-то большего. Примеры: Overcooked (хаотическая кооперация), Journey (анонимная кооперация), ММО-рейды (слаженная командная работа).

**6. Открытие (Discovery)** — удовольствие от нахождения нового, от понимания того, как устроен мир. Открытие — это исследовательский инстинкт, желание узнать, что за горизонтом, как работает система, что скрывается в тени. Открытие может быть пространственным (новая территория), механическим (новая стратегия) или нарративным (новая информация о мире). Примеры: Outer Wilds, Return of the Obra Dinn, Hollow Knight, Breath of the Wild.

**7. Выражение (Expression)** — удовольствие от самовыражения, от создания и персонализации. Выражение — это творческий импульс: игрок хочет не просто играть, а оставлять след, создавать, персонализировать. Это может быть строительство (Minecraft), кастомизация (The Sims), творчество (Super Mario Maker) или отыгрыш (D&D). Примеры: Minecraft, The Sims, customize-системы в RPG.

**8. Подчинение (Submission)** — удовольствие от подчинения правилам игры, от отдыха в структурированной деятельности. Подчинение — это парадоксальный тип удовольствия: мы наслаждаемся тем, что правила снимают с нас бремя выбора. Фермы в ММО, гринд в Diablo, рутинные занятия в Animal Crossing — всё это подчинение, и оно удовлетворяет глубокую потребность в структурированном отдыхе.

#### 2.4.2. Эстетика как целевая функция генерации

В системе Gidede восемь типов эстетики — это **целевая функция** генерации. Когда AI определяет, какую игру создать, он сначала определяет целевые типы эстетики, а затем подбирает механики, которые их обеспечивают. Это основа Reverse MDA-процесса:

1. **Вход:** дизайнер указывает целевые эстетики (например: Вызов + Открытие + Фантазия) 2. **Генерация:** AI выводит, какая динамика нужна для этих эстетик 3. **Подбор:** AI выбирает механики из MechanicsDB, порождающие нужную динамику 4. **Валидация:** классический MDA — проверка, какие эстетики реально порождают сгенерированные механики

Маппинг «эстетика → динамика → механика» не однозначен: одна и та же механика может порождать разные эстетики в разных контекстах. Но существуют устойчивые связи: Вызов обычно требует баланса сложности и навыка (динамика потока) и механик с осмысленным выбором; Открытие требует скрытой информации и пространства для исследования; Фантазия требует нарративного контекста и эстетического оформления.

### 2.4.3. Сочность (Juiciness) и фидбэк

Шелл (Кн. 1, 2) вводит понятие «сочности» (juiciness) — богатого, многослойного фидбэка на каждое действие игрока. «Сочный» интерфейс даёт визуальные, звуковые и кинестетические подтверждения каждого действия, создавая ощущение силы и контроля. «Движения второго порядка» — производные эффекты, превышающие само действие: удар мечом → вспышка → частицы → тряска камеры → звук → число урона — каждый слой усиливает ощущение.

Сочность — это не «украшательство», а функциональный элемент геймдизайна. Богатый фидбэк помогает игроку понимать состояние игры, чувствовать свои действия и оставаться в потоке. Если фидбэк бедный или запаздывающий, игрок теряет связь с игрой — и теряет интерес.

Правило Шелла: если интерфейс не реагирует за 1/10 секунды, игрок считает, что что-то не так. Это порог восприятия: 100 миллисекунд — граница между «мгновенной» и «запаздывающей» реакцией.

---

## 2.5. История: нарратив, конфликт и агентивность

---

История — элемент, который даёт смысл действиям игрока, превращая абстрактные механические взаимодействия в нарративный опыт. Без истории шахматы — это абстрактная стратегия; с историей (даже минимальной — «я полководец, ведущий армию») — это драма противостояния.

### 2.5.1. Пять проблем интерактивного повествования

Шелл (Кн. 2) формулирует пять фундаментальных проблем, возникающих на стыке интерактивности и нарратива:

1. **Крутость vs свобода:** в хорошей истории герой страдает, но игрок не хочет страдать. Как заставить игрока переживать неприятности, не отталкивая его?
2. **Неудача vs удовольствие:** в хорошей истории герой сталкивается с неудачами, но игрок не хочет проигрывать. Как создавать драму неудачи, не фрустрируя игрока?
3. **Неожиданность vs контроль:** истории строятся на неожиданностях, но игрок, управляющий нарративом, лишает историю неожиданности. Как сохранить сюрприз, давая игроку свободу?

4. **Бесконечность vs завершённость:** игроки хотят играть вечно, но истории требуют концовки. как создать удовлетворяющее завершение, не обрывая игру?

5. **Темп vs контроль:** история разворачивается в своём темпе, но игрок хочет управлять временем. Как согласовать авторский темп с интерактивностью?

### 2.5.2. Два метода: Жемчужная нить и Машина истории

Шелл предлагает два практических метода решения этих проблем:

**Жемчужная нить (String of Pearls):** неинтерактивная история (нитка) чередуется с интерактивными сегментами (жемчужины). [Кат-сцена] → [Свободный геймплей] → [Кат-сцена] → [Свободный геймплей]. Этот метод критикуется за «недостаточную интерактивность», но он работающий и популярный: большинство AAA-игр используют именно его. Ключ к успеху: кат-сцены должны быть короткими, эмоционально насыщенными и мотивирующими, а геймплей — свободным и разнообразным.

**Машина истории (Story Machine):** игра создаёт систему, в которой истории возникают естественно из взаимодействия игроков с механиками. Никакого заранее написанного сценария — но каждый игровой сеанс порождает уникальную историю. Это эмерджентный нарратив, и он мощнее любого скриптового: игрок рассказывает «свою» историю, а не «чужую». Примеры: Dwarf Fortress, RimWorld, EVE Online, Crusader Kings.

Для AI-системы Gidede оба метода должны поддерживаться: Жемчужная нить — для игр с сильным авторским нарративом, Машина истории — для игр с эмерджентным нарративом. Комбинация обоих (как в Mass Effect: основная сюжетная линия + эмерджентные побочные истории) — оптимальная стратегия.

### 2.5.3. Лудонарративная гармония, ирония и диссонанс

Адамс и Дорманс (Кн. 4) расширяют концепцию лудонарративного диссонанса (Андрианова/Яковлева, Кн. 3) до трёх типов соотношения механики и нарратива:

**Гармония:** механика и нарратив говорят одно и то же. В Portal механика порталов буквально воплощает нарратив о «думать нестандартно». В This War of Mine механика дефицита ресурсов воплощает нарратив о тяжести войны. Гармония создаёт синергию — каждый элемент усиливает другие.

**Ирония:** механика и нарратив противоречат друг другу — но осознанно, ради сатиры или критики. В *Papers, Please* механика бюрократической рутины противоречит нарративу о человечности — и это создаёт мощную критику системы. Ирония — это инструмент, а не ошибка.

**Диссонанс:** механика и нарратив противоречат друг другу — неосознанно, из-за ошибки дизайна. В *Uncharted* Натан Дрейк — обаятельный авантюрист в кат-сценах, но массовый убийца в геймплее. Это диссонанс, который разрушает убедительность персонажа. Диссонанс — это баг, который нужно исправлять.

Для AI-системы: валидатор должен проверять соотношение механики и нарратива и отмечать: (1) гармонию — усилить; (2) иронию — осознанно использовать; (3) диссонанс — исправлять.

#### 2.5.4. Агентивность и структуры сюжета

Агентивность (Андрианова/Яковлева, Кн. 3) — способность игрока влиять на развитие событий. Три уровня: высокая (решения меняют мир), низкая (иллюзия влияния), нулевая (наблюдение без влияния). Геймплейная агентивность (доступные действия) и сюжетная агентивность (влияние на сюжет) — не одно и то же: игра может давать свободу действий, но не давать свободы в сюжете (*Far Cry*), и наоборот.

Четыре структуры сюжета: линейная (один путь), разветвлённая (несколько путей), открытая (хабы + квесты), кольцевая (возврат к началу с новым пониманием). *Кисётэнкэцу* — японская 4-частная структура (Введение, Развитие, Поворот, Развязка) — предлагает альтернативу западному трёхактному построению, особенно подходящую для игр, где «поворот» — это изменение восприятия игроком правил игры.

---

### 2.6. Взаимодействие элементов: от тетрады к системе

---

Четыре макроэлемента (механика, динамика, эстетика, история) не существуют изолированно — они образуют систему, где каждый элемент влияет на все остальные. Качество игры определяется не качеством отдельных элементов, а качеством их взаимодействия.

### 2.6.1. Принцип голографического дизайна

Шелл (Призма 10) формулирует принцип голографического дизайна: каждый элемент должен содержать «голографическое» отражение целого. Механика должна воплощать историю (боевая система Dark Souls воплощает нарратив о страдании и упорстве). История должна раскрывать механику (лор Portal объясняет, почему существуют порталы). Эстетика должна поддерживать технологию (пиксель-арт Celeste соответствует простоте управления). Технология должна определять эстетику (процедурная генерация Spelunky создаёт бесконечное разнообразие).

### 2.6.2. Тема как объединяющий принцип

Тема — это фильтр, который определяет, что принадлежит игре, а что нет. Кейс Pirates of the Caribbean VR (Шелл, Кн. 2) демонстрирует, как каждый элемент подчинён одной теме «Ощутить себя пиратом»: от формы виртуальной среды до штурвала и пушек вместо джойстика, от звука моря до пиратских сумок для вещей. Большинство тематических деталей не требуют больших затрат — это шрифты, цвета, звуки, мелочи. Сила темы — в деталях.

Резонансная тема — тема, которая «задевает за живое», апеллируя к глубинным человеческим переживаниям. «Любовь важнее жизни» (Титаник), «Будьте добры и великодушны» (Геркулес), «Ребёнок внутри вас противостоит взрослому» (Toontown) — темы, вызывающие эмоциональный отклик, потому что обращаются к универсальным истинам.

### 2.6.3. Матрица 4×3 Бонда: три уровня согласованности

Матрица 4×3 Бонда (Кн. 17) позволяет анализировать согласованность элементов на трёх уровнях:

**Фиксированный уровень:** согласованность того, что создал разработчик. Механика воплощает историю, история раскрывается через механику, эстетика поддерживает технологию, технология позволяет механику. Если на фиксированном уровне элементы не согласованы — игра не будет работать, как бы хорошо ни были выполнены отдельные элементы.

**Динамический уровень:** согласованность того, что возникает при взаимодействии. Эмерджентные стратегии поддерживают или противоречат замыслу? Игровые моменты (скриншоты, клипы) отражают лучшее в игре? Эмергентный нарратив обогащает или разрушает задуманную историю?

**Культурный уровень:** согласованность того, что выходит за пределы игры. Мемы, фан-арт, спидран — это «тест на прочность» дизайна. Если игра порождает богатый культурный уровень, значит, её элементы создали пространство для творчества. Если культурный уровень беден — значит, игра слишком линейна или слишком неглубока.

#### 2.6.4. Формальные элементы как чек-лист полноты

Восемь формальных элементов Фуллертон (Кн. 5) — это чек-лист, гарантирующий, что ни один аспект игры не упущен:

1. **Игроки** — определены ли? Сколько? Каковы их роли? 2. **Цели** — чёткие ли? Понятные ли? Мотивирующие ли? 3. **Процедуры** — какие действия доступны? Интуитивные ли? 4. **Правила** — какие ограничения? Понятные ли? Согласованные ли? 5. **Ресурсы** — какие ресурсы? Как распределены? Ограничены ли? 6. **Конфликт** — встроен ли? Достаточный ли? Справедливый ли? 7. **Границы** — определены ли? Понятные ли? Уместные ли? 8. **Результат** — неопределённый ли? Измеримый ли? Удовлетворяющий ли?

Каждый элемент — это слот, который должен быть заполнен. Если хотя бы один элемент не определён, генерация неполна. Для AI-системы это означает: генерация должна гарантировать заполнение всех восьми слотов, а валидация — проверять полноту и качество каждого.

---

### 2.7. Языковая метафора: существительные, глаголы и грамматика игры

---

Зубек (Кн. 6) и Гэзэуэй (Кн. 9) предлагают продуктивную метафору для проектирования и генерации элементов игры — **языковую**. Эта метафора особенно важна для AI-системы, потому что она создаёт мост между естественным языком (на котором общается дизайнер) и формальным языком игры (на котором работает система).

#### 2.7.1. Объекты = существительные

Объекты игры — это её «существительные»: меч, зелье, враг, дверь, юнит, ресурс, территория. Каждый объект имеет атрибуты (числовые, булевы, перечислимые) и состояния. Объекты — это «что» игры: из чего она состоит, с чем взаимодействует игрок.

При генерации концепции на естественном языке («игра про исследование подземелий») AI может разобрать описание на существительные: подземелье, сокровище, ловушка, монстр, corridors, комнаты. Каждое существительное — потенциальный объект игры, требующий определения атрибутов и поведения.

### 2.7.2. Действия = глаголы

Действия — это «глаголы» игры: исследовать, сражаться, собирать, строить, торговать, прятаться, убегать. Глаголы определяют, что игрок может делать с объектами. Чем больше глаголов и чем свободнее их комбинации, тем богаче пространство действий.

Различие между оперативными и результирующими действиями (Шелл) — это различие между базовыми глаголами и их комбинациями. «Идти» и «атаковать» — оперативные глаголы; «фланкировать» — результирующий, возникающий из комбинации «идти» + «атаковать» в определённом контексте. Дизайн глаголов — это дизайн глубины: несколько хорошо скомбинированных глаголов дают больше глубины, чем много изолированных.

### 2.7.3. Правила = грамматика

Правила — это «грамматика» игры: как глаголы и существительные сочетаются. «Можно атаковать врага мечом» — грамматически правильное предложение. «Можно атаковать зельем дверь» — возможно грамматически правильное, но бессмысленное (если только игра не разрешает бросать зелья в двери). Грамматика определяет, какие комбинации допустимы, а какие нет.

Хорошая грамматика игры — как хорошая грамматика языка — должна быть: (1) согласованной (правила не противоречат друг другу); (2) выразительной (позволяет описать множество ситуаций); (3) лаконичной (минимум правил при максимуме выразительности). Элегантность — это грамматическая экономия: каждое правило работает на несколько ситуаций.

### 2.7.4. Иерархия языковой модели

Гэззэуэй развивает метафору в иерархическую модель: существительные = атрибуты (данные), глаголы = механики (правила/код), предложения = микросистемы, главы = макросистемы, книги = игры. Каждый уровень строится на предыдущем:

- **Атрибуты** (уровень данных): HP, урон, скорость, стоимость - **Механики** (уровень правил): атака, защита, лечение, движение - **Микросистемы** (уровень взаимодействий): боевая система, экономика, крафтинг -



**Макросистемы** (уровень архитектуры): PvE-петля, PvP-баланс, система прогрессии - **Игра** (уровень целого): весь дизайн как единый текст

Для AI-системы эта иерархия — шаблон генерации: от данных к правилам к системам к архитектуре к целому. Каждый уровень генерируется на основе предыдущего, и валидируется на соответствие замыслу.

---

## Резюме главы

---

В этой главе мы детально рассмотрели четыре макроэлемента игры:

1. **Механика** — формальная система игры: 7 базовых механик Шелла (пространство, время, объекты, действия, правила, навык, шанс), 5 структурных типов Адамса/Дорманса (физика, экономика, прогрессия, тактическое маневрирование, социальное взаимодействие), типология систем Селлерса (двигатели, экономики, экологии), MechanicsDB с пятью уровнями таксономии.
2. **Динамика** — то, что происходит при взаимодействии: петли обратной связи (8 характеристик, 6 комбинаций), 4 главные петли игры, эмерджентность (4 уровня Фромма, дихотомия Юла, второпорядочный дизайн).
3. **Эстетика** — то, что чувствует игрок: 8 типов удовольствия ЛеБланка как целевая функция генерации, сочность и фидбэк как инструменты создания опыта.
4. **История** — то, что придаёт смысл: 5 проблем интерактивного повествования, 2 метода (Жемчужная нить и Машина историй), лудонарративная гармония/ирония/диссонанс, агентивность и структуры сюжета.
5. **Взаимодействие элементов** — голографический дизайн, тема как объединяющий принцип, матрица 4×3 Бонда для анализа согласованности, формальные элементы Фуллертона как чек-лист полноты, языковая метафора как мост между описанием и реализацией.

В следующей главе мы углубимся в MDA Framework — модель, которая связывает все элементы в единый генеративный и аналитический процесс.

---

# Глава 3. MDA Framework и модели анализа

---

> **Раздел Библии геймдизайна:** 2.3 — MDA Framework и модели анализа >

**Источники:** KB-01 (Фундамент геймдизайна), KB-03 (MDA-фреймворк), KB-05 (Типология систем), KB-06 (Core Loop), KB-13 (Системный подход Селлерса) >

**Книги:** Шелл (Кн. 1, 2), Адамс/Дорманс (Кн. 4), Фуллертон (Кн. 5), Зубек (Кн. 6), Селлерс (Кн. 13), Бонд (Кн. 17)

---

## 3.1. Зачем нужны аналитические модели

---

Геймдизайн — дисциплина на стыке искусства и инженерии. Художник может опираться на интуицию; инженер — на формулы. Геймдизайнер стоит между: он создаёт формальные системы, которые должны порождать эстетический опыт. Это принципиально более сложная задача, чем чистое искусство или чистая инженерия, потому что результат — опыт игрока — непредсказуем и не поддаётся прямому контролю. Зубек (Кн. 6) называет это **«проблемой второго порядка»**: дизайнер создаёт механизм и надеется, что игрок получит задуманный опыт, но не может гарантировать этого напрямую.

Аналитические модели — это инструменты, которые помогают решить проблему второго порядка. Они дают дизайнеру язык для описания происходящего, рамки для диагностики проблем и методы для целенаправленного проектирования. Без аналитических моделей геймдизайн превращается в слепой поиск: «сделай — протестируй — надеюсь, что работает». С ними — в осознанный процесс: «я хочу вызвать  $X \rightarrow$  для этого нужна динамика  $Y \rightarrow$  для этого нужны механики  $Z$ ».

Для AI-системы Gidede аналитические модели ещё важнее, чем для человека. Человек обладает интуицией, эмпатией и опытом тысяч игр; AI — нет. AI нуждается в формализованных моделях, которые превращают качественные цели (например, «игра должна быть увлекательной») в количественные параметры (целевая эстетика Вызов  $\rightarrow$  динамика потока  $\rightarrow$  механика с балансом навыка и шанса  $\rightarrow$  порог сложности и кривая обучения). Каждая модель в этой главе — это шаг от интуитивного геймдизайна к алгоритмическому.

---

## 3.2. Классический MDA Framework

---

MDA (Mechanics — Dynamics — Aesthetics) — самая влиятельная аналитическая модель в геймдизайне, предложенная Робинот Хьюником, Марком ЛеБланком и Робертом Зубеком в 2004 году. MDA описывает игру через три уровня, каждый из которых соответствует разной перспективе: дизайнер видит механику, система порождает динамику, игрок испытывает эстетику.

### 3.2.1. Три уровня MDA

**Механика (Mechanics)** — базовые правила, данные и алгоритмы игры.

Механика — это то, что создаёт дизайнер: формулы урона, таблицы вероятностей, правила перемещения, структуры ресурсов. Механика полностью определена и контролируема; это «исходный код» игры. Механика — уровень дизайнера.

**Динамика (Dynamics)** — поведение механики во время игры, при взаимодействии с игроком. Динамика — это не отдельные правила, а их совместный эффект: как петли обратной связи усиливают или ослабляют друг друга, как экономика развивается во времени, как эмерджентные стратегии возникают из комбинации простых правил. Динамика — уровень системы.

**Эстетика (Aesthetics)** — эмоциональный опыт игрока, его ощущения и впечатления от игры. Эстетика — не визуальный стиль (это «эстетика» в обычном смысле), а типы удовольствия, которые игра доставляет. ЛеБланк выделяет восемь типов эстетики, и каждый из них — самостоятельное измерение опыта, которое можно целенаправленно проектировать. Эстетика — уровень игрока.

### 3.2.2. Двухнаправленность MDA

Ключевой инсайт MDA: модель работает в двух направлениях. Дизайнер создаёт механику, из которой возникает динамика, которая порождает эстетику — это **аналитический путь** (от игры к опыту). Но игрок взаимодействует с игрой иначе: он чувствует эстетику, наблюдает динамику и изучает механику — это **интерпретирующий путь** (от опыта к правилам).

Эта двухнаправленность — не просто теоретическое наблюдение, а практический инструмент. Когда дизайнер говорит: «Я хочу, чтобы игрок чувствовал напряжение», — он движется в обратном направлении: от эстетики (напряжение) к динамике (какая динамика создаёт напряжение?) к

механике (какие правила порождают эту динамику?). Это генеративный путь MDA, который Бонд (Кн. 17) формализовал как Reverse MDA (раздел 3.4).

### 3.2.3. Восемь типов эстетики (ЛеБланк)

Восемь типов эстетики — это «алфавит» опыта. Они не взаимоисключающие: любая игра может доставлять несколько типов одновременно. Но каждая успешная игра имеет **доминантные** типы эстетики, которые определяют её идентичность.

#	Тип	Суть	Пример игры	--- ---- ----- -----	1	Чувственное удовольствие (Sensation)   Радость от чувственного восприятия — визуал, звук, кинестетика   Journey, Beat Saber, Rez		
2	Фантазия (Fantasy)   Удовольствие от пребывания в другом мире, отыгрыш роли	Skyrim, The Witcher, Animal Crossing	3	Нарратив (Narrative)   Удовольствие от драматического развёртывания истории	The Last of Us, Disco Elysium	4	Вызов (Challenge)   Удовольствие от преодоления трудностей и роста мастерства	Dark Souls, Celeste, Sekiro
5	Товарищество (Fellowship)   Удовольствие от связи с другими игроками	Overcooked, Journey (кооп.), MMO-рейды	6	Открытие (Discovery)   Удовольствие от нахождения нового, исследования	Outer Wilds, Hollow Knight, BotW	7	Выражение (Expression)   Удовольствие от самовыражения и персонализации	Minecraft, The Sims, Mario Maker
8	Подчинение (Submission)   Удовольствие от подчинения правилам, структурированного отдыха	Animal Crossing, Diablo, MMO-фермы						

Важно понимать: эти типы — не жанры, а измерения. Dark Souls — это прежде всего Вызов, но в нём есть и Фантазия (тёмный мир), и Открытие (секреты), и Нарратив (фрагментарный лор). Animal Crossing — это Подчинение + Выражение + Товарищество. Понимание эстетического профиля игры — первый шаг к осознанному дизайну.

### 3.2.4. Ограничения классического MDA

MDA — мощная модель, но она имеет ограничения, которые необходимо учитывать:

**Размытость «динамики».** Понятие «динамика» охватывает слишком широкий спектр явлений: от микро-паттернов (петля «атака → получение урона») до макро-паттернов (экономическая эволюция всей игры). Это затрудняет операциональное использование: «динамика» трудно измерима и трудно формализуема. Зубек (Кн. 6) — один из авторов оригинальной статьи MDA — сознательно отказался от терминов «динамика» и «эстетика» в своей более поздней работе, заменив их более конкретными понятиями.

**Однонаправленная каузальность.** Классический MDA подразумевает линейную причинность: механика → динамика → эстетика. Но в реальности связь двунаправленная: эстетика (ожидания игрока) влияет на то, как он взаимодействует с механикой, создавая разные динамики. Один и тот же набор механик может порождать разный опыт для разных игроков.

**Недостаточная формализация.** MDA описывает, что происходит, но не даёт формального аппарата для прогнозирования: по данному набору механик невозможно точно предсказать, какие эстетики возникнут. Это ограничение — прямое следствие проблемы второго порядка, и оно непреодолимо полностью, но его можно смягчить через дополнительные модели (Machinations, типология систем и т.д.).

---

### 3.3. Модель Зубека: Механика → Геймплей → Опыт игрока

---

Роберт Зубек (Кн. 6) — соавтор оригинальной статьи MDA — в своей более поздней работе предложил альтернативную трёхуровневую модель, сознательно отказавшись от терминов «динамика» и «эстетика». Его мотивация практична: эти термины не используются в реальной студийной работе и вызывают путаницу. Зубек заменил их на более конкретные, операциональные понятия.

#### 3.3.1. Три уровня модели Зубека

**Механика и системы (Mechanics & Systems)** — объекты, действия и правила игры. Системы — это группы взаимосвязанных механик. На этом уровне описывается что есть: объекты с атрибутами, допустимые действия, правила переходов между состояниями. Это уровень дизайнера — он полностью контролирует механику.

**Геймплей (Gameplay)** — динамический процесс взаимодействия игрока с механикой и системами. Геймплей — это не абстрактная «динамика», а конкретный, наблюдаемый процесс: игрок делает X → игра отвечает Y → игрок принимает решение Z. Геймплей — совместный продукт игры и игрока; он возникает в момент взаимодействия, а не существует заранее.

**Опыт игрока (Player Experience)** — субъективные ощущения и эмоции, которые игрок испытывает в процессе геймплея. Опыт — это то, что игрок чувствует: радость, фрустрацию, любопытство, скуку, напряжённость.

Дизайнер не может напрямую управлять опытом — только через манипуляцию механикой и системами.

### 3.3.2. Проблема второго порядка

Зубек формулирует фундаментальную проблему геймдизайна как **«проблему второго порядка»**: дизайнер создаёт динамический механизм и надеется, что игрок получит задуманный опыт, но не может гарантировать этого. Между намерением дизайнера и опытом игрока стоят два уровня опосредования: сначала механика порождает геймплей (первый порядок), затем геймплей порождает опыт (второй порядок). На каждом уровне могут возникнуть расхождения: механика может породить не тот геймплей, который задуман; геймплей может породить не тот опыт, который ожидался.

Это означает, что геймдизайн — это не проектирование (где результат детерминирован), а выращивание (где результат вероятен). Как садовник не может «сделать» дерево, но может создать условия, в которых дерево вырастет, так и геймдизайнер не может «сделать» опыт, но может создать систему, в которой опыт возникает.

### 3.3.3. Два подхода к дизайну

Зубек выделяет два подхода, соответствующих двум направлениям модели:

**Нисходящий (Top-down)**: начать с целевого опыта → спланировать геймплей → определить механику. Это генеративный подход — дизайнер знает, какой опыт хочет создать, и конструирует систему «от цели». В чистом виде нисходящий подход редко работает: слишком много неизвестных между замыслом и реализацией.

**Восходящий (Bottom-up)**: начать с механики → протестировать геймплей → наблюдать опыт. Это исследовательский подход — дизайнер экспериментирует, наблюдая, какой опыт возникает из данной механики. В чистом виде восходящий подход тоже ограничен: без целевой функции легко потерять направление.

**На практике**: гибридный итеративный подход. Дизайнер планирует нисходяще (определяет целевой опыт), экспериментирует восходяще (прототипирует и тестирует) и итерирует — уточняет и замысел, и реализацию на основе наблюдений.

### 3.3.4. Преимущества модели Зубека для AI-системы

Модель Зубека имеет ключевое преимущество для AI-генерации: её средний уровень — «Геймплей» — конкретен и наблюдаем. В отличие от абстрактной «динамики» MDA, геймплей можно описать как последовательность действий и реакций, что делает его формализуемым. Для AI-системы это означает:

- **Геймплей можно моделировать:** AI может симулировать взаимодействие игрока с механикой и наблюдать возникающий геймплей (через Machinations-модели или агентные симуляции). - **Геймплей можно валидировать:** AI может сравнить реальный геймплей (из симуляции) с целевым (из спецификации) и выявить расхождения. - **Геймплей связывает механику и опыт:** понимание того, как именно механика порождает геймплей, позволяет точнее подбирать механики для заданного опыта.

---

## 3.4. Обратный MDA (Reverse MDA)

---

Джереми Бонд (Кн. 17) — студент Шелла — формализовал генеративное использование MDA как **Reverse MDA**: Aesthetics → Dynamics → Mechanics. Если классический MDA — инструмент анализа (данная механика → какой опыт?), то Reverse MDA — инструмент генерации (желаемый опыт → какая механика?).

### 3.4.1. Процесс Reverse MDA

Reverse MDA — это шестиступенчатый процесс, который ведёт дизайнера от абстрактной цели к конкретной механике:

**Шаг 1: Определить целевую эстетику.** Какой опыт должен получить игрок? Выбрать из 8 типов ЛеБланка: Чувственное, Фантазия, Нарратив, Вызов, Товарищество, Открытие, Выражение, Подчинение. Обычно доминируют 2-3 типа. Пример: для RPG — Вызов + Фантазия + Открытие.

**Шаг 2: Вывести динамику, создающую эту эстетику.** Какие динамические процессы обеспечивают целевую эстетику? Для Вызова — баланс навыка и шанса, нарастающая сложность, петли обратной связи. Для Фантазии — нарративный контекст, роль, идентификация с аватаром. Для Открытия — скрытая информация, пространство для исследования.

**Шаг 3: Определить механику, порождающую эту динамику.** Какие конкретные правила, ресурсы и действия создадут нужную динамику? Для баланса навыка и шанса — механика проверки навыка с вероятностным

элементом (d20, критический шанс). Для нарративного контекста — система квестов, диалогов и лора. Для скрытой информации — туман войны, случайная генерация.

**Шаг 4: Выбрать механики из MechanicsDB.** На этом этапе конкретные механики подбираются из каталога: 7 фундаментальных типов Шелла → 5 структурных типов Адамса/Дорманса → 16+ паттернов дизайна → 127 дискретных механик из SW.BAND → жанровые шаблоны.

**Шаг 5: Валидация через многоуровневую тетраду.** Проверить выбранные механики через матрицу 4×3 Бонда (Механика/История/Эстетика/Технология × Фиксированный/Динамический/Культурный): согласованы ли все 12 ячеек? Не возникает ли лудонарративного диссонанса?

**Шаг 6: Проверка через цели дизайна.** Проверить результат через четыре цели дизайна Бонда: Уважение (ценит ли игра время игрока?), Волшебный круг (поддерживает ли иммерсию?), Поток (баланс вызова и навыка?), Интересные решения (обоюдоострые выборы?).

### 3.4.2. Reverse MDA как основа AI-генерации

Reverse MDA — это нативный процесс для AI-системы. В отличие от человека, который может начать с интуитивного «попробую вот это», AI нуждается в чёткой целевой функции. Восемь типов эстетики — это целевая функция: AI получает на вход целевую эстетику и генерирует механики, которые с высокой вероятностью её обеспечат.

Ключевое преимущество: Reverse MDA формализует интуитивный процесс, который опытный дизайнер выполняет неосознанно. Когда опытный дизайнер говорит: «Я хочу создать хардкорный экшен», — он уже знает, что нужны механики с высоким навыковым порогом, жёсткое наказание за ошибки, быстрый фидбэк. Reverse MDA делает это знание явным и доступным для алгоритмизации.

Но Reverse MDA — не панацея. Маппинг «эстетика → динамика → механика» неоднозначен: одна и та же механика может порождать разные эстетики в разных контекстах, а одна и та же эстетика может достигаться разными механиками. Поэтому Reverse MDA должен дополняться классическим MDA-анализом — это двойной процесс, описанный в разделе 3.5.

---



## 3.5. Двойной MDA-процесс Gidede

---

Интегрируя классический MDA, Reverse MDA и модель Зубека, Gidede использует **двойной MDA-процесс** — цикл генерации и валидации, в котором каждый проход уточняет результат:

### 3.5.1. Генеративный проход (Reverse MDA)

``` ВХОД: Целевая эстетика + Жанр + ЦА + Ограничения ↓ ШАГ 1: Определить целевую эстетику (2-3 типа из 8) ↓ ШАГ 2: Вывести целевую динамику (маппинг эстетика → динамика) ↓ ШАГ 3: Выбрать механики из MechanicsDB (7→5→16→127→жанр) ↓ ШАГ 4: Конструирование Core Loop (петли Кн. 6 + шаблоны Кн. 5) ↓ ВЫХОД: Набор механик + структура Core Loop + спецификация системы ```

3.5.2. Аналитический проход (Классический MDA)

``` ВХОД: Сгенерированный набор механик ↓ ШАГ 1: Смоделировать динамику (Machinations / агентная симуляция) ↓ ШАГ 2: Определить эстетику, порождаемую данной динамикой ↓ ШАГ 3: Сравнить с целевой эстетикой ↓ ШАГ 4: Если расхождение → вернуться к генеративному проходу ↓ ВЫХОД: Валидированный набор механик ```

### 3.5.3. Роль модели Зубека в двойном процессе

Модель Зубека добавляет критический промежуточный уровень — геймплей — между динамикой и опытом. Это позволяет AI-системе не просто сравнивать «целевая эстетика vs реальная эстетика», а анализировать путь от механики к опыту:

1. Данная механика → моделируемый геймплей → ожидаемый опыт 2. Целевой опыт → требуемый геймплей → необходимая механика

Если на шаге 2 аналитического прохода обнаруживается расхождение между ожидаемым и целевым опытом, AI может проанализировать геймплей — конкретный процесс взаимодействия — и определить, на каком этапе возникает расхождение. Это делает итерацию направленной, а не слепой.

### 3.5.4. Маппинг «Эстетика → Динамика → Механика»

Для работы генеративного прохода необходим маппинг — таблица соответствий между типами эстетики, типичными динамиками и релевантными механиками. Этот маппинг не однозначен (одна механика →

разные эстетики в разных контекстах), но содержит устойчивые связи, выявленные из анализа 17 книг:

| Эстетика | Типичная динамика | Релевантные механики (уровни MechanicsDB)  
| |-----|-----|-----| | Чувственное |  
Непосредственный фидбэк, синестезия | Действия (кинетические),  
Эстетическое оформление, «Сочность» фидбэка | | Фантазия | Идентификация  
с ролью, иммерсия | Прогрессия (нарративная), Роль/класс, Объекты (аватар) |  
| Нарратив | Драматическая арка, раскрытие | Прогрессия (сюжетная),  
Конфликт, Выбор с последствиями | | Вызов | Баланс навык/сложность,  
нарастание | Навык + Шанс, Петли ОС (эскалация), Кривая прогрессии | |  
Товарищество | Кооперация, социальные связи | Социальное взаимодействие,  
Роли, Зависимость (взаимная) | | Открытие | Исследование, скрытая  
информация | Пространство (открытое), Скрытые объекты, Неопределённость  
| | Выражение | Персонализация, творчество | Действия (конструктивные),  
Кастомизация, Ресурсы (строительные) | | Подчинение | Рутинная,  
структурированный отдых | Прогрессия (гринд), Ресурсы (цикл  
производство→потребление) |

Этот маппинг — отправная точка для AI-генерации, а не жёсткое правило. Конкретный контекст (жанр, платформа, ЦА) может модифицировать связи: например, Открытие в хорроре связано не с исследованием (как в BotW), а с напряжённым поиском пути к спасению.

---

## 3.6. Линзы Шелла: 113 перспектив анализа

---

Если MDA — это «анатомия» игры (из чего она состоит), то линзы Шелла — это «диагностика» (что с ней не так и как это исправить). Джесси Шелл (Кн. 1, 2) создал систему из 113 линз — структурированных наборов вопросов, каждая из которых представляет определённую перспективу, через которую дизайнер может рассмотреть свою игру.

### 3.6.1. Структура и назначение линз

Каждая линза — это не правило, а вопрос. Линзы не говорят, что делать; они спрашивают, думал ли ты об этом. Это принципиально отличает их от паттернов дизайна (которые дают готовые решения) и делает их идеальным инструментом для AI-валидации: AI может прогнать концепцию через все 113 линз и выявить слепые зоны — аспекты, которые дизайнер не учёл.

Линзы организованы в 16 групп, соответствующих разным аспектам игры:

| Группа | Линзы № | Фокус | Ключевые вопросы |  
|-----|-----|-----|-----| | Опыт игрока | 1-3, 68-70 | Суть опыта, эмоции, сеттинг | «Что игрок чувствует?», «Что его удивляет?» | | Удовольствие и мотивация | 4-8 | Веселье, любопытство, ценность | «Где здесь веселье?», «Зачем игрок продолжает?» | | Элементы и целостность | 9-12 | Тетрада, голограмма, единство, резонанс | «Все ли элементы согласованы?» | | Идея и итерация | 13-18 | Вдохновение, проблема, фильтры, риск | «Прошла ли идея 8 фильтров?» | | Игрок и мотивация | 19-25 | Типы, удовольствия, поток, потребности | «Кто твой игрок?», «В каком состоянии поток?» | | Механика | 26-36 | Пространство, время, состояния, действия | «Какие глаголы у игры?», «Сколько эмерджентности?» | | Баланс | 37-53 | 12+ типов баланса | «Есть ли доминантная стратегия?», «Есть ли треугольность?» | | Головоломки | 54-58 | Доступность, видимый прогресс, параллельность | «Понятно ли, что делать?» | | Интерфейс | 59-67 | Управление, фидбэк, сочность, каналы | «Отвечает ли интерфейс за 100мс?» | | Интерес и нарратив | 68-82 | Кривая интереса, история, свобода | «Какова кривая интереса?» | | Мир и персонажи | 83-91 | Фантазия, аватар, функции, отношения | «Зачем этот персонаж?» | | Окружение | 92-95 | Противоречия, атмосфера, зрелищность | «Что говорит пространство?» | | Социальность | 96-99 | Дружба, самовыражение, сообщество | «Как игроки помогают друг другу?» | | Команда и процесс | 100-103 | Любовь, команда, документация, тесты | «Сколько итераций сделано?» | | Технология и бизнес | 104-109 | Технология, клиент, пичинг, прибыль | «Какова бизнес-модель?» | | Этика | 110-113 | Трансформация, ответственность, скрытые цели | «Делает ли игра игрока лучше?» |

### 3.6.2. Ключевые линзы для AI-валидации

Не все 113 линз равноценны для AI-системы. Некоторые (например, линза команды или пичинга) относятся к организационному процессу и не формализуемы алгоритмически. Но значительная часть линз — это готовые валидационные чек-листы. Выделим наиболее формализуемые:

**Линза №40: Треугольность** — «Есть ли выбор: рискнуть ради большой награды или играть безопасно? Соответствует ли награда риску?» Шелл утверждает, что в 8 из 10 случаев, когда кто-то жалуется «здесь нет веселья», проблема — в отсутствии треугольности. Математическая основа:  $EV(\text{безопасный выбор}) \approx EV(\text{рискованный выбор})$ , но дисперсия разная. Треугольность — это осмысленный выбор между стратегиями с разным профилем риска, и она критически важна для Вызова и Открытия.

**Линза №30: Эмерджентность** — «Сколько оперативных действий доступно? Сколько результирующих действий возникает? Сколько путей достижения

цели?» Чем больше оперативных действий и свободнее их комбинации, тем больше результирующих — и тем глубже игра. Это измеримый критерий: AI может подсчитать количество глаголов и оценить фактор ветвления пространства состояний.

**Линза №69: Кривая интереса** — «Какова кривая интереса на протяжении игры? На уровне? На отдельном вызове?» Фрактальная структура: крючок → нарастающие пики с отдыхом → кульминация → завершение. AI может анализировать racing игры, сопоставляя интенсивность вызовов с кривой интереса.

**Линза №9: Тетрада** — «Все ли четыре элемента (Механика, История, Эстетика, Технология) согласованы? Усилен ли каждый элемент другими?» Это базовая проверка целостности — принцип резонанса, где элементы не просто сосуществуют, а усиливают друг друга.

**Линза №25: Пространство действий** — «Каково воспринимаемое пространство действий? Совпадает ли оно с реальным?» Скрытые действия (которые возможны, но игрок о них не знает) — это Открытие. Но если пространство действий меньше, чем кажется игроку, возникает фрустрация — ощущение, что «я хочу, но не могу».

### 3.6.3. Интеграция линз с моделью Зубека

Каждую линзу можно отнести к одному из трёх уровней модели Зубека:

- **Линзы опыта** (1-8, 19-25, 68-82, 96-99) → Уровень «Опыт игрока» — проверяют, какой опыт возникает - **Линзы геймплея** (9-12, 26-36, 37-58, 59-67) → Уровень «Геймплей» — проверяют, как игрок взаимодействует с механикой - **Линзы процесса и контекста** (13-18, 83-95, 100-113) → Мета-уровень — проверяют условия, в которых создаётся и существует игра

Эта классификация позволяет AI-системе применять линзы направленно: если проблема выявлена на уровне опыта — применить линзы опыта; если на уровне геймплея — линзы геймплея. Вместо слепого перебора 113 линз — целенаправленная диагностика.

---

## 3.7. Многоуровневая тетрада Бонда: матрица 4×3

---

Джереми Бонд (Кн. 17) расширил базовую тетраду Шелла (Механика, История, Эстетика, Технология), добавив три уровня владения: Фиксированный,

Динамический и Культурный. Результат — матрица 4×3, которая позволяет анализировать каждый элемент игры на трёх уровнях:

### 3.7.1. Матрица 4×3

| | Механика | История | Эстетика | Технология |  
|---|-----|-----|-----|-----| | **Фиксированный** (создано разработчиком) |  
Правила, формулы, алгоритмы | Сценарий, лор, мир | Визуальный стиль, звук, интерфейс | Движок, платформа, API | | **Динамический** (возникает при взаимодействии) | Эмерджентные стратегии, мета-игра | Эмергентный нарратив, моментальные истории | Игровые моменты, процедурный контент |  
Моды, настройки, пользовательский контент | | **Культурный** (вне контроля разработчика) | Спидран, турнирные правила, мета-стратегии | Фанфики, теории, мемная культура | Фан-арт, косплей, стримы | Моддинг, хаки, инструменты сообщества |

### 3.7.2. Принцип перехода владения

Ключевой инсайт Бонда: **ответственность дизайнера не заканчивается на фиксированном уровне**. Хотя дизайнер не контролирует динамический и культурный уровни напрямую, он создаёт условия, в которых они формируются. Игры с богатым динамическим уровнем (Minecraft, Dwarf Fortress) — это не случайность, а результат дизайна механик, допускающего эмерджентность. Игры с живым культурным уровнем (D&D, Among Us) — это результат дизайна, который поощряет социальное творчество.

Принцип перехода владения: Фиксированный → полностью контролируется разработчиком. Динамический → разделяется между разработчиком и игроком. Культурный → выходит за пределы контроля разработчика. Но **дизайнер несёт ответственность за все три уровня**, потому что именно его решения на фиксированном уровне определяют, что возможно на динамическом и культурном.

### 3.7.3. Практическое применение матрицы

Матрица 4×3 — инструмент валидации согласованности. Для каждой из 12 ячеек дизайнер (или AI) задаёт вопрос: «Что находится в этой ячейке? Согласовано ли оно с соседними ячейками?» Рассмотрим примеры:

**Лудонарративная гармония:** Фиксированная механика (бой в Doom) ↔ Фиксированная история (демоны, ад) ↔ Фиксированная эстетика (агрессивный визуал и звук) ↔ Фиксированная технология (быстрый движок). Все четыре элемента сообщают одно и то же: «ты — мощный воин, уничтожающий»

демонов». На динамическом уровне эмерджентные стратегии (glory kill для восстановления здоровья) согласуются с историей («Думсайер питается насилием»). На культурном уровне — спидраны и мемы (Rip and Tear) — органичное продолжение.

**Лудонарративный диссонанс:** Фиксированная история (Нейтан Дрейк — обаятельный авантюрист) ↔ Фиксированная механика (убийца сотен людей). На динамическом уровне: игрок наслаждается массовым убийством, но в кат-сценах Нейтан симпатичен. Диссонанс между ячейками «Фиксированная механика» и «Фиксированная история» в одной строке матрицы.

Для AI-системы матрица 4×3 — это **чек-лист согласованности**: AI проверяет все 12 ячеек на наличие содержимого и на согласованность между ячейками одной строки и одного столбца.

---

## 3.8. Системная модель Селлерса: Parts → Loops → Whole

---

Майкл Селлерс (Кн. 13) предлагает системную модель, которая описывает игру как систему, состоящую из двух подсистем: **игры** и **игрока**. Вместе они образуют систему **игра+игрок**, в которой возникает геймплей. В отличие от MDA, где причинность однонаправленна (механика → динамика → эстетика), модель Селлерса — двунаправленная: игра и игрок взаимно влияют друг на друга.

### 3.8.1. Три уровня подсистемы «игра»

Селлерс выделяет три уровня компонентов в подсистеме «игра»:

**Parts (Части)** — структурные компоненты: токены (игровые объекты) и правила (ограничения и возможности). Части — это «существительные» и «глаголы» игры, базовый словарь, из которого строится всё остальное. Шелл называет их оперативными действиями; Зубек — объектами и действиями. Части определяются дизайнером и статичны до начала игры.

**Loops (Петли)** — функциональные элементы, построенные из частей. Петля — это циклическая структура, в которой действие игрока изменяет состояние игры, что ведёт к обратной связи, которая информирует следующее действие. Петли — это «предложения» игры, осмысленные единицы динамики. Одна петля: атака → урон → смерть врага → лут → улучшение → атака сильного врага. Петли вкладываются друг в друга: быстрые петли внутри медленных.

**Whole (Целое)** — архитектурный/тематический уровень, возникающий из петель. Целое — это не просто сумма петель, а их синергия: как петли взаимодействуют, создавая единый опыт. Целое отвечает на вопрос: «Что это за игра?» — не в терминах механик, а в терминах идентичности и смысла. Например, в Civilization целое — «построить империю, проходящую через эпохи» — это не одна петля, а результат взаимодействия множества петель (исследование, строительство, война, дипломатия, экономика).

### 3.8.2. FBS и SBF — альтернативные рамки

Селлерс ссылается на две модели из других дисциплин, которые можно применять к анализу игр:

**FBS (Function-Behavior-Structure)** — из архитектурного дизайна. Structure (Структура) = Parts, Behaviour (Поведение) = Loops, Function (Функция) = Whole. Функция — это назначение системы: зачем она существует. Анализируя игру через FBS, дизайнер спрашивает: «Какова функция этой механики? Какое поведение она порождает? Какая структура нужна для этого поведения?»

**SBF (Structure-Behavior-Function)** — из когнитивной науки. Тот же набор элементов, но обратный порядок анализа: от структуры к функции. SBF полезна для анализа существующих игр (как работает данная структура?), а FBS — для синтеза новых (какая структура нужна для данной функции?).

Обе рамки — вариации модели Селлерса, и их ценность в том, что они подчёркивают направленность анализа. FBS — генеративная рамка (от цели к реализации), SBF — аналитическая рамка (от реализации к цели). Это параллель к двойному MDA-процессу Gidede.

### 3.8.3. Четыре главные петли

Селлерс выделяет четыре петли, описывающие динамику системы «игра+игрок»:

**1. Петля модели игры** — внутренняя динамическая модель мира игры. Если модель статична (ничего не меняется без действий игрока) — нет интерактивности, нет геймплея. Динамическая модель создаёт пространство для геймплея через **второпорядочный дизайн**: дизайнер создаёт не один путь, а пространство возможностей. Чем больше возможностей, тем больше потенциальных путей через игру.

**2. Петля ментальной модели игрока** — динамическая модель, которую игрок строит в своём сознании. Когда ментальная модель совпадает с моделью игры, игрок «понимает» игру и действует эффективно. Когда

ментальная модель неверна, возникает либо удивление (если расхождение в хорошем смысле — «О, я не знал, что так можно!»), либо фрустрация (если в плохом — «Это несправедливо!»). Цель дизайна — создавать ситуации, где ментальная модель постоянно уточняется и углубляется.

**3. Интерактивная петля** — цикл «действие → обратная связь → обновление ментальной модели → новое действие». «Заккрытие петли» — первый магический момент разработки: когда игрок впервые может действовать, видеть результат и учиться. Селлерс выделяет шесть временных масштабов этой петли: Action/Feedback (мс-с) → Short-term Cognitive (с-мин) → Long-term Cognitive (мин-ч) → Social (ч-дни) → Emotional (ч-недели) → Cultural (недели-месяцы). Петли вкладываются: быстрые внутри медленных.

**4. Петля дизайнера** — мета-петля, в которой дизайнер создаёт/изменяет игру на основе наблюдений за игроком. Это самая долгая петля, и её качество определяет качество всей игры. Правило цикла Шелла: чем больше итераций дизайнерской петли, тем лучше игра.

### 3.8.4. Второпорядочный дизайн

Второпорядочный дизайн — центральная концепция Селлерса, имеющая два значения:

**Первое значение:** дизайнер создаёт токены и правила, определяющие пространство состояний, а не один путь. Если существует только один путь — это не игра, а книга или фильм. Игрок определяет свой собственный путь через пространство возможностей. Дизайнер — архитектор пространства, а не маршрута.

**Второе значение:** механика (токены + правила) создаётся дизайнером напрямую (первый порядок), но динамика эмергирует из механики при игре (второй порядок). Опыт игрока — уже третий порядок: он возникает из динамики через восприятие и эмоции. Дизайнер не может напрямую создать опыт, но может создать механику, из которой опыт вероятно возникнет.

Для AI-системы Gidede второпорядочный дизайн — это **парадигма**: AI должен не генерировать «правильный» путь через игру, а создавать пространство возможностей, из которых эмерджентно возникает целевой опыт. Это качественно отличается от генерации линейного контента.

---



## 3.9. Сравнительный анализ моделей

Пять моделей, рассмотренных в этой главе, не конкурируют, а дополняют друг друга. Каждая решает свою задачу и имеет свою область применения:

### 3.9.1. Сравнительная таблица

Модель	Направление	Основной инструмент	Сильная сторона	Слабая сторона
<b>Классический MDA</b>	Аналитическое (от механики к опыту)	8 типов эстетики	Понятная структура, общий язык	«Динамика» размыта, однонаправленная
<b>Reverse MDA</b>	Генеративное (от опыта к механике)	Маппинг «эстетика → динамика → механика»	Целенаправленная генерация	Маппинг неоднозначен
<b>Зубек (М/Г/О)</b>	И то, и другое	Конкретный «геймплей» как средний уровень	Операционализация «динамики»	Менее известна, меньше инструментов
<b>Линзы Шелла</b>	Диагностическое	113 структурированных вопросов	Полный охват, готовые чек-листы	Неформализуема, зависит от интерпретации
<b>Селлерс (Parts/Loops/Whole)</b>	Системное	Петли обратной связи, типология систем	Системное мышление, FBS/SBF	Менее конкретна, требует домысливания
<b>Бонд (4×3)</b>	Валидационное	Матрица 12 ячеек	Проверка согласованности на 3 уровнях	Не генеративна, только проверка

### 3.9.2. Как модели взаимодействуют

Модели образуют **вложенную структуру**:



**Принцип взаимодействия:** Селлерс даёт язык для описания компонентов игры (parts, loops, whole). Зубек даёт язык для описания процесса (механика → геймплей → опыт). Классический MDA и Reverse MDA — два направления работы этого процесса. Линзы Шелла — диагностический инструмент, применяемый на любом этапе. Матрица Бонда — финальная проверка согласованности результата.

---

## 3.10. Практикум: применение моделей анализа в Gidede

---

Рассмотрим конкретный сценарий применения аналитических моделей в AI-системе Gidede: генерация концепции игры с целевой эстетикой «Вызов + Открытие» для жанра action-RPG.

### 3.10.1. Шаг 1: Reverse MDA — определение целевых компонентов

**Целевая эстетика:** Вызов + Открытие (вторичная: Фантазия)

**Целевая динамика:** - Вызов: баланс навыка и сложности, нарастающая кривая вызовов, петли с триангулярностью (риск/награда) - Открытие: скрытая информация, нелинейное пространство, процедурная генерация - Фантазия: отыгрыш роли, нарративный контекст, идентификация с аватаром

**Необходимые механики** (из MechanicsDB): - Level 0: Навык + Шанс + Пространство + Объекты - Level 1: Физика (бой) + Внутренняя экономика (прокачка) + Прогрессия (сюжет) + Тактическое маневрирование (позиционирование) - Level 2: Dynamic Engine (ресурсы → прокачка → больше ресурсов) + Escalating Challenge (нарастание сложности) + Multiple Feedback (многоуровневые петли) - Level 3: Бой в реальном времени, Система классов, Дерево навыков, Туман войны, Скрытые области, Босс-бои, Лут-система - Level 4: RPG-шаблон

### 3.10.2. Шаг 2: Конструирование через Parts → Loops → Whole

**Parts:** Персонаж (атрибуты, класс, навыки) + Враги (типы, поведение) + Предметы (оружие, броня, зелья) + Окружение (территории, подземелья) + Ресурсы (НР, мана, ХР, золото)

**Loops:** Микро-петля (атака → урон → смерть/победа) → Малая петля (найти врага → сражение → лут) → Средняя петля (исследование → бои → возврат →

прокачка) → Крупная петля (пройти подземелье → босс → награда → новый регион) → Макро-петля (пройти акт → раскрыть сюжет → новый мир)

**Whole:** «Путешествие героя через опасный мир, где каждый бой — испытание навыка, а каждый поворот — возможность открыть нечто новое»

### 3.10.3. Шаг 3: Валидация через модель Зубека

**Механика → Геймплей:** Боевая система с уклонениями и парированиями порождает геймплей «изучи паттерн врага → найди окно для атаки → уклонись от ответного удара». Это конкретный наблюдаемый процесс.

**Геймплей → Опыт:** Этот геймплей порождает опыт «напряжённое внимание → триумф мастерства → удовлетворение». Если геймплей «заспамить кнопку атаки → случайно победить» — опыт будет «скука, нет вызова».

**Расхождение?:** Если AI обнаруживает, что геймплей не порождает целевой опыт (например, бой слишком лёгкий), он модифицирует механику (увеличивает HP врагов, добавляет атаки с таймингом).

### 3.10.4. Шаг 4: Проверка через линзы Шелла

Применим ключевые линзы к сгенерированной концепции:

- **Линза треугольности (#40):** Есть ли выбор «рисковать или играть безопасно»? Да: агрессивная атака наносит больше урона, но оставляет уязвимым; безопасная стратегия медленнее, но надёжнее. EV обеих стратегий примерно равен. - **Линза эмерджентности (#30):** Сколько оперативных действий? (атака, парирование, уклонение, прыжок, использование предметов, магия). Сколько результирующих? (фланкирование, стун-лок, комбо, кайтинг, бэкстаб). Достаточно для глубины. - **Линза кривой интереса (#69):** Нарастающие пики (боссы) с отдыхом (безопасные зоны, торговля) → кульминация (финальный босс) → завершение (эпилог). Фрактальная структура на всех масштабах. - **Линза тетрады (#9):** Механика (бой в реальном времени) ↔ История (эпический квест) ↔ Эстетика (тёмное фэнтези) ↔ Технология (3D-движок). Все согласованы.

### 3.10.5. Шаг 5: Валидация через матрицу 4×3

Проверяем все 12 ячеек:

		Механика		История		Эстетика		Технология			
	---	-----		-----		-----		-----		<b>Фиксированный</b>	Боевая система, прокачка, лут
				Сюжет, лор, квесты		Визуал, звук, UI		3D-движок, платформа			

| **Динамический** | Эмерджентные стратегии (билды) | Игровые истории («я чуть не умер») | Процедурный лут, моменты | Моды, настройки | | **Культурный** | Мета-билды, спидраны | Фанфики, лор-теории | Фан-арт, косплей | Моддинг, вики |

**Проверка:** Согласована ли динамическая механика (эмерджентные билды) с фиксированной историей (эпический квест)? Да: разные билды = разные способы прохождения, что обогащает историю. Согласована ли культурная эстетика (косплей) с фиксированной эстетикой (тёмное фэнтези)? Да: яркие персонажи в тёмном мире = идеальный материал для косплея.

### 3.10.6. Шаг 6: Классический MDA — финальная валидация

**Сгенерированная механика → Моделируемая динамика → Реальная эстетика:**

Механика (бой + прокачка + лут + исследование) порождает динамику (нарастающая сила, нарастающие вызовы, открытие нового) порождает эстетику (Вызов + Открытие + Фантазия). Совпадает с целевой? Да. Процесс завершён.

Если бы расхождение обнаружилось (например, динамика «гринд» вместо «нарастающий вызов» порождает эстетику Подчинение вместо Вызова), AI вернулся бы к генеративному проходу и скорректировал механику: например, добавил эскалацию сложности вместо плоского гринда.

---

## 3.11. Итоги: аналитический арсенал Gidede

---

Аналитические модели, рассмотренные в этой главе, образуют многослойную систему, каждая часть которой решает свою задачу:

1. **Reverse MDA** — генеративный двигатель: от целевого опыта к конкретным механикам. Используется на этапе создания концепции.

2. **Модель Зубека** — операционализатор: превращает абстрактную «динамику» в конкретный, наблюдаемый «геймплей». Используется для промежуточной проверки: порождает ли данная механика нужный геймплей?

3. **Классический MDA** — валидатор: проверяет, какие эстетики реально порождают данные механики. Используется для финальной проверки: совпадает ли реальный опыт с целевым?

4. **Линзы Шелла** — диагност: выявляет слепые зоны и проблемы, которые не видны через MDA. Используется для глубокого анализа конкретных аспектов.

5. **Parts → Loops → Whole (Селлерс)** — системный язык: описывает структуру игры на трёх уровнях детализации. Используется для проектирования и документирования.

6. **Матрица 4×3 (Бонд)** — координационный чек-лист: проверяет согласованность всех элементов на всех уровнях. Используется для финальной валидации перед генерацией документации.

Вместе эти модели превращают геймдизайн из интуитивного ремесла в осознанную инженерию — и, что критически важно для Gidede, делают его алгоритмизируемым, позволяя AI-системе выполнять каждый шаг формально и проверяемо.

---

## Глава 4. Core Loop — ядро игрового опыта

---

> **Раздел Библии геймдизайна:** 2.4 — Core Loop — ядро игрового опыта >

**Источники:** KB-06 (Core Loop), KB-03 (MDA-фреймворк), KB-05 (Типология систем), KB-13 (Системный подход) > **Книги:** Шелл (Кн. 1, 2), Адамс/Дорманс (Кн. 4), Фуллертон (Кн. 5), Зубек (Кн. 6), Шрайбер (Кн. 7), Гэри (Кн. 10), Селлерс (Кн. 13), D&D (Кн. 16), Бонд (Кн. 17)

---

### 4.1. Что такое Core Loop и почему это важно

---

Core Loop — это минимально значимый цикл игровой активности, который приносит удовольствие и мотивирует продолжать. Это «сердцебиение» игры: повторяющийся паттерн действий, который игрок выполняет снова и снова, и каждый цикл приносит ему награду — удовольствие, прогресс или новый вызов. Если механика — скелет игры, а динамика — её дыхание, то Core Loop — это её пульс.

Концепция Core Loop пронизывает всю литературу по геймдизайну, хотя и под разными названиями: «игровой цикл» (game loop), «петля опыта» (experience loop), «кор-геймплей» (core gameplay), «30 секунд веселья» (30 seconds of fun). Разные авторы подчёркивают разные аспекты, но сходятся в главном: **игра** —

**это не набор механик, а цикл; и качество этого цикла определяет качество игры.**

Критическое различие, которое проводит Зубек (Кн. 6): **Core Loop ≠ микроцикл**. Самый быстрый цикл не обязательно является самым весёлым. В The Sims микроцикл (есть, спать) — это рутина, поддерживающая персонажа; Core Loop — пойти на работу → заработать → купить вещи → улучшить жизнь → повторить. Микроцикл обслуживает персонажа; Core Loop обслуживает мотивацию игрока. Это различие фундаментально: оптимизировать неправильный цикл — всё равно что полировать педали в автомобиле, у которого не работает двигатель.

Для AI-системы Gidede Core Loop — это **основной конструктивный элемент** генерации. Прежде чем проектировать механики, экономику, прогрессию или нарратив, AI должен определить Core Loop: что игрок делает снова и снова, и почему ему это нравится. Все остальные элементы игры выстраиваются вокруг Core Loop, поддерживая и обогащая его.

---

## 4.2. Анатомия Core Loop: петля взаимодействия

---

### 4.2.1. Базовая петля взаимодействия (Кук, через Фуллертон)

Дэниел Кук (через Фуллертон, Кн. 5) формализует минимальную единицу взаимодействия как **петлю взаимодействия** (interaction loop):

```
``` Ментальная модель → Решение → Действие → Правила → Фидбэк →  
Обновление модели ```
```

Каждая итерация этой петли — один акт взаимодействия игрока с игрой. Петля когнитивна прежде, чем механична: игрок формирует намерение (ментальная модель), принимает решение, действует, наблюдает результат, обновляет понимание. Петля замкнута, когда игрок может действовать, видеть результат и учиться на нём. Незамкнутая петля — это фрустрация: «я делаю что-то, но не понимаю, что происходит».

4.2.2. Цепочка навыков (Skill Chain, Кук)

Петли взаимодействия вкладываются друг в друга, образуя **цепочку навыков** (skill chain):

``` Микро-петля (нажать кнопку → атака) → Мезо-петля (убить врага → получить лут) → Макро-петля (пройти подземелье → повысить уровень) → Мета-петля (завершить игру → открыть новый режим) ```

Каждый уровень вкладывается в следующий: результаты микро-петли питают мезо-петлю, результаты мезо — макро, и так далее. Ключевой вопрос валидации: есть ли обратная связь на каждом уровне? Соединены ли уровни? Если микро-петля не связана с мезо-петлей (например, бой не даёт лута), цикл рвётся — и игрок теряет мотивацию.

### 4.2.3. Пять геймплейных циклов Зубека

Зубек (Кн. 6) описывает пять концентрических циклов с разной периодичностью:

Уровень	Длительность	Пример действий
<b>Микро</b>	Секунды	Двигаться, атаковать, поднять предмет
<b>Малый</b>	1-2 минуты	Найти врагов → сражение → сбор лута
<b>Средний</b>	5-10 минут	Экспедиция → зачистка → возврат → продажа
<b>Крупный</b>	15-30 минут	Зачистить зону, сменить команду, прокачать навыки
<b>Макро</b>	Часы	Завершить регион, продвинуть сюжет

Зубек визуализирует это как «**луковую**» **диаграмму** (concentric schema) — вложенные кольца, где самый быстрый цикл в центре, самый медленный — на периферии. Это одновременно коммуникационный и диагностический инструмент: пробелы в диаграмме = пробелы в дизайне. Если у игры есть микро- и макро-циклы, но нет среднего — игрок будет дезориентирован между «что делать прямо сейчас» и «к чему это всё ведёт».

Принцип расширения от Core Loop: (1) начать с микро-уровня и протестировать фундамент; (2) итеративно найти Core Loop; (3) добавлять более крупные циклы — они должны поддерживать меньшие (результаты макро-цикла питают выбор в микро-цикле).

---

## 4.3. Шесть временных масштабов интерактивной петли

---

Селлерс (Кн. 13) расширяет модель петли взаимодействия, выделяя шесть временных масштабов, на которых работает интерактивная петля:

| Масштаб | Длительность | Тип вовлечения | |-----|-----|-----| |  
**Action/Feedback** | мс-с | Рефлексивное внимание, моментальный геймплей | |  
**Short-term Cognitive** | с-мин | Тактические решения | | **Long-term Cognitive** |  
мин-ч | Стратегические решения | | **Social** | ч-дни | Социальные  
взаимодействия | | **Emotional** | ч-недели | Эмоциональная привязанность | |  
**Cultural** | недели-месяцы | Культурная идентификация с игрой |

Петли **наслаиваются**: быстрые внутри медленных. «30 секунд веселья» из Halo — это 3-секундная петля внутри 30-секундной внутри 5-минутной внутри 20-минутной. Каждый масштаб обслуживает разные когнитивные и эмоциональные потребности: рефлекс на микро-уровне, тактика на мезо-уровне, стратегия на макро-уровне, социальная идентичность на мета-уровне.

**Интерактивный бюджет** (Селлерс) — важное ограничение: каждый человек имеет ограниченную способность к интеракции. Слишком много интерактивности = когнитивная перегрузка; слишком мало = скука. Дизайнер должен управлять когнитивной нагрузкой одновременно на всех вложенных петлях. Это особенно критично для Core Loop: если базовая петля перегружена (слишком много решений за слишком короткое время), игрок не сможет войти в состояние потока.

---

## 4.4. Три структурных типа Core Loop

---

Селлерс (Кн. 13) предлагает типологию игровых систем, которая одновременно является типологией Core Loop. Каждая игра имеет доминирующий тип Core Loop, хотя реальные игры комбинируют все три:

### 4.4.1. Двигатель (Engine) — однонаправленный ресурсный поток

**Структура:** Усиливающая или балансирующая петля с одним ресурсом. Игрок вкладывает ресурс → получает больше ресурса → вкладывает ещё → получает ещё больше (или тормозящая версия: тратит ресурс для предотвращения runaway).

**Boosting Engine** (усиливающий): Добавляет ресурсы в игру, создавая рост. Игрок выбирает: потратить ресурс сейчас или инвестировать для увеличенного будущего потока. Пример: Монополия — деньги → недвижимость → больше денег. Dominion — карта → ресурсы → лучшая карта → больше ресурсов.



**Braking Engine** (тормозящий): Удаляет ресурсы из игры, предотвращая runaway. Пример: налоги, износ оружия, расход маны, затрата действий.

#### **Три патологии двигателей:**

1. **Runaway** (саморазгон): Усиливающий двигатель без тормозящего. «Богатые богатеют» — игрок, получивший раннее преимущество, экспоненциально отрывается. Диагностика: усиливающий двигатель без тормозящего = runaway. Решение: добавить тормозящий двигатель.

2. **Deadlock** (тупик): Ресурс цикла нужен для запуска самого цикла. Конструктор, для работы которого нужны детали, которые производит этот же конструктор. Решение: обеспечить начальный «затравочный» ресурс.

3. **Stall** (остановка): Недостаточно ресурсов для продолжения цикла. Производство слишком медленное по сравнению с потреблением. Решение: увеличить производство или уменьшить потребление.

### **4.4.2. Экономика (Economy) — конвертация ресурсов**

**Структура:** Усиливающая петля с обменом/конвертацией ресурсов. Игрок обменивает один ресурс на другой с нелинейным приростом ценности.

**Ключевое свойство — развёртывание сложности** (unfolding complexity): экономика начинает просто (хлеб + дерево) и постепенно добавляет новые ресурсы и цепочки конвертации (руда → железо → сталь → оружие). Каждая новая валюта увеличивает глубину решений. Игрок начинает с тривиальных обменов и постепенно осваивает всё более сложные цепочки.

**Валюты** — каталитические ресурсы, которые не потребляются при обмене (золото, серебро), но подвержены инфляции (слишком много валюты = обесценивание) и стагнации (слишком мало оборота = бессмысленность).

**Пример RPG:** Убить монстра → получить опыт и золото → повысить уровень → стать сильнее → убить более сильного монстра → получить больше опыта и золота. Это экономика с встроенным двигателем: XP → уровень → способности → больше XP.

### **4.4.3. Экология (Ecology) — балансирующая конвертация**

**Структура:** Балансирующая петля с обменом/конвертацией ресурсов. Целое стремится к **динамическому равновесию** (метастабильности), хотя части постоянно меняются.

**Метастабильность, а не стазис:** Здоровая экология — это не неподвижное равновесие, а динамическое. Подсистемы постоянно колеблются, но система в целом остаётся устойчивой. Пример: бой в RPG — два противника обмениваются ударами, HP каждого колеблется, но бой в целом имеет тенденцию к завершению (победа одного).

**Устойчивость (resilience) vs Хрупкость (brittleness):** Устойчивая экология восстанавливается после возмущения (игрок потерял много HP, но использовал зелье и восстановил позицию). Хрупкая — разрушается (потеря HP → каскад неудач → смерть). Переход между устойчивостью и хрупкостью трудно обнаружить — система может казаться устойчивой, пока не столкнётся с определённым типом возмущения.

### Три типа экологий в играх:

1. **Инвентарь:** Больше предметов → меньше места → давление освободить → баланс. Конфликт между «хранить на всякий случай» и «освободить для нового».

2. **Бой:** Два противника уравнивают друг друга. HP как ресурс, который тратится и восстанавливается. Атака и защита как противоположные потоки.

3. **Социальная:** Трёхфракционная система — если одна фракция усиливается, две другие объединяются против неё (Dark Age of Camelot). Двухфракционные системы склонны к runaway: одна сторона начинает побеждать → привлекает больше игроков → побеждает ещё больше. **Три фракции — минимальное число для экологической стабильности.**

#### 4.4.4. Комбинирование типов

Реальная игра — иерархия вложенных систем. RPG: экономика (ядро прокачки) + экология (бой) + двигатель (ресурсы) — все вложены друг в друга. Стратегия: несколько двигателей (добыча + производство) + экономика (торговля) + экология (баланс сил). Разные типы Core Loop требуют разных подходов к балансировке: двигатель — контроль runaway, экономика — контроль инфляции, экология — контроль хрупкости.

---

## 4.5. Цикл прогрессии: физика мотивации

---

Шрайбер (Кн. 7) формализует Core Loop через **цикл прогрессии** — базовую структуру, которая управляет темпом и мотивацией игрока:

### 4.5.1. Структура цикла прогрессии

``` Мотивация → Прогрессия → Награда → Удовлетворение ```

Каждый проход через этот цикл — один «шаг» прогрессии. Мотивация (почему игрок действует) → Прогрессия (движение к цели) → Награда (подтверждение прогресса) → Удовлетворение (позитивное подкрепление) → новая мотивация (следующий шаг).

4.5.2. Формальная динамика

Шрайбер описывает прогрессию через формальные переменные:

- **Скорость прогрессии** = $1 / \text{сопротивление}$. Сопротивление — сумма всех факторов, замедляющих прогресс: сложность врагов, дефицит ресурсов, ограничения времени. Высокое сопротивление = медленная прогрессия; низкое = быстрая.

- **Ускорение прогрессии** = dv/dt . Когда ускорение положительное — игрок чувствует рост («я становлюсь сильнее!»). Когда ускорение становится отрицательным — игрок чувствует замедление и начинает рассматривать выход из цикла (сменить деятельность, бросить игру).

- **Отрицательное ускорение** — критический момент дизайна: именно здесь игрок решает, продолжать или нет. Дизайнер должен управлять ускорением, создавая «провалы» (отрицательное ускорение, за которым следует положительное — ощущение прорыва) и избегая «плато» (нулевое ускорение без просвета — ощущение застоя).

4.5.3. Эффект Зейгарник

Шрайбер ссылается на психологический феномен — **эффект Зейгарник**: незавершённые действия запоминаются лучше и вызывают тревогу, подталкивая к завершению. Это психологический двигатель, который удерживает игрока в Core Loop. В Civilization «ещё один ход» — это классический эффект Зейгарник: всегда есть незавершённое действие (строящееся здание, исследуемая технология, незахваченный город), и мозг тревожно сигнализирует: «Доведи до конца!»

Для AI-системы Gidede эффект Зейгарник — **конструктивный принцип**: Core Loop должен всегда содержать незавершённые под-циклы, создающие «тягу» к продолжению. Если все под-циклы завершены одновременно — мотивация падает, и игрок выходит из игры.

4.6. Конверсионные цепи и циклы (Зубек)

Зубек (Кн. 6) предлагает экономическую модель внутренней структуры Core Loop через **конверсионные цепи** — последовательности механик, преобразующих одни ресурсы в другие:

4.6.1. Конверсионные цепи

Механики игры образуют цепочки «производитель → потребитель»:

- **Бой** конвертирует монстров + здоровье → лут - **Продажа** конвертирует лут → золото - **Лечение** конвертирует золото → здоровье - **Покупка** конвертирует золото → снаряжение - **Снаряжение** конвертирует → эффективность боя → больше лута

Каждое звено цепи — одна механика. Цепочка в целом — один проход Core Loop.

4.6.2. Конверсионные циклы

Когда цепочка замыкается (последний ресурс конвертируется обратно в первый), образуется **конверсионный цикл** — повторяющийся Core Loop с анализируемыми свойствами:

Прибыльный цикл → линейный рост ресурсов → риск **гринда**. Если каждый проход через цикл приносит чистую прибыль, ресурсы растут линейно. Это предсказуемо и удовлетворяюще на ранних этапах, но со временем превращается в рутину — каждый проход одинаков, мотивация снижается.

Убыточный цикл → истощение ресурсов → риск **фрустрации**. Если каждый проход отнимает больше, чем даёт, ресурсы убывают. Это может создавать напряжение и значимость решений, но при затяжном убыточном цикле игрок теряет мотивацию.

Динамическая настройка — ключевой механизм: масштабирование кривой прибыльности во времени, так что цикл переходит от прибыльного к убыточному. На ранних этапах — прибыльный цикл (ощущение роста, мотивация продолжать). На поздних — убыточный (напряжение, необходимость оптимизации, выбор приоритетов). Этот переход — конкретный дизайнерский рычаг, предотвращающий и гринд, и фрустрацию.

4.6.3. Применение к RPG: макро-модель

Шрайбер (Кн. 7) формализует макро-модель RPG как систему вложенных циклов:

Т часов → L уровней → L-1 переходов → $\sim L/T$ переходов/час → L/2 стадий контента → $\geq 3 \times (L/2)$ конфигураций противника

Каждый «переход» — один проход Core Loop, завершающийся повышением уровня. Каждая «стадия контента» — диапазон уровней с однотипными вызовами. Каждая «конфигурация противника» — уникальная комбинация врагов и механик. Формула показывает: для 40-часовой RPG с 20 уровнями нужно ≈ 30 конфигураций противника — астрономически меньше, чем если бы каждый бой был уникальным. Core Loop обеспечивает реиграбельность через вариацию контекста, а не уникальность каждого прохода.

4.7. Пять вопросов кор-геймплея (Гэри)

Джастин Гэри (Кн. 10) предлагает самый предписывающий фреймворк для определения Core Loop — пять вопросов, которые должен задать себе дизайнер:

4.7.1. Какой игровой цикл?

Вопрос: Можно ли описать Core Loop в 1-2 предложениях? Если нет — либо цикл не определён, либо он слишком сложен. Хороший Core Loop описывается просто: «Найди врагов → сразись → собери лут → улучшись → найди более сильных врагов». Если описание требует абзаца — дизайн, скорее всего, перегружен.

Этот вопрос заставляет дизайнера (или AI) **сформулировать суть игры** до того, как будут добавлены детали. Это как «elevator pitch» для Core Loop: если вы не можете объяснить его кратко, вы его не понимаете.

4.7.2. В чём главный конфликт?

Вопрос: Какое ограничение стоит на пути к цели? Конфликт — суть игры: без конфликта нет вызова, без вызова нет мотивации. Каждый дизайнерский вопрос должен проверяться: «служит ли это решение главному конфликту или подменяет его?»

Примеры главных конфликтов: Dark Souls — выживание в мире, который пытается вас убить. StarCraft — ограничение ресурсов при неограниченных амбициях. Portal — пространство, которое не позволяет пройти прямо. Главный конфликт определяет, какие механики нужны, а какие — лишние.

4.7.3. Какие ресурсы?

Вопрос: Какие ресурсы управляют Core Loop? Гэри выделяет три типа:

- **Масштабируемые** (Scalable) — растут со временем. Мана, золото, опыт. Управляют долгосрочной прогрессией. - **Фиксированные** (Fixed) — неизменны. 1 действие за ход в настольной игре. Создают жёсткие ограничения и осмысленный выбор. - **Накапливаемые** (Accumulating) — растут, если не используются. Эликсир в Clash Royale. Создают срочность: «потратить сейчас или дождись максимума, но упусти момент».

Тип ресурсов определяет динамику Core Loop: масштабируемые ресурсы порождают прогрессию, фиксированные — осмысленный выбор, накапливаемые — тайминг и срочность.

4.7.4. Как происходит взаимодействие?

Вопрос: Как игроки взаимодействуют друг с другом (или с игрой)? Пять типов:

- **Горячие общие ресурсы** (Hot shared resources) — видимый захват: «ты взял моё!». Пример:  ление территорий в Risk. - **Холодные общие ресурсы** (Cold shared resources) — скрытый выбор: «ты взял то, что я хотел, но я не вижу». Пример: драфт карт. - **Прямой урон** — атака, разрушение, устранение. Пример: бой в Street Fighter. - **Социальная дедукция** — блеф, скрытые роли, подозрение. Пример: Among Us, Werewolf. - **Интеллектуальная дедукция** — логический вывод, анализ информации. Пример: шахматы, Mastermind.

Тип взаимодействия определяет эмоциональный профиль Core Loop: горячие ресурсы → агрессия и возмущение; холодные → обида и стратегия; прямой урон → адреналин; социальная дедукция → паранойя; интеллектуальная → удовлетворение от озарения.

4.7.5. В чём цель игры?

Вопрос: Как игра завершается? Три типа терминации:

- **Элиминация** — последний выживший побеждает. Пример: Battle Royale, Poker. - **Ограничение по времени** — побеждает тот, кто достиг больше к моменту истечения. Пример: спортивные игры, раундовые соревнования. -

Достижение цели — первый достигший целевого состояния побеждает. Пример: гонки, квесты, сбор определённого количества очков.

Тип терминции определяет структуру racing: элиминация → нарастающее напряжение (игроков всё меньше); ограничение по времени → пиковое напряжение в конце; достижение цели → рваный темп (кто-то может «выстрелить» в любой момент).

4.8. «30 секунд веселья»: Core Loop в действии

4.8.1. Происхождение концепции

Концепция «30 секунд веселья» приписывается Джейми Грисемеру, дизайнеру Halo (через Шрайбера, Кн. 7). Суть: базовый цикл боевого encounters в Halo занимает ~30 секунд и сам по себе является весёлым. Встретить врагов → выбрать подход → выполнить бой → оценить результат → повторить с вариацией. Весь Halo строится на вариации этого 30-секундного цикла: разные враги, разное оружие, разная тактика, но одна и та же базовая структура.

4.8.2. Вложенность петель в Halo

Селлерс (Кн. 13) разворачивает структуру Halo на вложенные петли:

- **3-секундная петля:** Прицелиться → выстрелить → попасть/промахнуться → скорректировать. Это рефлексивный уровень — чистый навык и реакция. - **30-секундная петля:** Встретить группу врагов → выбрать тактику → выполнить серию боевых действий → зачистить область. Это тактический уровень — выбор подхода. - **5-минутная петля:** Пройти секцию уровня → встретить новый тип врага → адаптировать тактику → достичь checkpoint. Это уровень вызова — нарастание сложности. - **20-минутная петля:** Завершить уровень → увидеть кат-сцену → получить новый навык/оружие → начать следующий уровень. Это уровень прогрессии — рост и нарратив.

«30 секунд веселья» — это минимальный уровень, на котором Core Loop уже доставляет удовольствие. Не 3 секунды (слишком малы для осмысленного выбора) и не 5 минут (слишком долго для быстрой обратной связи). 30 секунд — золотая середина между действием и рефлексией.

4.8.3. Обобщение: нахождение «правильного» масштаба

Принцип «30 секунд веселья» обобщается: для каждой игры существует оптимальный масштаб Core Loop — минимальная длительность, при которой цикл уже приносит удовольствие. Этот масштаб зависит от жанра:

| Жанр | Оптимальный масштаб Core Loop | Что делает цикл |
|-----|-----|-----| | Шутер | 10-30 секунд | Боевой encounter | | RPG | 2-5 минут | Бой → лут → прокачка | | Стратегия | 5-15 минут | Ход → решение → результат | | ММО-рейд | 15-30 минут | Попытка → анализ → коррекция | | Настольная игра | 5-15 минут | Ход → взаимодействие → результат | | Idle-игра | 30-60 секунд | Нажатие → награда → улучшение |

AI-система Gidede должна определять оптимальный масштаб Core Loop на основе жанра и целевой эстетики: Вызов требует коротких циклов (быстрая обратная связь), Открытие — средних (время на исследование), Подчинение — длинных (рутина с регулярным подкреплением).

4.9. Game Loop D&D: эталон формализованного цикла

D&D (Кн. 16) описывает свой игровой цикл как четырёхшаговый ритм:

1. **Мастер описывает сцену** — окружение, NPC, ситуацию 2. **Игроки описывают действия** — что их персонажи делают 3. **Мастер описывает результаты** — через правила (броски d20) или нарратив 4. **Возврат к шагу 1** — новая ситуация на основе результатов

4.9.1. Экономия действий как архитектура цикла

D&D формализует ограничения через **экономия действий**: 1 Action + 1 Bonus Action + 1 Reaction + Movement за ход. Это жёсткий бюджет, который заставляет игрока выбирать: что из доступного я сделаю этим ходом? Принцип: **ограниченные действия = осмысленный выбор**. Если игрок может сделать всё — нет выбора, нет стратегии, нет веселья.

Это переносимый дизайнерский паттерн: любой Core Loop можно проанализировать через его «бюджет действий». Что игрок может сделать за один цикл? Слишком много — перегрузка, нет фокуса. Слишком мало — ограниченность, скука. Оптимально — 3-5 значимых вариантов, из которых можно выбрать 1-2.

4.9.2. Четыре этапа прогрессии D&D

D&D разделяет прогрессию на четыре макро-цикла, каждый с собственным масштабом конфликтов:

Этап Уровни Масштаб конфликтов	----- ----- -----	1-й 1-4
Локальные угрозы; начинающие авантюристы	2-й 5-10	Региональные угрозы; опытные герои
3-й 11-16	Мировой масштаб; магия, меняющая реальность	4-й 17-20
Мультивселенский масштаб; эпические архетипы		

Каждый этап — макро-цикл с собственной драматической аркой и масштабом последствий. Это модель масштабирования Core Loop: базовая структура цикла сохраняется, но масштаб растёт — от «спасти деревню» до «спасти мультивселенную». AI-система может использовать эту модель для генерации долгосрочной прогрессии: Core Loop остаётся неизменным, но контекст и ставки эскалируют.

4.10. Патологии Core Loop: диагностика и лечение

Объединяя диагностику из всех источников, мы можем выделить семь патологий Core Loop — типичных отказов, которые приводят к тому, что цикл «не работает»:

4.10.1. Runaway (Саморазгон)

Симптом: Игрок, получивший раннее преимущество, экспоненциально отрывается от остальных (PvP) или от сложности (PvE).

Причина: Усиливающая петля без балансирующей. Каждый проход через Core Loop увеличивает ресурсы, которые делают следующий проход легче, что увеличивает ресурсы ещё быстрее.

Лечение: Добавить тормозящий двигатель (налог, износ, расход), усиливающую сложность (враги растут быстрее, чем игрок), или отрицательную обратную связь (резиновая лента — отстающим легче).

4.10.2. Deadlock (Тупик)

Симптом: Игрок не может начать Core Loop, потому что ресурс, нужный для входа, производится только внутри цикла.

Причина: Круговая зависимость: для боя нужно снаряжение → для снаряжения нужны деньги → для денег нужно продать лут → для лута нужно победить врага → для победы нужно снаряжение.

Лечение: Обеспечить «затравочный» ресурс — начальное снаряжение, стартовый капитал, первый квест с гарантированной наградой. В AI-системе: проверять, что у каждого ресурса есть как минимум один источник, не зависящий от самого цикла.

4.10.3. Stall (Остановка)

Симптом: Core Loop замедляется до остановки — ресурсов недостаточно для продолжения.

Причина: Потребление превышает производство в устойчивом состоянии. Или: стоимость входа в цикл растёт быстрее, чем способность игрока платить.

Лечение: Увеличить производство, уменьшить потребление, или добавить альтернативные источники ресурсов. В AI-системе: моделировать потоки ресурсов через Machinations и проверять, что производство $\geq$ потребление при любом устойчивом состоянии.

4.10.4. Grind (Гринд)

Симптом: Каждый проход через Core Loop идентичен предыдущему — нет вариации, нет новых решений, нет роста.

Причина: Прибыльный конверсионный цикл без динамической настройки. Ресурсы растут, но геймплей не меняется.

Лечение: Ввести динамическую настройку прибыльности (цикл переходит от прибыльного к убыточному), добавить новые механики на определённых этапах прогрессии, ввести эмерджентные элементы (рандомизация, адаптивный AI врагов). В AI-системе: генерировать «кривую инновации» — график, показывающий, когда в Core Loop добавляются новые элементы.

4.10.5. Frustration Plateau (Фрустрационное плато)

Симптом: Игрок застревает на одном уровне сложности, не прогрессируя и не регрессируя.

Причина: Убыточный конверсионный цикл без «прорыва». Ресурсы убывают, но нет альтернативного пути к прогрессии.

Лечение: Добавить альтернативные стратегии (другой путь через Core Loop), снизить сложность для конкретного этапа (динамическая настройка сложности), или предоставить «аварийный» ресурс (компенсация за повторные неудачи — как в Celeste, где после нескольких смертей появляются дополнительные платформы).

4.10.6. Disconnected Loops (Разрыв петель)

Симптом: Микро-циклы не связаны с макро-циклами. Бой приносит удовольствие, но не влияет на прогрессию; прокачка влияет на прогрессию, но не на бой.

Причина: Отсутствие конверсионных цепей между уровнями. Результаты микро-цикла не «поднимаются» к макро-циклу.

Лечение: Проектировать Core Loop снизу вверх, обеспечивая, что каждый уровень вкладывается в следующий. Использовать метод концентрических диаграмм Зубека для визуализации связей. В AI-системе: проверять, что каждый ресурс, производимый на микро-уровне, потребляется на мезо- или макро-уровне.

4.10.7. Loop Overload (Перегрузка цикла)

Симптом: Слишком много решений за один проход Core Loop — когнитивная перегрузка, паралич выбора.

Причина: Нарушение интерактивного бюджета. Слишком много типов ресурсов, слишком много вариантов действий, слишком много информации для отслеживания.

Лечение: Упростить Core Loop до 3-5 ключевых решений за цикл. Использовать принцип «фиксированных действий» (как экономия действий в D&D). Скрыть сложность: базовый Core Loop прост, продвинутые опции открываются постепенно. В AI-системе: генерировать Core Loop с контролируемым фактором ветвления.

4.11. Конструирование Core Loop: метод для Gidede

Объединяя все модели, мы формулируем пошаговый метод конструирования Core Loop для AI-системы Gidede:

4.11.1. Шаг 1: Определить целевую эстетику и жанр

Вход: целевая эстетика (из Reverse MDA) + жанр + целевая аудитория.

Выход: профиль Core Loop — доминирующий тип эстетики определяет тип Core Loop: - Вызов → Engine (нарастающая сложность) - Открытие → Economy (новые ресурсы и цепочки) - Товарищество → Ecology (социальный баланс) - Подчинение → Engine (ритуальный цикл)

4.11.2. Шаг 2: Сформулировать Core Loop в одном предложении

Пройти пять вопросов Гэри: Какой цикл? В чём конфликт? Какие ресурсы? Какое взаимодействие? В чём цель? Если Core Loop не описывается в одном предложении — упростить.

4.11.3. Шаг 3: Определить масштаб Core Loop

На основе жанра определить оптимальную длительность одного прохода (от 10 секунд для шутера до 15 минут для стратегии). Это определяет «частоту пульса» игры.

4.11.4. Шаг 4: Спроектировать конверсионный цикл

Определить ресурсы и их конверсионные цепи (Зубек). Убедиться, что: - Каждый ресурс имеет как минимум один источник и один сток - Цикл не является чисто прибыльным (риск гринда) или чисто убыточным (риск фрустрации) - Динамическая настройка прибыльности обеспечена

4.11.5. Шаг 5: Построить вложенные петли

Используя концентрические диаграммы Зубека, спроектировать микро → малый → средний → крупный → макро циклы. Убедиться, что: - Каждый уровень вложен в следующий (результаты микро-цикла питают мезо-цикл) - Нет разрывов между уровнями - Обратная связь присутствует на каждом уровне

4.11.6. Шаг 6: Проверить на патологии

Пройти чек-лист семи патологий: Runaway? Deadlock? Stall? Grind? Frustration Plateau? Disconnected Loops? Loop Overload? При обнаружении патологии — применить соответствующее лечение.

4.11.7. Шаг 7: Прототип и валидация

«Закрывать петлю» (Селлерс): обеспечить, что игрок может действовать, видеть результат и учиться. Если базовая петля не приносит удовольствия — не строить поверх неё. Итерировать: цикл дизайнера (Селлерс) → цикл модели игры → цикл ментальной модели игрока → интерактивная петля → уточнение.

4.12. Примеры Core Loop из реальных игр

Для справки — каталог Core Loop из успешных игр, формализованных через модель конверсионных циклов:

Игра	Core Loop (1 предложение)	Тип	Ресурсы	Конфликт
Halo	Встретить врагов → выбрать подход → выполнить бой → оценить результат	Engine + Ecology	Здоровье, боеприпасы, позиция	Выживание против превосходящих сил
Diablo	Убить монстра → получить лут → продать/купить → улучшить снаряжение → убить сильнее	Economy	HP, мана, золото, XP, предметы	Дефицит лучшего снаряжения
The Sims	Пойти на работу → заработать → купить вещи → улучшить жизнь → повторить	Engine	Деньги, настроение, навыки	Ограниченные деньги vs бесконечные желания
Civilization	Основать город → добыть ресурсы → построить армию → расширить империю → повторить	Economy + Ecology	Золото, наука, производство, территория	Ограниченная территория и ресурсы
Minecraft	Добывать ресурсы → крафтить инструменты → строить → исследовать → добывать лучше	Economy	Блоки, здоровье, голод, инструменты	Ночь и враги vs необходимость укрытия
Dark Souls	Изучить паттерн врага → найти окно атаки → атаковать → уклониться → повторить до победы → продвигаться	Engine + Ecology	HP, stamina, души	Один шанс → один удар → одно неосторожное движение = смерть
Clash of Clans	Собирать ресурсы → строить/обучать → сражаться (два переплетённых Core Loop)	Engine + Economy	Золото, эликсир, трофеи	Время строительства vs давление атак
D&D	Мастер описывает → игроки действуют → мастер adjudicates → новая ситуация	Ecology	HP, заклинания, действия, золото	Ограниченные действия за ход vs неограниченные угрозы
Monopoly	Бросить кубики → купить недвижимость → собирать аренду → покупать ещё	Engine	Деньги, недвижимость	Дефицит денег на покупку
Among Us	Выполнять задачи → наблюдать за другими → подозревать → голосовать → исключать	Ecology + Social	Задачи, подозрения, алиби	Кто эмитент? Доверять или нет?

4.13. Итоги: Core Loop как ядро генерации

Core Loop — не один из многих элементов игры; это фундамент, на котором строится всё остальное. Без работающего Core Loop не работают ни прогрессия, ни баланс, ни нарратив — потому что всё это существует внутри Core Loop, обогащая и направляя его.

Для AI-системы Gidede Core Loop — **первый артефакт генерации**: прежде чем генерировать механики, экономику, прогрессию или нарратив, AI должен сгенерировать Core Loop. И прежде чем добавлять сложность — убедиться, что базовая петля замкнута и приносит удовольствие.

Модели, рассмотренные в этой главе, дают AI полный инструментарий для конструирования Core Loop: - **Петля взаимодействия Кука** — для моделирования когнитивного цикла - **Концентрические циклы Зубека** — для проектирования вложенных петель - **Типология Селлерса** — для определения структурного типа (Engine/Economy/Ecology) - **Цикл прогрессии Шрайбера** — для управления темпом и мотивацией - **Конверсионные цепи Зубека** — для проектирования экономики цикла - **Пять вопросов Гэри** — для верификации чёткости замысла - **«30 секунд веселья»** — для определения оптимального масштаба - **Диагностика патологий** — для выявления и лечения типичных отказов

Глава 5. Баланс и математические модели

> **Раздел Библии геймдизайна**: 2.5 — Баланс и математические модели > **Источники**: KB-07 (Баланс), KB-04 (MechanicsDB), KB-05 (Типология систем), KB-13 (Системный подход) > **Книги**: Шелл (Кн. 1), Адамс/Дорманс (Кн. 4), Гэззэуэй (Кн. 9), Шрайбер (Кн. 7), Роллингс/Моррис (Кн. 12), Селлерс (Кн. 13), Бонд (Кн. 17)

5.1. Что такое баланс и почему он важен

Баланс — одно из самых многозначных понятий в геймдизайне. В узком смысле — это отсутствие доминантной стратегии: когда несколько путей к

победе равноценны и выбор между ними нетривиален. В широком смысле — это согласованность всех элементов игры, при которой ни один не разрушает опыт других. Шелл (Кн. 1) выделяет 12 типов баланса, Шрайбер (Кн. 7) — 7, Роллингс/Моррис (Кн. 12) — 3. Селлерс (Кн. 13) определяет баланс как свойство системы «игра+игрок»: сбалансированная игра не имеет доминантной стратегии, даёт игроку пространство для исследования и ощущается справедливой. Гэззэуэй (Кн. 9) добавляет перспективу агентивности: баланс — это настройка данных, при которой игрок чувствует контроль над ситуацией, а выбор не является очевидным.

Ключевой инсайт, пронизывающий все источники: **математический баланс ≠ воспринимаемый баланс**. Можно идеально сбалансировать числа и при этом создать игру, которая ощущается несправедливой — и наоборот. Игроки подвержены когнитивным искажениям: ошибка игрока («после серии неудач обязательно повезёт»), предвзятость подтверждения (искать доказательства своей правоты), апофения (видеть закономерности там, где их нет). Селлерс подчёркивает: **воспринимаемая справедливость важнее математической**. Это означает, что балансировка — не чисто математическая задача, а задача на стыке математики и психологии.

Для AI-системы Gidede баланс — это **формализуемый артефакт**: AI должен уметь генерировать числовые системы, которые математически сбалансированы, и при этом создавать ощущение справедливости. Это двойная задача: формальные модели (кривые, формулы, матрицы) обеспечивают математическую основу, а эвристики восприятия (формат чисел, прозрачность случайности, кривая сложности) — психологическую.

5.2. Типология баланса: 12 измерений Шелла

Шелл (Кн. 1) предлагает самую полную типологию баланса — 12 типов, каждый из которых описывает отдельное напряжение в дизайне:

1. Справедливость — симметричные vs асимметричные стартовые условия. В соревновательной игре равные стартовые условия — базовое требование; в кооперативной — разная сила персонажей обогащает командную динамику.

2. Сложность и успех — канал потока: слишком сложно → фрустрация, слишком легко → скука. Баланс сложности — не статическое состояние, а динамический процесс: сложность растёт вместе с мастерством игрока.

3. Осмысленный выбор — нетривиальные решения; нет доминантной стратегии; треугольность (выбор: безопасный малый выигрыш vs рискованный большой). Шелл утверждает: в 8 из 10 случаев, когда кто-то жалуется «нет веселья», проблема — в отсутствии треугольности.

4. Навык vs шанс — баланс мастерства и случайности. Игры чистого навыка (шахматы) детерминированы; игры чистого шанса (рулетка) лишены агентивности. Большинство игр — на спектре между.

5. Голова vs руки — ментальный вызов vs физическая ловкость. Стратегия → голова; экшен → руки. Гибридные жанры балансируют оба.

6. Конкуренция vs кооперация — PvP vs совместная игра. Лучшие многопользовательские игры совмещают оба: команды конкурируют, но внутри команды кооперируются.

7. Короткая vs длинная игра — длительность сессии, racing, момент завершения. Слишком короткая — не успеваешь погрузиться; слишком длинная — устаёшь.

8. Награды — слава, ресурсы, доступ, зрелищность. Фиксированные vs переменные графики подкрепления. Переменное отношение — наиболее стойкий паттерн (оперантное научение).

9. Наказание — последствия неудачи. Наказание должно отговаривать от плохой игры, но не фрустрировать. Слишком лёгкое — нет напряжения; слишком суровое — страх неудачи парализует.

10. Свобода vs контролируемый опыт — агентивность игрока vs замысел дизайнера. Полная свобода = хаос; полный контроль = коридор.

11. Простота vs сложность — элегантность: минимум правил с эмерджентной глубиной. «Совершенная игра не может быть улучшена добавлением; совершенная игра не может быть улучшена удалением».

12. Детализация vs воображение — сколько показать vs сколько оставить для додумывания. Меньше деталей → больше воображение → больше вовлечённость.

Для AI-системы эти 12 типов — **валидационный чек-лист**: после генерации числовой системы AI проверяет каждый тип баланса и выявляет проблемы.

5.3. Три модели баланса: транзитивная, нетранзитивная, ситуационная

5.3.1. Транзитивный баланс («получаешь за то, что платишь»)

Транзитивные отношения: если $A > B$ и $B > C$, то $A > C$. Это «экономический» тип баланса: каждый элемент имеет стоимость, и его сила пропорциональна стоимости. RPG-уровни — транзитивны: персонаж 10-го уровня сильнее 5-го, который сильнее 1-го.

Принцип: Сумма преимуществ = Сумма затрат. Если меч стоит 100 золота и даёт +10 урона, а топор стоит 200 золота и даёт +20 урона — это транзитивный баланс: каждый платит пропорционально получаемой силе.

Ограничение: Транзитивный баланс сужает воспринимаемое пространство возможностей. Если каждый элемент точно стоит своей цены, «правильный» выбор очевиден — и осмысленность выбора исчезает. Транзитивный баланс необходим, но недостаточен.

5.3.2. Нетранзитивный баланс (Камень-Ножницы-Бумага)

Нетранзитивные отношения: A бьёт B , B бьёт C , C бьёт A . Классический пример — Камень-Ножницы-Бумага (КНБ). Нетранзитивность расширяет воспринимаемое пространство возможностей: нет единственного лучшего выбора — оптимальная стратегия зависит от выбора оппонента.

Формализация через матрицу выигрышей (Шрайбер, Кн. 7):

Для симметричной игры с нулевой суммой три теоремы: 1. Решение одинаково для обоих игроков 2. Все жизнеспособные стратегии дают одинаковый выигрыш 3. Выигрыш каждой жизнеспособной стратегии = 0

Алгоритм решения нетранзитивной игры: 1. Построить матрицу выигрышей (строки = ваши варианты, столбцы = варианты оппонента, ячейки = ваш выигрыш) 2. Записать уравнения выигрыша для каждого варианта 3. Добавить ограничение: сумма вероятностей = 1 4. Приравнять все выигрыши (к 0 для симметричной игры с нулевой суммой) 5. Решить систему уравнений

КНБ со стоимостью — контринтуитивный результат (Шрайбер): если усилить камень (выигрыш +2 вместо +1), частота выбора бумаги увеличивается, а не камня! Это потому, что оппонент знает об усилении камня и чаще выбирает бумагу — а значит, чаще выбирает ножницы, чтобы бить бумагу. Усиление

элемента может снизить его использование в равновесии Нэша. Это имеет прямое следствие для патчинга: бафф опции может уменьшить её популярность.

Матричное решение: $s = M^{-1} \times p$, где M — матрица выигрышей, s — вектор вероятностей, p — вектор выигрышей.

Расширения: КНБ-Ящерица-Спок (5 элементов, каждый бьёт 2 и проигрывает 2), китайская система У-Син (5 элементов с нетранзитивными связями), биологический пример — ящерицы с оранжевыми/синими/жёлтыми горлами (побеждают друг друга по кругу).

5.3.3. Ситуационный баланс

Элементы не являются универсально «сильнее» или «слабее» — их ценность зависит от контекста. Формула (Шрайбер):

$$EV_{\text{ситуационный}} = \sum P(\text{ситуация}_i) \times \text{Ценность}(\text{объект в ситуации}_i)$$

Ценность универсальности = $\text{Max_cost}(\text{полная неопределённость}) - \text{Min_cost}(\text{полная информация})$. Чем выше неопределённость ситуации, тем дороже универсальность.

АоЕ-ценность = Урон $\times$ Среднее_число_целей. Атака по площади ситуационно сильнее в толпе врагов и слабее против одной цели.

Стоимость переключения: Высокая $\rightarrow$ специализация ценнее (инвестиции в один путь окупаются). Низкая $\rightarrow$ универсальность ценнее (можно адаптироваться на ходу).

5.3.4. Идеальный дисбаланс (Портноу, через Шрайбера)

Джеймс Портноу предлагает парадоксальный принцип: **совершенный баланс = скучно**. Умеренные отклонения (~10-15%) от кривой стоимости создают «идеальный дисбаланс», который делает игру увлекательнее через мета-игру поиска эксплойтов. Условия: (1) игрок не может оптимально использовать всё; (2) разработчик понимает кривую; (3) широкий выбор с контр-стратегиями. Кривая стоимости — слуга, а не хозяин: **нужно знать кривую, прежде чем от неё отклоняться**.

5.4. Кривые стоимости и мощности

5.4.1. Три модели кривых стоимости (Шрайбер)

| Тип игры | Модель | Критерий баланса | |-----|-----|-----| | PvP с ресурсной стоимостью | Кривая тождества ($y=x$) | Сумма преимуществ = Сумма затрат | | PvP без стоимости | Сдвинутая тождественная | Сумма преимуществ – Сумма затрат = $\text{const} > 0$ | | PvE | Прогрессия ценности | $\text{Value}[n+1] = \text{Value}[n] \times (1 + \text{прирост})$ |

Математика поддержки (support math): каждый параметр выражается в единицах якорного ресурса. Пример: +1 урон = 5 единиц преимущества; стихийная атака = 40 единиц; регенерация = 100 единиц. Ограничения = отрицательная стоимость.

Реверс-инжиниринг кривых стоимости из существующих игр: 1. Найти «ванильные» карты (только стоимость + базовые статьи) → система уравнений 2. Найти пары, отличающиеся на один атрибут → вычесть → стоимость разницы 3. Карты с одной неизвестной способностью → определить стоимость способности 4. Валидация через турнирные данные

Пример — стоимость маны в MtG (эмпирическая кривая Шрайбера): для $6+N$ маны: $\lceil 11 + 3N \rceil$. Цветная мана $\approx 2 \times$ бесцветной; вторая единица цветной маны = +3 стоимости.

5.4.2. Шесть типов кривых прогрессии (Селлерс)

1. Линейная: $y = mx + b$. Простая, предсказуемая, но может стать скучной. Одинаковая скорость роста на всех этапах — нет ни ускорения, ни замедления.

2. Полиномиальная: $y = ax^2 + bx + c$. Ускоряющийся рост. Подходит для игр, где поздние уровни должны ощущаться «эпичнее» ранних. Риск: на высоких уровнях числа становятся астрономическими.

3. Экспоненциальная: $y = a \times b^x$. Очень быстрый рост. Опасна: runaway на высоких уровнях. Используется для XP-таблиц (каждый следующий уровень требует экспоненциально больше опыта).

4. Логистическая (S-кривая): Медленный старт → быстрый рост → плато.

Самая естественная кривая для прогрессии: обучение (медленно) → мастерство (быстро) → совершенствование (медленно). Рекомендуется как значение по умолчанию.

5. Кусочно-линейная: Разные линейные сегменты для разных диапазонов. Гибкая, но менее элегантная. Позволяет «настроить» каждый этап прогрессии отдельно.

6. NAP (Near-Arithmetic Progression): Близкая к арифметической с настраиваемым шагом. Простая в понимании, достаточная для большинства игр.

5.4.3. Семь типов кривых Шрайбера

| Кривая | Формула | Применение | |-----|-----|-----| | Тожественная | $y = x$ | Прямой обмен 1:1; деревья навыков | | Линейная | $y = mx$ | Стоимость от урона; HP от маны | | Экспоненциальная | $y = C \times b^x$ | XP-кривая; ускоряющийся финал | | Логарифмическая | $y = \log_b(x)$ | Кривая уровней (обратная к XP-кривой) | | Треугольная | $y = (x^2 - x) / 2$ | **Наиболее используемая формула возрастающей отдачи** | | Пользовательская | Произвольная | Контекстуальные зависимости | | Обфускации стоимости | Нелинейная с разрывами | F2P: соккрытие реальной стоимости |

Правило Сиды Мейера: Если нужно изменить число и не знаешь насколько — удвой или раздели пополам. Это значительно эффективнее итеративных изменений на 10%.

Эвристика: Нужна кривая между линейной и экспоненциальной? Попробуй треугольную первой.

5.4.4. Формула экспоненциального роста (Гэззэуэй)

``` (Предыдущий\_шаг × Mult) + Add = Новый\_шаг ```

Переменные: Init (начальное значение), Mult (множитель итерации, >1 для роста), Add (аддитивная константа).

Рекомендуемые начальные значения: Iterations=100, Init=100, Add=10, Mult=1.02.

Пять типичных проблем и решений: 1. Mult=1 → линейный рост → использовать Mult>1, но малый 2. Большой Mult → астрономические числа → использовать 1.02–1.10 3. Дробные значения → столбец ROUND для отображения 4. Ранние уровни почти линейные → увеличить Add 5. Слишком резкий ранний скачок → увеличить Init

**Ключевое правило:** Больше итераций → меньше множитель.

Экспоненциальные кривые имеют **высший приоритет** в иерархии дизайна — фиксировать первыми.

---

## 5.5. Формула балансировки диапазона и фулкрум

---

### 5.5.1. Формула диапазона (Гэзэуэй)

``` ((Max - Min) × %) + Min = Числовое\_значение ```

Разделение ответственности: Системный баланс (Min, Max) — ведущий дизайнер — ощущение системы в целом. **Индивидуальный баланс** (%) — команда балансировки — ощущение конкретных объектов. Изменение только Min/Max мгновенно обновляет ВСЕ объекты, сохраняя пропорции.

Это мощный архитектурный паттерн для AI-системы: AI может варьировать Min/Max на уровне системы (настройка общей мощности) и % на уровне объектов (настройка индивидуальных различий), не ломая одно другим.

5.5.2. Фулкрум (опорная точка)

Фулкрум — референтный игровой объект, базовая линия для сравнения всех остальных. Часто не появляется в самой игре — чисто технический инструмент.

Транзитивность баланса через фулкрум: Если А сбалансирован относительно фулкрума, и В сбалансирован относительно фулкрума → $A \approx B$. Каждый объект тестируется **только против фулкрума**, что снижает сложность с $O(n^2)$ попарных сравнений до $O(n)$.

Прогрессия фулкрума: Каждый уровень имеет свой фулкрум. Безопаснее начинать со среднего уровня (5-й из 9), затем ослабить для минимума и усилить для максимума.

5.5.3. Взвешенная сумма и DPS

``` Взвешенное\_значение = Сырое\_значение × Вес Итоговый\_балл =  
 $\sum (\text{Значение}_i \times \text{Вес}_i)$  DPM = Урон\_оружия × Атак\_в\_минуту DPS = DPM / 60  
Атак\_в\_минуту = 60 / Задержка(секунды) ```

Контринтуитивный вывод: чем важнее атрибут, тем выше его вес — и, следовательно, тем **ниже** сырые значения у сбалансированных объектов по этому атрибуту. Это объясняет, почему «самый важный» параметр часто имеет наименьшие числа: он и так вносит наибольший вклад через вес.

---

## 5.6. Петли обратной связи и экономическая стабильность

---

### 5.6.1. 8 характеристик петли ОС (Адамс/Дорманс)

Любая петля обратной связи в игре может быть описана через 8 измерений:

|   |                                                             |                                   |                 |                     |                                     |   |
|---|-------------------------------------------------------------|-----------------------------------|-----------------|---------------------|-------------------------------------|---|
| # | Характеристика                                              | Варианты                          | --- ----- ----- | 1                   | <b>Тип</b>                          |   |
|   | Положительная (усиливающая) / Отрицательная (балансирующая) |                                   | 2               |                     |                                     |   |
|   | <b>Эффект</b>                                               | Конструктивный / Деструктивный    | 3               | <b>Инвестиция</b>   | Сколько нужно вложить для активации | 4 |
|   | <b>Отдача</b>                                               | Сколько возвращается              | 5               |                     |                                     |   |
|   | <b>Скорость</b>                                             | Мгновенная / Задержанная          | 6               | <b>Длительность</b> | Одноразовая / Постоянная            | 7 |
|   | <b>Косвенность</b>                                          | Прямая / Опосредованная           | 8               |                     |                                     |   |
|   | <b>Определённость</b>                                       | Детерминированная / Вероятностная |                 |                     |                                     |   |

**Диагностический паттерн:** Положительная, конструктивная, низкая инвестиция, высокая отдача, мгновенная, постоянная, прямая, детерминированная петля — почти гарантированно вызовет runaway. Каждое измерение — рычаг для коррекции: добавить задержку → замедлить runaway; увеличить инвестицию → сделать петлю стратегическим выбором; добавить вероятность → смягчить нарастание.

### 5.6.2. Шесть комбинаций сумма × ОС (Шрайбер)

|                       |                            |                                             |                                        |                                              |
|-----------------------|----------------------------|---------------------------------------------|----------------------------------------|----------------------------------------------|
|                       | Положительная ОС           | Отрицательная ОС                            | Обе                                    |                                              |
| --- ----- ----- ----- |                            | <b>Положительная сумма</b>                  | Экспоненциальный разрыв (4X-стратегии) | «Косичка» — лидерство чередуется (Catan)   — |
|                       | <b>Нулевая сумма</b>       | Быстрое завершение (армрестлинг)            | «Качели» — бесконечное чередование     | Минус рано, плюс поздно (покер)              |
|                       | <b>Отрицательная сумма</b> | Порочный круг (шахматы — оба теряют фигуры) | «Косичка» на нисходящем тренде         | —                                            |

Каждая комбинация создаёт определённый тип PvP-динамики. Экспоненциальный разрыв уместен в PvE (ощущение роста), но разрушителен в PvP. «Качели» создают напряжение, но могут фрустрировать. Стабильное

равновесие хорошо для долгих игр, но может быть скучным. Дизайнер должен сознательно выбирать комбинацию.

### 5.6.3. Q-фактор (Роллингс/Моррис)

**Q-фактор** — субъективная оценка компонента от 1 до 10 по каждому атрибуту. Сортирует компоненты в Q-иерархию. Компоненты, которые хуже всех по всем атрибутам — кандидаты на удаление. Если каждый компонент доминирует хотя бы по одному атрибуту — хороший знак.

**Золотое правило баланса:** Случайный выбор никогда не должен быть оптимальной стратегией. Если это так — плохой дизайн.

**SPS-правило** (Scissors-Paper-Stone): Если отношения между компонентами следуют модели КНБ, каждый компонент динамически (не статически) лучше другого. Это заставляет постоянно менять тактику. Комбинация SPS + Q-фактор создаёт матрицу для валидации динамического баланса.

---

## 5.7. Вероятность и ожидаемое значение в балансе

---

### 5.7.1. Базовые формулы

```  
P = благоприятные_исходы / общие_исходы
 $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
 $P(A \cap B) = P(A) \times P(B)$ [независимые события]
 $P(\text{not } A) = 1 - P(A)$
 $EV = \sum(\text{исход}_i \times P_i)$ [ожидаемое значение]
 $EV(dn) = (n + 1) / 2$ [ожидаемое значение кубика dn]
```

### 5.7.2. Инсайт волатильности (Шрайбер)

При равных EV, но разной дисперсии, **более волатильная опция выигрывает в краткосрочных сценариях**. Воин: 1 урон/ход (стабильно) vs Маг: 6 урона с  $P=1/6$  (волатильно) → при цели в 6 суммарного урона маг выигрывает ~60% боёв. Это объясняет жизнеспособность «стеклянных пушек» даже при равном DPS: в конкретном бою высокая дисперсия — преимущество.

### 5.7.3. Входная vs выходная случайность

**Входная случайность** (до решения игрока — генерация карты, раздача карт) → поддерживает реиграбельность. Игрок адаптируется к обстоятельствам.

**Выходная случайность** (после решения — бросок на попадание) → добавляет напряжённость, но может фрустрировать. Игрок принимает оптимальное решение, но случайность отменяет результат.

Ключевой инсайт: Покер вознаграждает предсказание случайности; блэкджек — только следование оптимальному алгоритму. Это различие определяет, является ли случайность обогащающим или фрустрирующим элементом.

#### 5.7.4. Парадокс редкого лута

При 10% шансе выпадения и 50,000 игроках в день, 4 одинаковых выпадения подряд произойдут ~5 раз в день. Это **статистически нормально, не баг**. Игроки будут жаловаться в любом случае. Это нужно учитывать при проектировании систем лута: всегда добавлять «страховку» (pity timer, гарантированная награда после N попыток).

#### 5.7.5. Эмоциональный дизайн через числа (Гэзэуэй)

| Тип чисел                       | Эмоциональный эффект     | Пример                                              |
|---------------------------------|--------------------------|-----------------------------------------------------|
| «Лёгкие» (круглые, кратные)     | Спокойствие, уверенность | 20 HP — 5 — 5 — 5 = 5 (ясная четверть)              |
| «Тяжёлые» (некруглые, неровные) | Напряжённость, страх     | 20 HP — 6 — 6 — 6 = 2 (больше красного на полоске!) |

AI может **осознанно выбирать формат чисел** для создания желаемых эмоций — это формализация эмоционального дизайна через числовые системы. Большие числа → более гладкое распределение вероятностей (10 HP + 1D6 очень «качается»; 100 HP + 1D6 — стабильно).

---

### 5.8. Цепи Маркова и рекурсивное EV

#### 5.8.1. Марковские цепи (Шрайбер)

Матрица переходов  $M$  × вектор состояний  $V$  = следующее состояние.  
Итеративно:  $V_{n+1} = M \times V_n$ . Стационарное состояние:  $V = M \times V$ .

Применение: моделирование вероятностей состояний игры во времени. Например, вероятности выигрыша/проигрыша в зависимости от текущего счёта, вероятности нахождения в определённой экономической фазе.



### 5.8.2. Рекурсивное EV для бесконечных процессов

Заклинание, наносящее 1 урон с  $P=0.2$  повторения:  $D = 1 + 0.8D \rightarrow D = 1/0.2 = 5$ . Это мощный инструмент для балансировки цепных механик: способностей с шансом повторного срабатывания, комбо-систем, пульсирующих эффектов.

### 5.8.3. Формула воспринимаемой сложности (Шрайбер)

```` Воспринимаемая\_сложность =  $(C_v + C_s) - (P_v + P_s)$  ````

Где C_v = виртуальный вызов (сила врагов), C_s = стратегический вызов (необходимость новых стратегий), P_v = виртуальная сила игрока (уровень персонажа), P_s = реальный навык игрока.

Положительная воспринимаемая сложность $\rightarrow$ ощущение вызова.
Отрицательная $\rightarrow$ ощущение лёгкости. Нулевая $\rightarrow$ ощущение баланса. Эта формула — основа для динамической настройки сложности (DDA): AI подстраивает C_v и C_s так, чтобы воспринимаемая сложность оставалась в «зоне потока».

5.9. Рейтинговые системы и матчмейкинг

5.9.1. Система Elo

````  $E = 1 / (1 + 10^{((R_{opp} - R_{self})/400)})$  [ожидаемый выигрыш]  $R_{new} = R_{old} + K \times (A - E)$  [обновление рейтинга] ````

$A$  = фактический результат (1/0/0.5),  $K$  = константа волатильности (32 для новых, 16 для установившихся).

**Elo с поправкой на удачу** (Шрайбер):  $E_{adjusted} = (1 - L) \times E_{skill} + 0.5 \times L$ , где  $L$  = коэффициент удачи. Единственный источник, дающий формальный метод корректировки рейтингов в играх со значительной случайностью.

### 5.9.2. Система Glicko

Рейтинг + RD (Ratings Deviation — отклонение рейтинга). 95% доверительный интервал =  $R \pm 2RD$ . Больше RD  $\rightarrow$  больше неопределённость  $\rightarrow$  большие изменения рейтинга после матча. По мере игры RD уменьшается  $\rightarrow$  рейтинг стабилизируется.

### 5.9.3. TrueSkill (Microsoft)

$\mu$  (среднее = рейтинг) и  $\sigma$  (стандартное отклонение = неопределённость). Гауссово распределение. Командные игры:  $\mu_{team} = \Sigma \mu_i$ ,  $\sigma_{team} = \Sigma \sigma_i$ . Преимущество: поддерживает командные форматы и неопределённость нового игрока.

---

## 5.10. Макро-модель RPG и формулы прогрессии

---

### 5.10.1. Макро-модель RPG (Шрайбер)

``` Длительность = Т часов Число\_уровней = L Переходов = L – 1 Переходов/час  $\approx L/T$  (больше в начале, меньше в конце) Стадий\_контента  $\approx L/2$  Конфигураций\_врагов  $\geq 3 \times (L/2)$  Очков\_характеристик\_за\_уровень = (Финальное – Начальное) / (L – 1) ```

Пример: 40-часовая RPG с 20 уровнями $\rightarrow$ 19 переходов $\rightarrow$ ~ 0.5 переходов/час $\rightarrow$ 10 стадий контента $\rightarrow$ ≥ 30 конфигураций врагов. Формула показывает, что для RPG с Core Loop «бой $\rightarrow$ лут $\rightarrow$ прокачка» нужно астрономически меньше уникального контента, чем если бы каждый бой был уникальным: Core Loop обеспечивает реиграбельность через вариацию контекста.

5.10.2. Четыре этапа прогрессии D&D (Кн. 16)

| Этап | Уровни | Масштаб | Динамика баланса |
|------|--------|---------------------|--|
| 1-й | 1-4 | Локальные угрозы | Базовые способности, простые комбинации |
| 2-й | 5-10 | Региональные угрозы | Расширенный арсенал, многовариантные тактики |
| 3-й | 11-16 | Мировой масштаб | Высшая магия, сложные взаимодействия систем |
| 4-й | 17-20 | Мультивселенский | Эпические способности, баланс хрупок |

Каждый этап — собственная задача балансировки: на низких уровнях важен транзитивный баланс (сила пропорциональна уровню), на высоких — нетранзитивный (взаимные контры способностей).

5.10.3. Математика гачи (Шрайбер)

10 редких + 10 необычных + 10 обычных предметов $\rightarrow$ в среднем нужно купить **293** персонажей для полной коллекции. Это прямая формула для балансировки F2P-экономики: зная размер пула и вероятности, можно рассчитать среднюю стоимость полной коллекции и настроить монетизацию.

5.11. Шесть методов макросистемной балансировки (Гэзэуэй)

| Метод | Когда уместен | Когда неуместен | |-----|-----|-----| |
Чистый баланс | Серьёзные соревнования | Веселье-ориентированные игры | |
Положительная ОС | Разрушение тупика | Догоняющие (усиливает разрыв) | |
Отрицательная ОС (резинка) | Казуальные игры | Серьёзные соревнования | |
DDA (динамическая сложность) | Мобильные F2P, AAA-экшен | Песочницы, открытые RPG | |
Многоуровневая сложность | Агентивность игрока | Ограниченные ресурсы разработки | |
Перекрёстная калибровка | Большие игры с множеством механик | Сложность балансировки |

Бинарный поиск для балансировки (Гэзэуэй): Когда правильное значение неизвестно — начать с любого числа, удваивать, пока «слишком много», затем бинарный поиск. Для 1024 возможных значений — всего 11 guesses. Этот алгоритм可直接 формализован для автоматической балансировки в AI-системе.

5.12. Диагностика: 6 признаков общего баланса (Гэзэуэй)

1. **Время выбора:** Мгновенный выбор = дисбаланс (опция очевидно лучше/хуже); раздумье = хороший знак 2. **Разногласие среди игроков:** О том, какая опция лучше = хороший знак (нет консенсуса = нет доминанты) 3. **Разнообразие частоты использования:** Ни одна опция не используется подавляюще чаще других 4. **Симметрия жалоб:** Жалобы о разных вещах = баланс (все немного недовольны = все немного довольны) 5. **Нестабильность тир-листов:** Тир-листы меняются со временем = баланс (мета-эволюция) 6. **Разногласие экспертов:** Топ-аналитики расходятся во мнениях = баланс

Для AI-системы эти признаки — **аналитические метрики:** AI может отслеживать частоту выбора, распределение жалоб, стабильность мета-игры и формально определять, сбалансирована ли система.

5.13. Патологии баланса и их лечение

Объединяя диагностику из всех источников, формулируем каталог из восьми патологий:

5.13.1. Доминантная стратегия

Симптом: Один путь к победе очевидно лучше остальных. Игроки выбирают одно и то же.

Диагностика: Частота выбора одной стратегии $> 50\%$. Жалобы только об одной тактике. Тир-лист стабилен и возглавляется одним элементом.

Лечение: Ввести нетранзитивные отношения (КНБ-структуру), повысить стоимость доминанта, добавить контр-стратегии, ослабить синергии доминантной стратегии.

5.13.2. Runaway (саморазгон)

Симптом: Ранний лидер экспоненциально отрывается. «Богатые богатеют».

Диагностика: Усиливающая петля без балансирующей (8-мерный профиль: положительная, конструктивная, низкая инвестиция, высокая отдача, мгновенная, постоянная, прямая, детерминированная).

Лечение: Добавить Dynamic Friction (сопротивление, масштабирующееся с уровнем ресурса), Stopping Mechanism (жёсткий потолок), или отрицательную ОС (резиновая лента).

5.13.3. Мёртвая зона (ничего не работает)

Симптом: Несколько опций очевидно бесполезны. Игроки никогда их не выбирают.

Диагностика: «Параметр-мусор» (safely ignorable attribute) — если атрибут можно игнорировать без потери эффективности, он мёртв. Частота использования опции $\approx 0\%$.

Лечение: Усилить опцию (повысить сырые значения), снизить стоимость (сделать «дешёвой альтернативой»), добавить уникальную ситуационную ценность (контекст, в котором опция сильна).

5.13.4. Обязательный выбор (божественный параметр)

Симптом: Один атрибут или опция обязательна для любого оптимального билда. «Божественный параметр» (mandatory for any optimal build).

Диагностика: 100% оптимальных билдов включают элемент. Исключение элемента делает билд заведомо неоптимальным.

Лечение: Усилить альтернативы, добавить затраты на обязательный элемент (сделать его дорогим, чтобы отказ от него давал ресурсы для других путей), ввести нетранзитивность (элемент силён в одном контексте, слаб в другом).

5.13.5. Разрыв билдов

Симптом: С ростом уровня разрыв между оптимальным и субоптимальным билдом увеличивается.

Формула: $\Delta(t) = \text{Power_optimal}(t) - \text{Power_competitive}(t)$. Если Δ монотонно возрастает — дисбаланс растёт с уровнем.

Лечение: Добавить «пересечения» кривых мощности (субоптимальный билд силён на одних уровнях, оптимальный — на других), ввести затухающие доходы (diminishing returns) для доминантного пути.

5.13.6. Инфляция / стагнация экономики

Симптом (инфляция): Валюта обесценивается от перепроизводства. Покупки теряют значимость.

Лечение: Добавить стоки (sinks) — отрицательно-суммарные элементы, удаляющие валюту. Экономические ресеты (сезонные обнуления).

Симптом (стагнация): Валюта накапливается без возможностей траты.

Лечение: Добавить значимые стоки (улучшения, косметика, доступ к контенту). Каждый источник должен иметь парный сток.

5.13.7. Хрупкость экологии

Симптом: Малое возмущение вызывает каскадный коллапс. Система кажется устойчивой, пока не столкнётся с определённым типом возмущения.

Диагностика: Статистическое управление процессами: если ресурс отклоняется на 3+ сигмы от среднего — система «вышла из-под контроля». Переход от устойчивости к хрупкости трудно обнаружить.

Лечение: Добавить резервные механизмы (backup loops), диверсифицировать источники ресурсов (чтобы отказ одного не обрушил систему), ввести отрицательные ОС с разными временными масштабами.

5.13.8. Воспринимаемая несправедливость

Симптом: Математика правильна, но игроки жалуются на нечестность.

Диагностика: Разрыв между математическим и воспринимаемым балансом. Когнитивные искажения: ошибка игрока, предвзятость подтверждения, апофения.

Лечение: Использовать «скрытую» случайность (резиновая лента, незаметная помощь проигрывающим), pity timer (гарантированная награда после N неудач), прозрачность вероятностей (показать реальные шансы — снижает жалобы), «иллюзия возможной победы» (Кроуфорд): игрок должен верить, что может победить; при неудаче винить себя, а не игру.

5.14. Четыре метода балансировки (Шрайбер) + клонирование

1. **Опыт разработчика** — эвристики, данные из предыдущих игр. Быстро, но неточно. 2. **Маломасштабное тестирование** — наблюдение за реальными игроками. Качественно, но мало данных. 3. **Аналитика** — крупномасштабный сбор данных. Количественно, но корреляция $\neq$ каузация. 4. **Математика** — таблицы, кривые, вероятностные модели. Точно, но не учитывает психологию. 5. **Клонирование** (неформально) — заимствовать числа из существующей сбалансированной игры с пониманием, почему она сбалансирована.

Принципы модификации чисел: - Большие изменения ($\times 2$), если не знаешь правильное значение (Правило Сиды Мейера) - $\geq 10\%$, если есть примерное представление - Начинать с экстремальных значений при поиске

Два подхода к масштабу чисел: «Всё сверхмощное» (эпичное ощущение) vs «Ничто не сверхмощно» (скромные числа). **Никогда** не делать «почти всё посредственное + пара сверхмощных предметов» — это худший из обоих миров.

5.15. Итоги: баланс как двойная задача

Баланс в геймдизайне — это **двойная задача**: математическая формализация + психологическое восприятие. AI-система Gidede должна:

1. **Генерировать** числовые системы, математически сбалансированные через кривые стоимости, формулы диапазонов и нетранзитивные структуры 2. **Валидировать** сгенерированные системы через 12 типов баланса Шелла, 6 признаков Гэззюэя и диагностику 8 патологий 3. **Проверять** воспринимаемый баланс через формулу воспринимаемой сложности, эмоциональный дизайн чисел и когнитивные искажения 4. **Итерировать** через бинарный поиск, A/B-тестирование и аналитику

Инструментарий, рассмотренный в этой главе, даёт AI полный арсенал: - **Кривые стоимости** — для транзитивного баланса - **Матрицы выигрышей** — для нетранзитивного баланса - **Формула диапазона + фулкрум** — для системной настройки - **8-мерный профиль петель ОС** — для диагностики динамики - **6 комбинаций сумма × ОС** — для выбора типа PvP-динамики - **Формулы вероятности и EV** — для случайности - **Рейтинговые системы** — для матчмейкинга - **Диагностика патологий** — для выявления и лечения отказов

Глава 6. Экономика игры и системы прогрессии

> **Раздел Библии геймдизайна**: 2.6 — Экономика игры и системы прогрессии
> **Источники**: KB-04 (Machinations), KB-07 (Баланс), KB-13 (Системный подход), KB-06 (Циклы конверсии) > **Книги**: Адамс/Дорманс (Кн. 4), Шрайбер (Кн. 7), Селлерс (Кн. 13), Зубек (Кн. 6), Шелл (Кн. 1), Бонд (Кн. 17), Роллингс/Моррис (Кн. 12)

6.1. Что такое игровая экономика и почему она критична

Игровая экономика — это система производства, распределения и потребления ресурсов внутри игры. Термин «экономика» здесь понимается

расширительно: это не только деньги и торговля, но и любые дискретные ресурсы — здоровье, опыт, боеприпасы, юниты, территории, действия, время. Адамс и Дорманс (Кн. 4) определяют внутреннюю экономику как один из пяти фундаментальных типов механики наряду с физикой, прогрессией, тактическим маневрированием и социальным взаимодействием. При этом именно внутренняя экономика поддается **строгой формализации** — она дискретна и может быть описана математически, в отличие от физики (непрерывной) или социального взаимодействия (неформализуемого).

Селлерс (Кн. 13) углубляет это понимание, вводя типологию из трёх типов игровых систем: **двигатели** (engines), **экономики** (economies) и **экологии** (ecologies). Двигатели оперируют одним ресурсом через усиливающие или тормозящие петли; экономики добавляют обмен и конвертацию ресурсов с нелинейным приростом ценности; экологии стремятся к динамическому равновесию через балансирующие петли с обменом. Большинство реальных игр — иерархия вложенных систем: RPG содержит экономику (ядро прокачки), экосистемы (бой) и двигатели (ресурсные потоки), вложенные друг в друга.

Зубек (Кн. 6) предлагает операциональный инструмент: **цепочки конверсии** и **циклы конверсии**. Механики образуют цепочки «производитель→потребитель»: сражение конвертирует монстров + здоровье в добычу; продажа конвертирует добычу в золото; исцеление конвертирует золото в здоровье. Повторяющиеся цепочки образуют циклы, чья прибыльность определяет динамику всей игры. Прибыльный цикл → линейный рост ресурсов → риск гринда; неприбыльный цикл → истощение → риск разочарования. Динамическая настройка — масштабирование кривой прибыли со временем — позволяет циклу из прибыльного становиться неприбыльным, создавая естественные фазы игры.

Для AI-системы Gidede игровая экономика — это **формализуемый каркас**, на который натягиваются все остальные системы. AI должен уметь: (1) генерировать экономические модели на основе описания игры; (2) анализировать их на типичные патологии; (3) балансировать через симуляцию; (4) визуализировать через Machinations-подобные диаграммы.

6.2. Элементы внутренней экономики

6.2.1. Базовые элементы по Адамсу/Дормансу

Книга Адамса и Дорманса (Кн. 4) вводит формальную систему элементов, из которых строится любая игровая экономика. Эта система составляет основу

визуального языка Machinations и позволяет документировать, анализировать и симулировать экономику без написания кода.

| Элемент | Обозначение | Функция | |-----|-----|-----| | **Источник (Source)** | Треугольник ↑ | Генерирует ресурс из ниоткуда (автоматически или по действию игрока) | | **Сток (Drain)** | Треугольник ↓ | Уничтожает ресурс (траты, потери, смерть) | | **Пул (Pool)** | Круг/прямоугольник | Хранит ресурс, показывает текущее количество | | **Конвертер (Converter)** | Треугольник с вертикалью ⊥ | Превращает N ресурсов типа A в M ресурсов типа B | | **Торговец (Trader)** | Двойной треугольник $\rightleftharpoons$ | Обменивает ресурсы между двумя пулами |

Каждый из этих элементов имеет чёткую математическую семантику: источник описывается скоростью генерации (ресурсов за тик), сток — скоростью потребления, пул — текущим значением и ёмкостью, конвертер — коэффициентом преобразования, торговец — курсом обмена. Это позволяет автоматически строить системы уравнений для анализа экономического поведения.

6.2.2. Продвинутое узлы Machinations

Базовые элементы дополняются продвинутыми, которые добавляют условную логику и динамику:

| Узел | Функция | |-----|-----| | **Регистр (Register)** | Хранит вычисленное значение — формулу от других узлов. Позволяет выражать производные величины | | **Ворота (Gate)** | Условный распределитель ресурсов — перенаправляет поток по вероятности или условию. Создаёт ветвление в экономике | | **Активатор (Activator)** | Включает/выключает другой узел в зависимости от состояния. Моделирует условия блокировки | | **Триггер (Trigger)** | Запускает немедленное действие при выполнении условия. Реализует пороговые эффекты | | **Задержка (Delay)** | Откладывает действие на заданное число тиков. Моделирует время производства, кулдауны | | **Очередь (Queue)** | Ресурсы выстраиваются в порядке поступления (FIFO). Моделирует лист ожидания, очереди строительства |

Два типа связей между узлами: **связь ресурсов** (сплошная стрелка — физический поток ресурсов) и **связь состояний** (пунктирная стрелка — влияние состояния одного узла на другой без перемещения ресурсов). Это различие критически важно: связь состояний позволяет моделировать петли обратной связи, при которых информация о количестве ресурса влияет на скорость его генерации, не перемещая сам ресурс.

6.2.3. Системы «источник — хранилище — сток» по Селлерсу

Селлерс (Кн. 13) предлагает более интуитивную метафору для описания тех же элементов — «ванна с водой»: источник (кран) наполняет хранилище (ванну), сток (слив) опустошает. Поток (flow) — связь между элементами, по которой перемещается ресурс. Эта метафора делает экономическое моделирование доступным для дизайнеров без математической подготовки.

Ключевое дополнение Селлерса — концепция **поведения (behaviors)**: каждая часть системы инкапсулирует своё состояние и воздействует на другие части через поведение — создание, изменение или уничтожение ресурсов. Это прямо заимствовано из объектно-ориентированного программирования и позволяет мыслить об элементах экономики как об автономных агентах, а не только как о пассивных контейнерах.

6.3. Экономические структуры и их динамика

6.3.1. Пять экономических структур (Адамс/Дорманс)

| Структура | Описание | Эффект на геймплей | |-----|-----|-----| |
Петля обратной связи | Выход системы влияет на её вход | Усиление (положительная) или балансировка (отрицательная) | | **Взаимная блокировка (Deadlock)** | А нужен Б, Б нужен А | Застой и фрустрация — или стратегическая задача | | **Равновесие** | Производство = Потребление | Стабильная экономика, предсказуемый рост | | **Дефицит** | Потребление > Производство | Напряжённость, конкуренция за ресурсы | | **Профицит** | Производство > Потребление | Накопление, экспансия, «золотой век» |

Каждая структура порождает определённый тип игрового опыта. Равновесие создаёт ощущение стабильности и предсказуемости — идеально для строителей и менеджеров. Дефицит генерирует напряжение и заставляет принимать трудные решения — основа survival и хардкорных игр. Профицит даёт ощущение мощи и свободы — ключ к power fantasy. Взаимная блокировка может быть как проблемой (фрустрация от невозможности начать), так и дизайнерским инструментом (стратегическая задача: как разорвать deadlock?).

6.3.2. Три типа систем по Селлерсу

Селлерс (Кн. 13) вводит типологию, которая стала одной из центральных моделей для Gidede, поскольку она связывает структуру экономической системы с типом порождаемого геймплея:

1. Двигатели (Engines) — петли с одним ресурсом. Усиливающие (boosting) двигатели добавляют ресурсы в игру: игрок выбирает, использовать ресурс сейчас или инвестировать для увеличения потока в будущем. Тормозящие (braking) двигатели удаляют ресурсы, предотвращая runaway. Три типичные проблемы двигателей: runaway («богатые богатеют»), deadlock (ресурс, нужный для запуска, можно получить только из работающего двигателя), stall (недостаточно ресурсов для продолжения).

2. Экономии (Economies) — усиливающие петли с обменом/конвертацией ресурсов. Игрок получает способность, обменивая один ресурс на другой с нелинейным приростом ценности. Ключевое свойство — **развёртывание сложности (unfolding complexity)**: экономика начинается просто (хлеб + дерево) и постепенно добавляет новые ресурсы, здания, цепочки (руда → железо → сталь → оружие). Каждый новый уровень увеличивает глубину решений. Валюты — каталитические ресурсы, которые не потребляются при обмене и могут использоваться в любой форме обмена.

3. Экологии (Ecologies) — балансирующие петли с обменом/конвертацией ресурсов. Целое стремится к динамическому равновесию (метастабильности), хотя части постоянно меняются. Классический пример: хищник-жертва (олени/волки). В играх: инвентарь (больше предметов → меньше места → баланс), бой (два уравнивающих друг друга противника), социальные экологии (3 фракции — если одна усиливается, две другие объединяются). Критический инсайт Селлерса: **две фракции → runaway** (WoW: Alliance > Horde), **три фракции → динамическое равновесие** (Dark Age of Camelot).

6.3.3. Статическая vs развивающаяся экономика

Адамс/Дорманс (Кн. 4) проводят важное различие:

| Характеристика | Статическая экономика | Развивающаяся экономика |
|------------------|---|--|
| Структура | Фиксированная от старта | Изменяется решениями игрока |
| Примеры | Монополия, шахматы Civilization, SimCity, StarCraft | Паттерн Machinations Static Engine Engine Building |
| Задача дизайнера | Сбалансировать начальную конфигурацию | Сбалансировать ВСЕ возможные конфигурации |

Развивающаяся экономика — фундаментально более сложная задача балансировки, но она же создаёт наиболее глубокий и вовлекающий геймплей. Игрок САМ строит свою экономику, и это даёт ощущение авторства и контроля, которое невозможно получить в статической системе.

6.4. Паттерны дизайна экономик

6.4.1. Библиотека паттернов Machinations (Адамс/Дорманс)

Адамс и Дорманс создали каталог из 16+ структурных паттернов, формализованных в Machinations. Каждый паттерн описывает типовую экономическую структуру, условия применения и типичные проблемы. Для AI-системы Gidede это **генеративный алфавит**: AI может выбирать, комбинировать и проверять паттерны автоматически.

Паттерны-двигатели (Engines)

| Паттерн | Суть | Когда использовать | |-----|-----|-----| | **Static Engine** | Постоянный, предсказуемый поток ресурсов | Стабильная базовая экономика (зарплата каждый ход, фиксированный доход) | | **Dynamic Engine** | Поток ресурсов, который можно улучшать/апгрейдить | Игрок инвестирует в рост (RTS-харвестеры, фабрики, апгрейды добычи) | | **Converter Engine** | Два конвертера в цикле, создающие излишек | Петля роста через преобразование (руд → золото → постройка → больше рудника) | | **Engine Building** | Игрок сам конструирует свою экономику | Экономика — часть творчества (Civilization, Factorio, Stellaris) |

Паттерны-трения (Friction)

| Паттерн | Суть | Когда использовать | |-----|-----|-----| | **Static Friction** | Постоянное сопротивление/сток ресурсов | Фиксированные расходы (налог каждый ход, содержание войск, аренда) | | **Dynamic Friction** | Сопротивление, масштабирующееся с уровнем ресурсов | Замедление лидера (подходящий налог, усложнение на высоких уровнях, rubber-banding) | | **Stopping Mechanism** | Жёсткий потолок, останавливающий производство | Абсолютный лимит (максимум юнитов, население города, вместимость склада) | | **Attrition** | Ресурсы потребляются с нарастающей скоростью | Эскалация давления (износ оборудования, рост расходов, старение юнитов) |

Паттерны-эскалации (Escalation)

| Паттерн | Суть | Когда использовать | |-----|-----|-----| | **Escalating Challenge** | Препятствия становятся сложнее | Прогрессирующий racing (волны врагов, уровни сложности, рейды) | | **Escalating Complexity** | Система становится сложнее со временем | Давление принятия решений (Tetris — скорость растёт, RTS — больше юнитов) | | **Arms Race** | Конкурирующие игроки эскалируют друг против друга | PvP-баланс (многоуровневые стратегии, гонка технологий) |

Прочие паттерны

| Паттерн | Суть | |-----|-----| | **Play-Style Reinforcement** | Долгосрочная стратегическая приверженность вознаграждается — игроки, выбравшие специализацию, получают бонусы за углубление, а не за переключение | | **Multiple Feedback** | Несколько одновременных петель обратной связи — создаёт богатую динамику, но сложнее в балансировке | | **Trade** | Обмен между игроками или ресурсными системами — создаёт социальную динамику и возникающие курсы обмена | | **Worker Placement** | Распределение ограниченных действий — конкуренция за дефицитные слоты создаёт напряжение | | **Slow Cycle** | Периодические, предсказуемые фазовые изменения (день/ночь, сезоны, экономические циклы) — добавляет долгосрочную стратегическую глубину |

6.4.2. Рабочий процесс проектирования экономики

Адамс/Дорманс (Кн. 4) описывают четырёхшаговый процесс, который прямо формализуем для AI:

1. **Начать с базового паттерна** — выбрать подходящий из библиотеки на основе жанра и целей игры 2. **Уточнить** — добавить детали, специфичные для конкретной игры (конкретные ресурсы, коэффициенты, условия) 3. **Комбинировать** — соединить несколько паттернов для создания сложной экономики (двигатель + трение + эскалация) 4. **Упростить** — убрать избыточные элементы, сохранить элегантность

Процесс Lunar Colony (сквозной пример из книги) демонстрирует этот подход: базовый двигатель (добыча минерала) → добавить трение (затраты на поддержку базы) → добавить эскалацию (нарастающая сложность операций) → комбинировать паттерны → протестировать через симуляцию → итерировать. Это **шаблон проектирования** для AI-модуля генерации экономик.

6.4.3. Шесть типов экономических систем (Шрайбер)

Шрайбер (Кн. 7) классифицирует экономики по механизму ценообразования и включению реальных денег:

1. **Фиксированная** — разработчик устанавливает все цены. Наиболее распространённый тип. Дизайнер контролирует каждый аспект, но теряет гибкость: если цена неверна, не существует механизма самокоррекции.
2. **С определяемой игроками динамикой** — аукцион WoW, торговля в Catan. Цены формируются спросом и предложением. Более реалистичная и самобалансирующаяся, но может страдать от спекуляций и арбитража.
3. **F2P-экономика** — часть ресурсов за реальные деньги/рекламу. Двойная валюта: софтверная (зарабатывается в игре) и хардвалюта (покупается за реальные деньги). Ключевой принцип: средние ресурсы игрока намеренно недостаточны для задачи → давление на донат.
4. **С ориентацией на престиж** — косметика за деньги, не влияющая на геймплей. Демонстрация статуса без нарушения баланса. Примеры: скины в League of Legends, косметика в Fortnite.
5. **Использующая реальные деньги** — азартные игры, игры с выводом. Строго регулируется, требует лицензий.
6. **Смешанная** — комбинация нескольких типов. Большинство современных AAA-игр используют смешанную экономику: фиксированные цены NPC + аукцион игроков + косметический магазин + battle pass.

6.5. Ресурсы: типология, свойства и проектирование

6.5.1. Пять классов ресурсов (Шрайбер)

Шрайбер (Кн. 7) выделяет пять фундаментальных классов ресурсов, каждый из которых имеет уникальные свойства и требует особых подходов к балансировке:

1. **Время** — самый важный ресурс. Продолжительность игры определяет количество ВСЕХ остальных ресурсов. Формула RPG-прогрессии: `Т часов → L уровней → L-1 переходов → ~L/T переходов/час → L/2 стадий контента → ≥3×(L/

2) конфигураций противника`. Время необратимо и одинаково для всех игроков — это делает его идеальным якорем для балансировки.

2. Валюта — универсальный ресурс обмена. Внутриигровая, донатная, реальные деньги. Курс обмена часто скрывается для обфускации реальной стоимости (F2P-дизайн). Валюта подвержена инфляции и стагнации — двум противоположным патологиям, которые требуют разных методов лечения.

3. Специфичные игровые объекты — ценность меняется по ходу игры. Железный меч бесценен на 1-м уровне и бесполезен на 50-м. Алмазы обесценивают железо. Это создаёт естественную прогрессию ценности и проблему «мусорного лута» на поздних этапах.

4. Жизни / HP — редко рассматривается как ресурс в отрыве, но один из самых важных. Общий уровень сложности во многом зависит от управления этим ресурсом. HP — «якорный ресурс» (anchor) для боевых игр: все боевые параметры можно выразить в единицах HP.

5. Опыт и уровни — очки опыта (награда), уровни (пороговые значения), навыки, ранги. Верхний предел уровня критичен: без потолка в PvP новые игроки никогда не смогут конкурировать со старыми. Слишком низкий потолок → ощущение ограниченности; слишком высокий → недостижимость.

6.5.2. Ядерные и подсобные ресурсы (Селлерс)

Селлерс (Кн. 13) разделяет ресурсы по степени связи с core loop:

- **Ядерные (core resources)**: наиболее важные, напрямую связанные с базовыми игровыми циклами. HP и XP в RPG, деньги в tycoon, территория в стратегии. Ядерные ресурсы — главные «существительные» игры; их потоки определяют структуру экономики. - **Подсобные (subsidiary resources)**: поддерживают ядерные. Армия, технологии, фортификации — всё для удержания территории. Подсобные ресурсы создают глубину: они нужны для эффективного использования ядерных, но не являются самоцелью.

Это разделение критически важно для AI-системы: при генерации экономики AI сначала определяет ядерные ресурсы (их количество обычно невелико — 2-4), затем проектирует подсобные (их может быть много), и только потом — связи между ними.

6.5.3. Абстракция ресурсов

Важный принцип, проходящий через все источники: **ресурсом может быть любая дискретная величина, которую игра отслеживает.** Адамс/Дорманс

приводят примеры: очки (Pac-Man), жизни, ключи, патроны, время хода, территория, действия, приоритет хода, информация. Селлерс добавляет: «даже пространство — это ресурс» (в стратегиях территория — ключевой ресурс; в inventory management — ячейки инвентаря; в tower defense — клетки для размещения башен). Чем шире дизайнер мыслит о ресурсах, тем богаче экономика.

6.6. Прогрессия: от замка-и-ключа к эмерджентной прогрессии

6.6.1. Дихотомия Юла: эмерджентность vs прогрессия

Книга Адамса/Дорманса (Кн. 4) строится на фундаментальном различии, введённом Джеспером Юлом:

| | | | | |
|---|----------|------------------|--------------|----------------------------|
| Парадигма | Описание | Структура правил | Игровой опыт | |
| ----- | ----- | ----- | ----- | Игры эмерджентности |
| Небольшое число правил порождает множество вариаций геймплея Немного правил → много взаимодействий Шахматы, Go, покер, RTS | | | | Игры прогрессии |
| Дизайнер создаёт заранее спланированную последовательность вызовов Много правил → однонаправленный путь Adventure, Portal, FPS-кампания | | | | |

Эмерджентность обеспечивает реиграбельность, но трудно контролирует опыт. Прогрессия обеспечивает контролируемый опыт, но исчерпывается. Большинство успешных игр **совмещают** обе парадигмы: Mass Effect (прогрессия сюжета + эмерджентность боёвки), Skyrim (основная сюжетная линия + открытый мир), Dark Souls (линейная структура уровней + эмерджентная боевая система).

Интеграция двух парадигм — не простое сложение, а проектирование **моста** между ними. Дизайнер решает, где механики создают свободу, а где направляют игрока. Для AI это означает: при генерации концепции — определять, какая парадигма доминирует, и подстраивать структуру механик и контента соответственно.

6.6.2. Механизм «замка и ключа»

Классический механизм прогрессии (Адамс/Дорманс, Кн. 4): для продвижения дальше нужен определённый предмет или навык (ключ), который открывает блокирующий элемент (замок).

Простая модель: Ключ А → Замок А → Ключ Б → Замок Б — линейная цепочка. Каждый ключ открывает ровно один замок, каждый замок блокирует ровно один путь. Это простейший и наиболее контролируемый тип прогрессии.

Метроидвания: Нелинейная структура — несколько ключей, несколько замков, обратное отслеживание. Игрок получает ключ в одной области, который открывает замок в другой, ранее посещённой области. Это создаёт исследование и ощущение растущей свободы, но требует тщательного проектирования маршрутов.

Динамические замки: Замки, меняющие состояние в зависимости от контекста. Например, дверь открывается только ночью, или проход доступен только при определённом уровне репутации. Это связывает прогрессию с другими системами игры (циклы, экономика, социальные механики).

6.6.3. Эмерджентная прогрессия — революционная концепция

Революционная идея Адамса/Дорманса: прогрессию можно переопределить не как «пространственное движение от начала к концу», а как **изменение состояния игры в направлении целевого состояния**. В этом смысле:

- **Традиционная прогрессия:** состояние «замок закрыт» → действие «найти ключ» → состояние «замок открыт» → новая область. Дизайнер явно задаёт последовательность. - **Эмерджентная прогрессия:** экономика игры естественным образом проходит фазы (начальная → рост → зрелость → завершение), и эти фазы — тоже прогрессия. Система правил создаёт естественное направление развития без скриптов.

Граница между эмерджентностью и прогрессией — не жёсткая. «Эмерджентная прогрессия» — это мост: система правил создаёт естественное направление развития без прямого скриптования. Для AI-системы это означает возможность генерировать системы прогрессии двух типов: скриптовые (замок-и-ключ) и эмерджентные (экономические фазы). Комбинация обоих — наиболее мощная стратегия.

6.6.4. Четыре этапа прогрессии D&D (Кн. 16)

Система Dungeons & Dragons предоставляет эталонную модель прогрессии с четырьмя чёткими этапами, каждый из которых является собственной задачей балансировки:

| Этап | Уровни | Масштаб | Ключевая динамика |
|-----|-----|-----|-----| | 1-й | 1-4 | Локальные угрозы | Базовые способности, простые комбинации, ограниченные ресурсы | | 2-й | 5-10 | Региональные угрозы | Расширенный арсенал, многовариантные тактики, больше ресурсов | | 3-й | 11-16 | Мировой масштаб | Высшая магия, сложные взаимодействия систем, огромная мощь | | 4-й | 17-20 | Мультивселенский | Эпические способности, баланс хрупок, игра на грани сломанности |

Каждый этап — качественный скачок, а не просто количественное увеличение. На низких уровнях важен транзитивный баланс (сила пропорциональна уровню); на высоких — нетранзитивный (взаимные контры способностей). AI может использовать эту модель как шаблон: для любой системы прогрессии определить этапы, каждый из которых имеет свою механическую доминанту.

6.7. Кривые прогрессии и математика прокачки

6.7.1. Цикл прогрессии (Шрайбер)

Шрайбер (Кн. 7) формулирует базовый цикл прогрессии: **Мотивация → Прогрессия → Награда → Удовлетворение**. Скорость прогрессии = $1/\text{сопротивление}$; ускорение = dv/dt . Отрицательное ускорение → игрок задумывается о смене цикла. Это простая, но мощная модель: любой момент, когда прогрессия замедляется, — точка риска оттока.

Эффект Зейгарник — незавершённые действия запоминаются лучше завершённых → игроки испытывают беспокойство, пока задача не завершена. Это ключ к аддиктивности Civilization и подобных игр: всегда есть «ещё один ход», потому что всегда есть незавершённые действия (строящееся здание, почти исследованная технология, почти завоёванная территория).

6.7.2. Пять типов кривых прогрессии (Шрайбер)

| Кривая | Характеристика | Где уместна | |-----|-----|-----| |
Линейная | Постоянная скорость роста | Казуальные игры, ранние этапы RPG |
Экспоненциальная | Ускоряющийся рост | Финальные этапы (ускорение завершения), XP-кривая | |
Замедляющаяся | Быстрый старт, потом торможение | F2P (быстрый начальный прогресс → hook), обучение | |
Прерывистая | Попеременное усиление/ослабление | Эмоциональные качели, драматический racing | |
Произвольная | Ручная настройка каждого этапа | Уникальные игровые ситуации, специфический racing |

Формула воспринимаемой сложности (Шрайбер):

$\text{Воспринимаемая_сложность} = (C_v + C_s) - (P_v + P_s)$, где C_v = виртуальный вызов (сила врагов), C_s = стратегический вызов (необходимость новых стратегий), P_v = виртуальная сила игрока (уровень персонажа), P_s = реальный навык игрока. Положительная воспринимаемая сложность → ощущение вызова; отрицательная → ощущение лёгкости; нулевая → баланс.

6.7.3. Семь типов кривых прогрессии (Шрайбер, расширенно)

| Кривая | Формула | Применение | |-----|-----|-----| | Тожественная | $y = x$ | Прямой обмен 1:1; деревья навыков | | Линейная | $y = mx$ | Стоимость от урона; НР от маны | | Экспоненциальная | $y = C \times b^x$ | Кривая опыта; ускоряющийся финал | | Логарифмическая | $y = \log_b(x)$ | Кривая уровней (обратная к кривой опыта) | | Треугольная | $y = (x^2 - x) / 2$ | Наиболее часто используемая формула возрастающей отдачи | | Пользовательская | Произвольная | Контекстные зависимости | | Обфускации стоимости | Нелинейная с разрывами | F2P: запутывание реальной стоимости |

Ключевой принцип: Для монстров и предметов плавная кривая — скорее исключение. Резкие пики и впадины, созданные вручную поверх плавных таблиц, создают лучшие моменты игры. «Идеальная кривая = скучная кривая». Селлерс добавляет: **логистическая (S-кривая)** — наиболее естественная кривая для прогрессии: обучение (медленно) → мастерство (быстро) → совершенствование (медленно).

6.7.4. Макро-модель RPG-прогрессии (Шрайбер)

`` Длительность = T часов Число_уровней = L Переходов = L - 1 Переходов/час $\approx L/T$ (больше в начале, меньше в конце) Стадий_контента $\approx L/2$
Конфигураций_врагов $\geq 3 \times (L/2)$ Очков_характеристик/уровень = (Финальное - Начальное) / (L - 1) ``

Пример: 40-часовая RPG с 20 уровнями → 19 переходов → ~ 0.5 переходов/час → 10 стадий контента → ≥ 30 конфигураций врагов. Эта формула — инструмент для AI: задай длительность и число уровней → получи требования к контенту.

6.7.5. Прогрессия разрыва билдов (Шрайбер)

Разница между оптимальной и конкурентоспособной сборками растёт с уровнем: $\Delta(t) = \text{Power_optimal}(t) - \text{Power_competitive}(t)$ — монотонно возрастающая функция. Для сужения разрыва: мощность определяется в основном базовыми параметрами (не зависящими от игрока), а атрибуты/

навыки дают лишь небольшие бонусы. Несколько уровней сложности как компенсация.

6.8. Петли обратной связи в экономике

6.8.1. Усиливающие и балансирующие петли (Селлерс)

Селлерс (Кн. 13) подробно разбирает два типа петель и их роль в игровых экономиках:

Усиливающие (reinforcing) петли: каждая часть увеличивает ресурс следующей, создавая экспоненциальный рост. «Богатые становятся богаче». Примеры: банковский процент (баланс → процент → баланс), Монополия (деньги → недвижимость → аренда → деньги). Усиливающие петли награждают победителей, усиливают ранний успех и дестабилизируют геймплей. В PvE они создают ощущение роста и мощи; в PvP — разрушают баланс.

Балансирующие (balancing) петли: каждая часть уменьшает активность следующей, создавая равновесие. Примеры: термостат (разница → нагрев → уменьшение разницы), XP-прогрессия (новый уровень → увеличение XP для следующего → замедление прокачки). Балансирующие петли поддерживают равновесие и продлевают геймплей.

Метастабильность: здоровая экология = динамическое равновесие, не стазис. Части постоянно меняются, но система в целом остаётся сбалансированной. Пример: хищник-жертва (рысь-заяц), Йеллоустонские волки. Это ключевая концепция для проектирования живых, динамичных игровых экономик.

6.8.2. Восемь характеристик петли ОС (Адамс/Дорманс)

Любая петля обратной связи в игре может быть описана через 8 измерений, образующих полный «профиль» петли (подробно описано в главе 5, здесь — применение к экономике):

| | | | |
|-----|--|-------------------------------|-----------------------------|
| # | Характеристика | Варианты | Экономический пример |
| --- | ----- | ----- | ----- |
| 1 | Тип | Положительная / Отрицательная | |
| | Доход от недвижимости / Налог на роскошь | 2 | Эффект |
| | Конструктивный / Деструктивный | Рост производства / Инфляция | 3 |
| | Инвестиция | Сколько вложить | Стоимость постройки фабрики |
| 4 | Отдача | Сколько возвращается | |

| Доход от фабрики за ход | | 5 | Скорость | Мгновенно / Задержка |
Немедленная прибыль / Доход через N ходов | | 6 | Длительность |
Одноразовая / Постоянная | Бонус за завершение / Постоянный апгрейд | | 7 |
Косвенность | Прямая / Опосредованная | Прямой доход / Доход через цепочку
конверсий | | 8 | Определённость | Детерминированная / Вероятностная |
Фиксированная награда / Шанс дропа |

Диагностический паттерн: Положительная, конструктивная, низкая инвестиция, высокая отдача, мгновенная, постоянная, прямая, детерминированная петля — почти гарантированно вызовет runaway. Каждое измерение — рычаг для коррекции: добавить задержку → замедлить runaway; увеличить инвестицию → сделать петлю стратегическим выбором; добавить вероятность → смягчить нарастание.

6.8.3. Паттерны обратной связи по жанрам

| Жанр | Доминирующий тип петель | Пример | |-----|-----|-----| |
RPG | Положительная конструктивная (инвестиция → прокачка → больше возможностей) | Опыт → уровень → новые навыки → больше опыта | | **RTS** |
Положительная конструктивная + Отрицательная деструктивная (гонка вооружений) | Больше ресурсов → больше войск → захват территорий | | **FPS** |
Отрицательная деструктивная (урон снижает здоровье) + Положительная (подбор аптечек) | Урон → меньше HP → аптечка → восстановление | | **Puzzle** |
Положительная (решение части → прогресс) + Отрицательная (ошибки → откат) | Найдена часть решения → открыта новая область | | **Survival** |
Отрицательная балансирующая (голод → поиск еды → насыщение) + Усиливающая (лучшая экипировка → эффективнее добыча) | Голод → охота → мясо → выживание | | **ММО** | Сложная иерархия петель | Микро- (бой) → Мезо- (данжен) → Макро- (прогрессия персонажа) → Мета- (рейд-прогресс) |

6.9. Цепочки и циклы конверсии

6.9.1. Цепочки конверсии (Зубек)

Зубек (Кн. 6) вводит операциональный инструмент анализа экономики: механики образуют цепочки «производитель→потребитель». Сражение конвертирует монстров + здоровье в добычу; продажа конвертирует добычу в золото; исцеление конвертирует золото в здоровье. Расчёт **курсов обмена** позволяет анализировать прибыльность цепочек:

``` Курс\_обмена = Выход / Вход Прибыльность = Курс\_обмена ×  
Частота\_использования – Альтернативные\_издержки ```

Для AI-системы это — прямой алгоритм: построить граф конверсий, вычислить курсы обмена, найти прибыльные и убыточные цепочки, предложить корректировки.

### 6.9.2. Циклы конверсии и их динамика

Повторяющиеся цепочки образуют циклы. Зубек выделяет три типа динамики:

**Прибыльный цикл** → линейный рост ресурсов → риск гринда. Каждый проход по циклу приносит чистую прибыль, и игрок может повторять его бесконечно. Это основа гринда — механики, которая и привлекает (ощущение стабильного прогресса), и отталкивает (монотонность). Решение: динамическая настройка — масштабирование прибыли со временем, чтобы цикл из прибыльного становился неприбыльным.

**Неприбыльный цикл** → истощение → риск разочарования. Каждый проход по циклу приносит чистый убыток. Игрок вынужден искать внешние источники ресурсов. Это создаёт напряжение, но может фрустрировать, если внешние источники недоступны.

**Сбалансированный цикл** → равновесие → риск скуки. Прибыль и убытки сбалансированы. Стабильность, но отсутствие прогресса. Нужна внешняя прогрессия (новые области, враги, ресурсы), чтобы дать направление.

### 6.9.3. Диаграммы потоков ресурсов (Шрайбер)

Шрайбер (Кн. 7) предлагает визуальный инструмент: прямоугольники = ресурсы, стрелки = зависимости/обмен. Инструмент для анализа и выявления дисбалансов. Диаграмма показывает: (1) все ресурсы в игре; (2) все пути их получения и траты; (3) узкие места (ресурсы с одним источником); (4) избыточные ресурсы (ресурсы без стоков); (5) мёртвые циклы (циклы, не связанные с остальной экономикой).

Для AI-системы диаграмма потоков — это **граф**, который можно автоматически анализировать: находить несвязанные компоненты, узкие места, избыточные ресурсы, потенциальные deadlocks.

---

## 6.10. Инфляция, стагнация и другие экономические патологии

---

### 6.10.1. Инфляция

**Определение:** валюта теряет ценность из-за переизбытка. Больше денег → каждый отдельный золотой стоит меньше. Шрайбер выделяет два типа: инфляция за счёт предложения (слишком много валюты генерируется) и инфляция за счёт спроса (покупательная способность игроков растёт).

**Борьба с инфляцией:** элементы с отрицательной суммой (денежные стоки — налоги, ремонт, расходники), удаление элементов с положительной суммой (деструкция предметов, bind-on-equip), экономические сбросы (new expansion обесценивает старый контент). Селлерс добавляет: **динамическое трение** — налог, пропорциональный богатству, — наиболее элегантное решение, поскольку оно самобалансирующееся.

### 6.10.2. Стагнация

**Определение:** валюта накапливается без возможностей траты. Обратная сторона инфляции: игроки богатеют, но не могут ничего купить. Это особенно опасно в поздней игре: все апгрейды куплены, все постройки возведены, деньги некуда девать.

**Борьба:** постоянные стоки (содержание, налоги, расходники), бесконечные апгрейды (всегда есть что улучшить, пусть и с убывающей отдачей), социальные механики (подарки, торговля, аукцион), престижные сбросы (prestige system — обнулить прогресс ради постоянного бонуса).

### 6.10.3. Арбитраж

**Определение:** игроки используют разницу цен для безрисковой прибыли. Покупают у NPC за 10, продают на аукционе за 15 — чистая прибыль без риска. Это признак **несбалансированной экономики**, где курс обмена не отражает реальную ценность.

**Борьба:** синхронизация цен (NPC покупает дешевле, чем продаёт), комиссия на аукционе, биндинг предметов (нельзя перепродать), динамическое ценообразование (цена меняется в зависимости от спроса/предложения).

#### 6.10.4. Runaway (саморазгон)

**Определение:** усиливающая петля без балансирующей → ранний лидер экспоненциально отрывается. «Богатые богатеют». Наиболее распространённая и разрушительная патология в игровых экономиках.

**Диагностика:** положительная, конструктивная, низкая инвестиция, высокая отдача, мгновенная, постоянная, прямая, детерминированная петля (8-мерный профиль).

**Лечение** (в порядке приоритета): (1) Dynamic Friction — сопротивление, масштабирующееся с уровнем ресурса (налог на роскошь); (2) Stopping Mechanism — жёсткий потолок (максимум юнитов, вместимость склада); (3) отрицательная ОС (резиновая лента — отстающие получают бонус); (4) Attrition — нарастающее потребление (расходы растут быстрее доходов).

#### 6.10.5. Deadlock (взаимная блокировка)

**Определение:** А нужен Б, Б нужен А → ни один не может начаться. В экономике: для покупки фабрики нужны ресурсы, которые производит фабрика. В прогрессии: для доступа к области нужен предмет, который находится в этой области.

**Лечение:** (1) предоставить стартовый капитал (минимальное количество ресурса для запуска); (2) альтернативный путь (не-оптимальный, но рабочий); (3) «кредит» (взять в долг с условием возврата); (4) таймер (через N ходов deadlock разрешается автоматически).

#### 6.10.6. Stall (остановка)

**Определение:** недостаточно ресурсов для продолжения. В отличие от deadlock (где цикл не может начаться), при stall цикл запускался, но ресурсы исчерпались. Типично для игр с необратимыми тратами: потратил всё на войну → нечего восстанавливать → не можешь воевать → ещё больше теряешь.

**Лечение:** (1) минимальный гарантированный доход (даже при нулевых ресурсах — базовая регенерация); (2) «спасательный круг» при критическом уровне (скрытая механика помощи); (3) альтернативные источники (ресурсы, не зависящие от основного цикла).

---



## 6.11. Конструирование экономики: Faucets, Drains и баланс источников/стоков

---

### 6.11.1. Модель «кран и слив» (Селлерс, Шрайбер)

Метафора Селлерса (Кн. 13): экономика — это система резервуаров, соединённых трубами. **Краны (faucets)** — источники ресурсов, наполняющие резервуары. **Сливы (drains)** — стоки ресурсов, опустошающие резервуары. Баланс экономики = баланс кранов и сливов.

Шрайбер (Кн. 7) формализует: если суммарный поток из кранов > суммарный поток в слив → инфляция (ресурсы накапливаются). Если наоборот → дефляция (ресурсы иссякают). Если равны → стационарное состояние.

### 6.11.2. Открытая vs закрытая экономика

**Закрытая экономика** — самодостаточна: все ресурсы циркулируют внутри, ничего не создаётся и не уничтожается (с нулевой суммой). Пример: настольные игры с фиксированным количеством денег. Преимущество: предсказуемость и простота балансировки. Недостаток: ограниченность — нет ощущения роста.

**Открытая экономика** — внешние факторы влияют: ресурсы генерируются и уничтожаются. Пример: ММО, F2P. Преимущество: ощущение роста и прогрессии. Недостаток: сложность балансировки, риск инфляции/стагнации.

**Балансировка открытой экономики** (Шрайбер): (1) ограничение расходов (caps на количество ресурсов); (2) ограничение мощности (мощность не растёт бесконечно); (3) конкурентный шлюз (доступ к ресурсам через конкуренцию с другими игроками).

### 6.11.3. Алгоритм конструирования экономики для AI

На основе всех источников формулируем пошаговый алгоритм, который AI-система Gidede может использовать для генерации и балансировки игровой экономики:

**Шаг 1: Определить ядерные ресурсы** (2-4 ресурса, напрямую связанных с core loop). Использовать модель Селлерса: что является «НР» игры? Что — «ХР»? Что — «золотом»?

**Шаг 2: Построить базовый двигатель** (выбрать паттерн из библиотеки Machinations). Static Engine для предсказуемой экономики; Dynamic Engine для

экономики роста; Converter Engine для экономики преобразования; Engine Building для развивающейся экономики.

**Шаг 3: Добавить трение** (Static Friction для базовых расходов; Dynamic Friction для замедления лидера; Stopping Mechanism для абсолютных лимитов). Без трения любой двигатель runaway.

**Шаг 4: Добавить эскалацию** (Escalating Challenge для нарастающего вызова; Escalating Complexity для нарастающего давления; Arms Race для PvP). Эскалация создаёт направление прогрессии.

**Шаг 5: Определить кривые прогрессии** (выбрать тип кривой для каждого ядерного ресурса). XP: экспоненциальная или треугольная. Мощность: линейная или полиномиальная. Стоимость: транзитивная кривая стоимости.

**Шаг 6: Построить граф конверсий** (как ресурсы превращаются друг в друга). Использовать цепочки конверсии Зубека. Проверить: все ли ресурсы имеют и источники, и стоки? Нет ли deadlocks? Нет ли мёртвых циклов?

**Шаг 7: Добавить петли обратной связи** (положительные для роста; отрицательные для баланса). Профилировать каждую петлю по 8 характеристикам Адамса/Дорманса. Проверить: нет ли runaway-петлей без балансирующих?

**Шаг 8: Симулировать и итерировать.** Построить Machinations-модель или эквивалентную систему уравнений. Запустить тысячу итераций. Собрать статистику: средние значения ресурсов, скорость роста, точки дисбаланса. Скорректировать.

---

## 6.12. Экономика в разных жанрах

---

### 6.12.1. RPG

**Ключевые ресурсы:** HP, мана, опыт, золото, предметы, навыки.

**Доминирующая структура:** экономика с конвертерами — опыт конвертируется в уровни, уровни — в навыки, навыки — в боевую мощь, боевая мощь — в больше опыта (положительная петля). **Типичная патология:** runaway на поздних уровнях — при экспоненциальной XP-кривой и линейном росте стоимости контента игрок становится слишком мощным. **Решение:** убывающая отдача навыков, масштабирование врагов, пороги эффективности (diminishing returns).

### 6.12.2. RTS

**Ключевые ресурсы:** золото/минералы, дерево/газ, юниты, территории, технологии. **Доминирующая структура:** Engine Building + Arms Race — игрок строит экономику, одновременно соревнуясь с противником. **Типичная патология:** runaway через положительную петлю «больше ресурсов → больше войск → больше побед → больше ресурсов». **Решение:** Dynamic Friction (содержание войск), Stopping Mechanism (limiter на юнитов), Attrition (расходы растут с размером армии).

### 6.12.3. FPS

**Ключевые ресурсы:** здоровье, боеприпасы, оружие, позиция. **Доминирующая структура:** дефицит ресурсов + стоки через урон. **Типичная патология:** stall — боеприпасы закончились, здоровья нет, продолжать невозможно. **Решение:** гарантированные респавы боеприпасов, регенерация здоровья, динамический спавн лута.

### 6.12.4. Survival

**Ключевые ресурсы:** голод, жажда, здоровье, материалы, энергия. **Доминирующая структура:** экология — балансирующие петли стремятся к равновесию, но ресурсы ограничены. **Типичная патология:** порочный круг — мало здоровья → не можешь охотиться → нет еды → ещё меньше здоровья. **Решение:** минимальная регенерация, «безопасные зоны» с базовыми ресурсами, не-оптимальные, но рабочие альтернативы.

### 6.12.5. MMO

**Ключевые ресурсы:** золото, опыт, снаряжение, репутация, социальный капитал. **Доминирующая структура:** иерархия вложенных систем — двигатели (добыча), экономики (крафт, аукцион), экологии (PvP-баланс, социальные системы). **Типичная патология:** инфляция — с течением времени количество золота в мире только растёт (краны > сливов). **Решение:** масштабируемые стоки (ремонт, налоги, расходники для рейдов), bind-on-equip, сезонные сбросы экономики.

### 6.12.6. F2P / Мобильные

**Ключевые ресурсы:** энергия (ваучеры времени), софтвалута, хардвалута, очки опыта, cosmetic токены. **Доминирующая структура:** фиксированная экономика с двойной валютой и energy-gating. **Типичная патология:** стагнация у платящих (купил всё — нечего делать) и фрустрация у

бесплатных (слишком долго ждать). **Решение:** battle pass (и платящим, и бесплатным есть что делать), престижные сбросы, UGC (пользовательский контент).

---

## 6.13. Формализация экономики для AI-системы Gidede

---

### 6.13.1. Формальная модель игровой экономики

Объединяя модели из всех источников, формулируем единую формальную модель, которую можно напрямую транслировать в программную архитектуру:

```
``` GameEconomy = { resources: [Resource], nodes: [Node], flows: [Flow],  
feedback_loops: [FeedbackLoop], progression_curves: [Curve] }
```

```
Resource = { name: String, type: Enum(Core, Subsidiary, Currency, Consumable,  
Meta), class: Enum(Time, Currency, GameObject, HP, Experience), bounds: (min:  
Number, max: Number), initial_value: Number }
```

```
Node = Enum(Source, Drain, Pool, Converter, Trader, Gate, Delay, Queue)
```

```
Flow = { from: Node, to: Node, type: Enum(Resource, State), rate: Expression, //  
формула скорости потока condition: Expression | null, // условие активации  
probability: Float | null // вероятность (для Gate) }
```

```
FeedbackLoop = { nodes: [Node], type: Enum(Reinforcing, Balancing), effect:  
Enum(Constructive, Destructive), profile: { investment: Float, return: Float, speed:  
Enum(Instant, Delayed), duration: Enum(OneShot, Permanent), indirectness:  
Enum(Direct, Indirect), determinism: Enum(Deterministic, Probabilistic) } }
```

```
Curve = { target_resource: Resource, formula: Expression, curve_type:  
Enum(Identity, Linear, Exponential, Logarithmic, Triangular, Logistic, Custom,  
Obfuscation) } ```
```

6.13.2. Алгоритм автоматического анализа экономики

1. **Построить граф зависимостей:** ресурсы → узлы → потоки 2. **Найти компоненты связности:** несвязанные подсистемы = потенциальные проблемы 3. **Найти циклы:** классифицировать как прибыльные/убыточные/сбалансированные 4. **Профилировать петли ОС:** для каждой петли вычислить 8-мерный профиль 5. **Проверить баланс кранов/сливов:** для каждого ресурса сравнить суммарный inflow и outflow 6. **Найти патологии:**

runaway (положительная петля без балансирующей), deadlock (цикл без стартового ресурса), stall (убыточный цикл без внешних источников), инфляция (краны > сливов), стагнация (сливы > кранов или нет сливов) 7.

Предложить корректировки: на основе типа патологии предложить паттерн Machinations для лечения

6.13.3. Ключевые формализуемые модели для Gidede

| Модель | Формализация | Применение в Gidede | |-----|-----|-----|
| Элементы экономики | Source/Drain/Pool/Converter/Trader | Внутренний формат представления экономик | | Паттерны Machinations | 16+ паттернов = генеративный алфавит | AI выбирает и комбинирует паттерны для генерации экономики | | Цепочки конверсии | DiGraph(resource, conversion_rate) | Анализ прибыльности и связности экономики | | Петли обратной связи | 8-мерный профиль петли | Диагностика и коррекция экономических патологий | | Кривые прогрессии | 7 типов кривых + формулы | Генерация XP-таблиц, стоимости, мощности | | Типология систем | Engine/Economy/Ecology | Классификация генерируемых систем | | Баланс кранов/сливов | $\Sigma(\text{faucets}) \approx \Sigma(\text{drains})$ для каждого ресурса | Автоматическая проверка баланса | | Цикл прогрессии | Мотивация → Прогрессия → Награда → Удовлетворение | Проектирование racing и удержания | | Формула воспринимаемой сложности | $(C_v + C_s) - (P_v + P_s)$ | Динамическая настройка сложности | | Макро-модель RPG | $T \rightarrow L \rightarrow$ переходы → стадии → конфигурации | Планирование контента |

6.13.4. Чек-лист валидации экономики

AI-система должна автоматически проверять:

- [] **Связность:** Все ресурсы достижимы из стартового состояния? Нет ли «островов»? - [] **Баланс кранов/сливов:** Для каждого ресурса суммарный $\text{inflow} \approx \text{outflow}$ (с учётом желаемой динамики)? - [] **Нет runaway:** Каждая положительная петля имеет балансирующий противовес? - [] **Нет deadlock:** Для каждого цикла есть стартовый ресурс или альтернативный путь? - [] **Нет stall:** Для каждого убыточного цикла есть внешние источники? - [] **Прогрессия определена:** Для каждого ядерного ресурса есть кривая прогрессии? - [] **Фазы экономики:** Есть ли естественные фазы (начальная → рост → зрелость → завершение)? - [] **Инфляция под контролем:** Есть ли стоки для каждой валюты? - [] **Стагнация предотвращена:** Есть ли долгосрочные цели траты для каждого ресурса? - [] **Арбитраж невозможен:** Курс покупки $\leq$ курс продажи для всех торговых пар? - [] **Глубина решений:** Есть ли значимые выборы в управлении ресурсами? - [] **Доступность:** Может ли новичок понять базовый экономический цикл за 5 минут?

6.14. Кейс: моделирование экономики Pac-Man

Адамс/Дорманс (Кн. 4) подробно разбирают экономику Pac-Man как пример применения Machinations:

Ресурсы: точки (dots), энергизаторы (power pellets), очки, жизни, призраки-опасности.

Источники: точки на уровне генерируют очки при поедании (Source → Pool «очки»). Энергизаторы — особый источник, который активирует временное состояние.

Стоки: потеря жизни при контакте с призраком (Pool «жизни» → Drain). Каждый контакт — сток одной жизни. Без жизней — конец игры.

Конвертеры: энергизатор конвертирует «опасность» в «уязвимость призраков» на ограниченное время. Это временный конвертер с Delay-узлом: активация → N тиков уязвимости → возврат к норме.

Петля: поедание точек → приближение завершения уровня → новый уровень с теми же правилами, но большей скоростью призраков. Это Escalating Challenge + Slow Cycle (фазы «нормальный» и «энергизатор»).

Диагностика: экономика Pac-Man — пример хорошо сбалансированной системы. Положительная петля (поедание точек → очки) ограничена количеством точек на уровне (Stopping Mechanism). Отрицательная петля (контакт с призраком → потеря жизни) балансируется положительной (энергизатор → возможность поедать призраков). Каждая фаза имеет ясную цель и чёткую обратную связь.

6.15. Кейс: экономика Монополии

Ещё один классический кейс Адамса/Дорманса:

Ресурсы: деньги, недвижимость, дома/отели.

Источники: стартовые деньги при начале игры, зарплата за проход GO (\$200 каждый круг).

Стоки: аренда (плата другим игрокам), налоги, штрафы, покупка недвижимости и домов.

Конвертеры: покупка недвижимости (деньги → собственность), постройка домов (деньги + собственность → улучшенная собственность).

Положительная петля: больше собственности → больше арендного дохода → больше покупок → ещё больше собственности. Это классический Dynamic Engine без достаточного трения.

Проблема: Положительная петля без достаточного трения → runaway → снежный ком → затяжные игры. Симуляция Machinations подтверждает: средняя игра в Монополию длится значительно дольше, чем оптимально, из-за несбалансированной положительной обратной связи.

Лечение: добавить Dynamic Friction (налог, пропорциональный богатству), Stopping Mechanism (максимум домов), Attrition (износ собственности).

Исторически: в турнирных правилах Монополии добавлен аукцион при отказе от покупки — это частичное решение, ускоряющее циркуляцию собственности.

6.16. Резюме: архитектура модуля «Экономика и прогрессия» в Gidede

На основе всего вышеизложенного формулируем архитектуру модуля:

Вход: описание игры (жанр, целевая аудитория, длительность, core loop).

Шаг 1 — Определение ресурсов: AI идентифицирует ядерные и подсобные ресурсы на основе жанра и core loop. Использует типологию Шрайбера (5 классов) и разделение Селлерса (ядерные/подсобные).

Шаг 2 — Генерация экономической структуры: AI выбирает паттерны Machinations из библиотеки (16+ паттернов), комбинирует их в экономическую модель. Использует процесс Lunar Colony: двигатель → трение → эскалация → комбинация.

Шаг 3 — Определение прогрессии: AI выбирает кривые прогрессии для каждого ядерного ресурса, определяет этапы (по модели D&D), вычисляет макро-параметры (по модели Шрайбера).

Шаг 4 — Построение графа конверсий: AI строит граф цепочек конверсии (по Зубеку), проверяет связность, находит прибыльные/убыточные циклы.

Шаг 5 — Валидация: AI проверяет экономику по чек-листу из 12 пунктов, профилирует петли ОС, выявляет патологии.

Шаг 6 — Симуляция: AI запускает автоматическую симуляцию (тысячи итераций с искусственными игроками), собирает статистику, корректирует параметры.

Выход: Machinations-подобная диаграмма экономики, таблицы кривых прогрессии, граф конверсий, отчёт о валидации, рекомендации по балансировке.

Глава 7. Level Design — принципы и паттерны

> **Раздел Библии геймдизайна:** 2.7 — Level Design: принципы и паттерны >

Источники: KB-14 (LD Better Levels), KB-11 (Кадиков), KB-08 (Роджерс), KB-04 (Machinations гл. 10), KB-12 (Роллингс/Моррис) > **Книги:** Кадиков (Кн. 11), LD: Better Levels (Кн. 14), Роджерс (Кн. 8), Адамс/Дорманс (Кн. 4), Шелл (Кн. 1), Роллингс/Моррис (Кн. 12)

7.1. Что такое левел-дизайн и его место в проектировании игры

Левел-дизайн — это искусство и наука создания игрового пространства, в котором механики, нарратив и эстетика объединяются в целостный опыт. Кадиков (Кн. 11) проводит фундаментальное различие между **core gameplay** и **level gameplay**: core gameplay — набор базовых правил и механик, доступных по умолчанию; level gameplay — геймплей, создаваемый дизайнером уровней через оперирование пространством, объектами и настройками виртуального мира. Чистое поле без level gameplay — это «песочница» в худшем смысле: механики есть, но негде их применить интересно. Задача левел-дизайнера — обеспечить достаточное количество интересных игровых ситуаций для применения каждой базовой механики.

Адамс/Дорманс (Кн. 4) дополняют это понимание моделью «от игрушки к игре»: **игрушка** — объект, с которым можно свободно играть без цели; **игровая площадка** — структурированное пространство, где игрушки используются по правилам; **игра** — игровая площадка + условия победы/поражения. Прогрессия от игрушки к игре — это добавление структуры к

свободе, и именно этим занимается левел-дизайнер: он превращает механики (игрушки) в структурированный опыт (игру) через пространство (игровую площадку).

Кадиков подчёркивает: роль дизайнера уровней принципиально отлична от роли художника по окружению. Дизайнер уровней фокусируется на создании **геймплея через пространство**, а не на визуальной составляющей. Его обязанности: (1) проектная документация, (2) трёхмерный прототип, (3) настройка логики событий, (4) сотрудничество с художниками, (5) улучшение игрового опыта. Это различие критически важно для AI-системы: генерация уровня $\neq$ генерация визуала; сначала — функциональное пространство, потом — эстетическое оформление.

Для Gidede левел-дизайн — это **пространственная проекция механик и экономики**: уровень — это место, где механики становятся геймлеем, где экономические потоки материализуются в ресурсы, где прогрессия обретает форму пути. AI должен уметь генерировать уровни, которые обслуживают механики, экономику и нарратив, а не просто выглядят красиво.

7.2. Концепция уровня: от идеи до плана

7.2.1. Три столпа концепции уровня (LD: Better Levels)

Прежде чем проектировать уровень, необходимо определить три столпа, которые задают рамки проектирования:

1. Ограничения (Restrictions) — вещи, которые нужно сделать. Часто не определяются левел-дизайнером: «ввести новую механику», «использовать нового врага», «встреча с NPC в конце». Это внешние требования, которые нельзя игнорировать.

2. Цели (Goals) — вещи, которые хочется сделать. Тема уровня, вызовы, вертикальность, альтернативные маршруты. Это дизайнерские амбиции, которые превращают уровень из обязательного в запоминающийся.

3. Контекст (Context) — вещи, которые нужно учитывать. Позиция уровня в игре (7-й уровень — это середина; 1-й — обучение; последний — кульминация), какие типы уровней уже запланированы, какая механика была введена на предыдущем уровне.

Для AI-системы три столпа — это **входные параметры генератора уровней**: AI получает ограничения (какие механики/враги должны появиться), цели

(тема, эмоция, структура) и контекст (позиция в игре, предыдущие уровни), и на их основе генерирует концепцию.

7.2.2. Модель «Основа + Изюминка» (Кадиков)

Любой уровень = основная идея (набор клише, знакомый игроку) + специальный ингредиент (изюминка, обеспечивающая запоминаемость). Без изюминки уровень — безликий набор банальностей; без основы — непонятный эксперимент. Кадиков выделяет **6 типов изюминок**:

1. **Контраст тем** — совмещение 2+ контрастирующих тематик (Fallout: ретрофутуризм + постапокалипсис; Nuketown: идиллический городок + ядерный полигон) 2. **Достопримечательности** — узнаваемые ориентиры (башня Нового Вегаса, Эйфелева башня, засыпанный Дубай) 3.

Выразительный образ — спецэффекты и освещение (горящий отель L4D2, ливень Hard Rain, концерт Dark Carnival) 4. **Геймплейная находка** — уникальные механики (бал-маскарад Dishonored, фабрика Titanfall 2, прыжки во времени Dishonored 2) 5. **Необычное в обычном** — бассейн на крыше (Uncharted 2), голова Статуи Свободы на улице (Crysis 2) 6. **«Три в одном»** — комбинация выразительного образа + геймплейной находки + сюжетного повествования — максимум запоминаемости

Для AI «Основа + Изюминка» — это **генеративный паттерн**: выбрать знакомую основу из базы клише, затем добавить изюминку одного из 6 типов. Роджерс (Кн. 8) дополняет: **10 клише-тем** (космос, огонь/лёд, подземелье, фабрика, джунгли, жуткий особняк, пираты, грязный город, космическая станция, канализация) и техника **«Мексиканская пицца»**: комбинация двух клише даёт оригинальную тему (огненное кладбище, ледяные пираты).

7.2.3. Ключевые вопросы планирования

Объединяя вопросы из обоих источников, формулируем исчерпывающий чек-лист для планирования уровня:

- Где происходит уровень? (Локация, сеттинг) - Когда? (Время суток, позиция в игре) - Каковы механики игры? (Уровень обслуживает геймплей, не наоборот) - Почему игрок запомнит этот уровень? (Изюминка) - Соответствует ли локация геймплею? (Снайперский бой в тесных коридорах = плохо) - Какова «история» локации? (Что здесь произошло? Кто живёт?) - Каков Critical Path? (Кратчайший маршрут прохождения) - Каков Golden Path? (Предпочтительный маршрут дизайнера — оптимальный опыт)

7.3. Анатомия игрового пространства

7.3.1. Шесть слоёв пространства (Кадиков)

Кадиков (Кн. 11) вводит модель из шести слоёв, из которых строится любое игровое пространство. Эта модель — чек-лист комплектности уровня и порядок проектирования:

| Слой | Описание | Роль | |-----|-----|-----| | **1. Структурная геометрия** | Крупноформные архитектурные элементы (стены, полы, потолки) | Каркас уровня — определяет пространственную структуру | | **2. Детали игрового окружения** | Мелкие объекты (мебель, декорации, пропсы) | Наполнение — функциональные и атмосферные элементы | | **3. Задний план** | Фоновые элементы за пределами игрового пространства | Глубина и контекст — мир не заканчивается за краем карты | | **4. Освещение** | Ключевой инструмент направления внимания и атмосферы | Управление взглядом и настроением | | **5. Аудио и визуальные эффекты** | Звук + частицы + погода | Атмосферность и обратная связь | | **6. Игровой функционал** | Интерактивные элементы, триггеры, зоны | Геймплей — механики в действии |

Принцип проектирования «От большого к малому»: сначала структурная геометрия → потом детали → потом декор. Это прямо противоположно интуиции многих начинающих дизайнеров, которые начинают с деталей. Нарушение этого порядка ведёт к уровням, которые красиво выглядят, но плохо играют.

7.3.2. Функциональность в архитектуре

Кадиков формулирует четыре принципа, которые связывают архитектуру с геймплеем:

1. «Думай, как архитектор» — форма следует функции: архитектура реальных зданий подчинена назначению (жильё, склад, церковь), и виртуальная архитектура должна аналогично отражать своё назначение. Казарма выглядит как казарма, лаборатория — как лаборатория.

2. Функциональность + геймплей — пространство обязано быть средой для функционирования игровых механик: если враги прячутся за укрытиями, архитектура должна предусматривать укрытия; если игрок прыгает — должны быть платформы.

3. Логика и здравый смысл — двери должны вести куда-то, комнаты иметь назначение, лестницы — перила. Нарушение этого принципа разрушает иммерсию.

4. От большого к малому — проектирование сверху вниз: сначала структурная геометрия → потом детали → потом декор.

7.3.3. Масштаб и пропорции

Масштаб — одно из самых тонких измерений левел-дизайна. Кадиков и Роджерс сходятся в ключевом принципе: **габариты персонажа — базовая единица для всех размеров**. Роджерс (Кн. 8) конкретизирует: рост персонажа = базовая единица; ширина прохода > рост; дальность прыжка $\approx 5 \times \text{рост}$; высота одиночного прыжка $\approx 0.5 \times \text{рост}$; двойного $\approx 2 \times \text{рост}$.

Масштабирование и геймплей: тесные коридоры → клаустрофобия, огромные залы → трепет, низкие потолки → давление, высокие → свободу. **Искажение пропорций ради геймплея** — стандартная практика: двери шире реальных, потолки выше, коридоры просторнее. Ключевой принцип: начинаем с реалистичного масштаба (1 юнит = 1 метр), затем искажаем в пользу геймплея, а не наоборот.

Метрики (Gym) — тестовый уровень с эталонными измерениями (LD: Better Levels): линейка (расстояния), рамка (окна, двери, укрытия), сфера (AoE, дистанции), заметки. Gym позволяет тестировать значения в быстром прототипе и гарантирует правильность укрытий, прыжков и других действий с самого начала.

7.3.4. Модульность

Принцип модульности = разбиение на повторно используемые компоненты.

Преимущества: скорость производства, консистентность, оптимизация.

Недостатки: монотонность, ограниченность вариаций. Решение: творческий подход к комбинированию модулей + декорирование уникальными элементами.

Для AI-системы модульность — это **основа процедурной генерации**: определяя набор модулей и правила их комбинирования, AI может генерировать бесконечное количество уникальных уровней.

7.4. Навигация и читаемость

7.4.1. Проблема восприятия глубины

LD: Better Levels формулирует фундаментальную проблему: **восприятие глубины и масштаба в играх принципиально отличается от реального**. Игрок видит экран с камерой, а не мир напрямую. Камера может быть отсоединена (3-е лицо) — ещё сложнее оценить расстояния. 2D-платформер легче для оценки расстояний, чем 3D (горизонтальное измерение точнее, чем глубинное).

Решения: - Объекты известного размера (футбольный мяч, автомобиль) для оценки расстояний - Линейные элементы с повторяющимися сегментами (забор, труба, ковёр) — каждый сегмент = единица расстояния - Осторожность с объектами переменного размера (дерево — маленькое близко или большое далеко?) - Параллакс (foreground / midground / background) для передачи глубины - Предметы в коридорах, указывающие на продолжение пути (свет из-за поворота, ковёр за углом)

7.4.2. Читаемость окружения

Читаемость = способность игрока различать элементы окружения. LD: Better Levels выделяет три основные проблемы: (1) всё сливается (зелёно-коричневая растительность с тенями), (2) визуальный шум (vegetation + shadows = перегруз), (3) необходимость UI-элементов как «костыль».

Решения: - **Снижение визуальной сложности:** минимализм, десатурация окружения, высокая насыщенность персонажей - TF2: бледное окружение + яркие персонажи; Rainbow Six: Siege: тёмные персонажи + светлое окружение

- **Цветовая гармония:** низкая насыщенность = спокойствие, высокая = шум -

Контраст: важные элементы должны выделяться — не тонуть в визуальном шуме

Кадиков добавляет **6 принципов честного мультиплеера**, первые два из которых прямо относятся к читаемости: (1) читаемость персонажа в окружении (модель должна контрастировать с фоном), (2) «меньше значит больше» — минимизация визуального шума (сложная детализация убирается за пределы геймплейного пространства).

7.4.3. Ориентиры: микро / мезо / макро

LD: Better Levels вводит трёхуровневую систему ориентиров, которая обеспечивает навигацию на всех масштабах:

Микро-ориентиры — объекты для локальной навигации (знак, ящик, дверь). Указывают: «сюда», «здесь можно укрыться», «тут поворот».

Мезо-ориентиры — ориентиры области (часовая башня, кран). Помогают ориентироваться внутри одной области уровня: «я около башни», «я у крана».

Макро-ориентиры — ориентиры всей карты (гора, небоскрёб). Видны из любой точки, дают общее направление: «гора на севере», «небоскрёб в центре».

Роджерс называет макро-ориентиры «**винни**» (**weenies**) — термин, заимствованный из Disney: замки, горы, статуи — объекты, притягивающие игрока от одного маяка к другому. Без макро-ориентиров игрок теряется в открытом мире; без микро-ориентиров — не может найти путь в локальном пространстве.

7.4.4. Девять навигационных инструментов (Кадиков)

Кадиков систематизирует все способы навигации в играх:

Группа 1 — Графический интерфейс: (1) Карты (глобальная и локальная), (2) Целеуказатели/маркеры (расстояние до цели в UI), (3) Дополненная реальность (голокарта The Division, линия-путь Dead Space).

Группа 2 — Путевые знаки (интегрированные в окружение): (4) Композиция, контраст и свет (световые акценты направляют движение), (5) Ориентиры (видимые издали объекты-маяки), (6) Персонажи (NPC как путеводные нити), (7) Движение (анимированные объекты привлекают внимание), (8) Подбираемые предметы (разбросанные предметы = тропа из хлебных крошек), (9) Знаки и цветовое кодирование (стрелки, надписи, цветовые маркеры).

Ключевой принцип: лучшие уровни направляют игрока **без UI** — через группу 2. UI-навигация — это «костыль», который говорит о том, что пространство спроектировано недостаточно понятно. AI-система должна стремиться генерировать уровни, навигабельные без UI-подсказок.

7.4.5. Инструменты управления вниманием

LD: Better Levels описывает конкретные техники направления игрока:

Saliency (заметность): яркое, движущееся, контрастное — привлекает внимание. Это базовый механизм: важные элементы должны быть заметнее фона.

Визуальный язык: белый шёлк/царапины в Uncharted, жёлтые метки в God of War, красные бочки = взрывоопасные. Согласованный визуальный язык позволяет игроку интуитивно понимать возможности среды.

Nope zone: области, которые визуально «отталкивают» (опасность, тупик, неправильный путь). Тёмные, узкие, угрожающие пространства = «сюда не надо».

Хлебные крошки (breadcrumbs): мелкие объекты, ведущие по пути (монетки, трупы, капли крови). Подсознательно направляют, не нарушая иммерсию.

Дразнение (teasing): показ цели издалека, чтобы создать желание добраться (синий свет вдали, силуэт здания). Это «винни» в действии: видимый, но ещё недостижимый ориентир.

Prospect & Refuge: открытые пространства для наблюдения + укрытия для безопасности. Чередование создаёт естественный ритм: осмотреться из укрытия → перебежать → осмотреться.

Контрастирующие пути: светлый vs тёмный, широкий vs узкий — визуально указывают на разный тип пути. Главный путь = широкий и светлый; секретный = узкий и тёмный.

7.5. Темп и ритм уровня (Pacing)

7.5.1. Три элемента темпа на трёх масштабах

LD: Better Levels формулирует **матрицу темпа 3×3**: три ключевых элемента (интенсивность, разнообразие, время) на трёх масштабах (микро, мезо, макро).

Интенсивность: насколько напряжён текущий момент. Бой = высокая; исследование = средняя; отдых = низкая. Монотонная интенсивность = главный враг вовлечения: постоянный экшен утомляет, постоянное исследование = скучно.

Разнообразие: чередование типов активности. Бой → головоломка → исследование → нарратив → бой. Роджерс добавляет: «Walking is NEVER, EVER gameplay!» — 10 альтернатив ходьбе: прыжки, бой, полёт, коллекция, побег, лазание, стелс, плавание, исследование, раскачка.

Время: длительность каждого сегмента. Критический вопрос: «Сколько это длится?» Слишком долго при одной интенсивности = монотонность; слишком мало = невозможность погрузиться.

Микро-масштаб (секунды-минуты): момент-к-моментному геймплею. Один бой, одна головоломка, один отрезок пути.

Мезо-масштаб (минуты-десятки минут): последовательность активностей внутри уровня. Цикл «бой → лут → отдых → бой».

Макро-масштаб (весь уровень/игра): общая арка напряжения. Начало (крючок) → развитие (нарастание) → кульминация (пик) → завершение (разрешение).

7.5.2. Ворота и клапаны (Gates & Valves)

LD: Better Levels вводит три инструмента управления продвижением игрока:

Hard Gate — полностью останавливает прогресс (найти ключ, нажать выключатель, убить босса). Дольше обходить. Используется для принуждения к исследованию или обязательного обучения механике.

Soft Gate — кратковременно замедляет (свалить дерево, ждать NPC, мини-катсцена). Для паузы, диалога, замедления темпа. Не нарушает поток, но даёт передышку.

Valve — область, закрываемая за игроком (прыжок с обрыва, разрушение пути, лаз, который можно пройти только в одном направлении). Продвигает вперёд, предотвращает возврат, позволяет загружать/выгружать части уровня.

Для AI-системы Gates и Valves — это **инструменты управления потоком прогрессии**: Hard Gate = «замок» в модели Адамса/Дорманса; Soft Gate = временная задержка; Valve = односторонний поток ресурсов (аналог Valve в Machinations).

7.5.3. Петли (Loops) и минимизация бэктрекинга

Минимизация бессмысленного возврата через пустые области — ключевая задача. Уровень должен замыкаться сам на себя: кольцевая структура

позволяет вернуться к началу новым путём, не проходя уже зачищенную область. Примеры: подземелье с кольцевой структурой, Void Bastards и Hitman 2 — «швейцарский сыр» (множество отверстий-проходов).

7.5.4. Beats, Charts и Timelines

Инструмент планирования темпа: график интенсивности по времени. Цель: чередовать высокие и низкие точки, избегать монотонности. Роджерс (Кн. 8) дополняет: **Beat Chart** — таблица из 15 столбцов (Level Name, Mechanics, Player's Objective, Story Beat, Play Style, Enemies, NPCs, Bonus Materials, Time of Day, Color Mapping и др.), которая позволяет увидеть всю игру как единый ритмический паттерн. **Beat Chart Audit Checklist** — 8 проверок: дублирование имён, подряд идущие темы одной темы, позднее введение систем, сгруппированные враги, неиспользуемые механики (<3 использований — убрать), одинаковые награды, монохромная палитра, 3+ ночных уровня подряд.

7.6. Атмосфера, эмоции и нарратив в уровне

7.6.1. Три принципа атмосферы (Кадиков)

1. Имитация реальности — 4 подкатегории: природные явления и погодные эффекты (смена суток, отражения, дождь, туман); имитация движения (колыхание листвы, частицы, работающие механизмы — «дышащий» мир); художественный беспорядок (объекты «не по сетке», хаотичность = жизнь); звуковой фон (глобальные и локальные шумы, эхо, озвучка взаимодействия).

2. Постоянство и достоверность — правило постоянства: если объект выглядит интерактивным, он должен быть интерактивным; нарушение правила = потеря доверия. Визуальный язык: каждый тип объекта имеет предсказуемое поведение (красные бочки = взрываются, зелёные ящики = здоровье). Достоверность: мир должен «убеждать» игрока в своей реальности через внутреннюю логику.

3. Симуляция жизни — следы присутствия NPC (разбитые лагеря, брошенная еда, потухшие костры); следы недавних событий (тела, крови, гильзы, разрушения); динамические элементы (мигающие лампы, капающая вода, периодические звуки); «живой» фон (NPC-статисты, птицы, животные, далёкие звуки боя).

7.6.2. Психология восприятия пространства

Кадиков выделяет **6 архитектурных паттернов**, каждый из которых порождает определённую эмоцию:

| Паттерн | Эмоция | Пример применения | |-----|-----|-----| | **Узкие коридоры** | Клаустрофобия, напряжение | Хоррор, стелс, саспенс | | **Широкие залы** | Свобода, трепет, уязвимость | Бои на арене, вау-моменты | | **Вертикальность** | Мощь, величие, восхождение/падение | Боссы, кульминации, тревверсинг | | **Низкие потолки** | Давление, дискомфорт | Подземелья, бункеры, тайные лаборатории | | **Открытые пространства** | Свобода, но потеря ориентации | Открытый мир, снайперские позиции | | **Замкнутые комнаты** | Безопасность, уют, засада | Хабы, магазины, засады врагов |

LD: Better Levels добавляет эмоциональные техники: **страх** (тесные коридоры = клаустрофобия, затопление = паника); **облегчение** (после тесноты → открытое пространство); **акрофобия** (страх высоты — Half-Life 2: под мостом); **вторжение в личное пространство** (враги рядом = дискомфорт); **форма пространства** (квадратная комната = задержка, узкий коридор с целью вдали = движение вперёд).

7.6.3. Цвет и освещение

Кадиков выделяет **3 функции цвета** в левел-дизайне:

1. Эмоциональная температура: тёплые (оранжевый = безопасность, красный = опасность/ярость), холодные (синий = одиночество/изоляция, голубой = прохлада).

2. Пространственная коррекция: холодные тона расширяют пространство, тёплые сужают; светлые = лёгкость, тёмные = тяжесть. Это позволяет визуально «корректировать» размеры помещений без изменения геометрии.

3. Идентификация уровня: уникальная цветовая палитра = запоминающийся образ (Mirror's Edge = красный + белый, BioShock = арт-деко палитра).

7.6.4. Нарратив через окружение

Три типа нарратива в играх (LD: Better Levels): (1) **Явный (Explicit):** кат-сцены, диалоги, цели миссий; (2) **Неявный (Implicit):** следы крови, трофеев на стене, окружающий сторителлинг; (3) **Эмерджентный (Emergent):** «истории игроков», возникающие из взаимодействия систем.

Для левел-дизайна наиболее эффективен **окружающий сторителлинг** (implicit narrative) — он не прерывает игрока, делает мир живым, вознаграждает внимательность. Кадиков формализует **4 техники сюжетного окружения**: (1) фоновое восприятие (подсознательный анализ деталей), (2) говорящие улики (декоративные элементы, отражающие следы событий), (3) фокусировка внимания (световые акценты, композиционные рамки, контраст), (4) обучение через окружение (Dead Space: надписи на стенах, Half-Life 2: убитый зомби лезвием пилы = подсказка к гравипушке).

7.7. Боевое пространство

7.7.1. Четыре стиля игры (LD: Better Levels)

| Стил | Описание | Пространственные потребности |
|-----|-----|-----| | **Рашер (Rusher)** | Агрессивный ближний/средний бой по основному пути | Широкие коридоры, укрытия вдоль пути, быстрый доступ к врагу | | **Снайпер (Sniper)** | Хорошие линии видимости издалека | Возвышенности, длинные коридоры, открытые пространства | | **Ниндзя (Ninja)** | Фланговые маршруты, обходные пути | Боковые проходы, вентиляция, тени, альтернативные маршруты | | **Оппортунист (Opportunist)** | Использует интерактивные объекты | Взрывные бочки, газовые трубы, сталактиты, окружение как оружие |

Ключевой принцип: хорошее боевое пространство предоставляет выбор для всех четырёх стилей. Плохое — заставляет всех играть одинаково. AI-система должна генерировать боевые пространства, учитывающие все стили, и проверять: есть ли маршрут для ниндзя? позиция для снайпера? объекты для оппортуниста? путь для рашера?

7.7.2. Фокус сражения и линия фронта (Кадиков)

Фокус сражения (Combat Focus) — ключевое понятие Кадикова: место на уровне, на котором фокусируется внимание игрока в ходе боя. Три ошибки слабого фокуса: (1) враги вне поля зрения, (2) укрытия разбросаны далеко → огонь со всех сторон, (3) выход за кадром. Три правила сильного фокуса: (1) враги в рамках фокуса, (2) укрытия рядом → нет уязвимости с нескольких сторон, (3) выход в фокусе.

Линия фронта — воображаемая линия между игроком и противником. Фланговые атаки заставляют смещать линию фронта. Дизайн уровня должен предусматривать смену направления атаки.

LD: Better Levels дополняет: **Combat Fronts** — чёткое разделение сторон: фронт, фланг, тыл. Без этого = неконтролируемый хаос. **Нейтральная зона (No Man's Land)**: опасная область между сторонами без явных преимуществ. **Перемещение фронта**: во время боя фронт смещается (враги отступают, подкрепления прибывают с нового направления).

7.7.3. Фланговые маршруты

LD: Better Levels формулирует **5 правил фланговых маршрутов**:

1. Должны служить «мостами» через нейтральную зону 2. Укрытия вдоль фланговых маршрутов не должны быть хорошими постоянными позициями (иначе игрок засядет там вместо продвижения) 3. Укрытия вдоль флангов должны быть со стороны врага (чтобы атаковать его фланг) 4. Маршрут должен быть замечен с позиции игрока 5. Не должен быть мгновенно доступен — игрок должен сначала пройти через свой фронт

7.7.4. Система укрытий

Кадиков определяет **4 критерия идеального укрытия**: (1) защищает от огня, (2) позволяет наблюдать за полем боя, (3) позволяет вести огонь из-за него, (4) имеет путь отхода.

Разновидности: низкие (присед за ними), высокие (стоя за ними), укрытия-препятствия, укрытия-углы. Правила размещения: каждое укрытие = тактическая возможность; расстояние между укрытиями = дистанция перебежки под огнём; не все укрытия равноценны — лучшие позиции = награда за тактическое мышление.

7.7.5. Динамическое боевое пространство

Оба источника подчёркивают: лучшее боевое пространство **меняется во время боя**. Разрушение (обрушение укрытий меняет тактическую ситуацию), затопление (ограничивает пространство), подкрепления (новые враги с новых направлений), перемещение фронта. Статическое пространство предсказуемо; динамическое заставляет адаптироваться.

LD: Better Levels предостерегает: избегать предсказуемого паттерна «Enter room → Door closes → Enemies spawn → Defeat → Door open». Это не динамика, а рутина. Настоящая динамика — непредсказуемая смена условий, которая заставляет пересматривать тактику на ходу.

7.7.6. Представление противника

Кадиков формулирует правило: враги не должны «появляться из ниоткуда». Правильное представление: (1) группа врагов эффектно появляется (высадка десанта, взрыв стены), (2) быстро занимает выгодные позиции, (3) только потом открывает огонь. Игрок получает время для подготовки и оценки ситуации — это создаёт ощущение справедливости и тактической глубины.

7.8. Мультиплеерный левел-дизайн

7.8.1. Шесть принципов честного мультиплеера (Кадиков)

1. **Читаемость персонажа** — модель игрока должна контрастировать с фоном 2. **Меньше значит больше** — минимизация визуального шума в зоне боя 3. **Освещение и погода** — равные условия видимости для всех 4. **Простота планировки** — легко запоминающаяся структура, ключевые маршруты очевидны 5. **Скорость движения** — размеры пространства определяются скоростью перемещения и временем реакции 6. **Пространство для движения** — достаточная ширина коридоров, отсутствие неконтролируемых «бутылочных горлышек»

7.8.2. Пять структур мультиплеерных карт (LD: Better Levels)

| Структура | Описание | Режимы | |-----|-----|-----| | **Круг (Circle)** | Кольцевая структура основного пути | Deathmatch — пикапы вдоль кольца = постоянное движение | | **Восьмёрка (Figure 8)** | Два кольца, соединённые пересечением | CTF, бомба — пересечение = точка встречи | | **Столкновение (Clash Point)** | Точки, где команды встречаются при выходе со спавна | Team DM, domination | | **Двойное столкновение (Double Clash)** | 2 точки встречи, обычно в форме восьмёрки | Round-based режимы | | **Перетягивание каната (Tug of War)** | Команды борются за центральную зону, продвигаясь к базе противника | Payload, assault |

Кадиков дополняет **4 классическими планировками**: (1) Циркуляция (круговое движение по периметру), (2) Столкновение (два стартовых лагеря + центральная зона), (3) Перетягивание каната (одна ключевая точка в центре), (4) Оборона (одна команда защищает, другая атакует — асимметричные маршруты).

Ключевой принцип: простота = популярность. Популярные МР-карты имеют простую структуру (2–3 основных маршрута). Сложный лабиринт = трудно

запомнить = стресс. Ночные уровни = плохая видимость = непопулярные карты (исключение: King's Row из Overwatch — ночной, но хорошо освещённый = популярен).

7.8.3. Баланс мультиплеерных карт

Симметрия vs асимметрия: симметрия = проще балансировать, асимметрия = интереснее, но сложнее. **Время:** время достижения ключевых точек от спавна должно быть сопоставимым для обеих команд. **Спавн-поинты:** не должны быть уязвимыми — защита через клапаны + блокировку линий видимости. **PPP (Potential Player Positions):** все возможные позиции игрока в любой момент — инструмент анализа угроз и укрытий. **Фокальные точки:** каждая область должна иметь фокальную точку, место для 4–8 игроков, вариацию рельефа и 2 (лучше 3) входа/выхода. **Вертикальность:** разница высот 3м и 6м делает уровни интереснее, увеличивая «геймплей на квадратный метр».

7.9. Прогрессия механик в уровне

7.9.1. «Музыка механик» (Роджерс)

Роджерс (Кн. 8) предлагает 7-шаговую последовательность введения механик в уровне — «модель Пети и Волка»:

1. Движение + простые вызовы (изучение базового управления) 2. +1 новая механика ×2–3 ситуации (освоение) 3. +2-я механика + комбо с первой (сочетание) 4. +Хазард (опасность среды — усложнение) 5. +Враги (живой противник — динамика) 6. Комбо: враги + хазарды (сложные ситуации) 7. Переворот одной механики (инверсия — новая перспектива)

Это формализованный алгоритм конструирования уровня, который AI может использовать напрямую: задаётся набор механик → генерируется последовательность введения → проверяется по Rule of Threes (максимум 2–3 новых элемента на уровень).

7.9.2. Обучение через уровень (Адамс/Дорманс)

Адамс/Дорманс (Кн. 4) формулируют принцип **постепенного обучения**, идентичный подходу Valve: (1) **Представить** — показать механику в безопасной среде; (2) **Практиковать** — дать задание, использующее механику; (3) **Комбинировать** — объединить с ранее изученными

механиками; (4) **Усложнить** — добавить усложняющие факторы. Каждый уровень вводит одну новую механику и даёт игроку освоить её перед добавлением следующей.

7.9.3. Модель Disneyland (Роджерс)

Роджерс предлагает иерархическую модель: **World → Land → Attraction → Scene = World → Level → Experience → Moment-to-moment Gameplay**.

Каждый уровень — «аттракцион» в «земле» (акте) «мира» (игры). Эта модель задаёт иерархию проектирования: сначала определяется место уровня в мире, затем его уникальный опыт, затем момент-к-моментный геймплей.

7.10. Прототипирование и производство уровней

7.10.1. Блокинг и Grayboxing

Блокинг = грубая геометрия для тестирования геймплея до финального искусства. Принцип: **form follows function** — сначала геймплей, потом визуал. Роджерс: Grayboxing — уровень из примитивных форм для проверки масштаба, камеры, пейсинга. Смена эмоции каждые 15–20 минут. Формула времени: $walk-through\ time \times 2 \approx \text{финальное время уровня}$.

7.10.2. Четырёхфазный процесс производства (Кадиков)

1. **Планирование** — документация, зарисовки, концепт-арт 2. **Играбельный прототип** — базовая геометрия + основные механики + тест геймплея 3. **Прототип геометрии** — уточнённая геометрия + освещение + базовое декорирование 4. **Финальная сборка** — полный контент + полировка + оптимизация

7.10.3. Переработка уровней (Recycling)

LD: Better Levels описывает техники повторного использования уровней с изменениями: (1) **Сдвиг силы** — атака укрепления → использование пулемёта против подкрепления; (2) **Проход в обратном направлении** — проникновение в подземелье → побег обратно; (3) **Повторное посещение** — уровень изменился (открытый мир, God of War — уровень воды в озере). Это важный инструмент оптимизации производства и создания ощущения живого мира.

7.11. Формализация левел-дизайна для AI-системы Gidede

7.11.1. Формальная модель уровня

```
``` Level = { concept: LevelConcept, space: SpatialStructure, navigation:
NavigationSystem, pacing: PacingPlan, encounters: [Encounter], atmosphere:
AtmosphereConfig }
```

```
LevelConcept = { restrictions: [String], // обязательные элементы goals: [String], //
дизайнерские амбиции context: LevelContext, // позиция в игре foundation:
String, // клише-основа hook: HookType, // изюминка (6 типов) weenies:
[Landmark] // макро-ориентиры }
```

```
SpatialStructure = { layers: [SpatialLayer], // 6 слоёв Кадикова scale:
ScaleConfig, // базовая единица → все размеры modules: [Module], // повторно
используемые компоненты paths: { critical: Path, // кратчайший маршрут golden:
Path, // предпочтительный маршрут alternative: [Path] // секретные/боковые
пути }, gates: [Gate], // hard/soft gates + valves loops: [Loop] // петли для
минимизации бэктрекинга }
```

```
NavigationSystem = { landmarks: { micro: [Landmark], meso: [Landmark], macro:
[Landmark] }, signposting: SignpostingType, breadcrumbs: [Breadcrumb],
visual_language: VisualLanguage, nope_zones: [Area], teasing_points: [TeasePoint] }
```

```
PacingPlan = { beats: [Beat], // график интенсивности timeline: Timeline,
gates_and_valves: [GateValve], intensity_curve: Curve, // аналогия с кривыми
прогрессии variety_sequence: [ActivityType] }
```

```
Encounter = { space: CombatSpace, enemies: [EnemyConfig], fronts: CombatFronts,
flank_routes: [FlankRoute], cover_positions: [Cover], dynamic_elements:
[DynamicElement] }
```

```
AtmosphereConfig = { color_palette: ColorConfig, lighting: LightingConfig, ambient:
AmbientConfig, narrative_elements: [NarrativeElement], wow_moments:
[WowMoment] } ```
```

### 7.11.2. Алгоритм генерации уровня для AI

**Шаг 1: Определить концепцию** — по трём столпам (Restrictions, Goals, Context). Выбрать основу из клише-тем + изюминку одного из 6 типов.



**Шаг 2: Определить структуру путей** — Critical Path, Golden Path, альтернативные маршруты. Выбрать модель: Alley (линейный) или Island (исследование), или гибрид.

**Шаг 3: Разместить ориентиры** — макро-ориентиры (WEENIES) для глобальной навигации, мезо-ориентиры для областей, микро-ориентиры для локальной навигации. Обеспечить навигабельность без UI.

**Шаг 4: Спроектировать темп** — построить график Beats (интенсивность × время). Расставить Gates/Valves. Обеспечить чередование интенсивности, разнообразия и отдыха.

**Шаг 5: Спроектировать боевые пространства** — для каждого энкаунтера определить Combat Fronts, фланговые маршруты (по 5 правилам), укрытия (по 4 критериям), позиции для 4 стилей игры.

**Шаг 6: Определить прогрессию механик** — по «музыке механик» (7 шагов) + Rule of Threes. Каждый уровень = максимум 2–3 новых элемента.

**Шаг 7: Настроить атмосферу** — цвет, освещение, звук, нарративные элементы. Вау-моменты в ключевых точках (начало, середина, конец).

**Шаг 8: Валидация** — проверить по чек-листу из 15 пунктов.

### 7.11.3. Ключевые формализуемые модели

| Модель | Формализация | Применение в Gidede | |-----|-----|-----|  
| Три столпа концепции | Restrictions + Goals + Context | Генератор концепции уровня | | Основа + Изюминка | Foundation(cliche) + Hook(6 types) | Генератор запоминаемости | | 6 слоёв пространства | Checklist для комплектности | Валидатор уровня | | 3 уровня ориентиров | Micro / Meso / Macro | Валидатор навигации | | 9 навигационных инструментов | UI-группа + Путевые знаки | Рекомендация навигации | | Матрица темпа 3×3 | Интенсивность × Разнообразие × Время × Масштаб | Планировщик темпа | | Gates & Valves | Hard / Soft / Valve | Управление потоком | | 4 стиля игры | Rusher / Sniper / Ninja / Opportunist | Генератор боевых пространств | | Combat Fronts | Фронт + Фланг + Тыл + No Man's Land | Модель боевого пространства | | 5 структур MP-карт | Circle / Figure 8 / Clash / Double Clash / Tug | Классификатор мультиплеера | | «Музыка механик» | 7-шаговая последовательность | Генератор прогрессии механик | | Beat Chart | 15 столбцов + 8 проверок | Валидатор структуры игры | | Вау-моменты | 5 категорий + 3 структурных роли | Генератор эмоциональных пиков |

#### 7.11.4. Чек-лист валидации уровня

AI-система должна автоматически проверять:

- [ ] **Концепция:** 3 столпа определены? (Restrictions, Goals, Context) - [ ] **Запоминаемость:** Изюминка определена (одна из 6 типов)? - [ ] **Пути:** Critical Path и Golden Path задокументированы? - [ ] **Метрики:** Базовая единица определена? Все размеры от неё? - [ ] **Читаемость:** Окружение десатурировано? Важные элементы контрастны? - [ ] **Ориентиры:** Есть микро/мезо/макро ориентиры? - [ ] **Навигация:** Направление без UI возможно? Визуальный язык установлен? - [ ] **Темп:** Интенсивность, разнообразие и время сбалансированы на всех масштабах? - [ ] **Gates/Valves:** Используются для управления темпом? Нет ли монотонности? - [ ] **Боевое пространство:** Учитывает 4 стиля игры? Combat fronts определены? - [ ] **Фланги:** Маршруты соответствуют 5 правилам? - [ ] **Укрытия:** Соответствуют 4 критериям идеального укрытия? - [ ] **Прогрессия механик:** «Музыка механик» соблюдена? Rule of Threes? - [ ] **Атмосфера:** 3 принципа (реальность, постоянство, жизнь) соблюдены? - [ ] **Vau-моменты:** Есть в начале, середине и конце уровня?

---

## Глава 8. Нарратив и эмоциональный дизайн

---

> **Раздел Библии геймдизайна:** 2.8 — Нарратив и эмоциональный дизайн >

**Источники:** KB-11 (Нарратив), KB-13 (Опыт и мотивация), KB-03 (MDA Framework), KB-14 (Валидация) > **Книги:** Андрианова/Яковлева (Кн. 3), Шелл (Кн. 1, 2), Адамс/Дорманс (Кн. 4), Зубек (Кн. 6), Селлерс (Кн. 13), Фуллертон (Кн. 5), Бонд (Кн. 17)

---

### 8.1. Три термина: сюжет, повествование, нарратив

---

Одна из главных причин, по которым нарративный дизайн в играх остаётся трудноформализуемой областью, — терминологическая путаница. Слова «сюжет», «повествование» и «нарратив» часто используются как синонимы, однако за каждым из них стоит принципиально разное понятие с разным

фокусом и разными инструментами проектирования. Андрианова и Яковлева (Кн. 3) предлагают наиболее чёткое разделение, которое для AI-системы Gidede становится основой архитектуры нарративного модуля: каждый термин определяет отдельный субмодуль с собственными входными данными, методами генерации и критериями валидации.

**Сюжет** (plot) — запланированная история, созданная сценаристами. Это конкретные локации, кат-сцены, встречи, квесты и диалоги — всё, что написано автором до того, как игрок нажмёт кнопку «Start». Фокус сюжета — на том, что написал автор. Сюжет линейен по своей природе (даже разветвлённый сюжет — это набор линейных веток, написанных заранее) и полностью контролируется разработчиком. Для AI-системы сюжет — это генерируемый контент: деревья диалогов, структуры квестов, кат-сцены, лор. Входные данные для генерации: жанр, сеттинг, тема, целевая аудитория.

**Повествование** (narration) — процесс создания рассказа истории, при котором у реципиента возникает образ рассказчика. Фокус повествования — на том, кто рассказывает и как. Повествование отвечает за форму подачи: устный рассказ NPC, текстовый лог, аудиодневник, надпись на стене, кадр камеры, environmental storytelling. Один и тот же сюжет может быть рассказан тысячей разных способов, и каждый создаст разные впечатления. Для AI-системы повествование — это выбор формата и стиля подачи: как информация доносится до игрока. Ключевой вопрос: «Через какой канал и от чьего лица игрок узнаёт о событиях?»

**Нарратив** (narrative) — полнота игрового опыта игрока, полученного путём уникального прохождения игры. Фокус нарратива — на том, кто получает (игрок), а не на том, кто рассказывает или что написано. Нарратив — это то, что остаётся у игрока в голове после завершения игры: не сценарные строки, а прожитый опыт. Критически важный инсайт: **нарратив присутствует в любой игре, даже если у неё нет сюжета**. Игрок всегда будет стремиться выстроить историю из своих действий, даже если игра не предоставляет для этого никаких сценарных заготовок. Рас-Ман для одного игрока — колобок, убегающий от привидений; для другого — пицца без кусочка, поедающая точки.

Для Gidede это разделение — не просто теоретическая изящность, а архитектурное решение. Когда пользователь говорит «нарратив», AI должен понимать, к какому из трёх уровней относится запрос: генерация сюжета (события, квесты, диалоги), проектирование повествования (формат подачи, рассказчик, канал) или валидация нарратива (целостность опыта игрока, лудонарративный диссонанс).

---

## 8.2. Два типа нарратива: включённый и эмергентный

---

Андрианова и Яковлева выделяют два фундаментальных типа нарратива, которые определяют всю стратегию проектирования истории в игре.

### 8.2.1. Включённый (embedded) нарратив

Включённый нарратив — это история, созданная сценаристами и встроенная в игру. Она предопределена, фиксирована и одинакова для всех игроков (с точностью до ветвлений). Игрок «раскапывает» встроенную историю, проходя уровни, смотря кат-сцены, читая диалоги и обнаруживая лор. Классические примеры: RPG (The Witcher 3, Disco Elysium), визуальные новеллы (Doki Doki Literature Club), сюжетные приключения (The Last of Us, God of War).

Проектирование включённого нарратива — это классическая сценарная работа: создание персонажей, выстраивание арки конфликта, написание диалогов, проектирование квестовых цепочек. Вся ответственность за качество нарратива лежит на авторе, и все инструменты классической драматургии (трёхактная структура, путь героя, повышение ставок) здесь применимы полностью. Для AI-системы это означает: генерация включённого нарратива — задача генеративного модуля, который комбинирует структурные шаблоны (4 типа сюжета, кисётэнкэцу) с контентными генераторами (персонажи, диалоги, квесты).

### 8.2.2. Эмергентный (emergent) нарратив

Эмергентный нарратив — это история, формирующаяся в сознании игрока во время прохождения. Она не написана заранее, а возникает из взаимодействия игрока с игровыми системами. Каждый игрок создаёт свой уникальный нарратив, и два прохождения одной и той же игры могут породить совершенно разные истории. Классические примеры: песочницы (Dwarf Fortress, RimWorld), стратегии (Civilization, Crusader Kings), аркады (Pac-Man, Tetris), симуляторы (The Sims).

Эмергентный нарратив зависит от ментальности и культурного контекста игрока: одна и та же игровая ситуация будет интерпретирована по-разному игроками из разных культур. Проектирование эмергентного нарратива — это не написание истории, а создание механик, которые **порождают** интересные ситуации. Шелл (Кн. 1, 2) называет это «Машиной историй» (Story Machine):

игра создаёт систему, в которой истории возникают естественно из взаимодействия игроков с механиками, без заранее написанного сценария. Селлерс (Кн. 13) развивает эту идею через концепцию «второпорядочного дизайна»: дизайнер создаёт пространство состояний, а не один путь; нарратив эмерджентно возникает из взаимодействия систем.

### 8.2.3. Стратегия выбора для AI-системы

Выбор типа нарратива определяется жанром и целевой аудиторией. Для Gidede это решение принимается на этапе определения концепции:

Жанр	Тип нарратива	Подход Gidede
RPG, Adventure	Включённый	Генерация сюжета, ветвлений, диалогов, квестов
Песочница, Strategy	Эмергентный	Генерация механик, порождающих истории
Action, Shooter	Гибридный	Базовый сюжет + эмергентные боевые нарративы
ММО, Survival	Гибридный	Событийный каркас + социальные эмергентные истории

Большинство успешных современных игр комбинируют оба подхода: включённый нарратив задаёт рамки и мотивацию, а эмергентный — наполняет живым опытом между сюжетными точками. Адамс и Дорманс (Кн. 4) формализуют это через дихотомию Юла: игры эмерджентности (мало правил → много взаимодействий) и игры прогрессии (много правил → заданный путь). Лучшие игры — гибриды, где эмерджентность возникает внутри структуры прогрессии.

---

## 8.3. Агентивность: свобода игрока в нарративе

---

Агентивность (agency) — степень влияния игрока на развитие игры — ключевое понятие, определяющее архитектуру нарративной системы. Андрианова и Яковлева выделяют два типа агентивности, которые необходимо проектировать независимо:

**Геймплейная агентивность** — все доступные игроку действия в игровом мире. Это ширина пространства действий (сколько «глаголов» доступно), глубина комбинаторики (сколько результирующих стратегий возникает из оперативных действий) и свобода экспериментирования. Геймплейная агентивность напрямую связана с механиками и динамикой игры и проектируется через MechanicsDB и Core Loop.

**Сюжетная агентивность** — степень влияния игрока непосредственно на сюжет. Это способность игрока менять ход истории, принимать решения, влияющие на развитие событий и финал. Сюжетная агентивность — отдельный измеритель, который может не коррелировать с геймплейной: игра может давать огромную свободу действий (Minecraft), но нулевую сюжетную агентивность; и наоборот — визуальная новелла может давать минимальную геймплейную агентивность, но высокую сюжетную (Detroit: Become Human).

Три уровня агентивности по Андриановой/Яковлевой:

- **Высокая** — игрок сам формирует историю. Игры с выборами, последствиями, множественными финалами. RPG с ветвящимся сюжетом, песочницы с эмергентным нарративом. Дизайнер проектирует систему, а не линию. -

**Низкая** — игрок наблюдает за predetermined историей, но проживает её интерактивно. Линейные приключения (Uncharted), визуальные новеллы без ветвлений. Дизайнер контролирует нарратив, а игрок — темп и эмоции. -

**Нулевая** — игрок только проживает интерактивный процесс без влияния на сюжет. Экзистенциальные игры (Passage), арт-игры (Journey). Дизайнер использует отсутствие агентивности как инструмент для создания специфического опыта (ощущение бессилия, смирение).

Для Gidede агентивность — это **настраиваемый параметр** генерации. AI получает на вход желаемый уровень геймплейной и сюжетной агентивности и подбирает соответствующие нарративные структуры: для высокой сюжетной агентивности — разветвлённый сюжет с ромбовидными квестами; для низкой — жемчужную нить; для нулевой — атмосферный эмергентный нарратив без кат-сцен.

---

## 8.4. Лудонарративный диссонанс и его спектр

---

Лудонарративный диссонанс — одна из наиболее критических проблем в проектировании нарратива для видеоигр. Термин введён Клинтом Хокингом в 2007 году в размышлениях над BioShock, где сюжетная философия объективизма вступала в противоречие с линейной структурой игры, не дававшей игроку реального выбора. Андрианова и Яковлева определяют это как «несоответствие сюжета и игрового процесса, конфликт между этими сущностями, который разрушает состояние потока игрока».

### 8.4.1. Три состояния: гармония, ирония, диссонанс

Адамс и Дорманс (Кн. 4) расширяют бинарную модель «диссонанс/гармония», добавляя третье состояние — иронию. Это расширение критически важно для AI-системы, которая должна различать не только ошибку, но и осознанный дизайнерский приём:

**Гармония** — механика и нарратив работают в одном направлении, взаимно усиливая друг друга. Пример: в *The Last of Us* механика дефицита ресурсов (мало патронов, нужно выбирать, тратить ли на врага или сохранить) гармонично поддерживает нарратив выживания в постапокалиптическом мире. Механика и история говорят одно и то же: «этот мир жесток, ресурсы ограничены, каждый выбор имеет значение». Для AI-системы гармония — целевое состояние: генерируемые механики должны поддерживать генерируемый нарратив, и наоборот.

**Ирония** — механика и нарратив противоречат друг другу, но это противоречие **осознанно используется как художественный приём**. Пример: в *Spec Ops: The Line* игрок как бы «вынужден» совершать ужасные действия механически (расстрел толпы), при этом игра подвергает это критике через нарратив. Контраст между тем, что игрок делает, и тем, что игра говорит об этом, создаёт мощный эмоциональный эффект — диссонанс работает на игру, а не против неё. Для AI-системы ирония — сложный для генерации, но возможный режим: система должна знать, когда противоречие является ошибкой, а когда — приёмом.

**Диссонанс** — механика и нарратив противоречат друг другу неосознанно, разрушая иммерсию и доверие игрока. Пример: в *Tomb Raider* (2013) сюжет показывает уязвимую Лару Крофт, впервые взявшую в руки оружие и испытывающую травму от первого убийства, но механика через 10 минут превращает её в штурмовика, без счёта укладывающего врагов. Сюжет говорит «хрупкая девушка в беде», механика — «неубиваемый терминатор». Для AI-системы диссонанс — это **ошибка, которую нужно обнаруживать и исправлять**.

### 8.4.2. Валидатор лудонарративного диссонанса

Для Gidede формализуется алгоритм обнаружения диссонанса. Система проверяет согласованность на трёх уровнях:

**Уровень 1: Механика ↔ Тема.** Соответствуют ли ключевые механики заявленной теме игры? Если тема — «выживание», но механика даёт неограниченные ресурсы → диссонанс. Если тема — «одиночество», но

механика постоянно окружает игрока NPC → диссонанс. Проверка: для каждой ключевой темы (из концепции) определить, какие механики её поддерживают, а какие — подрывают.

**Уровень 2: Геймплей ↔ Персонаж.** Соответствует ли поведение персонажа под управлением игрока его описанию в сюжете? Если персонаж описан как пацифист, но геймплей принуждает к насилию → диссонанс. Если персонаж — опытный воин, но геймплей делает его неуклюжим → диссонанс. Проверка: для каждого значимого персонажа сопоставить его нарративный профиль с доступными геймплейными действиями.

**Уровень 3: Прогрессия ↔ Арка.** Соответствует ли геймплейная прогрессия нарративной арке персонажа? Если персонаж должен расти от слабости к силе, но механика даёт максимум возможностей с самого начала → диссонанс. Если нарративная арка — «падение», но геймплей постоянно вознаграждает → диссонанс. Проверка: сопоставить кривую геймплейной мощности с нарративной аркой трансформации.

---

## 8.5. Структуры сюжета

---

Структура сюжета — это каркас, на котором держится весь включённый нарратив. Андрианова и Яковлева выделяют четыре базовые структуры, каждая из которых определяет архитектуру нарративного модуля в AI-системе.

### 8.5.1. Четыре базовые структуры

**Линейная** — ключевые события следуют в единой последовательности, ведущей к единому финалу. Это наиболее контролируемая структура: дизайнер полностью управляет темпом, эмоциональной аркой и раскрытием информации. Примеры: *Uncharted*, *Call of Duty*, большинство линейных приключений. Для AI-системы линейная структура — простейшая для генерации: достаточно определить последовательность сюжетных точек (story beats) и соединить их геймплеем.

**Разветвлённая** — несколько путей развития сюжета, определяемых выборами игрока, ведущих к разным финалам. Это структура с высокой сюжетной агентивностью, но она значительно сложнее в производстве: каждое ветвление умножает количество контента. Примеры: *Detroit: Become Human* (три protagonists, десятки концовок), *Heavy Rain*. Ключевая проблема: комбинаторный взрыв — если каждый выбор создаёт 2 ветки, то 10 выборов =



1024 варианта. Решение: **слияние веток** (branches merge), где разные выборы ведут к одним и тем же ключевым точкам, но с разными эмоциональными оттенками.

**Открытая** — игрок свободен в порядке прохождения, но существует основная сюжетная линия (critical path). Примеры: Skyrim, The Witcher 3, Breath of the Wild. Открытая структура требует **модульного нарратива**: каждый квест — самодостаточная история, но все вместе они образуют общую арку. Проблема: контроль темпа. Игрок может прийти к финальному боссу на первом уровне или после 100 часов крафта. Решение: мягкие ограничения (уровневые требования, сюжетные триггеры) и **квантовое** 叙事 — мир реагирует на действия игрока, но не нарушает общую арку.

**Кольцевая** — начало = конец; игрок возвращается на стартовую точку, но на новом уровне понимания или мощности. Примеры: MOBA (LoL, DotA — каждый матч — кольцо), Dark Souls (мир закольцован, финал возвращает к началу), roguelike (Hades — каждый забег — кольцо, но нарративная арка progresses). Кольцевая структура идеально подходит для игр с повторяемым геймплеем: каждый цикл — мини-история с завязкой, кульминацией и развязкой.

### 8.5.2. Кисётэнкэцу — альтернативная структура

Андрианова и Яковлева приводят восточный метод «Кисётэнкэцу» (起承転結) — четырёхчастную структуру повествования, распространённую в японской литературе и играх:

1. **Ki** (Введение) — завязка, представление персонажей и мира. Установление статус-кво, нормального состояния, которое будет нарушено. 2. **Shō** (Развитие) — развитие событий, углубление конфликта. Персонажи действуют, обстоятельства усложняются. 3. **Ten** (Поворот) — неожиданный поворот, перелом. То, что меняет всё: раскрытие тайны, предательство, катастрофа. Ключевой элемент: поворот должен быть неожиданным, но логичным в ретроспективе. 4. **Ketsu** (Развязка) — разрешение, новый статус-кво. Не обязательно «хэппи-энд» — но новое равновесие, отличное от начального.

В отличие от западной трёхактной модели, кисётэнкэцу подчёркивает важность **поворота** (Ten) как структурообразующего элемента. Если в трёхактной модели кульминация — это точка максимального напряжения, то в кисётэнкэцу поворот — это точка максимального сюрприза. Это различие практически значимо для AI-генерации: система должна не только генерировать напряжение (повышение ставок), но и обеспечивать неожиданность (твисты).

### 8.5.3. Жемчужная нить Шелла

Шелл (Кн. 1, 2) предлагает прагматичный метод совмещения линейной истории с интерактивностью — «жемчужную нить» (String of Pearls):

``` [Кат-сцена] → [Свободный геймплей] → [Кат-сцена] → [Свободный геймплей] → ... ```

Неинтерактивная история (нитка) чередуется с интерактивными сегментами (жемчужины). Этот метод часто критикуется за «недостаточную интерактивность», но Шелл справедливо отмечает: игрокам он нравится. Причина в том, что жемчужная нить решает фундаментальное противоречие между свободой и контролем: кат-сцены дают контекст и мотивацию, а геймплей — агентивность и веселье. Без контекста геймплей бессмыслен; без агентивности контекст скучен. Жемчужная нить — это мост между двумя мирами.

Для AI-системы жемчужная нить — **основной паттерн генерации нарратива для большинства жанров**: система генерирует последовательность сюжетных точек (кат-сцен, диалогов, событий), между которыми размещает геймплейные сегменты, соответствующие текущему нарративному контексту.

8.6. Персонажи: типы, архетипы, библии

Персонажи — главный носитель эмоциональной связи игрока с игрой. Без убедительных персонажей даже блестящий сюжет останется абстракцией; с ними — даже слабый сюжет может стать запоминающимся.

8.6.1. Шесть типов персонажей

Андрианова и Яковлева систематизируют типы игровых персонажей, каждый из которых требует разного подхода к проектированию:

Тип	Описание	Проектирование для AI
Игровой персонаж	Управляется игроком	Геймплейная механика + нарративная арка
NPC	Неигровой персонаж	Диалоги + AI-поведение + квестовые функции
Заданный персонаж	С predetermined внешностью и характером	Сильный нарративный профиль + ограниченная кастомизация
Нулевой персонаж	«Пустышка» — минимальная заданность, максимальная проекция	Минимум предыстории + максимум свободы действий
Кастомизируемый	Игрок меняет внешность и	

характеристики | Модульная система создания + совместимость с анимациями
| | **Наблюдающий «из-за четвёртой стены»** | Общается напрямую с игроком
| Мета-нарративные механики + разрушение четвёртой стены |

Выбор типа персонажа — стратегическое решение, влияющее на весь дизайн: заданный персонаж (Джоэл из The Last of Us) позволяет написать глубокую арку трансформации, но ограничивает проекцию игрока; нулевой (Гордон Фримен) максимизирует проекцию, но требует «тихого протагониста», чья арка разворачивается через окружение, а не диалоги.

8.6.2. Архетипы и Циркумплекс

Для генерации персонажей AI-система использует два инструмента.

Архетипы Юнга—Кэмпбелла дают универсальные модели трансформации: Герой, Наставник, Тень, Трикстер, Оборотень — каждый архетип определяет нарративную функцию в истории. Шелл (Кн. 2) дополняет это **межличностным циркумплексом** — двумерной моделью (дружелюбный/враждебный × доминантный/подчинённый), которая позволяет создавать персонажей с различными социальными позициями и выявлять «пустые места» на сетке отношений.

Для AI-системы комбинирование архетипа (нарративная функция) и циркумплекса (социальная позиция) даёт матрицу генерации: архетип определяет, какую роль персонаж играет в истории, а позиция на циркумплексе — как он взаимодействует с другими. Например, Наставник на позиции «дружелюбный-доминантный» = мудрый учитель; Наставник на «враждебный-подчинённый» = неохотный советчик, дающий информацию под принуждением.

8.6.3. Библия персонажа

Для каждого значимого персонажа создаётся **библия персонажа** — структурированный документ, содержащий: внешность и возраст; предысторию; мировоззрение и мотивацию; цели (осознанные и неосознанные); отношения с другими персонажами (сетка связей); речевые особенности (словарный запас, синтаксис, любимые фразы); арку развития (трансформацию от начала к концу). Для Gidede генерация библии персонажа — отдельный субмодуль, который на основе архетипа, позиции на циркумплексе и роли в сюжете генерирует полную документацию персонажа, включая арку трансформации (Призма 91 Шелла: «Как персонажи изменяются? Достаточно ли изменений? Неожиданные?»).

8.7. Конфликт — движок сюжета

Без конфликта нет истории — это аксиома драматургии, действующая и в играх. Андрианова и Яковлева выделяют восемь типов конфликта, каждый из которых порождает свой тип нарративной динамики и свои механические требования:

| Тип конфликта | Описание | Механическое воплощение |
|-----|-----|-----| | **Внутренний** | Герой против самого себя |
Выборы с моральными последствиями, двойные механики | | **С окружающим миром** | Герой против среды/обстоятельств | Сурвайвал, платформинг, environmental puzzles | | **Социальный** | Герой против общества | Репутация, фракции, социальные механики | | **Отношений** | Герой против другого персонажа | PvP, дуэли, соперничество, диалоги-переговоры | | **«На развилке»** | Выбор между двумя равноценными вариантами | Дилеммы, branching, моральные выборы | | **Межрасовый** | Конфликт рас/народов | Фракционные войны, расовые механики | | **Целей** | Разные цели у разных персонажей | Асимметричный мультиплеер, конкуренция за ресурсы | | **Методов** | Одна цель, разные методы достижения | Классовая система, стелс/экшен, разные стили прохождения |

Для AI-системы тип конфликта — это **входной параметр нарративной генерации**: система получает целевой тип конфликта и генерирует механики, персонажей и квестовые структуры, которые его поддерживают. Ключевой принцип: конфликт должен быть **разрешим** (иначе — фрустрация) и **осмыслен** (иначе — тривиальность). Фуллертон (Кн. 5) формулирует четыре типа решений по значимости: тривиальное (нет реального выбора), безразличное (выбор есть, но исходы одинаковы), дилемма (оба варианта имеют значимые плюсы и минусы), значимое (выбор влияет на последующий опыт). Дилемма и значимое решение — целевые типы; тривиальное и безразличное — антипаттерны.

8.8. Сценарные инструменты

Сценарные инструменты — конкретные техники, которые сценаристы используют для поддержания интереса и управления эмоциями игрока. Андрианова и Яковлева выделяют шесть основных инструментов, каждый из которых может быть формализован для AI-генерации.

Твист (twist) — неожиданный поворот сюжета, нарушающий ожидания игрока. Твист работает только тогда, когда он неожиданен, но логичен в ретроспективе: игрок должен иметь возможность оглянуться и увидеть подсказки, которые он пропустил. Правило твиста: никогда не обманывайте игрока — он должен иметь возможность предвидеть поворот, если достаточно внимателен. Для AI-системы твист — это генеративный шаблон: «дано: текущие ожидания → сгенерировать: событие, нарушающее ожидания, но подтверждаемое предшествующими подсказками».

Клиффхэнгер (cliffhanger) — сюжетный крючок, обрыв на самом интересном месте. Клиффхэнгер заставляет игрока хотеть продолжения и является одним из мощнейших инструментов удержания. Механизм: прервать нарративную арку в точке максимального напряжения, перед самым разрешением. Для AI-системы клиффхэнгер — инструмент управления сессией: если аналитика показывает, что игрок заканчивает сессию в определённый момент, генерация клиффхэнгера перед этим моментом может увеличить retention.

Повышение ставок (raising the stakes) — с каждым шагом последствия становятся серьёзнее, риск растёт. Что сначала было личным делом, становится делом команды, потом города, потом мира. Повышение ставок создаёт нарративное ускорение: чем выше ставки, тем сильнее эмоциональная вовлечённость. Для AI-системы повышение ставок — **градиент нарративной интенсивности**: система должна генерировать арку, в которой каждая следующая сюжетная точка имеет более высокие ставки, чем предыдущая.

Эмоциональные горки (emotional roller coaster) — чередование напряжения и расслабления, аналогичное кривой интереса Шелла (Глава 16 Кн. 1, 2). Постоянное напряжение истощает; постоянное расслабление усыпляет. Идеальный racing — волнообразная кривая с нарастающими пиками: небольшой пик → спад → больший пик → спад → ещё больший пик → кульминация → разрешение. Для AI-системы эмоциональные горки — это **параметрическая модель темпа**: система получает целевую кривую эмоциональной интенсивности и генерирует последовательность событий, ей соответствующую.

Чередование сцен — переключение между разными линиями повествования для поддержания интереса. Когда одна линия достигает пика напряжения, переключение на другую даёт передышку и одновременно создаёт дополнительный слой интереса. Это классический приём сериального сторителлинга, и для AI-системы он означает: при проектировании многолинейного сюжета система должна чередовать линии, не позволяя одной доминировать слишком долго.

Триггеры — события, запускающие определённые сюжетные реакции. Триггеры — мост между геймплеем и нарративом: достижение игроком определённого уровня, убийство определённого босса, обнаружение определённого предмета — всё это триггеры, которые продвигают сюжет. Для AI-системы триггеры — это **связующие звенья** между MechanicsDB и нарративным модулем: система должна автоматически связывать геймплейные события с сюжетными последствиями.

8.9. Сеттинг, лор и миростроение

Прежде чем проектировать персонажей и квесты, необходимо создать мир, в котором они существуют. Андрианова и Яковлева (Кн. 3) формализуют три уровня мира, каждый из которых имеет свою функцию в нарративной системе и свои требования к AI-генерации.

8.9.1. Сеттинг

Сеттинг — совокупность всех игровых характеристик вымышленного мира: место, время, антураж, правила и законы. Сеттинг — это «декорации» нарратива: он определяет, что возможно в мире, а что нет, какие технологии доступны, какая магия работает, какие социальные структуры существуют. Авторы подчёркивают: определиться с сеттингом необходимо ещё до старта разработки, потому что сеттинг ограничивает дизайнерское пространство решений. Если сеттинг — средневековое фэнтези без пороха, то огнестрельное оружие исключено; если сеттинг — реалистичный современный мир, то магия исключена. Эти ограничения — не помеха, а ресурс: ограничения стимулируют креативность (как отмечает и Селлерс, Кн. 13).

Для AI-системы сеттинг — это **система ограничений генерации**: все генерируемые механики, персонажи и квесты должны соответствовать сеттингу. Нарушение сеттинга — форма лудонарративного диссонанса: если в средневековом фэнтези вдруг появляется плазменная винтовка без сюжетного обоснования, это разрушает иммерсию.

8.9.2. Лор

Лор (lore) — предыстория мира, его история и детали, которые делают мир живым и убедительным. Лор необходим для: (1) создания последовательного мира — каждая деталь должна быть согласована с остальными; (2) обоснования механик и правил — почему в мире есть магия? почему некоторые расы враждуют? почему экономика устроена именно так?; (3)

погружения игрока — мир, у которого есть история, ощущается реальнее; (4) работы команды разработки — лор служит единой точкой отсчёта для всех членов команды.

Для AI-системы лор — это **контекстная база данных**, из которой генератор черпает детали: имена, географические названия, исторические события, культурные особенности. Ключевой принцип: лор должен быть **внутренне непротиворечив** — AI должен проверять, не противоречат ли новые генерируемые элементы уже установленным фактам мира.

8.9.3. Миростроение (Worldbuilding)

Миростроение — неотъемлемая часть работы над большим игровым миром. Если система мира разваливается — сломается вся история. Андрианова и Яковлева выделяют два подхода к миростроению:

Мир → Герой: сначала создаётся мир со своей историей, географией, культурой, а затем в него помещается персонаж. Этот подход типичен для RPG, ММО и открытых миров: мир первичен, история персонажа — вторична. Преимущество: мир получается глубоким и убедительным. Риск: персонаж может «потеряться» в огромном мире без чёткой мотивации.

Герой → Мир: сначала создаётся персонаж с его историей и мотивацией, затем мир подстраивается под него. Этот подход типичен для линейных приключений и визуальных новелл: персональная история первична, мир — фон для неё. Преимущество: сильная эмоциональная связь с протагонистом. Риск: мир может ощущаться «декорацией», а не живым пространством.

Авторы также приводят **7 ошибок миростроения** (по Чарли Джейн Андерс), которые AI-система должна обнаруживать и предотвращать: (1) однобокие версии рас — выдуманные народы без индивидуальности; (2) случайные события — сюжет строится на фактах без объяснения причин; (3) заигрывание с суперсилами — технологии/магия без логичных последствий; (4) отсутствие инфраструктуры — сложное общество, оторванное от бытовых проблем; (5) повальная солидарность — единодушие всех групп (нереалистично); (6) отсутствие внутренних конфликтов; (7) мир-фон вместо мира-системы.

8.10. Темпоральность: время как нарративный инструмент

Одна из уникальных возможностей видеоигр как медиа — манипуляция временем. Андрианова и Яковлева выделяют специфические инструменты, которые не доступны литературе или кино в той же степени:

Сжатие времени — дни проходят за секунды. Классический пример: цикл день/ночь в RPG (Skyrim), где игровой час равен нескольким реальным минутам. Сжатие позволяет моделировать долгосрочные процессы (рост урожая, строительство, старение) в игровом масштабе. Для AI: параметр `time_scale` настраивается в зависимости от жанра и целевого темпа.

Растяжение времени — замедление, bullet time. Max Payne, Superhot — игры, где манипуляция временем является основной механикой. Растяжение создаёт эффект напряжённого внимания и даёт игроку время на стратегические решения в критических ситуациях. Для AI: система должна определять, когда растяжение времени усиливает нарратив (сцена с высоким эмоциональным накалом), а когда — разрушает (рутинный бой).

Параллельные временные линии — игрок переключается между персонажами в разное время, или играет в нескольких эпохах. Примеры: Braid (механика перемотки), Day of the Tentacle (три персонажа в разных временных периодах), Assassins Creed (прошлое и настоящее). Параллельные линии создают многослойный нарратив, где информация из одной линии обогащает понимание другой.

Временные петли — игрок проживает один и тот же период многократно, каждый раз узнавая больше и меняя исход. Примеры: Outer Wilds (22-минутный цикл), Majora's Mask (3-дневный цикл), Returnal, Hades. Временные петли — мощнейший нарративный инструмент, потому что они естественно сочетают повторяемый геймплей с нарастающим пониманием: каждый цикл — и повторение, и прогресс. Для AI-системы временная петля — это **структурный паттерн**, который генерирует: (1) набор событий, повторяющихся каждый цикл; (2) набор условий, которые игрок может менять между циклами; (3) нарративные открытия, которые раскрываются постепенно.

8.11. Квестовая система: типы × паттерны

Квесты — основной структурный элемент включённого нарратива в играх. Андрианова и Яковлева предлагают систематизацию, которая позволяет AI-системе генерировать квесты комбинаторным путём.

8.11.1. Восемь типов квестов

#	Тип	Описание	Жанровое применение	1
1	Поиск	Найти локацию, предмет или персонажа	RPG, Adventure	2
2	Сбор предметов	Добыть определённое количество ресурсов	RPG, Survival	3
3	Доставка	Доставить предмет из точки А в точку Б	RPG, ММО	4
4	Сопровождение	Сопроводить NPC через опасную зону	Shooter, RPG	5
5	Убийство	Устранить цель	Action, Shooter	6
6	Оборона	Защитить точку или персонажа от волн врагов	Strategy, Shooter	7
7	Активация	Активировать предметы или NPC	Puzzle, Adventure	8
8	Гибридные	Комбинация нескольких типов	Любой	

8.11.2. Четыре паттерна квестов

Паттерн	Описание	Подходящие жанры
1	Линейное усложнение	Каждый следующий квест сложнее предыдущего
2	Платформеры, экшен	Отложенные последствия
3	Выбор в квесте	проявляется намного позже
4	RPG	Моральная дилемма
5	Выбор между «плохим» и «хорошим» или «серой зоной»	RPG с сюжетом
6	Турнир	Обойти соперников или поставить рекорд
7	Арены, спортивные	

8.11.3. Матрица генерации: 8 × 4 = 32 комбинации

Комбинирование 8 типов квестов с 4 паттернами даёт 32 базовые комбинации для генерации. Например: «Доставка + Отложенные последствия» = квест, где доставленный предмет спустя несколько глав окажется причиной конфликта; «Убийство + Моральная дилемма» = квест, где цель — не очевидный враг, а персонаж с собственными мотивами.

Для AI-системы матрица квестов — это **комбинаторный генератор**: система выбирает тип и паттерн на основе жанра, позиции в игре (начало/середина/финал) и целевой эмоции, а затем генерирует конкретное содержание квеста с учётом сеттинга и персонажей. Авторы рекомендуют практический метод: «выпишите в одну колонку типы квестов, в другую — паттерны, затем соединяйте до получения удачной комбинации» — это буквально алгоритм, который может выполнять AI.

8.11.4. Структуры квестовых цепочек

Две основные структуры организации квестов внутри уровня или главы:

Ромбовидная (бриллиантовая) — от стартовой точки тянутся минимум два пути (добро/зло, стелс/экшен), они расходятся, затем сближаются к финалу. Внутри ромба — побочные квесты. Преимущество: даёт агентивность (выбор пути), но контролирует финал (обе ветки сходятся).

Линейный хаб — игрок доходит до точки, где нужно выполнить несколько квестов для разблокировки следующей части истории. Хаб даёт свободу порядка, но не свободы пути. Эффективен для управления темпом: игрок выбирает, за какой квест взяться, но не может пропустить обязательные.

8.12. Диалоги: функции и принципы

Диалоги — один из ключевых инструментов нарративного дизайна, и их проектирование требует системного подхода. Андрианова и Яковлева предлагают наиболее полную типологию.

8.12.1. Два вида и четыре формата диалогов

Внешний (эмбиентный) диалог — разговор, происходящий во время игры без остановки геймплея. Фоновые фразы, комментарии NPC, реакции на действия игрока. Этот тип диалога усиливает атмосферу и не прерывает поток, но не может передавать сложную информацию.

Интерактивный диалог — диалог, приостанавливающий игру для выбора ответов. Может быть линейным (одна реплика за другой) или нелинейным (ветвящийся, с выборами). Нелинейный диалог обычно имеет алмазную структуру: от **открытой** реплики расходятся ветки, которые затем сливаются к конечной точке.

Четыре формата: (1) диалог с другим человеком (классический формат); (2) диалог с самим собой (внутренний монолог — раскрывает мысли персонажа); (3) разговор с игроком (нарушение четвёртой стены —/meta-нарратив); (4) монолог (информативный или эмоциональный — один персонаж говорит, другой слушает).

8.12.2. Семь приёмов улучшения диалогов

Для AI-системы эти семь приёмов — не просто рекомендации, а **правила генерации**, которые должны соблюдаться при создании диалогов:

1. **Добавляем характер** — уникальные речевые особенности для каждого персонажа. В AI-генерации: каждому персонажу присваивается речевой профиль (длина фраз, словарный запас, синтаксис, любимые слова), который соблюдается во всех его репликах. 2. **Создаём тайну** — умалчивания и белые пятна подзадоривают фантазию. В AI-генерации: система должна не только отвечать на вопросы, но и оставлять пространство для догадок. 3. **Добавляем эмоции** — короткие, ёмкие, прямолинейные фразы. Эмоциональный диалог ≠ длинный диалог; часто одна реплика сильнее страницы текста. 4. **Контролируем глубину** — не все жанры нуждаются в многослойных диалогах. Кооперативный шутер не требует философских бесед; RPG — требует. 5. **Используем подтекст** — герои говорят одно, подразумевают другое. Подтекст — один из мощнейших инструментов драматургии, но трудный для AI-генерации: система должна создавать двойной смысл, где «явный» и «скрытый» уровни не совпадают. 6. **Редактируем** — «одна мысль — одна реплика», убираем трудночитаемый текст. Это правило контроля качества генерации: AI должен проверять, что каждая реплика содержит одну и только одну значимую мысль. 7. **Избегаем повторов** — правило трёх раз: важная информация должна быть подана трижды, но в разной форме. Это баланс между повторением (для закрепления) и разнообразием (для удержания интереса).

8.13. Эмоциональный дизайн: от эстетики к опыту

Эмоциональный дизайн — это целенаправленное проектирование эмоционального опыта игрока через все доступные средства: механику, нарратив, эстетику, социальные взаимодействия. Если нарратив отвечает на вопрос «какую историю рассказывает игра», то эмоциональный дизайн отвечает на вопрос «что чувствует игрок».

8.13.1. Восемь типов эстетики (ЛеБланк)

Восемь типов эстетики ЛеБланка (Кн. 2) — это **целевая функция** эмоционального дизайна. Они определяют, какие эмоции игра должна

вызывать, и являются входными данными для Reverse MDA-процесса (от желаемого опыта к механикам):

1. **Чувственное** (Sensation) — удовольствие от ощущений. Красота графики, музыка, кинестетика управления, тактильный фидбэк. Эмоции: восторг, восхищение, наслаждение. 2. **Фантазия** (Fantasy) — удовольствие от пребывания в другом мире. Отыгрыш роли, проживание невозможного в реальности. Эмоции: удивление, трепет, ощущение могущества. 3. **Нарратив** (Narrative) — удовольствие от истории и драмы. Сопереживание персонажам, интрига, катарсис. Эмоции: грусть, радость, гнев, надежда. 4. **Вызов** (Challenge) — удовольствие от преодоления трудностей. Рост мастерства, достижение сложной цели. Эмоции: напряжённость, триумф, гордость. 5. **Товарищество** (Fellowship) — удовольствие от связи с другими. Сотрудничество, общение, совместное преодоление. Эмоции: доверие, благодарность, принадлежность. 6. **Открытие** (Discovery) — удовольствие от нахождения нового. Исследование, разгадывание тайн, понимание системы. Эмоции: любопытство, озарение, удовлетворение. 7. **Выражение** (Expression) — удовольствие от самовыражения. Творчество, персонализация, строительство. Эмоции: гордость за создание, уникальность, свобода. 8. **Подчинение** (Submission) — удовольствие от следования правилам и погружения в структурированную деятельность. Медитативный grind, рутина как отдых. Эмоции: спокойствие, сосредоточенность, расслабление.

Для Gidede эта типология — операциональный инструмент: AI получает на вход целевые эстетики (например, «Нарратив + Вызов + Фантазия» для RPG) и подбирает механики, динамики и нарративные структуры, которые их обеспечивают. Обратный MDA-процесс: Aesthetics → Dynamics → Mechanics.

8.13.2. Четыре атрибута веселья (Селлерс)

Селлерс (Кн. 13) углубляет модель ЛеБланка, формулируя четыре атрибута веселья, которые связывают эмоциональный дизайн с нейрохимией и теорией систем:

Вызов (Challenge) — веселье возникает на границе возможностей. Кривая Йеркса-Додсона: оптимальный уровень возбуждения = максимальная производительность. Слишком мало — скука, слишком много — тревога. Вызов — самый прямой путь к эмоциональной вовлечённости: преодоление трудности вознаграждается эндорфинами.

Новизна (Novelty) — мозг вознаграждает обнаружение нового выбросом дофамина. Новизна — конечный ресурс: когда игрок изучил все паттерны, она исчерпывается. Поэтому система должна обеспечивать постоянное

поступление новизны через прогрессию механик, генерацию вариантов и твисты.

Обучение и мастерство (Learning/Mastery) — удовольствие от роста компетентности. Костер формулирует: «веселье — это обучение». Когда игрок овладевает навыком, возникает чувство мастерства; когда мастерство подтверждается сложным вызовом — возникает поток. Серотонин — нейрохимический коррелят статуса и мастерства.

Эмоциональная связь (Emotional Connection) — веселье усиливается, когда оно разделено с другими или связано с нарративом. Окситоцин — нейрохимический коррелят привязанности и доверия. Персонажи, которые вызывают эмпатию; сообщества, которые дают чувство принадлежности; нарративы, которые резонируют с личным опытом — всё это источники эмоциональной связи.

8.13.3. Кривая интереса и фрактальный racing

Шелл (Кн. 1, 2) формализует эмоциональный racing через **кривую интереса** — модель, описывающую, как интерес игрока меняется на протяжении игры. Успешный паттерн: начальный интерес (ожидания) → крючок (ранний захват внимания) → нарастающие пики с отдыхом между ними → кульминация (максимальная точка) → удовлетворяющее завершение. Ключевое свойство: остаточный интерес в конце должен быть выше начального — иначе игрок не захочет продолжать.

Кривая интереса **фрактальна**: она повторяется на трёх масштабах — игра целиком, каждый отдельный уровень, каждый отдельный вызов внутри уровня. Это означает, что эмоциональный racing должен проектироваться рекурсивно: каждый сегмент игры должен иметь свой крючок, свою кульминацию и своё завершение, при этом вкладываясь в общую кривую.

Три фактора интереса по Шеллу: (1) **Изначальный интерес** — некоторые вещи естественно интереснее (опасность > безопасность, фантазия > реальность, необычное > рутинное); (2) **Поэтика подачи** — эстетическое качество преподнесения; (3) **Проецирование** — насколько глубоко игрок проецирует себя в опыт. У игр уникальное преимущество: игрок **является** protagonистом, что максимизирует проецирование.

8.14. Нетекстовый нарратив: сторителлинг через окружение

Нарратив в играх не ограничивается текстом и диалогами. Одно из главных преимуществ видеоигры перед другими медиа — способность рассказывать истории через пространство, механики и эстетику. Андрианова и Яковлева перечисляют пять инструментов нетекстового нарратива, каждый из которых особенно ценен, потому что не прерывает геймплей:

Окружение (environmental storytelling) — расположение объектов в пространстве рассказывает историю. Разбитый лагерь говорит о недавнем побеге; запёртая дверь с царапинами — о том, что кто-то пытался выбраться; пустой детский стульчик среди развалин — о потерянной семье. Кадиков (Кн. 11) формализует это через «говорящие улики» — декоративные элементы, отражающие следы событий, и «фоновое восприятие» — подсознательный анализ деталей, создающий у игрока ощущение живого мира.

Звук и музыка — звуковой дизайн — мощнейший инструмент эмоционального проектирования. Тревожная музыка = опасность; тишина = затишье перед бурей; знакомая мелодия = ностальгия. Звук может рассказывать истории, которые визуал не может: далёкий крик, шаги за стеной, лязг цепи в темноте.

Анимация персонажей — то, как персонаж двигается, говорит о его состоянии без единого слова. Пригнутая поза = усталость; нервный взгляд = тревога; расслабленная походка = уверенность. Для AI-системы: описания анимаций должны входить в библию персонажа.

Освещение и атмосфера — Кадиков (Кн. 11) выделяет три функции освещения: эмоциональная температура (тёплый = безопасность, холодный = опасность), пространственная коррекция (свет направляет движение), идентификация уровня (уникальная световая палитра = запоминаемость). Освещение — самый тонкий инструмент эмоционального дизайна, потому что оно действует подсознательно.

Цветовая палитра — Кадиков формализует три функции цвета: эмоциональная температура (оранжевый = безопасность, синий = одиночество), пространственная коррекция (холодные тона расширяют, тёплые сужают) и идентификация уровня (уникальная палитра = запоминающийся образ). Mirror's Edge (красный + белый), BioShock (арт-деко), Limbo (чёрно-белый) — примеры игр, где цвет — главный нарративный инструмент.

8.15. Narrative Generator Pipeline для Gidede

На основе всех рассмотренных моделей формализуем алгоритм генерации нарратива для AI-системы Gidede. Этот пайплайн решает Пробел 4, выявленный в синтезе (1.3.3): отсутствие модели генерации нарратива от механик.

Шаг 1: Определение типа нарратива

Вход: жанр, целевая аудитория, модель мотивации Йи. Выход: тип нарратива (Embedded / Emergent / Hybrid) и уровень агентивности (High / Low / Zero) для каждого измерения (геймплейного и сюжетного).

Жанр	Тип нарратива	Сюжетная агентивность	Геймплейная агентивность	
-----	-----	-----	-----	RPG Embedded High High
Shooter	Hybrid	Low High	Strategy	Emergent Zero High Adventure
Embedded	Medium	Medium	Sandbox	Emergent Zero Very High Visual
Novel	Embedded	High	Low	

Шаг 2: Проектирование структуры сюжета (для Embedded/Hybrid)

Вход: тип нарратива, уровень агентивности, целевые эстетики. Выход: структура сюжета (линейная / разветвлённая / открытая / кольцевая), количество актов, ключевые сюжетные точки (story beats).

Алгоритм: (1) определить структуру на основе жанра и агентивности; (2) распределить story beats по кривой интереса Шелла; (3) для каждого beat определить тип (кат-сцена / диалог / геймплей / environmental); (4) проверить фрактальную согласованность: каждый beat содержит свою подкривую интереса.

Шаг 3: Создание персонажей

Вход: структура сюжета, целевой тип конфликта, сеттинг. Выход: набор персонажей с полными библиями.

Алгоритм: (1) определить архетипы, необходимые для структуры сюжета (Герой, Наставник, Тень и т.д.); (2) для каждого архетипа назначить позицию на циркумфлексе; (3) проверить полноту: все ли архетипы покрыты? есть ли «пустые места» на сетке отношений?; (4) сгенерировать библию персонажа; (5) проверить арку трансформации: изменяется ли персонаж? достаточно ли глубоко? неожиданно ли?

Шаг 4: Генерация квестов

Вход: story beats, персонажи, механики из MechanicsDB. Выход: квестовые цепочки, привязанные к story beats.

Алгоритм: (1) для каждого story beat определить целевой тип квеста (из матрицы 8×4); (2) выбрать паттерн квеста на основе позиции в игре (начало = линейное усложнение; середина = моральная дилемма; финал = турнир); (3) привязать квест к персонажам и механикам; (4) проверить 4 принципа дизайна побочных квестов: отражает ли основной сюжет? не лучше ли основного? связан ли с прогрессией? вариативен ли?

Шаг 5: Валидация лудонарративного согласования

Вход: механики (из MechanicsDB), нарратив (из Шагов 2-4), персонажи. Выход: отчёт о согласованности на трёх уровнях.

Алгоритм: (1) Уровень 1 — Механика ↔ Тема: проверить, что каждая ключевая тема поддерживается минимум одной механикой; (2) Уровень 2 — Геймплей ↔ Персонаж: проверить, что действия игрока согласуются с нарративным профилем персонажа; (3) Уровень 3 — Прогрессия ↔ Арка: проверить, что кривая мощности согласуется с аркой трансформации персонажа. Для каждого обнаруженного диссонанса предложить конкретное исправление.

Шаг 6: Документирование

Вход: все результаты предыдущих шагов. Выход: нарративная документация.

Типы документов (по Андриановой/Яковлевой): синопсис, расширенный синопсис, сторилайн, библия персонажей, блок-схемы диалогов, таблицы квестов, поэпизодный план. Каждый документ генерируется автоматически из данных пайплайна.

8.16. Интеграция с другими разделами Библии

Нарратив и эмоциональный дизайн не существуют изолированно — они пронизывают все слои пятислойной модели Gidede:

Слой 1 (Фундамент): Определение игры включает «эстетический опыт» как обязательный элемент. Магический круг — пространство, где нарратив обретает значимость. Принцип приоритета исключений — основа для твистов и неожиданных поворотов.

Слой 2 (Механика): Каждая механика должна проверяться на лудонарративное согласование. MechanicsDB должна включать нарративные теги: какие эстетики поддерживает данная механика, какой тип конфликта создаёт, какую роль играет в истории. Языковая метафора (существительные = объекты, глаголы = действия) — мост между механикой и нарративом.

Слой 3 (Динамика): Core Loop — это одновременно геймплейная и нарративная структура. Петли обратной связи могут усиливать нарратив (положительная ОС = ощущение роста) или подрывать его (отрицательная ОС, сводящая на нет нарративные достижения). Эмерджентность — источник эмергентного нарратива.

Слой 4 (Опыт): Эмоциональный дизайн — целевая функция всей системы. Восемь типов эстетики ЛеБланка определяют, что система должна генерировать. Четыре атрибута веселья Селлерса определяют, как оценивать результат. Кривая интереса Шелла определяет pacing.

Слой 5 (Процесс): Итеративный цикл генерации нарратива: сгенерировать → проверить на диссонанс → исправить → повторить. Плейтест нарратива: не спрашивать «понравилась ли история», а наблюдать, какие эмоции испытывает игрок и совпадают ли они с целевыми.

Для Gidede эта интеграция означает, что нарративный модуль — не отдельная надстройка, а **горизонтальный слой**, который взаимодействует со всеми остальными модулями. Генерация концепции должна включать определение целевых эстетик. Генерация механик должна проверяться на лудонарративное согласование. Генерация уровней должна учитывать сторителлинг через окружение. Генерация баланса должна поддерживать арку трансформации. Только при такой интеграции нарратив становится не «прикреплённым» к игре, а **встроенным** в неё — а это и есть условие лудонарративной гармонии.

Глава 9. Монетизация и Retention

> **Раздел Библии геймдизайна:** 2.9 — Монетизация и Retention >

Источники: KB-10 (Монетизация), KB-13 (Опыт и мотивация), KB-05 (Экономика), KB-03 (MDA Framework), KB-14 (Валидация) > **Книги:** Гэри (Кн. 10), Селлерс (Кн. 13), Гэззэуэй (Кн. 9), Зубек (Кн. 6), Адамс/Дорманс (Кн. 4), Шрайбер (Кн. 7), Шелл (Кн. 1, 2), Фуллертон (Кн. 5)

9.1. Философия монетизации: деньги как часть дизайна

Монетизация в играх — не финансовая надстройка, добавляемая после завершения дизайна, а неотъемлемая часть геймдизайна, которая должна проектироваться одновременно с механиками, экономикой и прогрессией. Гэри (Кн. 10) формулирует это предельно ясно: «механики должны быть тесно связаны со стратегией монетизации». Это не циничный призыв к коммерциализации, а прагматичное признание того факта, что способ, которым игрок платит за игру, определяет его отношение к ней, его ожидания и его поведение. Одноразовая покупка создаёт одни отношения между игроком и игрой; подписка — принципиально другие; свободная игра с микротранзакциями — третьи. Каждый тип монетизации формирует свой «контракт» с игроком, и нарушение этого контракта (например, премиумная игра с микротранзакциями) вызывает *resentment* и отток.

Гэри предлагает ключевой вопрос для валидации любой монетизационной модели: «Будь я целевой аудиторией, хотел бы я заплатить за возможность сыграть в эту игру?» Если ответ отрицательный — проблема не в цене, а в дизайне. Хорошо монетизируемая игра — это игра, которую игрок хочет поддержать финансово, потому что она доставляет ему ценность. Принцип: «зарабатывать на играх — не стыдно; больше денег = больше игр и больше игроков». Это утверждение не оправдывает эксплуативные практики, но устанавливает здоровую рамку: монетизация — это обмен ценности на ценность.

Селлерс (Кн. 13) подходит к монетизации с позиции системного мышления: экономика игры и реальная экономика монетизации — это **две взаимосвязанные системы**, которые должны быть сбалансированы друг с другом. Если игровая экономика генерирует слишком много ресурсов (инфляция), то платные ускорения теряют ценность. Если реальная экономика слишком агрессивна (*pay-to-win*), то игровая экономика теряет смысл — зачем зарабатывать ресурсы в игре, если можно их купить? Дизайнер должен управлять обоими потоками: *faucet/drain* внутриигровой экономики и *faucet/drain* реальной монетизации, обеспечивая их согласованность.

Для Gidede это означает: модуль монетизации не может быть отдельным «добавлением» к основной системе генерации. Он должен быть интегрирован в пайплайн проектирования: когда AI генерирует механики, экономику и прогрессию, он одновременно генерирует и монетизационную структуру,

согласованную с ними. Выбор модели монетизации — такое же архитектурное решение, как выбор типа Core Loop или кривой прогрессии.

9.2. Модели монетизации: таксономия и выбор

Гэзэуэй (Кн. 9) представляет наиболее формализованную таксономию моделей монетизации, которую AI-система может использовать как каталог с критериями выбора для конкретного проекта.

9.2.1. Шесть базовых моделей

Одноразовая покупка (Premium/B2P). Игрок платит один раз и получает полный доступ к игре. Это старейшая модель, создающая наиболее «честные» отношения с игроком: заплатил — играй. Плюсы: высокий средний чек, завершённость проекта, отсутствие давления на дизайн со стороны микротранзакций, высокая лояльность аудитории. Минусы: нет повторного дохода, единственная возможность произвести впечатление (первый запуск — всё или ничего), высокий порог входа для новых игроков, зависимость от скидок и распродаж. Классические примеры: The Witcher 3, God of War, почти все AAA-игры для консолей. Для AI-системы: премиумная модель означает, что весь фокус дизайна — на качестве первого впечатления и полноте контента. Прогрессия должна быть завершённой, контент — достаточным для ощущения «стоимости», повторная играемость — встроенной, но не обязательной.

Покупка расширений (DLC/Expansions). Базовая игра приобретается отдельно, а дополнительный контент — за дополнительную плату. Модель, сочетающая преимущества премиума (высокое качество базового продукта) с элементами повторного дохода. Плюсы: возврат игроков при выходе DLC, возможность исправлять баланс и расширять вселенную, сегментация аудитории (не все покупают все DLC). Минусы: меньшая аудитория для каждого следующего DLC (только владельцы базовой игры), diverted ресурсы разработки (команда делится между поддержкой и созданием нового контента), риск фрагментации игрового сообщества. Примеры: Civilization VI (система DLC и Season Pass), Paradox Games (модель постоянных расширений). Для AI-системы: модель DLC означает, что базовая игра проектируется как расширяемая платформа — система должна быть модульной, механики — расширяемыми, экономика — адаптируемой к новому контенту.

Микротранзакции (IAP — In-App Purchases). Игра распространяется бесплатно (или по низкой цене), а доход генерируется через покупки виртуальных товаров и услуг внутри игры. Это доминирующая модель в

мобильном геймдеве и всё более распространённая в ПК/консольных играх. Плюсы: постоянный доход, низкий порог входа для игроков, гибкость ценообразования, возможность монетизировать разных игроков по-разному (киты, дельфины, минно). Минусы: восприятие как pay-to-win, риск превращения игры в «дойную корову», постоянное давление на дизайн в сторону монетизации, этические проблемы (лутбоксы, тёмные паттерны). Примеры: Genshin Impact, Clash of Clans, Fortnite. Для AI-системы: микротранзакции требуют глубокой интеграции с игровой экономикой — каждая покупка должна ощущаться как ценность, а не как неизбежность.

Подписка (Subscription). Игроки платят регулярный взнос за доступ к игре или к премиум-контенту. Плюсы: стабильный предсказуемый доход, предсказуемое поведение игроков (регулярные сессии), стимул к постоянному обновлению контента. Минусы: высокие ожидания игроков (контент должен обновляться регулярно), порог входа для новых игроков (барьер регулярной оплаты), зависимость от удержания (один плохой месяц = массовый отток). Примеры: World of Warcraft, Final Fantasy XIV, Xbox Game Pass. Для AI-системы: подписная модель требует проектирования с фокусом на долгосрочное удержание и регулярный контент — прогрессия должна быть бесконечной, контент — обновляемым, социальные механики — глубокими (гильдии, рейды).

Рекламная модель (F2P + Ads). Игра бесплатна, доход генерируется через рекламу: баннеры, interstitial, rewarded video. Плюсы: нулевой порог входа, пассивный доход (не требует от игроков активных покупок), rewarded video — наименее раздражающий формат. Минусы: раздражение игроков (особенно баннеры и interstitial), низкий ARPU по сравнению с IAP, зависимость от объёма трафика, негативное влияние на иммерсию. Примеры: мобильные casual-игры, гиперказуальные игры. Для AI-системы: рекламная модель требует проектирования сессии с «рекламными паузами» — rewarded video интегрируется как выбор игрока (посмотри рекламу = получи награду), а не навязывается.

Гибридная модель. Комбинация нескольких моделей: F2P + IAP + Battle Pass + косметические покупки + опциональная подписка. Это наиболее распространённая модель в современных успешных играх: она диверсифицирует доход и позволяет монетизировать разные сегменты аудитории. Примеры: Fortnite (бесплатная игра + Battle Pass + косметика), Genshin Impact (F2P + гача + Battle Pass + подписка), Path of Exile (F2P + косметика + табы хранилища). Для AI-системы: гибридная модель — рекомендуемая по умолчанию, так как она даёт максимальную гибкость и минимум рисков, связанных с зависимостью от одного источника дохода.

9.2.2. Матрица «Жанр → Модель монетизации»

Выбор модели монетизации зависит от жанра, платформы и целевой аудитории. Гэззэуэй формализует это в шаблон профиля аудитории, включающем тип монетизации как входной параметр. На основе анализа индустриальных паттернов:

Жанр	Рекомендуемая модель	Обоснование
AAA Action/Adventure	Premium + DLC	Высокие ожидания качества, один сюжет
RPG (открытый мир)	Premium + DLC или F2P + IAP	Зависит от платформы и аудитории
ММО	Подписка + IAP	Бесконечный контент, социальные обязательства
Мобильная casual	F2P + Ads + IAP	Низкая готовность платить, массовая аудитория
Мобильная midcore	F2P + IAP + Battle Pass	Готовность платить за прогресс, регулярная игра
Стратегия (PC)	Premium + DLC	Глубокий геймплей, нишевая аудитория
MOBA / Hero Shooter	F2P + косметика	Конкуренция, социальная природа
CCG / Gacha	F2P + гача	Коллекционирование = естественная монетизация
Roguelike	Premium или F2P + IAP	Зависит от платформы и глубины
Симулятор / Tycoon	Premium или F2P + IAP + Ads	Зависит от платформы

Для Gidede эта матрица — входной инструмент: AI получает жанр и платформу и предлагает рекомендуемую модель монетизации с обоснованием, затем пользователь может скорректировать выбор.

9.3. Виртуальные экономики и монетизация

Связь между виртуальной экономикой и монетизацией — один из самых сложных аспектов геймдизайна, требующий одновременного управления двумя экономическими системами: внутриигровой (виртуальные ресурсы, их производство и потребление) и реальной (деньги игроков, их готовность платить и воспринимаемая ценность). Адамс и Дорманс (Кн. 4) и Селлерс (Кн. 13) предоставляют формальный аппарат для этой задачи.

9.3.1. Двойная валюта: soft и hard

Большинство F2P-игр используют систему из двух или более валют, разделённых по способу получения:

Soft-валюта — зарабатывается через игровой процесс (геймплейные faucet'ы). Золото, монеты, кристаллы маны — всё, что игрок получает, просто играя. Soft-валюта — основа внутриигровой экономики, обеспечивающая

ощущение прогресса и вознаграждения за усилия. Она должна быть достаточной для базового прогресса, но недостаточной для всего контента — иначе монетизация не работает. Ключевой принцип: игрок, использующий только soft-валюту, должен иметь полноценный игровой опыт, но с естественным трением (friction) в темпе прогрессии.

Hard-валюта — приобретается преимущественно за реальные деньги (реальные faucet'ы). Гемы, кристаллы, премиум-монеты — всё, что покупается. Hard-валюта обеспечивает доступ к ускорению прогресса, эксклюзивному контенту и косметике. Ключевой принцип: hard-валюта не должна быть единственным путём к контенту, критичному для геймплея (иначе — pay-to-win), но может быть путём к контенту, ускоряющему или украшающему опыт.

Соотношение soft/hard — ключевое проектное решение. Если soft-валюты слишком много — монетизация не работает (зачем платить, если всё зарабатывается?). Если слишком мало — игрок чувствует себя обделённым и уходит. Зубек (Кн. 6) формализует это через циклы конверсии: прибыльный цикл (soft-валюты становятся больше) vs неприбыльный (soft-валюты становятся меньше). Идеал для F2P: **микроцикл прибыльный** (игрок постоянно чувствует награду), но **макроцикл сбалансированный** (долгосрочная прогрессия требует patience или оплаты).

9.3.2. Faucets и Drains: инженерия экономики монетизации

Селлерс (Кн. 13) формализует экономику через Sources (источники/faucets) и Sinks (стоки/drains). Для монетизации эта модель применяется дважды:

Виртуальные faucet'ы и drain'ы: - Faucet: награды за квесты, дроп с врагов, ежедневные бонусы, пассивный доход - Drain: улучшения, крафт, ремонт, налоги, ремонт, расходники

Реальные faucet'ы и drain'ы: - Faucet: покупки hard-валюты, Battle Pass, подписки, косметика - Drain: скидки, бесплатные бонусы, мероприятия с наградами (для игрока это faucet, для разработчика — drain)

Баланс faucet/drain определяет «давление» на кошелёк игрока. Если виртуальные faucet'ы перекрывают drain'ы (инфляция) — монетизация не работает. Если drain'ы слишком агрессивны (стагнация) — игрок уходит. Адамс и Дорманс (Кн. 4) формализуют это через паттерн **Dynamic Friction**: сток, скорость которого зависит от объёма ресурса — чем больше у игрока soft-валюты, тем больше расход (например, стоимость улучшений растёт экспоненциально). Это создаёт естественный тормоз инфляции и мягко направляет к монетизационным предложениям.

9.3.3. Paywall и трение

Paywall — точка в игровом процессе, где прогресс без оплаты становится затруднительным или невозможным. Жёсткий paywall (без оплаты — дальше нельзя) создаёт resentment и отток. Мягкий paywall (без оплаты — можно, но значительно медленнее) — стандарт индустрии, но требует тонкой настройки: слишком мягкий — не монетизирует, слишком жёсткий — отталкивает.

Шрайбер (Кн. 7) формализует это через **cost-power curves**: каждая способность в игре имеет «стоимость» (сколько времени или денег нужно инвестировать) и «мощность» (насколько она эффективна). В хорошо сбалансированной F2P-игре кривая стоимости для бесплатного игрока логарифмическая (быстрый начальный рост, затем замедление), а для платящего — близкая к линейной (равномерный рост за счёт оплаты). Разница между этими двумя кривыми — это **ценностное предложение монетизации**: «заплати, чтобы пройти этот участок быстрее».

Для Gidede формализуется алгоритм проектирования paywall: 1. Определить кривую прогрессии для бесплатного игрока (базовый темп) 2. Определить кривую прогрессии для платящего игрока (ускоренный темп) 3. Разница между кривыми = ценность оплаты 4. Если разница слишком мала — монетизация не работает 5. Если разница слишком велика — игра воспринимается как P2W 6. Рекомендуемый диапазон: платящий игрок продвигается в 1.5–3 раза быстрее бесплатного

9.4. Retention: метрики и модели

Retention (удержание) — способность игры возвращать игрока снова и снова. Это фундаментальная метрика здоровья игры: без удержания не работает ни одна модель монетизации (премиум не получит позитивные отзывы и DLC-продажи, F2P не конвертирует игроков в платящих, подписка не продлится). Селлерс (Кн. 13) систематизирует метрики удержания, которые для Gidede становятся целевыми функциями оптимизации дизайна.

9.4.1. Ключевые метрики удержания

Retention Rate (D1/D7/D30) — процент игроков, вернувшихся в игру через 1, 7 и 30 дней после установки. Это базовая «температура» игры: D1 показывает качество первого впечатления, D7 — устойчивость Core Loop, D30 — долгосрочную ценность.

Индустриальные бенчмарки (мобильные игры): - Плохо: D1 < 25%, D7 < 10%, D30 < 3% - Средне: D1 25–35%, D7 10–20%, D30 3–8% - Хорошо: D1 35–45%, D7 20–30%, D30 8–15% - Отлично: D1 > 45%, D7 > 30%, D30 > 15%

DAU/MAU (Daily/Monthly Active Users) — количество уникальных пользователей в день/месяц. Отношение DAU/MAU («липкость» — stickiness) показывает, какую часть месячной аудитории игра удерживает ежедневно: 20% — средне, 50%+ — отлично (ежедневная привычка).

Средняя длительность сессии — время, которое игрок проводит за одной сессией. Зависит от жанра: 3–5 минут для гиперказуальных, 15–30 минут для midcore, 30–60+ минут для core-игр. Критически важна не абсолютная длительность, а **соответствие жанровым ожиданиям**: если midcore-игра даёт сессию 3 минуты — дизайн не соответствует аудитории.

Частота сессий — сколько раз в день/неделю игрок возвращается. Высокая частота (2–5 раз в день) характерна для мобильных F2P с короткими сессиями; низкая (2–3 раза в неделю) — для ПК/консольных игр с длинными сессиями.

Churn Rate (отток) — процент игроков, покинувших игру за период. $\text{Churn} = 1 - \text{Retention}$. Обратная метрика: чем ниже, тем лучше. Ключевой инсайт: churn не линейен — большинство игроков уходят в первые 3 дня (D1 retention определяет судьбу игры), затем отток замедляется. Если игрок остался на D7, вероятность его ухода на D30 значительно ниже.

9.4.2. Воронка удержания (Retention Funnel)

Селлерс описывает воронку удержания как последовательность этапов, на каждом из которых игра теряет часть игроков:

``` Установка → FTUE (First-Time User Experience) → Core Loop → Mid-Game → End-Game → Loyal Player 100% D1: 30–45% D7: 15–30% D30: 5–15% D90: 2–8% 1–3% ```

Каждый этап воронки имеет свои причины оттока и свои инструменты удержания:

**FTUE → D1** (потеря 55–70%): главные причины — непонятный интерфейс, скучное начало, технические проблемы, несоответствие ожиданиям (misleading реклама). Инструменты: плавный туториал, «крючок» в первые 30 секунд, минимизация фрикции при установке, соответствие рекламных материалов реальному геймплею.



**D1 → D7** (потеря 30–55% оставшихся): главные причины — отсутствие глубины после первого впечатления, повторяющийся геймплей, отсутствие социальной привязки. Инструменты: раскрытие глубины Core Loop, введение новых механик, социальные функции (гильдии, друзья), ежедневные бонусы.

**D7 → D30** (потеря 40–70% оставшихся): главные причины — исчерпание контента, отсутствие долгосрочных целей, ощущение «гринда». Инструменты: эндгейм-контент, сезонные события, PvP/рейтинги, коллекционные механики.

**D30+ → Loyal** (потеря 60–80% оставшихся): главные причины — выгорание, появление конкурентных игр, жизненные обстоятельства. Инструменты: LiveOps (события, сезоны), социальное сообщество, контент-обновления, Nostalgia-механики.

### 9.4.3. Модель когорт

Селлерс подчёркивает важность когортного анализа: игроки, установившие игру в разные периоды, ведут себя по-разному. Когорта — группа игроков, присоединившихся в один период (обычно один день или одна неделя). Сравнение когорт позволяет выявить: (1) влияние обновлений на удержание (когорта до обновления vs когорта после), (2) сезонные эффекты (когорта праздников vs когорта обычных дней), (3) качество трафика (когорта из рекламной кампании А vs когорта из кампании В). Для Gidede когортный анализ — инструмент валидации: AI может симулировать когорты с разными профилями мотивации и прогнозировать удержание для каждой.

---

## 9.5. Нейрохимия удержания: почему игроки возвращаются

---

Селлерс (Кн. 13) вводит нейрохимическую модель вовлечения, которая для AI-системы становится набором «рычагов» управления удержанием. Каждая нейрохимическая система связана с определённым типом геймдизайнерских решений и определёнными временными масштабами.

### 9.5.1. Четыре нейрохимических системы

**Дофамин (dopamine)** — нейромедиатор предвкушения, а не награды. Это критическое различие: дофамин выделяется не когда игрок получает награду, а когда он ожидает её. Слот-машина максимально дофаминергична не в момент выигрыша, а во время вращения барабанов — когда исход ещё

неизвестен. Дофамин отвечает за мотивацию «попробовать ещё раз» и работает на коротких временных масштабах (секунды–минуты). Для геймдизайна это означает: система наград должна быть **непредсказуемой** — переменное отношение (variable ratio schedule) по Зубку (Кн. 6) генерирует максимальный дофаминовый ответ и создаёт наиболее стойкую поведенческую реакцию. Каждое убийство монстра в Diablo — дофаминовый микровсплеск: «а вдруг выпадет легендарка?»

**Серотонин (serotonin)** — нейромедиатор статуса и социального доминирования. Серотонин выделяется, когда игрок чувствует себя значимым, успешным, признанным. Таблицы лидеров, ранги, достижения, редкие титулы — всё это серотониновые триггеры. Серотонин работает на средних временных масштабах (минуты–часы) и отвечает за мотивацию «быть лучшим». Для геймдизайна это означает: игра должна предоставлять **видимые маркеры статуса**, которые другие игроки могут распознать. Редкий маунт в WoW — не просто транспорт, а серотониновый сигнал: «я достиг чего-то, что большинству недоступно».

**Окситоцин (oxytocin)** — нейромедиатор социальной связи и доверия. Окситоцин выделяется при кооперации, взаимопомощи, принадлежности к группе. Гильдии, кооперативные рейды, механики взаимной помощи — всё это окситоциновые триггеры. Окситоцин работает на долгих временных масштабах (часы–недели) и создаёт самую стойкую форму привязанности: **социальную**. Игроки, у которых есть друзья в игре, удерживаются в 3–5 раз дольше, чем одиночные игроки. Для геймдизайна это означает: социальные механики — самый мощный инструмент долгосрочного удержания, и они должны проектироваться с самого начала, а не добавляться постфактум.

**Эндорфины (endorphins)** — нейромедиаторы преодоления и обезболивания. Эндорфины выделяются при преодолении трудностей, физическом или ментальном напряжении, после достижения сложной цели. Сложные боссы, пермадейт, хардкорный контент — эндорфиновые триггеры. Эндорфины работают на средних временных масштабах (минуты–часы) и создают глубокое чувство удовлетворения. Для геймдизайна это означает: игра должна предоставлять **достижимые, но трудные вызовы**, которые дают ощущение триумфа при преодолении. Отсутствие вызова = отсутствие эндорфинов = скука.

### 9.5.2. Временные масштабы и наслаивание петель

Селлерс описывает 6 временных масштабов интерактивных петель, каждый из которых связан с определённой нейрохимической системой:

| Масштаб | Время | Нейрохимия | Геймдизайн | |-----|-----|-----|-----| |  
Action/Feedback | мс-с | Дофамин | Моментальный геймплей, визуальная/  
звуковая обратная связь | | Short-term Cognitive | с-мин | Дофамин +  
Эндорфины | Тактические решения, микросражения | | Long-term Cognitive |  
мин-ч | Эндорфины + Серотонин | Стратегические решения, прохождение  
контента | | Social | ч-дни | Окситоцин + Серотонин | Кооперация, PvP, гильдии  
| | Emotional | ч-недели | Окситоцин + Эндорфины | Эмоциональная  
привязанность, нарратив | | Cultural | недели-месяцы | Окситоцин + Серотонин  
| Культурная идентификация, сообщество |

Петли **наслаиваются** друг на друга: быстрые петли внутри медленных. «30 секунд веселья» (от Halo, через Шрайбера, Кн. 7) — это 3-секундная петля внутри 30-секундной внутри 5-минутной внутри 20-минутной. Каждая петля — микроподкрепление, удерживающее игрока до завершения следующей, более крупной петли. Наиболее устойчивое удержание возникает, когда все 6 масштабов активны одновременно: моментальный геймплей (дофамин) поддерживает тактические решения, которые ведут к стратегическим целям, которые реализуются в социальном контексте, который создаёт эмоциональную привязанность, которая формирует культурную идентификацию с игрой.

---

## 9.6. Мотивация как фундамент удержания

---

Удержание невозможно без понимания, почему игроки играют. Мотивация — фундамент, на котором строится весь Retention-дизайн. Если игра не удовлетворяет базовую мотивацию игрока, никакие нейрохимические трюки и экономические ухищрения не заставят его вернуться.

### 9.6.1. Внутренняя и внешняя мотивация

Зубек (Кн. 6) проводит критическое различие между внутренней и внешней мотивацией. **Внутренняя мотивация** — деятельность сама по себе приносит удовольствие: танцевать, исследовать, решать головоломки, соревноваться. Игрок играет ради самого процесса. **Внешняя мотивация** — интересуется результатом, а не процесс: grind ради награды, прокачка ради уровня, фарм ради лута. Игрок играет ради того, что получит.

Идеал — сочетание обеих: внутреннее удовольствие от процесса + внешняя «морковка» за достижение. Если только внешняя мотивация — игра превращается в рутину (grind ради гринда). Если только внутренняя — игра может не удерживать в долгосрочной перспективе (когда внутреннее

удовольствие притупляется). Гэри (Кн. 10) формализует это через 5 типов мотивации, каждый из которых имеет внутреннюю и внешнюю составляющую:

| Тип | Внутренняя | Внешняя | Инструмент удержания |  
|-----|-----|-----|-----| | Победа | Соревнование как процесс |  
Рейтинг, ранг, достижения | Матчмейкинг равных, таблицы лидеров | |  
Самовыражение | Отыгрыш роли как творчество | Индивидуализация,  
кастомизация | Система скинов, эмблем, классов | | Связь | Социализация как  
ценность | Бонусы за кооперацию | Гильдии, рейды, мьютуал-хелп | |  
Погружение | Исследование мира | Сюжетные награды | Открытый мир,  
квесты, лор | | Развитие | Обучение как процесс | Уровни, навыки,  
разблокировка | Дерево навыков, прогрессия |

### 9.6.2. Модель Йи: 12 типов мотивации

Зубек (Кн. 6) приводит модель мотивации Йи (Yee) — наиболее научно обоснованную типологию игровых мотиваций, основанную на эмпирических исследованиях тысяч игроков. 12 типов группируются в 3 кластера:

**Кластер «Экшен-социальный»** (связан с экстраверсией): - Разрушение (Destruction) — стремление к хаосу, разрушению, агрессии - Возбуждение (Excitement) — стремление к адреналину, острым ощущениям, быстрым действиям - Состязание (Competition) — стремление победить других, доказать превосходство - Общение (Socializing) — стремление к общению, кооперации, принадлежности

**Кластер «Мастерство-достижения»** (связан с сознательностью): - Вызов (Challenge) — стремление к трудным задачам, требующим мастерства - Стратегия (Strategy) — стремление к планированию, оптимизации, глубокой тактике - Завершение (Completion) — стремление собрать всё, достичь 100%, закрыть все квесты - Влияние (Power) — стремление к контролю, доминированию, управлению другими

**Кластер «Погружение-творчество»** (связан с открытостью): - Фантазия (Fantasy) — стремление быть кем-то другим, жить в другом мире - Сюжет (Story) — стремление к нарративной глубине, персонажам, драме - Дизайн (Design) — стремление к созиданию, кастомизации, творчеству - Открытие (Discovery) — стремление к исследованию, нахождению скрытого, сюрпризам

Ключевой инсайт Йи: игры не позволяют «убежать от себя», а наоборот — усиливают выражение реальных черт личности. Игры, в которые мы играем, отражают то, что мы находим привлекательным. Для Gidede это означает: при проектировании удержания AI должен знать мотивационный профиль целевой

аудитории и гарантировать, что для каждого из доминирующих мотивов в игре есть соответствующий контент и механика. Если целевая аудитория мотивирована «Завершением» — нужны коллекции и достижения; если «Стратегией» — нужны сложные экономические системы; если «Фантазией» — глубокий сеттинг и нарратив.

### 9.6.3. Режимы подкрепления и удержание

Зубек (Кн. 6), опираясь на оперантное научение Скиннера, формализует четыре режима подкрепления, каждый из которых по-разному влияет на удержание:

**Фиксированное отношение (Fixed Ratio)** — награда за каждые N действий. Каждые 10 убитых монстров = сундук. Предсказуемо, создаёт ритм, но при прекращении награды поведение быстро затухает. Подходит для: крафта, повторяющихся заданий, «убей 10 кабанов».

**Фиксированный интервал (Fixed Interval)** — награда каждые T секунд. Ежедневный бонус, обновление магазина каждые 24 часа. Создаёт паттерн возвращения (прихожу раз в день за бонусом), но не мотивирует играть дольше. Подходит для: ежедневных наград, регулярных событий.

**Переменное отношение (Variable Ratio)** — награда в среднем за N действий, но непредсказуемо. Может выпасть с 1-го монстра, может с 20-го. Наиболее стойкая поведенческая реакция — принцип слот-машин. Игрок продолжает «крутить», потому что «следующий раз может быть выигрышным». Это наиболее мощный инструмент удержания, но и наиболее этически проблемный (лежит в основе лутбоксов и гачи). Подходит для: лута, гачи, редких дропов.

**Переменный интервал (Variable Interval)** — награда через непредсказуемые промежутки. Случайные мировые события, внезапные вторжения. Создает ощущение «живого мира», но менее стойкая реакция, чем переменное отношение. Подходит для: живых событий, редких появлений.

Для Gidede: при генерации систем наград AI должен комбинировать все четыре режима, обеспечивая: предсказуемую базовую награду (фиксированное отношение) + ежедневные ритуалы (фиксированный интервал) + эмоциональные пики от непредсказуемых наград (переменное отношение) + ощущение живого мира (переменный интервал). Баланс между предсказуемостью и сюрпризом — ключ к долгосрочному удержанию.

---

## 9.7. Три проблемы вовлечения и их решение

---

Гэри (Кн. 10) формулирует три ключевых проблемы вовлечения, каждая из которых напрямую связана с удержанием. Для Gidede это — диагностический фреймворк: если Retention-метрики падают, AI должен определить, какая из трёх проблем является причиной, и предложить соответствующее решение.

### 9.7.1. Проблема 1: Слишком много даунтайма

Даунтайм — время, когда игрок ничего не делает (ждёт хода, ждёт респауна, фармит без вызова). Даунтайм убивает ощущение агентивности и создаёт скуку. Решения:

- **Уменьшить ожидание:** дать игроку действовать в чужой ход (система «горячих» ресурсов, как в Clash Royale); параллельные действия вместо последовательных; автоматизация рутинных операций
- **Уменьшить grind:** 5 кабанов вместо 10; награда посреди квеста, а не только в конце; награда за процесс, а не только за результат
- **Заполнить даунтайм смыслом:** мини-игры во время ожидания; социальное взаимодействие; лор и environmental storytelling

### 9.7.2. Проблема 2: Слишком высокая или низкая сложность

Кривая сложности — ключевой параметр удержания, непосредственно связанный с теорией потока (Csikszentmihalyi, через Кн. 2, 5, 6, 13). Слишком просто — скука; слишком сложно — фрустрация. Канал потока — диапазон, где вызов соответствует навыку. Решения:

- **Прямая настройка:** выбор уровня сложности; динамическая настройка сложности (DDA); гандикап (только в казуальных играх)
- **Матчмейкинг:** система Эло/Глико/TrueSkill (Кн. 7) — подбор соперников примерно равного уровня; при правильном матчмейкинге ~50% побед = оптимальное удержание
- **Личные цели и «маленькие победы»:** статистика, достижения, мини-цели — даже при проигрыше игрок получает микро-награды, поддерживающие мотивацию
- **«Вдвое больше, вдвое меньше»** (Гэри): при калибровке сложности — не подкручивать на 10%, а радикально ( $\times 2$  или  $/2$ ) — быстрее найти золотую середину

### 9.7.3. Проблема 3: Игрок перестаёт верить в победу

Ощущение безнадёжности — смертельный приговор для удержания. Если игрок чувствует, что не может победить (или хотя бы достичь значимого прогресса), он уходит. Гэри предлагает четыре инструмента:

**Механика отыгрыша (catch-up mechanic)** — возможность вернуться из проигрышной позиции. Больше очков в поздних раундах; каждый этап = мини-партия с возможностью победы; компенсирующие бонусы для отстающих. Пример: Mario Kart — отстающие получают лучшие предметы.

**Непредсказуемое завершение** — неизвестно, когда игра закончится = постоянное напряжение. Если игрок знает, что исход решён, но матч продолжается — это мучительно. Если момент завершения неизвестен — даже отстающий сохраняет надежду. Пример: Ra — аукцион с неизвестным концом.

**Скрытая информация** — скрытые победные очки; нельзя точно просчитать итог. Если лидер очевиден — остальные теряют мотивацию. Если позиция неясна — все играют до конца. Примеры: скрытые очки в Settlers of Catan, секретные задания в Cosmic Encounter.

**Отрицательная обратная связь** — отстающие получают лучшие бонусы; саморегуляция игроков (отказ торговать с лидером). Адамс и Дорманс (Кн. 4) формализуют это через паттерн **Stopping Mechanism**: условие, которое останавливает усиливающую петлю до того, как она станет runaway. В контексте удержания: отрицательная ОС предотвращает доминирование одного игрока и сохраняет конкуренцию.

---

## 9.8. Жизненный цикл игрока: от FTUE до лояльности

---

Удержание — не единый процесс, а последовательность фаз, каждая из которых имеет свои дизайнерские требования и свои «места утечки» игроков. Концепция жизненного цикла игрока (player lifecycle) интегрирует все ранее описанные модели в единый пайплайн проектирования удержания.

### 9.8.1. Фаза 1: Привлечение и FTUE (0-15 минут)

Первые минуты игры — самый критический период. Селлерс описывает «воронку приобретения», где каждый клик теряет ~50% игроков: от просмотра рекламы до установки — 50% потерь; от установки до первого



запуска — ещё 50%; от первого запуска до завершения tutorials — ещё 50%.  
Итог: от 1000 кликов по рекламе до завершения tutorials доходят ~125 игроков.

FTUE (First-Time User Experience) — совокупность первого впечатления игрока.  
Ключевые принципы проектирования FTUE:

- **«Крючок» в первые 30 секунд:** принцип «30 секунд веселья» (Шрайбер, Кн. 7, от Halo) — минимально значимая FUN-активность, мотивирующая продолжать. Игрок должен ~~опытаться~~ что-то увлекательное в первые 30 секунд, даже если это упрощённая версия основного геймплея - **Минимум текста, максимум действия:** не рассказывать, как играть — давать играть. Интерактивный tutorial вместо текстовых инструкций. Бонд (Кн. 17): 7 методов косвенного руководства — научить через дизайн уровня, а не через текст - **Один навык за раз:** пошаговое раскрытие сложности (Гэри: «1-2 новые способности за раз», WoW: одна способность за уровень). Когнитивная перегрузка в первые минуты = отток - **Мгновенная награда за действие:** первая награда — в первые 60 секунд, даже если она тривиальна. Дофаминовый «крючок» — показать, что игра вознаграждает

### 9.8.2. Фаза 2: Освоение Core Loop (15 минут - 7 дней)

После FTUE задача — перевести игрока от «попробовал» к «освоил». Это фаза, где Core Loop (Глава 4 Библии) должен раскрыть свою глубину. Ключевые принципы:

- **Последовательное раскрытие механик:** «Музыка механик» Роджерса (Кн. 8) — 7-шаговая последовательность: движение → +1 механика → +2 + комбо → +хазард → +враги → комбо → инверсия. Каждая новая механика — дофаминовый всплеск новизны - **Знакомые паттерны + новизна:** Гэри — «изменение контекста» — повторное использование знакомой механики в новом контексте (Portal: одна порталная пушка, десятки применений). Новизна без необходимости учить новые правила - **Видимый прогресс:** полоска опыта, счётчик достижений, «раскрытие» карты — все индикаторы, подтверждающие, что игрок движется вперёд. Серотониновое подкрепление через маркеры статуса

### 9.8.3. Фаза 3: Средняя игра и социальная привязка (7-30 дней)

После освоения Core Loop задача — углубить вовлечённость через социальные механики и долгосрочные цели. Ключевые принципы:



- **Социальные механики:** гильдии, кооперативные рейды, PvP-рейтинги, таблицы лидеров. Окситоциновая привязка — самый мощный драйвер удержания на этом этапе. Игрок возвращается не ради игры, а ради людей - **Долгосрочные цели:** «путь к легенде» (30-дневные задания), сезонные награды, достижения с длительными критериями. Цели, которые требуют недель, а не минут, создают горизонт ожидания - **Регулярные события:** еженедельные турниры, ротация режимов, ограниченные по времени квесты. Переменный интервал (Variable Interval) — мир игры «живой», в нём всегда происходит что-то новое

#### 9.8.4. Фаза 4: Эндгейм и лояльность (30+ дней)

Эндгейм — фаза, где основное содержание пройдено, и задача — удержать игрока через контент, генерируемый системами (а не сценаристами). Селлерс (Кн. 13) формализует это через противопоставление **Content-driven** vs **Systemic** дизайна: контентный подход (ручное создание уровней и квестов) дорог и конечен; системный подход (эмерджентный контент из взаимодействия механик) масштабируем и бесконечен.

Инструменты эндгейм-удержания: - **Эмерджентный геймплей:** PvP с метой, которая постоянно эволюционирует; экономика с рыночными колебаниями; социальные системы с политическими интригами - **UGC (User-Generated Content):** редакторы уровней, моддинг, творческие режимы. Игроки создают контент друг для друга — бесконечный источник новизны - **Сезонный контент:** Battle Pass, сезонные лиги, ограниченные события. Создают «FOMO» (Fear Of Missing Out) — ещё один драйвер возвращения - **Мета-игра:** стратегическое планирование за пределами сессии; теорикрафт; сообщества вокруг игры. Культурный масштаб (недели-месяцы) — самая стойкая форма удержания

---

### 9.9. LiveOps: операции как часть дизайна

---

LiveOps (Live Operations) — практика постоянного управления живой игрой через обновления, события и коммуникацию. Для F2P-игр LiveOps — не опция, а необходимость: без регулярных обновлений Retention неизбежно падает. Для Gidede LiveOps — это проектируемая система, а не реактивная практика: AI должен генерировать план LiveOps одновременно с дизайном игры.

### 9.9.1. Типы LiveOps-активностей

**Сезоны** — крупные периодические обновления (обычно 1–3 месяца), привносящие новый контент, механики и нарратив. Сезоны создают «архитектуру ожидания» — игрок знает, что через N недель будет что-то новое, и это знание само по себе удерживает. Сезонная структура создаёт макро-петлю: каждый сезон — мини-арка с началом, развитием и кульминацией, а переход между сезонами — «крючок» для возвращения ушедших игроков.

**Battle Pass** — система прогрессии с двумя треками (бесплатный и платный), ограниченная по времени. Battle Pass — один из наиболее успешных монетизационных инструментов, потому что он продаёт не мощь (P2W), а **темп прогрессии и эксклюзивность**. Платящий игрок не становится сильнее — он получает косметические награды быстрее. Ключевой принцип Battle Pass: ценность должна ощущаться выше цены (перечень наград виден заранее, их суммарная стоимость >> цена Pass'a).

**События (Events)** — ограниченные по времени активности с уникальными наградами. События решают две задачи: (1) возвращают ушедших игроков (FOMO), (2) ломают рутину для активных игроков (новизна). Типы событий: тематические праздники, двойной XP, уникальные режимы, рейды с мировыми боссами.

**Обновления контента** — новые уровни, персонажи, механики. Основной инструмент долгосрочного удержания для Content-driven игр. Проблема: дорогой и медленный. Решение: системный дизайн (эмерджентный контент дешевле ручного).

### 9.9.2. Каденция обновлений

Каденция — ритм обновлений, определяющий ощущение «живой игры». Типичная каденция для успешной F2P-игры:

- **Ежедневно**: ежедневные задания, вращение магазина, kleine награды
- **Еженедельно**: новые квесты, ротация режимов, еженедельные бонусы
- **Ежемесячно**: крупное событие, новый персонаж/оружие, баланс-патч
- **Ежеквартально**: новый сезон, крупное обновление контента, новый Battle Pass

Для Gidede: AI генерирует рекомендуемую каденцию на основе жанра, ресурсов разработки и целевых Retention-метрик.

---

## 9.10. Этика монетизации

---

Монетизация — область с наибольшим этическим напряжением в геймдизайне. Линия между «продаём ценность» и «эксплуатируем слабости» тонка и часто размыта. Для AI-системы Gidede этические принципы монетизации — не благое пожелание, а **проектные ограничения**, нарушение которых система должна обнаруживать и предотвращать.

### 9.10.1. Спектр монетизации: от косметики до P2W

Спектр монетизационных практик от наименее к наиболее спорным:

1. **Косметика** — оплата за визуальную кастомизацию, не влияющую на геймплей. Скины, эмоции, аватары. Наименее спорная модель: платящий игрок не получает игрового преимущества. Пример: Fortnite, League of Legends
2. **Ускорение** — оплата за экономию времени: ускорение строительства, удвоение XP, пропуск гринда. Спорная модель: формально не P2W (всё достижимо бесплатно), но создаёт «налог на время», disproportionately влияющий на игроков с ограниченным игровым временем
3. **Дополнительный контент** — оплата за доступ к контенту: DLC, персонажи, карты. Приемлемая модель, если контент не является «основным» (базовая игра полноценна без DLC)
4. **Мощность** — оплата за внутриигровое преимущество: более сильное оружие, лучшие характеристики. Наиболее спорная модель: pay-to-win. Разрушает конкурентную честность и восприятие справедливости
5. **Гача/лутбоксы** — оплата за шанс получить желаемый предмет. Наиболее регулируемая модель: классифицируется как азартная игра в ряде юрисдикций (Бельгия, Нидерланды, Китай)

### 9.10.2. Тёмные паттерны и их обнаружение

Тёмные паттерны (dark patterns) — манипулятивные техники дизайна, эксплуатирующие когнитивные искажения игроков для увеличения монетизации. Типичные примеры:

- **Искусственный дефицит** — ограниченные по времени предложения, создающие FOMO-давление. «Только сегодня! Скидка 50%!» — даже если скидка перманентна
- **Иллюзия прогресса** — механики, создающие ощущение прогресса без реального продвижения (полоски, которые заполняются, но ничего не меняют)
- **Скрытые стоимости** — непрозрачные системы ценности, где реальная стоимость покупки скрыта за конвертацией валют (реальные деньги → гемы → ключи → сундуки → предметы)
- **Социальное давление** — механики, использующие социальное

**Экстракция** — целенаправленная монетизация уязвимых групп (несовершеннолетние, люди с зависимостями)

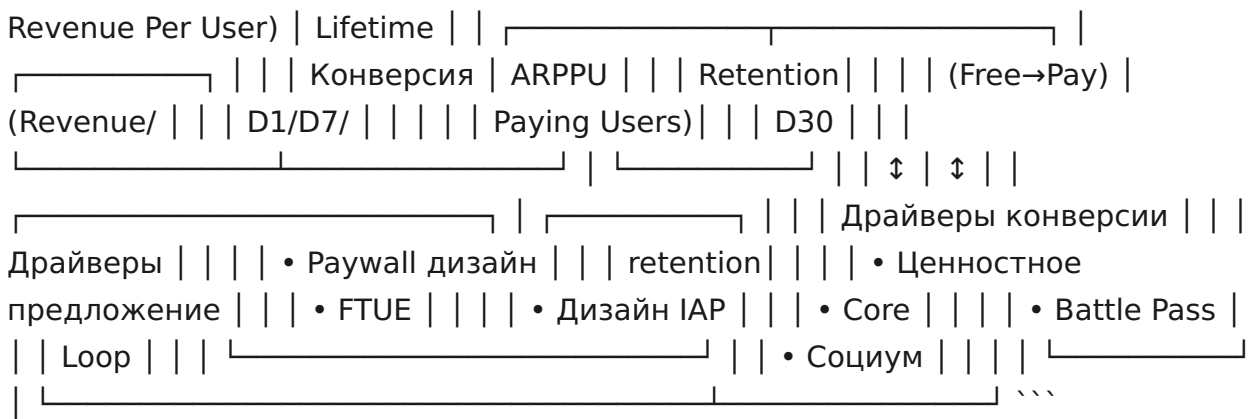
### 9.10.3. Принципы этичной монетизации

1. **Прозрачность** — игрок должен точно знать, что он покупает и сколько это стоит в реальных деньгах. Без скрытых конвертаций и обфускации стоимости
2. **Справедливость** — платящий игрок не должен иметь значимого игрового преимущества над бесплатным. Косметика и ускорение — да; мощность — нет
3. **Добровольность** — каждая покупка должна быть результатом осознанного выбора, а не манипулятивного давления. Без тёмных паттернов и искусственного дефицита
4. **Уважение к времени** — если grind — часть игры, он должен быть FUN (внутренняя мотивация), а не принуждением к оплате (внешняя мотивация через страдание)
5. **Защита уязвимых** — система монетизации не должна целенаправленно эксплуатировать несовершеннолетних и людей с зависимостями. Лимиты трат, прозрачная статистика, опциональные ограничения

### 9.11. KPI-фреймворк для AI-системы

### 9.11.1. Иерархия КРІ

(Lifetime Value) — главная целевая метрика |  $LTV = ARPU \times \text{Average Lifetime}$  |  $ARPU$  (Average Revenue Per User)



**LTV (Lifetime Value)** — суммарный доход с одного игрока за всё время его жизни в игре. Главная целевая метрика, объединяющая монетизацию и удержание:  $LTV = ARPU \times \text{Average Lifetime}$ . Условие окупаемости:  $LTV > CAC$  (Customer Acquisition Cost). Типичные бенчмарки:  $LTV/CAC > 3$  — здоровая экономика;  $LTV/CAC < 1$  — убыточная экономика.

**ARPU (Average Revenue Per User)** — средний доход на одного игрока (включая бесплатных).  $ARPU = \text{Конверсия} \times ARPPU$ . В мобильном F2P типичные значения: \$0.01-0.10 (гиперказуальные), \$0.10-1.00 (midcore), \$1.00-5.00 (core).

**ARPPU (Average Revenue Per Paying User)** — средний доход на одного платящего игрока. Показывает, сколько платящие игроки тратят в среднем. Типичные значения: \$5-50/месяц (мобильные), \$10-100/месяц (ПК/консоль).

**Конверсия (Conversion Rate)** — процент игроков, совершивших хотя бы одну покупку. Типичные значения: 1–3% (мобильные F2P), 5–10% (ПК F2P), 15–25% (F2P с сильной IP). Ключевой драйвер конверсии — «первая покупка» (стартовый пакет, который должен ощущаться как «невозможно пропустить»).

### 9.11.2. Формальная модель для Gidede

Для AI-системы формализуется модель прогнозирования KPI на основе параметров дизайна:

```

 `` Входные параметры дизайна: |— Жанр, платформа, целевая аудитория
 |— Модель монетизации и ценовые точки |— Тип Core Loop и его глубина |—
 Наличие социальных механик |— Каденция обновлений LiveOps |— Кривая
 прогрессии (тип и параметры) |— Этические ограничения

```

Прогнозируемые KPI: — D1/D7/D30 Retention (на основе качества FTUE, глубины Core Loop, соцмеханик) — Конверсия (на основе модели монетизации, ценностного предложения, first purchase) — ARPU/ARPPU (на

основе ценовых точек, глубины экономики, сегментации китов) — LTV (на основе retention × ARPU)

Валидация: — LTV > CAC (условие окупаемости) — D1 > 35%, D7 > 20%, D30 > 8% (минимальные бенчмарки) — Конверсия > 2% (минимальный бенчмарк для F2P) — Этический чеклист (нет тёмных паттернов, прозрачность, справедливость) ``

### 9.11.3. Чек-лист монетизации и удержания для AI-валидации

**Монетизация:** - [ ] Выбрана модель монетизации, соответствующая жанру и платформе? - [ ] Система валют (soft/hard) сбалансирована: soft достаточна для базового прогресса, но не покрывает весь контент? - [ ] Ценностное предложение каждой покупки понятно и честно? - [ ] Paywall мягкий: бесплатный игрок может пройти весь контент, но медленнее? - [ ] Первая покупка (starter pack) ощущается как «невозможно пропустить»? - [ ] Нет P2W-элементов в соревновательных режимах? - [ ] Ценообразование прозрачно (нет скрытых конвертаций)? - [ ] Этический валидатор не обнаружил тёмных паттернов?

**Retention:** - [ ] FTUE: «крючок» в первые 30 секунд? Минимум текста? Один навык за раз? - [ ] Core Loop: минимальный цикл FUN? Достаточная глубина для D7+? - [ ] Прогрессия: видимый прогресс? Краткосрочные и долгосрочные цели? - [ ] Социальные механики: гильдии, кооперация, PvP — хотя бы одна? - [ ] Режимы подкрепления: комбинация всех четырёх (FR, FI, VR, VI)? - [ ] Три проблемы вовлечения решены: нет избыточного даунтайма? Сложность в канале потока? Игрок верит в победу? - [ ] LiveOps-план: каденция обновлений определена? Сезоны, Battle Pass, события? - [ ] Эндгейм: есть системный контент, генерирующий реиграбельность?

**KPI:** - [ ] Прогноз LTV > CAC? - [ ] Прогноз D1 > 35%, D7 > 20%, D30 > 8%? - [ ] Прогноз конверсии > 2% (для F2P)? - [ ] Есть план A/B-тестирования ключевых гипотез монетизации?

---

## 9.12. Интеграция монетизации и удержания в пайплайн Gidede

---

Монетизация и удержание не могут быть спроектированы изолированно — они должны быть интегрированы в общий пайплайн проектирования игры. Для

Gidede это означает, что каждый модуль генерации должен учитывать монетизационные и Retention-импликации своих решений.

### 9.12.1. Модель двойной оптимизации

AI-система Gidede проектирует игру с двумя целевыми функциями:

1. **Целевая функция опыта:** максимизировать эстетический опыт игрока (через Reverse MDA: Aesthetics → Dynamics → Mechanics)
2. **Целевая функция удержания:** максимизировать Retention-метрики и LTV (через KPI-фреймворк)

Эти функции не всегда согласованы: опыт, максимизирующий краткосрочное удовольствие, может не способствовать долгосрочному удержанию (например, выдача всех наград сразу — весело, но убивает мотивацию возвращаться). И наоборот: дизайн, максимизирующий удержание через гринд, может ухудшить опыт. Задача AI — находить **парето-оптимальные** решения, где нельзя улучшить одну целевую функцию без ухудшения другой.

### 9.12.2. Архитектура модуля «Монетизация и Retention»

...

МОДУЛЬ «МОДЕЛЬ МОНЕТИЗАЦИИ»		• Каталог моделей (6 типов) с критериями выбора	• Генератор системы валют (soft/hard + faucet/drain)	• Конструктор paywall (мягкий/жёсткий + кривые стоимости)	• Ценовой калькулятор (first purchase, starter pack, IAP)	• Этический валидатор (тёмные паттерны, P2W-детектор)
МОДУЛЬ «RETENTION-ДИЗАЙН»		• FTUE-генератор (крючок, tutorial, первая награда)	• Мотивационный профайлер (модель Йи + Гэри)	• Генератор систем наград (4 режима подкрепления)	• Диагностика проблем вовлечения (3 проблемы Гэри)	• Жизненный цикл: FTUE → Core Loop → Mid-Game → End-Game
МОДУЛЬ «LIVEOPS-ПЛАННИРОВЩИК»		• Генератор каденции обновлений (ежедневно/еженедельно/...)	• Конструктор Battle Pass (2 трека, награды, прогрессия)	• Планировщик событий (сезоны, праздники, ивенты)	• Контент-календарь (N месяцев после релиза)	
МОДУЛЬ «KPI-ПРОГНОЗИРОВАНИЕ»		• Модель прогнозирования Retention (D1/D7/D30)	• Модель прогнозирования конверсии	• Модель прогнозирования ARPU/ARPPU	• Калькулятор LTV/CAC	• Валидатор: LTV >

---

## Резюме

---

Монетизация и удержание — две стороны одной медали: без удержания монетизация бессмысленна (некому платить), без монетизации удержание бессмысленно (нечем поддерживать игру). Их интеграция в геймдизайн — не финальный этап, а параллельный процесс, который должен начинаться одновременно с проектированием механик. Ключевые принципы:

- Монетизация — часть дизайна, а не надстройка; выбор модели определяет архитектуру игры
- Двойная экономика (виртуальная + реальная) должна быть сбалансирована через faucet/drain-модель
- Удержание строится на нейробиологии (дофамин/серотонин/окситоцин/эндорфины) и мотивации (модель Йи: 12 типов в 3 кластерах)
- Четыре режима подкрепления комбинируются для создания устойчивого паттерна возвращения
- Три проблемы вовлечения (даунтайм, сложность, безнадёжность) — диагностический фреймворк
- Жизненный цикл игрока (FTUE → Core Loop → Mid-Game → End-Game) определяет дизайнерские приоритеты на каждой фазе
- LiveOps — не реакция, а проектируемая система с каденцией обновлений
- Этика монетизации — проектное ограничение, а не благое пожелание
- KPI-фреймворк (LTV, Retention, ARPU, конверсия) — целевая функция для AI-оптимизации

---

## Глава 10. Плейтестинг и итерация

---

> **Раздел Библии геймдизайна:** 2.10 — Плейтестинг и итерация >

**Источники:** KB-14 (Валидация и чек-листы), KB-03 (MDA Framework), KB-13 (Опыт и мотивация), KB-05 (Экономика), KB-11 (Level Design) > **Книги:**

Фуллerton (Кн. 5), Шелл (Кн. 1, 2), Гэри (Кн. 10), Бонд (Кн. 17), Зубек (Кн. 6), Селлерс (Кн. 13), Гэззэуэй (Кн. 9), Адамс/Дорманс (Кн. 4), Роджерс (Кн. 8)

---



## 10.1. Философия плейтестинга: почему тестирование — не этап, а состояние

---

Шелл (Кн. 1) формулирует фундаментальный принцип геймдизайна: **«Чем больше раз вы тестируете и улучшаете свой дизайн, тем лучше ваша игра»** — Правило цикла (Rule of the Loop). Это не рекомендация, а аксиома: ни один дизайнер, сколь бы опытным он ни был, не может предсказать, как будет ощущаться игра в руках реального игрока. Зубек (Кн. 6) объясняет это через «проблему второго порядка»: дизайнер создаёт механики, но опыт игрока эмерджентен — он возникает из взаимодействия механик и игрока, и его нельзя спроектировать напрямую. Можно заложить предпосылки, но нельзя гарантировать результат. Поэтому плейтест — единственный способ узнать, что игрок на самом деле чувствует, думает и делает.

Фуллертон (Кн. 5) поднимает этот принцип до методологического уровня: **Playcentric Design Process** ставит плейтест в центр всего процесса разработки. Цель опыта → Прототип → Плейтест → Оценка → Итерация — и цикл повторяется снова и снова. Это не линейный процесс с плейтестом на финальном этапе; это спиральный процесс, в котором плейтест происходит на каждом витке, начиная с самых ранних прототипов. Фуллертон настаивает: «Дизайнер — это адвокат игрока. Все дизайнерские решения оцениваются через призму опыта игрока. Вы не можете предсказать, как будет ощущаться игра — вы должны протестировать её». Это радикальное утверждение лишает дизайнера иллюзии всезнания и передаёт власть над оценкой качества игроку.

Гэри (Кн. 10) дополняет: «Успех на 90% зависит от реализации и лишь на 10% от идеи». Это означает, что даже гениальная концепция бесполезна без итеративного цикла тестирования и улучшения. И наоборот: посредственная идея, прошедшая 100 итераций, превратится в отличную игру. Пример, который приводит Фуллертон — история Magic: The Gathering: Ричард Гартфилд начал с 300 карточек (слишком много), урезал до 120 (лучше, но несбалансированно), добавил мана-систему (революция), и каждая итерация сопровождалась плейтестом. MTG стал великим не потому, что Гартфилд был гением, а потому, что он был настойчивым итератором.

Для Gidede это означает: модуль плейтестинга и валидации не может быть «финальной проверкой» — он должен быть встроен в каждый этап генерации. Когда AI генерирует концепцию, он должен «плейтестировать» её через симуляцию. Когда генерирует механики — проверять их через Machinations-модель. Когда генерирует баланс — валидировать через количественный

анализ. Плейтест в AI-системе — это не фаза, а **функция валидации**, вызываемая после каждого шага генерации.

---

## 10.2. Прототипирование: от гипотезы к эксперименту

---

Прототипирование — мост между абстрактной идеей и проверяемой реальностью. Каждый прототип — это материализованная гипотеза, а каждый плейтест — эксперимент по её проверке. Бонд (Кн. 17) формулирует это предельно ясно: «Каждый прототип — гипотеза, каждый плейтест — эксперимент». Это научный подход к геймдизайну, который превращает интуитивные ощущения в проверяемые утверждения.

### 10.2.1. Бумажное прототипирование

Фуллертон (Кн. 5) и Бонд (Кн. 17) — ярые сторонники физического прототипирования до написания кода. Бумажный прототип позволяет тестировать правила игры без инвестиций в программирование, что критически важно на ранних этапах, когда дизайн наиболее изменчив. Процесс бумажного прототипирования по Фуллертону:

1. Визуализировать кор-геймплей — определить, что составляет минимальный игровой опыт
2. Построить физический прототип — бумага, карточки, фишки, монеты
3. Играть и тестировать — немедленно, не дожидаясь «завершённости»
4. Уточнять правила — после каждого теста, изменяя только один элемент
5. Добавлять сложность постепенно — не раньше, чем базовая механика работает

Гэри (Кн. 10) добавляет пять принципов прототипирования, которые для AI-системы становятся правилами генерации минимальных моделей:

1. **Первый прототип всегда отстой** — душийте перфекциониста. Цель прототипа — не быть красивым, а проверять гипотезу. AI должен генерировать «грубые» модели, не тратя ресурсы на полировку.
2. **Не переусердствуйте** — упрощайте до максимума. Пять минут теста могут дать не меньше, чем часовая партия. Минимальный прототип = минимальная проверяемая единица.
3. **Найдите подходящую аудиторию** — ранние прототипы лучше тестировать с гейм-дизайнерами (они понимают, что видят черновик), поздние — с целевой аудиторией.
4. **Мысленно продумайте весь игровой процесс перед созданием прототипа** — что я вижу? Какие

стимулы? Как отслеживать информацию? Это направляет генерацию прототипа от опыта к механике. **5. Первые прототипы делайте максимально гибкими** — карты, фигурки, переменные на стикерах — легче менять, чем код. В AI-системе это означает: генерировать параметрические модели, а не жёстко заданные структуры.

**Ограничения бумажного прототипирования** (Зубек, Кн. 6): бумажные прототипы хороши для явной механики (правила видны и обсуждаемы), но плохо подходят для неявной (скрытые в коде правила, физика, AI-поведение, кинестетика). Для неявной механики необходим цифровой прототип.

### 10.2.2. Цифровое прототипирование

Фуллerton (Кн. 5) выделяет четыре типа цифровых прототипов, каждый из которых проверяет определённый аспект дизайна:

**Механический прототип** — проверка работы ключевых механик. Чувствуется ли кор? Работают ли правила? Есть ли веселье в базовом цикле? Это самый важный тип: если механический прототип не работает, нет смысла двигаться дальше. Для AI-системы: Machinations-модель (Кн. 4) — идеальный инструмент механического прототипа. AI генерирует модель внутренней экономики, запускает симуляцию с искусственными игроками и анализирует результаты: есть ли runaway, deadlock, stall? Работает ли Core Loop?

**Эстетический прототип** — проверка визуального стиля и атмосферы. Соответствует ли атмосфера целям опыта? Передаёт ли эстетика нужные эмоции? Этот тип прототипа особенно важен для игр, где атмосфера — ключевая составляющая (хорроры, эксплорейшн, нарративные игры). Для AI-системы: генерация концепт-арт промптов и цветовых палитр для визуальной валидации.

**Кинестетический прототип** — проверка ощущений управления. Отзывчивость, game feel, juiciness — то, что Шелл (Кн. 1) называет «сочностью». Правило Шелла: если интерфейс не реагирует за 1/10 секунды, игрок считает, что что-то не так. Кинестетический прототип невозможно полностью валидировать в AI-системе, но можно проверять косвенно: есть ли фидбэк на каждое действие? Соответствует ли время отклика стандартам?

**Технологический прототип** — проверка технической осуществимости. Можно ли реализовать на целевой платформе? Хватит ли производительности? Этот тип выходит за рамки AI-системы геймдизайна, но AI может генерировать технические ограничения и проверять соответствие концепции платформе.

### 10.2.3. Прототип как минимальная проверяемая единица

Зубек (Кн. 6) подчёркивает: **играбельный прототип ≠ демо**. Демо — это демонстрация готового продукта; прототип — это минимальная проверка дизайнерской гипотезы. Разница принципиальна: демо должно выглядеть хорошо, прототип — давать ответ на вопрос. Прототип, который «выглядит хорошо, но не отвечает ни на какой вопрос» — потерянное время.

Принципы Келли (приведены у Зубека, Кн. 6) для перехода от концепции к прототипу:

1. **Начинать с очевидного** — не ищите изощрённые решения, реализуйте самое простое
2. **Защищать не идею, а прототип** — пусть прототип говорит за себя, не нужно оправдывать его словами
3. **Не влюбляться в первый прототип** — он почти наверняка неправильный
4. **Итерировать быстро** — скорость важнее качества на этом этапе
5. **Не тратить время на несущественные детали** — каждый элемент прототипа должен проверять конкретную гипотезу

Для Gidede это формализуется как **пайплайн генерации прототипов**:

1. Определить проверяемую гипотезу (например: «Core Loop с экономикой типа Engine даёт ощущение роста»)
2. Сгенерировать минимальную Machinations-модель
3. Запустить симуляцию с искусственными игроками
4. Проанализировать: подтверждается ли гипотеза?
5. Если нет — изменить один параметр и повторить
6. Если да — добавить следующий слой сложности

---

## 10.3. Методы плейтестирования

---

Плейтестирование — не просто «дать поиграть и посмотреть». Это структурированный процесс сбора данных о поведении и опыте игроков, имеющий свои методологии, протоколы и ловушки. Гэззэй (Кн. 9) подчёркивает, что геймдизайнер должен применять научный метод: формулировать вопросы, собирать информацию, строить гипотезы, проверять их экспериментом, анализировать данные, делать выводы — и повторять.

### 10.3.1. RITE-метод

Фуллerton (Кн. 5) описывает **RITE-метод (Rapid Iterative Testing and Evaluation)** — наиболее эффективную методологию плейтестирования для итеративного дизайна:

| Параметр | Значение | |-----|-----| | Количество тестировщиков за сессию  
| 4-5 | | Фикс после каждого раунда | Да — немедленно | | Повторный тест |  
После каждого фикса | | Цикл | Тест → Фикс → Тест → Фикс → ... |

RITE радикально отличается от традиционного QA-тестирования: вместо накопления баг-репортов и их массового исправления, RITE требует немедленного исправления после каждого обнаружения проблемы и повторного тестирования. Это гарантирует, что каждое исправление действительно решает проблему и не создаёт новых. Ключевой принцип: **быстрая итерация ценнее полного отчёта**. Лучше исправить одну проблему и убедиться, что фикс работает, чем задокументировать двадцать проблем и не исправить ни одной.

Для Gidede RITE-метод — это алгоритм для AI-модуля итеративного улучшения: сгенерировать дизайн → провести симуляцию (аналог плейтеста) → выявить проблему → исправить один параметр → повторить симуляцию → подтвердить улучшение. Цикл повторяется до тех пор, пока все выявленные проблемы не будут решены.

### 10.3.2. Процесс плейтест-сессии

Фуллerton (Кн. 5) описывает структурированный процесс плейтест-сессии:

| Этап | Длительность | Что происходит | |-----|-----|-----| | 1. Введение | 5 мин | Объяснить цели теста, правила поведения (не помогать, не подсказывать) | | 2. Разминка | 5 мин | Игроки знакомятся с управлением и интерфейсом | | 3. Игра | 30-60 мин | Тестирование без вмешательства дизайнера — наблюдение, не руководство | | 4. Обсуждение | 15 мин | Структурированные вопросы: что понравилось, что запутало, что скучно | | 5. Завершение | 5 мин | Благодарность, следующие шаги |

Критический принцип: **на этапе 3 дизайнер не вмешивается**. Это самое трудное правило — видеть, как игрок идёт не туда, и не подсказывать. Но именно это наблюдение — самое ценное: если игрок не понимает, куда идти, значит, навигация работает плохо, и это нужно исправить, а не компенсировать подсказками.

### 10.3.3. Наблюдение и сбор данных

Гэри (Кн. 10) формулирует пять правил тестирования, которые для AI-системы становятся принципами автоматической валидации:

1. **Забудьте про самолюбие** — представьте, что вы инопланетянин, наблюдающий за «гейм-дизайнером». Это принцип отстранённого

наблюдения: не защищайте свой дизайн, а наблюдайте за тем, что реально происходит.

**2. Внимательно наблюдайте** — выражения лиц, ошибки, неуверенные движения, точки концентрации, сильные эмоции. В AI-системе: анализировать данные о поведении — время принятия решений, количество ошибок, точки остановки, паттерны взаимодействия.

**3. Задавайте открытые вопросы** — структурированный опросник Гэри: (а) три понравившихся момента, (б) три не понравившихся, (в) опишите игру своими словами, (г) что непонятно, (д) что изменить, (е) что оставить. Ключевой принцип: вопросы должны быть открытыми, не наводящими. Не «Вам понравилась боевая система?» (да/нет), а «Что вы чувствовали во время боя?» (описание).

**4. На поздних стадиях используйте шкалу 1-5** — рекомендация, простота освоения, тематика, продолжительность, стратегическая глубина. Количественные данные позволяют отслеживать прогресс между итерациями.

**5. Игроки знают, что чувствуют, но не всегда знают почему** — цитата Нила Геймана: «Если читатель говорит, что с книгой что-то не так, он скорее всего прав. Если он говорит, как именно нужно её исправить, он скорее всего ошибается». Это критическое различие: игроки — эксперты в том, что они чувствуют, но не в том, как это исправить. Задача дизайнера — услышать чувство и найти решение.

#### 10.3.4. Научный метод в плейтестировании

Гэззэуэй (Кн. 9) предлагает восемь этапов научного метода, применённого к геймдизайну:

**1. Формулировка вопросов** — что конкретно мы хотим узнать? Плохой вопрос: «Достаточно ли увлекателен третий уровень?» Хороший вопрос: «Могут ли 80% игроков пройти третий уровень менее чем за 2 минуты, перезапустив его минимум 2, но не более 5 раз?» **2. Сбор информации и ресурсов** — данные о текущем состоянии дизайна, сравнение с аналогами **3. Формулировка объясняющей гипотезы** — «Если я увеличу здоровье босса на 20%, бой будет длиться  $\geq 5$  минут для 85% игроков» **4. Проверка гипотезы экспериментом** — воспроизводимый плейтест с контролем переменных **5. Анализ полученных данных** — количественный анализ: средние, медианы, распределения **6. Интерпретация данных и формулировка выводов** — подтверждается ли гипотеза? Если нет — почему? **7. Публикация результатов** — документирование для команды (в AI-системе:

автоматический лог) 8. **Повторное тестирование** — RITE: исправили → тестируем снова

Гэзэуэй также предупреждает о **предвзятости согласия (acquiescence bias)**: добровольные тестировщики заранее настроены отвечать положительно. Простые опросы («Вам понравилось?») искажают результаты. Решение: задавать конкретные поведенческие вопросы («Где вы застряли?», «Сколько раз вы умерли на этом уровне?», «Какие два оружия вы использовали чаще всего?»), а не оценочные.

---

## 10.4. Итеративный процесс: от теста к улучшению

---

Итерация — не просто «изменили и попробовали снова». Это структурированный процесс принятия решений на основе данных плейтеста. Каждая итерация должна иметь чёткую цель, проверяемую гипотезу и критерий успеха.

### 10.4.1. Четыре уровня тестирования

Фуллerton (Кн. 5) выделяет четыре уровня тестирования, каждый из которых фокусируется на разных аспектах игры:

**Уровень 1: Фундамент** — работают ли базовые механики? Есть ли кор-геймплей? Может ли игрок вообще взаимодействовать с игрой осмысленно? На этом уровне проверяются самые базовые гипотезы: «Понятна ли цель?», «Можно ли совершать осмысленные действия?», «Есть ли веселье в базовом цикле?». Если фундамент не работает — всё остальное бессмысленно.

**Уровень 2: Структура** — все ли системы связаны? Есть ли тупики? Поддерживают ли системы друг друга или конфликтуют? На этом уровне проверяется целостность: работают ли экономика, прогрессия, Core Loop как единая система? Не возникает ли ситуаций, когда игрок не может продвинуться?

**Уровень 3: Формальные детали** — баланс, числа, тайминги. На этом уровне проверяются конкретные параметры: слишком легко или слишком сложно? Слишком быстро или слишком медленно? Есть ли доминантная стратегия? Это уровень, где применяются математические модели из раздела 2.5 (Баланс) и экономический анализ из раздела 2.6 (Экономика и прогрессия).

**Уровень 4: Шлифовка** — game feel, сочность, полировка. На этом уровне проверяется «сочность» интерфейса (Шелл, Кн. 1): есть ли богатый многослойный фидбэк на каждое действие? Чувствуется ли сила в действиях игрока? Это уровень, который превращает функциональную игру в приятную.

Каждый уровень требует своих методов тестирования и своих критериев успеха. Нельзя тестировать баланс (уровень 3), если фундамент (уровень 1) ещё не работает. Нельзя полировать (уровень 4) то, что ещё не сбалансировано (уровень 3).

#### 10.4.2. Принцип «одно изменение за итерацию»

Бонд (Кн. 17) и Фуллертон (Кн. 5) настаивают на критически важном правиле: **после каждого плейтеста изменять только один элемент**. Это научный метод в чистом виде: если вы изменили три параметра одновременно и игра стала лучше — вы не знаете, какой из трёх изменений помог. Хуже того: возможно, одно изменение помогло, а другое навредило, и их эффекты скомпенсировали друг друга.

Гэри (Кн. 10) предлагает «калибровку не по чуть-чуть, а радикально» — принцип «вдвое больше, вдвое меньше»: если параметр кажется слишком маленьким, не прибавляйте 10% — удвойте. Если слишком большим — уменьшите вдвое. Это позволяет быстрее найти правильный диапазон, чем микро-корректировки. Сочетание двух принципов — «одно изменение за итерацию» и «радикальная калибровка» — создаёт эффективный процесс: меняем один параметр  $\times 2$ , тестируем, анализируем, при необходимости уточняем бинарным поиском.

Для Gidede это формализуется как алгоритм:

1. Выявить проблему в плейтесте (например: «игроки не используют оружие В»)
2. Сформулировать гипотезу: «Оружие В слишком слабое»
3. Изменить один параметр: увеличить урон оружия В в 2 раза
4. Провести плейтест
5. Проанализировать: используют ли теперь оружие В? Не стало ли оружие А бесполезным?
6. Если проблема решена — перейти к следующей проблеме
7. Если нет — скорректировать: увеличить урон оружия В в 1.5 раза (бинарный поиск)
8. Повторять до решения

#### 10.4.3. Три вывода после тестирования

Гэри (Кн. 10) формализует три возможных вывода после каждого плейтеста, которые определяют направление следующей итерации:



**Вывод 1: Основная механика работает** — игроки увлечены, Core Loop работает, есть веселье. Действие: углубить концепцию — добавить следующий слой механик, расширить контент, тестировать с ещё одной группой. Это зелёный свет: фундамент подтверждён, можно строить поверх.

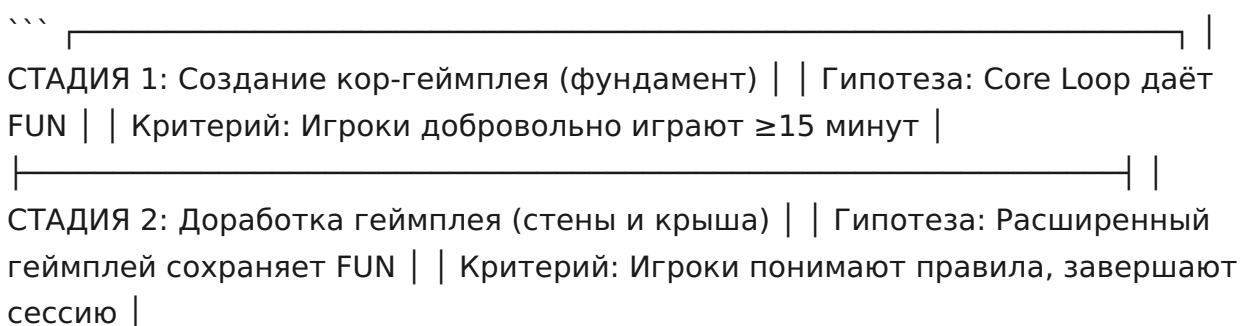
**Вывод 2: Основная механика не работает** — игроки скучают, не понимают цель, Core Loop не захватывает. Действие: конкретная критика задаёт вектор — вернуться к мозговому штурму, переосмыслить базовую механику, возможно, отказаться от текущего направления. Это красный свет: фундамент не подтверждён, нельзя строить поверх неработающего. Селлерс (Кн. 13) формулирует это как «поиск веселья» (finding the fun): «Ранняя итерация — создай прототип, закрой интерактивную петлю, проверь, есть ли веселье. Если нет — меняй, не строй поверх неработающего фундамента».

**Вывод 3: Неопределённый результат** — одни игроки увлечены, другие нет; данные противоречивы; непонятно, работает ли Core Loop. Действие: (а) попробовать ещё раз с усовершенствованным прототипом — возможно, проблема в прототипе, а не в механике; (б) отложить и вернуться со свежим взглядом — иногда нужно дистанцироваться от дизайна, чтобы увидеть его объективно.

Ключевой вопрос после каждого теста от Гэри: **«Это изменение работает на главный конфликт или против него?»** Главный конфликт — сердцевина игры, то, ради чего игрок играет. Каждое изменение должно усиливать главный конфликт, а не ослаблять его. Если изменение делает игру «удобнее», но ослабляет напряжение — это неправильное изменение.

## 10.5. Пять стадий дизайна и цикл итерации

Гэри (Кн. 10) предлагает модель из пяти стадий дизайна, каждая из которых содержит минимум один полный итеративный цикл. Это важная структура для AI-системы, потому что она определяет, какие гипотезы проверяются на каждой стадии и какие критерии успеха используются.



СТАДИЯ 3: Дизайн компонентов (интерьер)	Гипотеза: Компоненты разнообразят геймплей	Критерий: Игроки используют $\geq 3$ различных стратегии
СТАДИЯ 4: Доработка компонентов (мебель)	Гипотеза: Компоненты сбалансированы	Критерий: Нет доминантной стратегии, все стратегии жизнесп.
СТАДИЯ 5: Завершающие штрихи (лоск и украшения)	Гипотеза: Полировка улучшает опыт	Критерий: Высокие оценки по шкале «удовольствие» (4+/5)

Каждая стадия — это отдельный итеративный цикл. Два правила: (1) Стадии могут быть подвижными, без жёстких границ — если на стадии 3 обнаруживается проблема с Core Loop, нужно вернуться к стадии 1. (2) Базовый цикл разработки — основа всего; каждая стадия включает минимум один полный цикл «Задумка → Конкретизация → Штурм → Прототип → Тест → Итерация».

Для Gidede: AI должен отслеживать текущую стадию проектирования и адаптировать методы валидации. На стадии 1 — проверять Fun-фактор Core Loop через симуляцию. На стадии 4 — проверять баланс через математический анализ и статистику. На стадии 5 — проверять «сочность» через чек-лист Шелла.

## 10.6. Аналитические методы валидации

Плейтестирование с реальными игроками — золотой стандарт, но он не всегда доступен, особенно на ранних этапах. Кроме того, плейтест даёт качественные данные («игрокам скучно на третьем уровне»), но не всегда объясняет причину количественно. Для дополнения качественных наблюдений нужны аналитические методы.

### 10.6.1. Симуляция с искусственными игроками

Адамс и Дорманс (Кн. 4) описывают систему искусственных игроков (artificial players) в Machinations — автоматических агентов, которые играют по заданным стратегиям и позволяют запустить тысячи симуляций за минуты. Это аналог плейтеста, но с контролируруемыми переменными: дизайнер точно

знает, какую стратегию использует агент, и может сравнивать результаты разных стратегий.

Для AI-системы Gidede это ключевой инструмент валидации: Machinations-модель + искусственные игроки = автоматический плейтест. AI может: (1) сгенерировать модель экономики, (2) создать агентов с разными стратегиями (агрессивная, оборонительная, комбо), (3) запустить 1000 симуляций, (4) проанализировать результаты: какая стратегия побеждает? Есть ли runaway? Балансируются ли петли?

### 10.6.2. Математический анализ баланса

Гэззэуэй (Кн. 9) и Шрайбер (Кн. 7) предлагают набор математических инструментов для валидации баланса без плейтеста:

**Формула балансировки диапазона** (Гэззэуэй):  $((Max - Min) \times \%) + Min = \text{значение}$ . Каждый объект получает процентный ранг, и система автоматически пересчитывает все значения при изменении Min/Max. Это позволяет валидировать баланс аналитически: если все объекты распределены равномерно в диапазоне — базовый баланс обеспечен.

**Опорная точка (Fulcrum)** (Гэззэуэй): эталонный объект, относительно которого балансируются все остальные. Принцип транзитивности: если А сбалансирован с опорной и В сбалансирован с опорной, то  $A \approx B$ . Это снижает сложность балансировки с  $O(n^2)$  до  $O(n)$  — вместо тестирования каждой пары, тестируем каждый объект против эталона.

**Cost-power curves** (Шрайбер, Кн. 7): для каждого элемента строится кривая «стоимость → мощность». Если кривая для элемента А лежит строго выше кривой для элемента В при той же стоимости — элемент А доминирует и требует балансировки. AI может автоматически строить и сравнивать cost-power curves для всех элементов.

### 10.6.3. Аналитика поведения игроков

Селлерс (Кн. 13) систематизирует метрики, которые превращают плейтест из качественного наблюдения в количественный анализ:

**Приобретение (Acquisition)** — воронка приобретения: каждый клик теряет ~50% игроков. Метрики: установки, registrations, конверсия из просмотра в установку.

**Удержание (Retention)** — D1/D7/D30: процент игроков, вернувшихся через 1, 7, 30 дней. Это самый прямой показатель качества геймдизайна: если  $D1 <$

25% — фундамент проблемный (уровень 1 по Фуллертон), если  $D7 < 10\%$  — структурные проблемы (уровень 2), если  $D30 < 3\%$  — проблемы с долгосрочным контентом (уровни 3–4).

**Конверсия (Conversion)** — переход от бесплатных к платящим игрокам. Метрика монетизации, но также индикатор ценности: если игроки не платят — они не считают игру ценной.

**Использование (Usage)** — длительность сессии, частота, глубина (сколько контента потребляется). Позволяет идентифицировать проблемные зоны: если 80% игроков застревают на уровне 5 — уровень 5 нужно переработать.

**Сообщество (Community)** — социальные метрики: размер гильдий, активность чатов, пользовательский контент. Индикатор долгосрочного здоровья игры.

Для Gidede: аналитика — инструмент пост-релизной валидации. AI может не только анализировать данные, но и прогнозировать: на основе дизайна игры (тип Core Loop, модель монетизации, кривая прогрессии) предсказывать ожидаемые метрики и сравнивать их с реальными. Расхождение = сигнал к итерации.

#### 10.6.4. Четыре метода балансировки систем

Зубек (Кн. 6) формализует четыре способа балансировки, расположенных по возрастанию формализованности:

1. **Ручная балансировка** — играем → меняем → тестируем. Субъективно, но быстро. Подходит для ранних итераций, когда дизайн ещё формируется. Опасность: бесконечные споры в команде вместо прототипирования.
2. **Внешние модели** — электронные таблицы, расчёты, Machinations-диаграммы. Объективнее ручной балансировки, но отделены от реального геймплея. Могут не учитывать факторы, которые обнаруживаются только при реальной игре.
3. **Автоматизированная симуляция** — надстройка к игре, которая автоматически запускает тысячи боёв/раундов и собирает статистику. Позволяет быстро проверить: какая стратегия побеждает чаще? Какой класс имбалансен? Это самый мощный инструмент для балансировки чисел.
4. **Аналитика плейтестов** — сбор данных от реальных игроков. Самый надёжный источник, но требует большого количества играющих. На ранних этапах недоступен; становится ключевым после релиза.

Для Gidede: AI-система должна использовать все четыре метода, переключаясь между ними по мере развития проекта. На этапе генерации концепции — ручная балансировка (дизайнер и AI совместно). На этапе проверки механик — внешние модели (Machinations). На этапе балансировки — автоматизированная симуляция. На этапе живой игры — аналитика.

---

## 10.7. Дизайнерская петля и мета-итерация

---

Селлерс (Кн. 13) вводит концепцию **четырёх главных петель**, из которых четвёртая — петля дизайнера — является мета-петлёй, охватывающей все остальные:

1. **Петля модели игры** — внутренняя динамическая модель мира игры. Если модель статична — нет интерактивности, нет геймплея. 2. **Петля ментальной модели игрока** — динамическая модель, которую игрок строит в своём сознании. Должна соответствовать модели игры, иначе — фрустрация. 3. **Интерактивная петля** — цикл «действие → обратная связь → обновление модели» между игроком и игрой. «Закрытие петли» — первый магический момент: игрок может действовать, видеть результат и учиться. 4. **Петля дизайнера** — мета-петля: дизайнер создаёт/изменяет игру → игрок играет → дизайнер наблюдает → дизайнер меняет игру.

Петля дизайнера — это и есть процесс итеративного геймдизайна. Каждый виток этой петли — одна итерация: дизайнер вносит изменения на основе наблюдений, игрок взаимодействует с обновлённой игрой, дизайнер наблюдает и делает новые выводы. Качество итераций определяется тем, насколько точно дизайнер считывает данные из наблюдений и насколько эффективно превращает их в изменения.

Селлерс подчёркивает: для системных игр (см. раздел 2.4 о Core Loop) петля дизайнера особенно важна, потому что динамика системы **эмерджентна** — она не заложена напрямую, а возникает из взаимодействия частей.

Предсказать эмерджентную динамику аналитически крайне сложно; остаётся наблюдать и итерировать. Это делает плейтест не просто полезным, а **единственным** способом валидации системного дизайна.

---

## 10.8. Кривая интереса и валидация темпа

---

Шелл (Кн. 1) предлагает инструмент валидации не отдельных механик, а всего темпа игры — **кривую интереса**. Успешная кривая интереса имеет фрактальную структуру: крючок (hook) в начале → нарастающие пики с отдыхом между ними → кульминация → удовлетворяющее завершение. Три фактора интереса по Шеллу:

1. **Изначальный интерес** — некоторые вещи естественно интереснее других: опасность > безопасность, фантазия > реальность, необычное > рутинное. 2. **Поэтика подачи** — эстетическое качество преподнесения: как игра выглядит, звучит, ощущается. 3. **Проецирование** — насколько глубоко игрок проецирует себя в опыт. У игр уникальное преимущество: игрок ЯВЛЯЕТСЯ протагонистом.

Кривая интереса фрактальна на трёх уровнях: (1) игра целиком, (2) каждый уровень, (3) каждый вызов внутри уровня. Для валидации темпа AI должен проверять кривую интереса на всех трёх уровнях: есть ли крючок? Нарастает ли интерес? Есть ли отдых между пиками? Завершается ли кульминацией?

Роджерс (Кн. 8) дополняет инструментарием Beat Chart — графиком интенсивности по времени с чередованием высоких и низких точек. Beat Chart позволяет визуализировать темп и обнаруживать проблемы: затяжные плато без напряжения, отсутствие отдыха после кульминации, антикульминация (самый интенсивный момент — не финальный).

Пять типов Gameplay Progression по Роджерсу — ещё один инструмент валидации темпа:

1. **Новая механика** — впервые представляется 2. **Вариация** — знакомая механика в новом контексте 3. **Комбинация** — две механики вместе 4. **Инверсия** — знакомая механика работает наоборот 5. **Удаление** — механика убирается, оставляя вызов

Правильная прогрессия чередует все пять типов, создавая разнообразие и поддерживая интерес. AI может валидировать: использует ли дизайн все пять типов? Не повторяется ли один тип слишком часто? Достаточно ли вариативности?

---

## 10.9. Практика Гэзэуэя: научный подход к итерации

---

Гэзэуэй (Кн. 9) предлагает наиболее формализованный подход к итерации, основанный на научном методе и электронных таблицах. Его подход ценен для AI-системы тем, что каждый шаг алгоритмизируем.

### 10.9.1. Перевод качественных вопросов в количественные

Ключевой навык гейм-дизайнера — превращать расплывчатые ощущения в измеримые критерии. Гэзэуэй приводит примеры:

| Плохой вопрос | Хороший вопрос | |-----|-----| | «Достаточно ли увлекателен третий уровень?» | «Могут ли 80% игроков пройти третий уровень менее чем за 2 минуты, перезапустив его минимум 2, но не более 5 раз?» | | «Достаточно ли труден босс?» | «Закончился ли первый бой с боссом неудачей для  $\geq 85\%$  игроков? При победе — использовали  $\geq 70\%$  оружия и бой длился  $\geq 5$  минут?» |

Для Gidede: AI должен автоматически переформулировать качественные вопросы в количественные, используя измеримые критерии. Вход: «Игроки говорят, что третий уровень скучный» → Выход: «Метрика: процент игроков, завершивших уровень за  $> 5$  минут или со смертями  $< 1$ . Целевой показатель:  $< 20\%$  игроков в этом диапазоне».

### 10.9.2. Иерархический дизайн и приоритеты исправлений

Гэзэуэй вводит принцип иерархического дизайна: составить линейный список элементов от самых важных к наименее важным. При конфликте — менять тот, что ниже в иерархии. Пример иерархии:

1. Общая концепция игры (не менять!) 2. Игровой движок 3. Платформа 4. Игровое управление / камера 5. Игровой UI 6. Игровой персонаж 7. Атрибуты персонажа 8. Опорный уровень 9. Опорный враг 10. Все другие персонажи 11. Power-ups 12. Отдельные уровни 13. Награды макросистем 14. Звук / музыка

Это означает: если баланс оружия нарушен, не нужно менять концепцию игры — нужно поменять атрибуты оружия (пункт 7–10). Если уровень кажется скучным, не нужно менять механику — нужно перепроектировать уровень (пункт 12). Иерархия защищает фундамент от ненужных изменений и направляет итерации в правильное русло.

Для Gidede: AI должен знать иерархию дизайна и предлагать исправления на правильном уровне. Если проблема в балансе — менять числа, а не механики. Если проблема в механике — менять механику, а не концепцию. Если проблема в концепции — это красный свет, требующий серьёзного пересмотра.

### 10.9.3. Концепция «канарейки»

Гэзэуэй заимствует из шахтёрской практики концепцию «канарейки» — вычисление с заведомо известным результатом для проверки корректности данных. Примеры: сумма процентов = 100%, AVERAGE = SUM/COUNT (двойная проверка), общее количество элементов в каждом столбце. «Канарейки» — автоматические валидаторы: если канарейка «умерла» (результат не совпал), данные содержат ошибку.

Для AI-системы: при генерации любого набора данных AI автоматически включает проверочные вычисления. Если генерируются характеристики оружия — канарейка: DPS каждого оружия должен находиться в диапазоне  $\pm 20\%$  от среднего. Если генерируются уровни — канарейка: общее время прохождения всех уровней должно соответствовать целевому времени игры.

---

## 10.10. Убийцы удовольствия и валидация опыта

---

Фуллертон (Кн. 5) систематизирует «убийц удовольствия» (Fun Killers) — типичные проблемы, которые плейтиестирование призвано обнаружить и устранить. Для AI-системы это чек-лист автоматической валидации:

Убийца удовольствия	Описание	Решение	Как обнаружить в плейтесте
-----	-----	-----	-----
Микроменеджмент	Слишком много мелких решений, каждое из которых тривиально	Автоматизировать рутину, оставить только значимые решения	Игроки тратят больше времени на управление, чем на игру
Застой	Нет ощущения прогресса, действия не приводят к заметным изменениям	Добавить микро-цели, видимый прогресс, промежуточные награды	Игроки спрашивают «И что дальше?»
Непреодолимые препятствия	Невозможно продвинуться, нет альтернативных путей	Альтернативные пути, подсказки, динамическая сложность	Игроки застревают на одном месте >3 минуты
Произвольные события	Случайность без возможности влияния, несправедливые потери	Случайность + агентность: игрок влияет на исход, но не контролирует полностью	Игроки выражают фрустрацию: «Это нечестно!»
Предсказуемые пути	Один очевидный маршрут, нет причин исследовать	Ветвление, тайны,	



скрытые области, нелинейность | Игроки не исследуют, идут по прямой линии |

Фуллертон также классифицирует типы решений, которые игрок принимает в игре, — и эта классификация становится инструментом валидации:

Тип решения	Описание	Интересность	Пример в плейтесте
Тривиальное	Очевидный лучший выбор	Скучно — не выбор	Игроки всегда выбирают одно и то же
Безразличное	Нет разницы между вариантами	Бессмысленно — не выбор	Игроки выбирают случайно, равное распределение
Дилемма	Нет хорошего варианта	Напряжённо — но фрустрирующе	Игроки чувствуют себя запертыми
Значимое	Каждый вариант имеет уникальные последствия	Идеально — настоящий выбор	Игроки обсуждают, спорят, выбирают по-разному

Для Gidede: AI должен анализировать каждое решение в игре и классифицировать его по этой типологии. Если все решения тривиальные или безразличные — дизайн не создаёт значимого выбора, и это проблема уровня 2 (структура). Если решения — дилеммы без значимых вариантов — проблема в балансе (уровень 3).

---

## 10.11. Agile-мышление и управление итерациями

---

Бонд (Кн. 17) предлагает интеграцию Agile-методологии в процесс геймдизайна, что особенно ценно для AI-системы, которая должна управлять итерациями программно.

**Спринты** — короткие циклы разработки (1-2 недели), каждый из которых заканчивается проверяемым результатом. В контексте геймдизайна: каждый спринт — одна итерация цикла «прототип → тест → итерация».

**Ретроспективы** — после каждого спринта команда анализирует: что получилось? что не получилось? что улучшить в процессе? Это мета-итерация — итерация над самим процессом итерации.

**MVP (Minimum Viable Product)** для игр — минимальный играбельный прототип, который проверяет самую важную гипотезу: «FUN ли базовый цикл?». MVP — это не «урезанная версия финальной игры», а «минимальная проверка самой важной гипотезы». Разница принципиальна: MVP может выглядеть совершенно иначе, чем финальная игра, потому что его цель — не быть похожим на продукт, а отвечать на вопрос.

Для Gidede: Agile-мышление формализуется как система управления итерациями. AI ведёт трекер итераций: какая гипотеза проверяется, какой результат получен, какое изменение внесено, какой следующий шаг. Каждый цикл генерации → валидации → итерации — это аналог спринта. Ретроспектива — анализ паттернов: какие типы проблем возникают чаще всего? Какие изменения дают наибольший эффект? Это позволяет AI учиться и оптимизировать процесс генерации.

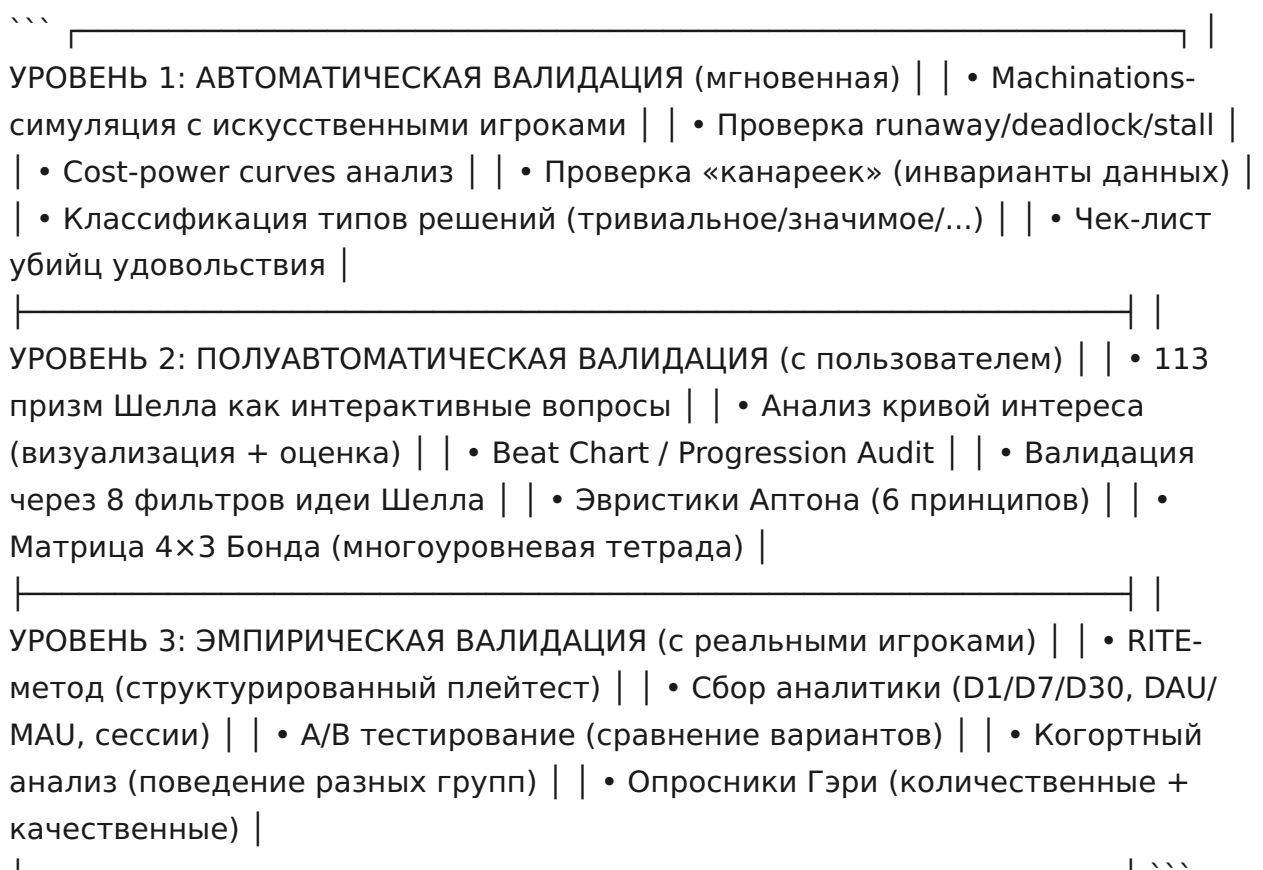
---

## 10.12. Интеграция в Gidede: модуль валидации и итерации

---

На основе всех проанализированных источников формируется архитектура модуля валидации и итерации для AI-системы Gidede.

### 10.12.1. Трёхуровневая система валидации



### 10.12.2. Цикл AI-итерации

Алгоритм итеративного улучшения, реализуемый AI:

1. **Генерация** — AI создаёт очередной вариант дизайна (механики, баланс, уровень, экономика) 2. **Автоматическая валидация** (уровень 1) — Machinations-симуляция, проверка инвариантов, анализ баланса 3. **Если проблемы найдены** — AI вносит одно изменение (принцип «одно изменение за итерацию») 4. **Повторный цикл** — переход к шагу 2 5. **Если автоматическая валидация пройдена** — представление результата пользователю 6. **Полуавтоматическая валидация** (уровень 2) — вопросы из призм Шелла, анализ кривой интереса 7. **Если пользователь вносит корректировки** — AI регенерирует с учётом обратной связи 8. **Повторный цикл** — переход к шагу 2 9. **Если полуавтоматическая валидация пройдена** — рекомендации по эмпирической валидации (уровень 3) 10. **Эмпирическая валидация** — RITE-метод с реальными игроками, сбор аналитики 11. **Анализ данных** — AI помогает интерпретировать результаты и формулировать гипотезы 12. **Возврат к шагу 1** — на основе данных плейтеста

### 10.12.3. Контрольный список для AI-валидации

На основе всех проанализированных источников формируется контрольный список, который AI применяет к каждому сгенерированному дизайну:

**Фундамент (Уровень 1 по Фуллертон):** - [ ] Определена цель опыта (Player Experience Goal по Фуллертон)? - [ ] Core Loop описан и работает (FUN в минимальном цикле)? - [ ] Главный конфликт идентифицирован (Гэри)? - [ ] Есть ли значимые решения (не тривиальные и не безразличные)? - [ ] Нет ли убийц удовольствия (микроманеджмент, застой, непреодолимые препятствия)?

**Структура (Уровень 2):** - [ ] Все системы связаны, нет изолированных механик? - [ ] Нет тупиков (ситуаций, где невозможно продолжить)? - [ ] Кривая интереса имеет крючок, нарастание, кульминацию, завершение? - [ ] Beat Chart показывает чередование интенсивности и отдыха? - [ ] Прогрессия использует все 5 типов (новая/вариация/комбинация/инверсия/удаление)?

**Баланс (Уровень 3):** - [ ] Нет доминантной стратегии (проверено через симуляцию)? - [ ] Cost-power curves не показывают доминирования? - [ ] «Канарейки» проходят (все инварианты данных корректны)? - [ ] Типы решений — значимые (не тривиальные и не безразличные)? - [ ] Баланс петель обратной связи (усиливающие + балансирующие)?

**Шлифовка (Уровень 4):** - [ ] Фидбэк на каждое действие (правило 1/10 секунды)? - [ ] «Сочность» интерфейса (многослойный фидбэк, juiciness)? - [ ]

Косвенное руководство (7 методов Бонда)? - [ ] Доступность (цветовая слепота, слух, моторика, когнитивная)?

**Процесс:** - [ ] Текущая стадия дизайна определена (1–5 по Гэри)? - [ ] Иерархия дизайна соблюдается (исправления на правильном уровне)? - [ ] Принцип «одно изменение за итерацию» соблюден? - [ ] Гипотеза сформулирована количественно (Гэзэуэй)? - [ ] Критерий успеха определен?

---

## 10.13. Резюме: плейтестинг как образ мышления

---

Плейтестинг и итерация — не фаза разработки, а **образ мышления** геймдизайнера. Шелл формулирует это как Правило цикла: итерация — не этап, а состояние. Фуллертон ставит плейтест в центр методологии. Селлерс описывает петлю дизайнера как мета-петлю, охватывающую весь процесс. Гэри сводит всё к базовому циклу из шести шагов, который повторяется снова и снова. Бонд добавляет Agile-мышление: спринты, ретроспективы, MVP. Гэзэуэй превращает итерацию в научный метод с измеримыми критериями.

Для AI-системы Gidede это означает: **модуль валидации не может быть опциональным**. Он должен быть встроен в каждый этап генерации, от концепции до финальной полировки. AI должен не только генерировать дизайн, но и проверять его, исправлять и проверять снова — в цикле, который повторяется до тех пор, пока все критерии качества не будут удовлетворены. Это превращает AI из генератора в **генератор-валидатор**, способный к самоулучшению — что является конечной целью проекта Gidede.

---

## Глава 11. Шаблоны GDD и чек-листы

---

> **Раздел Библии геймдизайна:** 2.11 — Шаблоны GDD и чек-листы >

**Источники:** KB-15 (Документация), KB-14 (Валидация), KB-03 (MDA Framework), KB-07 (Core Loop), KB-08 (Баланс), KB-11 (Наппатив), KB-12 (Level Design), KB-13 (Опыт и мотивация) > **Книги:** Шелл (Кн. 1), Фуллертон (Кн. 5), Роджерс (Кн. 8), Роллингс/Моррис (Кн. 12), Зубек (Кн. 6), Бонд (Кн. 17), Андрианова/Яковлева (Кн. 3), Гэри (Кн. 10), Селлерс (Кн. 13)

---

## 11.1. Философия документации: почему «волшебного шаблона» не существует — и что с этим делать

---

Шелл (Кн. 1) формулирует ключевое заблуждение начинающих дизайнеров: **«Волшебного шаблона не существует!»** Нет универсального формата GDD, который подошёл бы любой игре, любой команде и любому этапу разработки. GDD — это не формуляр, который нужно заполнить, а **теория** о том, что сделает игру хорошей. И, как любая теория, она устаревает в момент написания — потому что игра меняется с каждой итерацией, и документ, не обновлённый после последнего плейтеста, уже не описывает реальность.

Однако из этого не следует, что документация бесполезна. Шелл выделяет **две фундаментальные цели** любой документации в геймдизайне:

**Хранение** — человеческая память ненадёжна. Дизайнер, который «держит всё в голове», через три месяца не вспомнит, почему принял конкретное решение о балансе, и потратит часы на повторное осмысление уже решённой проблемы. Документация создаёт институциональную память, независимую от конкретных людей. В командах из пяти и более человек устная передача информации неизбежно искажается — «испорченный телефон» превращает «увеличить урон на 20%» в «изменить оружие», и два программиста реализуют разные интерпретации одного и того же решения. Документ устраняет неоднозначность.

**Передача** — документация должна не только хранить решения, но и передавать их контекст. Число «50 единиц урона» бессмысленно без контекста: это много или мало? Для какого уровня? Против какого врага? Хорошая документация объясняет не только «что», но и «почему» — каждое решение сопровождается обоснованием. Это критически важно для онбординга новых членов команды: документ, который отвечает на вопрос «почему мы сделали именно так», экономит недели адаптации.

Роллингс и Моррис (Кн. 12) поднимают аргументацию на уровень принципа: **«Документация — ось производственного процесса»**. Вокруг документации вращается весь процесс создания игры. В настольных играх правила видны и осязаемы — можно «потрогать» карточку, прочитать текст, обсудить механику. В компьютерных играх механика скрыта экраном — нельзя «потрогать» правила, нельзя увидеть формулу расчёта урона, нельзя «пощупать» баланс экономики. Поэтому письменное изложение проекта — не приятное дополнение, а жизненно важный элемент создания игры.

Недооценка документации = хаос в производстве: программисты реализуют не то, что задумал дизайнер, художники рисуют не тот стиль, продюсер не понимает скоуп.

Зубек (Кн. 6) добавляет прагматичную ноту: **дизайнерская документация — живой документ**, обновляемый с каждой итерацией. Не заменяет прототип, но дополняет его. Прототип показывает, как игра работает; документация объясняет, почему она работает именно так. Идеальный процесс: прототип → документация → плейтест → обновление документации → новая итерация прототипа. Документация, не обновлённая после последнего плейтеста, — мёртвый документ, который приносит больше вреда, чем пользы, потому что описывает не существующую игру.

Для Gidede этот принцип означает: модуль генерации документации не создаёт статический документ «один раз и навсегда», а генерирует **живую документацию**, которая автоматически обновляется при каждом изменении в модели игры. Каждое изменение механики, баланса или нарратива сразу отражается в соответствующих разделах GDD. Документация — это проекция текущего состояния модели, а не отдельный артефакт, требующий ручной синхронизации.

---

## 11.2. Три уровня документации: от манифеста до спецификации

---

Роллингс и Моррис (Кн. 12) предлагают **трёхуровневую модель документации**, которая покрывает весь жизненный цикл проекта — от первого вдохновения до детальной спецификации. Эта модель исключительно ценна для AI-системы, потому что каждый уровень имеет чёткую функцию, целевую аудиторию и объём, что позволяет генерировать документацию адаптивно: для питча — уровень 1, для препродакшена — уровень 2, для продакшена — уровень 3.

### 11.2.1. Техническое предложение (Gameplay Treatment)

**Объём:** 5–6 страниц. **Цель:** манифест, разъясняющий основную концепцию игры. **Аудитория:** издатель, инвестор, команда (для синхронизации видения). **Функция:** ответить на вопрос «Что это за игра и зачем она нужна?».

Роллингс и Моррис формулируют ключевые элементы технического предложения:

1. **Тип игры** — жанр, платформа, целевая аудитория, примерная длительность. Это не просто ярлык, а контракт с читателем: если заявлен «тактический RPG», читатель ожидает определённый набор механик и опыта. Тип игры задаёт систему ожиданий, которую остальная документация должна подтверждать, а не опровергать.

2. **Сюжет и оригинальность** — что делает игру уникальной? Это не пересказ истории (для этого есть отдельные документы), а ответ на вопрос: «Почему игрок выберет именно эту игру среди сотен других?» Уникальность может быть в сеттинге, механике, опыте, аудитории — но она должна быть сформулирована ясно и конкретно. «Уникальная боевая система» — не ответ; «Боевая система, где каждый удар требует выбора между тремя стойками, и правильная стойка в три раза эффективнее неправильной» — ответ.

3. **Реализуемость** — технически возможно создать эту игру? В какие сроки? С каким бюджетом? Какая команда нужна? Этот раздел отличает мечту от проекта: красивая идея, которую невозможно реализовать с имеющимися ресурсами — не проект, а фантазия. Техническое предложение должно честно оценивать реалистичность, иначе проект обречён на перерасход или провал.

**Принцип новизны** (Роллингс/Моррис): «Нет ничего нового под солнцем» — задача дизайнера не изобрести с нуля, а применить известные идеи совершенно новым способом. Техническое предложение должно показать, как известные механики комбинируются в новую, уникальную конфигурацию. Пример: Portal = Narbacular Drop (порталы) + Half-Life (сеттинг, физика) + GLaDOS (нарратив) = совершенно новая игра из известных элементов.

Для Gidede: техническое предложение генерируется на основе введённых данных — жанр, целевая эстетика (MDA), ключевые механики (MechanicsDB), уникальность (обратный MDA). AI заполняет шаблон, но оставляет пространство для ручной доработки — особенно в разделе «оригинальность», где требуется креативность.

### 11.2.2. Эскизный проект (Sketch Design)

**Объём:** 15–30 страниц. **Цель:** детализация концепции до уровня, на котором можно создать натурный макет. **Аудитория:** геймдизайнеры, программисты, художники. **Функция:** ответить на вопрос «Как именно игра работает?».

Эскизный проект расширяет техническое предложение, описывая:

1. **Игровые механики в деталях** — каждая механика описана отдельно: что делает игрок, что происходит в ответ, какие параметры настраиваются, как механика взаимодействует с другими. Это уровень, на котором абстрактная

идея «боевая система» превращается в конкретные правила: «Игрок выбирает атаку из трёх типов — быстрая (10 урона, 0.3 сек), средняя (25 урона, 0.7 сек), тяжёлая (50 урона, 1.5 сек). Враг блокирует с вероятностью 30%/50%/10% соответственно».

2. **Структуру уровней** — сколько уровней, какова их последовательность, какие механики представлены на каждом уровне, какова арка сложности. Для AI-системы: это уровень, на котором применяется модель «Основа + Изюминка» (Кн. 11) и Beat Chart (Роджерс, Кн. 8).

3. **Систему прогрессии** — как игрок развивается: кривая опыта, дерево навыков, открытие нового контента, разблокировка механик. Этот раздел напрямую связан с KB-09 (Прогрессия) и использует кривые из раздела 2.6.

4. **Пользовательский интерфейс** — как игрок взаимодействует с игрой: схема управления, расположение HUD, информационные панели, меню. Фуллертон (Кн. 5) формулирует 6 принципов дизайна интерфейса: форма следует функции, метафоры, визуализация, группировка, консистентность, фидбэк.

Ключевой принцип Роллингса/Морриса: **эскизный проект должен быть достаточно детальным, чтобы натурный макет игры можно было создать на его основе**. Это критерий готовности: если программист может построить прототип, читая только эскизный проект — документация достаточна. Если программист приходит с вопросами — документация неполна.

### 11.2.3. Полная документация проектировщика

**Объём:** 50–200+ страниц (зависит от масштаба проекта). **Цель:** исчерпывающее описание всех систем, механик и контента. **Аудитория:** вся команда разработки. **Функция:** эталонная спецификация, против которой проверяется реализация.

Полная документация включает:

1. **Все механики** с формулами, таблицами, диаграммами состояний. Каждое число обосновано: почему именно 50 единиц урона, а не 40 или 60? Какова формула расчёта? Какие параметры влияют? Это уровень, на котором применяются математические модели из раздела 2.5 (Баланс) и экономический аппарат из раздела 2.6.

2. **Все системы** с их взаимодействиями: как боевая система связана с экономикой, как прогрессия зависит от контента, как нарратив влияет на



механики. Для визуализации взаимодействий используется матрица взаимодействия свойств (Роллингс/Моррис, Кн. 12): свойства × свойства = связи, где каждая ячейка указывает тип взаимодействия (усиление, конфликт, независимость).

3. **Весь контент** — описания врагов, предметов, уровней, квестов с полными спецификациями. Это самый объёмный раздел, и в AI-системе он генерируется из MechanicsDB, Core Loop Model, Balance Data и Narrative Engine.

4. **Технические ограничения** — платформа, движок, производительность, память. Раздел, который часто упускают в геймдизайнерской документации, но который критически важен: дизайнер, предлагающий 10 000 NPC на экране, должен знать, что консоль потянет максимум 500.

Роллингс и Моррис подчёркивают: **форма документации может быть любой** — письменное изложение, эскизы, музыка, схемы, диаграммы, натурный макет — всё, что способствует донесению концепции. Нет требования «только текст» — если диаграмма передаёт мысль лучше абзаца, используйте диаграмму. Если прототип заменяет 10 страниц описания механики, прикладывайте прототип. Цель — не заполнить страницы, а обеспечить коммуникацию.

Для Gidede трёхуровневая модель формализуется как **pipeline генерации документации**:

``` ВХОД: Модель игры (Concept + Mechanics + Balance + Economy + Narrative + Levels)

ШАГ 1: Техническое предложение (5-6 стр.) → Автозаполнение: жанр, аудитория, уникальность, Core Loop, целевая эстетика → Ручная доработка: оригинальность, видение

ШАГ 2: Эскизный проект (15-30 стр.) → Автозаполнение: механики, прогрессия, UI, структура уровней → Диаграммы: Core Loop, геймплейные циклы, карта мира

ШАГ 3: Полная документация (50-200+ стр.) → Автозаполнение: все системы, формулы, таблицы, контент → Валидация: полнота через шаблон (38 секций Роджерса + 3 уровня Роллингса/Морриса) ```

11.3. Форматы документации: от One-Sheet до полного GDD

Роджерс (Кн. 8) предлагает наиболее детализированную систему форматов документации, практичных для индустрии. Его подход дополняет трёхуровневую модель Роллингса/Морриса конкретными шаблонами, которые можно напрямую использовать в AI-системе.

11.3.1. One-Sheet (Лист концепции)

Объём: 1 страница. **Цель:** мгновенная коммуникация сути игры. **Аудитория:** все — от инвестора до тестировщика.

One-Sheet — самый сжатый формат документации, эквивалент логлайна в киноиндустрии. Его задача — за 30 секунд чтения дать полное представление о том, что это за игра. Роджерс рекомендует включать:

| Элемент | Описание | Пример | |-----|-----|-----| | **Название** |
Запоминающееся, отражающее суть | «Hollow Knight» — пустота + рыцарь | |
Логлайн | 1 предложение: кто + где + что + почему | «Маленький жук-рыцарь исследует руины забытого королевства, сражаясь с заражёнными существами» | |
Жанр | Конкретный, с уточнением | «Metroidvania с уклоном в хардкорный экшн» | |
Целевая аудитория | Кто будет играть и почему | «Фанаты Dark Souls и Castlevania, ищущих челлендж в 2D» | |
Уникальность |
Одна ключевая отличительная черта | «Система амулетов, радикально меняющая стиль игры» | |
Визуальный крючок | Ключевой арт, передающий атмосферу | Концепт-арт мрачного подземного города |

Фуллerton (Кн. 5) добавляет к One-Sheet элементы питча: логлайн + ключевой арт + прототип + целевая аудитория + уникальность. По сути, One-Sheet — это документированный питч, который можно отправить по почте, распечатать для встречи или показать на экране за 15 секунд.

Для Gidede: One-Sheet генерируется автоматически из введённых данных. AI формулирует логлайн на основе жанра, сеттинга и ключевой механики. Визуальный крючок генерируется через image-generation промпт. Целевая аудитория определяется через мотивационный профиль (модель Йи, KB-13).

11.3.2. Ten-Pager (Десятистраничный обзор)

Объём: 5–10 страниц. **Цель:** достаточная детализация для принятия решения о финансировании / начале препродакшена. **Аудитория:** продюсер, издатель, ведущий дизайнер.

Ten-Pager — это расширенный One-Sheet, добавляющий конкретику о геймплее, но не уходящий в детали реализации. Роджерс рекомендует структуру:

1. **Название и логлайн** (из One-Sheet) 2. **Концепция** — подробное описание идеи: что делает игру уникальной, какой опыт она даёт, чем отличается от аналогов 3. **Core Loop** — визуальная диаграмма основного игрового цикла с кратким описанием каждой петли 4. **Уникальные механики** — 2–3 ключевые механики, которые определяют идентичность игры (не все механики, а только уникальные) 5. **Сеттинг и мир** — краткое описание вселенной, атмосферы, ключевых локаций 6. **Целевая аудитория и рынок** — кто будет играть, на какой платформе, какова конкуренция 7. **Монетизация** — модель (Premium / F2P / Подписка), примерная экономика 8. **Milestones** — ключевые этапы разработки с примерными сроками 9. **Риски и митигация** — главные риски проекта и способы их снижения 10. **Команда** — ключевые участники и их квалификация

Для Gidede: Ten-Pager — второй уровень генерации, автоматически заполняемый на основе One-Sheet + расширенных данных о механиках, экономике и прогрессии. AI добавляет визуальные диаграммы Core Loop, экономические модели и анализ рынка.

11.3.3. Полный GDD (Game Design Document)

Объём: 50–200+ страниц. **Цель:** исчерпывающая спецификация для реализации. **Аудитория:** вся команда разработки.

Роджерс предлагает наиболее полный шаблон GDD, состоящий из **38 секций**, организованных в 8 блоков:

... |
GDD по Роджерсу (38 секций) |
| | БЛОК
1: ОБЗОР ИГРЫ (1-6) | | 1. Название игры 2. Обзор | | 3. Жанр и платформа 4.
Целевая аудитория | | 5. Уникальность 6. Лицензия / IP |
| | БЛОК
2: ГЕЙМПЛЕЙ (7-14) | | 7. Core Loop 8. Управление | | 9. Механики 10. Камера и

| | |
|--|------|
| перспектива 11. Прогрессия 12. Баланс 13. Сложность 14. Режимы игры | |
| БЛОК 3: ПЕРСОНАЖИ И НАРРАТИВ (15-19) 15. Персонажи 16. Сюжет 17. Диалоги 18. Квесты 19. Лор и мир | БЛОК |
| 4: УРОВНИ И МИР (20-23) 20. Структура мира 21. Дизайн уровней 22. Навигация 23. Боевые пространства | БЛОК |
| 5: ЭКОНОМИКА И ПРОГРЕССИЯ (24-27) 24. Ресурсы 25. Экономика 26. Дерево технологий 27. Кривая сложности | БЛОК |
| 6: ИНТЕРФЕЙС И ВИЗУАЛ (28-31) 28. HUD и UI 29. Меню и навигация 30. Визуальный стиль 31. Звук и музыка | БЛОК |
| 7: МУЛЬТИПЛЕЕР И СОЦИАЛЬ (32-34) 32. Режимы 33. Социальные функции 34. Мета-игра | БЛОК |
| 8: ТЕХНИЧЕСКИЕ И БИЗНЕС (35-38) 35. Технические требования 36. Платформа и порты 37. Монетизация 38. Milestones и бюджет | ... |

Однако Шелл (Кн. 1) предупреждает: **«Лучше иметь несколько небольших связанных документов, чем один гигантский»**. Полный GDD из 200 страниц — это документ, который никто не прочитает целиком. Он устаревает в момент написания. Программист ищет конкретную формулу, художник — описание стиля, продюсер — сроки и бюджет. Каждый читает свой раздел и игнорирует остальные. Проблема: при изменении одного раздела (например, формулы расчёта урона) нужно обновить все зависимые разделы (баланс врагов, экономика, прогрессия), но кто-то забудет, и документация рассинхронизируется.

11.3.4. Модульная документация Gidede

Решение, объединяющее полноту Роджерса с прагматизмом Шелла — **модульная документация**, где 38 секций GDD реализованы как независимые модули, связанные через единую модель данных:

``` Модульная документация Gidede |— M-01: Concept Card (= One-Sheet) | |— Автогенерация из: жанр + эстетика + ключевая механика |— M-02: Core Loop Diagram | |— Автогенерация из: Core Loop Model (KB-07) |— M-03: Mechanics Bible | |— Автогенерация из: MechanicsDB (KB-04) |— M-04: Balance Spec | |— Автогенерация из: Balance Model (KB-08) |— M-05: Economy Spec | |—

Автогенерация из: Economy Model (KB-10) | — M-06: Progression Spec | —  
Автогенерация из: Progression Model (KB-09) | — M-07: Narrative Bible | —  
Автогенерация из: Narrative Engine (KB-11) | — M-08: Level Design Spec | —  
Автогенерация из: LD Model (KB-12) | — M-09: UI/UX Spec | — Ручная + чек-  
лист (6 принципов Фуллертон) | — M-10: Art Bible (промпты) | — AI-генерация  
промптов + референсы | — M-11: Audio Bible (промпты) | — AI-генерация  
промптов для звука и музыки | — M-12: Monetization Spec | — Выбор модели  
+ формулы LTV/CAC/ROAS | — M-13: Production Plan | — Milestones + бюджет +  
команда ```

**Ключевое преимущество модульности:** изменение в одном модуле автоматически обновляет все зависимые. Если дизайнер меняет формулу расчёта урона в M-04 (Balance Spec), это автоматически отражается в M-03 (Mechanics Bible), M-05 (Economy Spec — потому что урон влияет на скорость фарма), M-06 (Progression Spec — потому что баланс врагов определяет кривую сложности) и M-08 (Level Design — потому что боевое пространство проектируется под конкретные параметры боя). Единство модели данных гарантирует консистентность документации без ручной синхронизации.

---

## 11.4. Специализированные типы документации

---

### 11.4.1. Нарративная документация

Андрианова и Яковлева (Кн. 3) предлагают **11 типов нарративной документации**, каждый из которых обслуживает конкретную задачу:

| #  | Документ                    | Объём             | Цель                                         | Когда создаётся |
|----|-----------------------------|-------------------|----------------------------------------------|-----------------|
| 1  | <b>Логлайн</b>              | 1 предложение     | Суть истории в одном предложении             | Концепция       |
| 2  | <b>Синопсис</b>             | 1 абзац           | Краткое изложение сюжета                     | Концепция       |
| 3  | <b>Библия сеттинга</b>      | 5–20 стр.         | Правила мира, география, история, расы       | Препродакшен    |
| 4  | <b>Библия персонажа</b>     | 2–5 стр./персонаж | Мотивация, арка, голос, отношения            | Препродакшен    |
| 5  | <b>Карта сюжетных линий</b> | Визуальная        | Параллельные сюжеты и их пересечения         | Препродакшен    |
| 6  | <b>Сценарий кат-сцен</b>    | Полный            | Диалоги, действия, камера                    | Продакшен       |
| 7  | <b>База диалогов</b>        | Таблица           | Все реплики с контекстом и условиями         | Продакшен       |
| 8  | <b>Матрица квестов</b>      | Таблица 8×4       | 32 комбинации типа × цели квеста             | Дизайн          |
| 9  | <b>Лор-база</b>             | Вики              | Все элементы лора с перекрёстными ссылками   | Постоянно       |
| 10 | <b>Валидатор диссонанса</b> | Чек-лист          | Проверка согласованности нарратива и механик |                 |

Каждая итерация | | 11 | **Питч-дек** | 10–15 слайдов | Презентация нарратива для стейкхолдеров | Питчинг |

**Ключевой документ — Библия персонажа** — заслуживает отдельного внимания. Андрианова/Яковлева рекомендуют включать:

- **Внешность и характер** — визуальный образ и ключевые черты, которые определяют поведение - **Мотивация** — что персонаж хочет и почему (внутренняя и внешняя) - **Арка развития** — как персонаж изменяется от начала к концу - **Голос** — как персонаж говорит: словарь, тон, обороты - **Отношения** — как персонаж связан с другими (циркумплекс: дружелюбный ↔ враждебный, доминантный ↔ покорный) - **Противоречие** — что делает персонаж интересным: конфликт между долгом и желанием, между страхом и смелостью

Для Gidede: нарративная документация генерируется через Narrative Engine (KB-11), который на основе типа нарратива (Embedded/Emergent), структуры сюжета (4 типа) и квестовой матрицы (32 комбинации) автоматически создаёт M-07 (Narrative Bible). Валидатор лудонарративного диссонанса проверяет каждую итерацию.

#### 11.4.2. Дизайн-документы Фуллертон

Фуллертон (Кн. 5) рекомендует **модульные документы** вместо одного гигантского GDD (перекликаясь с Шеллом):

1. **One-Pager** (1 страница — концепция) — аналог One-Sheet Роджерса
2. **Design Macro** (5–10 страниц — обзор) — аналог Ten-Pager
3. **Детальные спецификации** (по модулю) — отдельные документы для каждой системы
4. **Диаграммы потоков** (flowcharts) — визуализация взаимодействия систем
5. **Концепт-арт** — визуальная коммуникация атмосферы и стиля

Фуллертон подчёркивает: «**AI — инструмент, а не дизайнер. Окончательные решения принимает человек**». Документация, сгенерированная AI, — это черновик, отправная точка для обсуждения, а не финальный артефакт. Роль AI — ускорить создание документации и обеспечить полноту, но ответственность за качество лежит на дизайнере.

#### 11.4.3. Концепт-документ Селлерса

Селлерс (Кн. 13) предлагает формат концепт-документа, фокусирующийся на **целевом опыте** (в духе Playcentric Design Process):

1. **Player Experience Goal** — целевое ощущение, которое должен получить игрок 2. **Core Mechanic** — основная механика, создающая это ощущение 3. **Core Loop** — визуальная диаграмма цикла с описанием каждой петли 4. **Системная карта** — как системы пересекаются и поддерживают друг друга 5. **Паттерны обратной связи** — какие петли усиливают, какие балансируют 6. **Метрики успеха** — как мы узнаем, что опыт достигнут (D1 Retention  $\geq$  40%, средняя сессия  $\geq$  15 мин и т.д.)

Концепт-документ Селлерса — наиболее «системный» из всех форматов: он не описывает игру как набор фич, а моделирует её как систему, производящую опыт. Это ближе всего к подходу Gidede, где игра = модель, а документация = проекция модели.

---

## 11.5. Система чек-листов: валидация геймдизайна

---

Чек-лист — не формальность, а **систематический инструмент валидации**, который предотвращает слепые зоны. Дизайнер, работающий без чек-листа, проверяет то, что помнит, и забывает проверить то, что не помнит. Чек-лист гарантирует полноту: каждый аспект дизайна рассмотрен, даже если ответ — «не применимо».

### 11.5.1. 113 линз Шелла как мега-чек-лист

Самый полный каталог проверочных вопросов — **113 линз (призм) Шелла** (Кн. 1), организованных в 16 категорий. Каждая линза — это перспектива, через которую дизайнер рассматривает свою игру. Линзы не являются формальными критериями — это вопросы, которые стимулируют мышление и выявляют проблемы.

Для AI-системы 113 линз — это **библиотека валидационных вопросов**, которую можно адаптивно применять:

| Категория | Линзы | Когда применять | |-----|-----|-----| | Опыт игрока | 1-3, 68-70 | Концепция: определение целевого опыта | | Удивление и мотивация | 4-8 | Идеация: проверка увлекательности | | Элементы и целостность | 9-12 | Каждая итерация: проверка единства | | Идея и итерация | 13-18 | Концепция: валидация идеи | | Игрок и мотивация | 19-25 | Профилирование аудитории | | Механики | 26-36 | Дизайн механик | | Баланс | 37-53 | Балансировка | | Головоломки | 54-58 | Дизайн головоломок | |

Интерфейс | 59–67 | UI/UX дизайн | | Интерес и нарратив | 68–82 | Нарративный дизайн | | Мир и персонажи | 83–91 | Дизайн мира | | Среда и архитектура | 92–95 | Level Design | | Социальные аспекты | 96–99 | Мультиплеер | | Команда и процесс | 100–103 | Управление | | Технология и бизнес | 104–109 | Производство | | Этика | 110–113 | Ответственность |

**Принцип адаптивного применения:** не все 113 линз одинаково важны для каждой игры. Линза справедливости (37) критична для PvP-игр, но почти не применима к single-player puzzle. Линза сюрприза (4) важна для нарративных игр, но менее актуальна для спортивных симуляторов. AI должен выбирать релевантные линзы на основе жанра, типа Core Loop и целевой аудитории.

### 11.5.2. Восемь фильтров идеи Шелла

Шелл (Кн. 1) предлагает компактный чек-лист для валидации концепции — **восемь фильтров**, через которые нужно пропустить каждую идею:

| # | Фильтр               | Ключевой вопрос                                    | Красный флаг                           |
|---|----------------------|----------------------------------------------------|----------------------------------------|
| 1 | Художественное чутьё | «Нравится ли мне эта игра?»                        | Нет эмоциональной связи                |
| 2 | Аудитория            | «Подойдёт ли эта игра моей целевой аудитории?»     | Несоответствие жанра и аудитории       |
| 3 | Создание опыта       | «Достаточно ли хорош дизайн игры?»                 | Механики не создают заявленный опыт    |
| 4 | Инновации            | «Достаточно ли игра инновационная?»                | Клон без уникальности                  |
| 5 | Бизнес и маркетинг   | «Будет ли игра продаваться?»                       | Нет рынка / перенасыщенный рынок       |
| 6 | Разработка           | «Технически возможно создать эту игру?»            | Нереализуемо с текущими ресурсами      |
| 7 | Социальный компонент | «Оправдывает ли социальная составляющая ожидания?» | Обязательный мультиплеер без аудитории |
| 8 | Плейтесты            | «Нравится ли игра игрокам?»                        | Игрокам скучно/непонятно               |

**Правило цикла:** если идея не проходит один фильтр — модифицируй и прогони через ВСЕ восемь заново, потому что исправление одного может сломать другое. Увеличение инновации (фильтр 4) может снизить реализуемость (фильтр 6). Добавление мультиплеера (фильтр 7) может нарушить бюджет (фильтр 5). Восемь фильтров — система, а не список: каждый фильтр влияет на остальные.

Для Gidede: восемь фильтров — автоматический валидатор концепции. AI проверяет каждую сгенерированную концепцию через все 8 фильтров и помечает слабые места. Пример вывода: «Фильтр 4 (Инновации): концепция использует стандартную формулу RPG без уникальных механик. Рекомендация: добавить механику X из MechanicsDB для дифференциации».



### 11.5.3. Контрольный список валидации баланса (Роллингс/Моррис)

Роллингс и Моррис (Кн. 12) предлагают конкретный чек-лист для валидации баланса, основанный на их модели 3 типов баланса + Q-фактор + SPS + золотое правило:

- [ ] **Баланс PvP**: Нет изначальных преимуществ? Случайные события равновероятны для всех? - [ ] **Баланс «игрок — геймдизайнер»**: Уровень владения соответствует вызову? Худший враг ≠ сама игра? - [ ] **Баланс «геймдизайнер — игрок»**: Все возможности сопоставимы по ценности? Доминирующие стратегии компенсированы? - [ ] **Q-фактор**: Каждый компонент лучше других хотя бы по одному атрибуту? Нет ли «мёртвых» компонентов (худших по всем атрибутам)? - [ ] **Правило SPS**: Взаимосвязи между компонентами образуют цикл «камень-ножницы-бумага»? Нет статически доминирующего выбора? - [ ] **Золотое правило**: Случайный выбор никогда не является оптимальной стратегией? - [ ] **Масштабируемость**: Баланс не нарушится при добавлении новых элементов?

### 11.5.4. Эвристики Аптона (через Зубека)

Зубек (Кн. 6) приводит 6 эвристик Аптона для валидации механик — универсальный чек-лист, применимый к любой механике:

- [ ] **Выбор**: Есть ли у игрока осознанный выбор из нескольких вариантов? - [ ] **Разнообразие**: Меняются ли доступные действия со временем? - [ ] **Последствия**: Приводят ли действия к приписываемым им результатам? - [ ] **Предсказуемость**: Может ли игрок понять связь действий и исходов? - [ ] **Неопределённость**: Не является ли исход полностью предрешённым? - [ ] **Удовлетворение**: Достижимы ли желанные результаты при разумных усилиях?

### 11.5.5. Чек-лист косвенного руководства (Бонд)

Бонд (Кн. 17) предлагает 7 методов косвенного руководства игроком — чек-лист для Level Design и навигации:

- [ ] **Указывающие линии**: Элементы среды направляют взгляд - [ ] **Камера**: Направление взгляда через положение камеры - [ ] **Контраст**: Яркость, текстура, цвет, направленность - [ ] **Звук**: Музыка для настроения, эффекты для локальных подсказок - [ ] **Имитация**: NPC демонстрируют действия - [ ] **Отсутствие альтернатив**: Закрытые двери, тупики - [ ] **Брендинг**: Визуальный язык, приучающий к ассоциациям

### 11.5.6. Убийцы удовольствия (Фуллертон)

Фуллертон (Кн. 5) формулирует 5 «убийц удовольствия» — анти-паттернов, которые разрушают игровой опыт:

| Убийца | Описание | Решение | |-----|-----|-----| | **Микроменеджмент** | Слишком много мелких решений без значимости | Автоматизировать рутину | | **Застой** | Нет ощущения прогресса | Добавить микро-цели и фидбэк | | **Непреодолимые препятствия** | Невозможно продвинуться без удачи | Альтернативные пути, подсказки | | **Произвольные события** | Случайность без возможности влияния | Случайность + агентность | | **Предсказуемые пути** | Один очевидный маршрут без альтернатив | Ветвление, тайны, секреты |

### 11.5.7. Чек-лист целей проектирования (Бонд)

Бонд (Кн. 17) формулирует 4 цели для игрока и 3 цели для игры:

**Для игрока:** - [ ] **Уважение:** Ценится ли время и вложения игрока? - [ ] **Магический круг:** Не нарушается ли погружение? - [ ] **Поток:** Сложность между скукой и разочарованием? - [ ] **Интересные решения:** Есть ли обоюдоострые выборы с последствиями?

**Для игры:** - [ ] **Формальная система:** Чёткие правила, определяющие структуру? - [ ] **Неопределённость:** Исход неизвестен заранее? - [ ] **Неравноценность результатов:** Есть градации успеха?

---

## 11.6. Интегрированный валидатор Gidede: Universal Design Validator

---

На основе синтеза всех чек-листов из 17 книг формируется **Universal Design Validator** — ядро валидационной системы Gidede. Валидатор работает на двух осях: **горизонтальной** (полнота — все ли аспекты проверены) и **вертикальной** (глубина — насколько глубоко проверен каждый аспект).

### 11.6.1. Структура валидатора

''' \_\_\_\_\_ |  
UNIVERSAL DESIGN VALIDATOR (Gidede) |  
|\_\_\_\_\_|||  
УРОВЕНЬ 1: ВАЛИДАЦИЯ КОНЦЕПЦИИ | | — 8 фильтров идеи (Шелл) | | —

Логлайн: кто + где + что + почему | | — Целевая эстетика: какие из 8 MDA-эстетик? | | — Профиль аудитории: мотивация по Йи | | — Уникальность: чем отличается от аналогов? | | — Реализуемость: технически + бюджет + сроки | | | УРОВЕНЬ 2: ВАЛИДАЦИЯ МЕХАНИК | | — 6 эвристик Аптона (выбор, разнообразие, последствия, | | | предсказуемость, неопределённость, удовлетворение) | | — Многоуровневая тетрада: все 12 ячеек заполнены? | | — MechanicsDB: совместимость выбранных механик? | | — Языковая метафора: существительные + глаголы + грамм. | | — Пространство состояний: фактор ветвления, плотность | | | УРОВЕНЬ 3: ВАЛИДАЦИЯ CORE LOOP | | — 5 вопросов кор-геймплея (Гэри) | | — «30 секунд веселья» (Halo) | | — Концентрические схемы: нет разрывов между циклами? | | — 4 главные петли (Селлерс): все работают? | | — Геймплейные циклы: микро→мезо→макро→мета | | | УРОВЕНЬ 4: ВАЛИДАЦИЯ БАЛАНСА | | — 3 типа баланса (PvP, игрок-геймдизайнер, геймдизайнер- | | | игрок) по Роллингу/Моррису | | — Q-фактор: нет мёртвых компонентов? | | — SPS: взаимосвязи образуют цикл? | | — Золотое правило: случайный выбор ≠ оптимальная стратегия? | | — Cost-power curves: нет доминирующих элементов? | | — Масштабируемость: добавление элементов не ломает баланс? | | | УРОВЕНЬ 5: ВАЛИДАЦИЯ ЭКОНОМИКИ И ПРОГРЕССИИ | | — Faucets/Drains: равновесие? | | — Циклы конверсии: прибыльность динамически настраивается? | | — Патологии: нет инфляции, стагнации, арбитража? | | — Кривая прогрессии: соответствует целевой? | | — Эмерджентная прогрессия: есть открытие помимо скрипта? | | | УРОВЕНЬ 6: ВАЛИДАЦИЯ НАРРАТИВА | | — Лудонарративный диссонанс: механики ↔ история? | | — Гармония/Ирония/Диссонанс (Адамс/Дорманс) | | — Агентивность: 3 уровня (выбор → влияние → смысл)? | | — Драматическая арка: завязка→нарастание→кульминация? | | — Triangle of Weirdness (Роджерс) | | | УРОВЕНЬ 7: ВАЛИДАЦИЯ LEVEL DESIGN | | — Читаемость: игрок понимает пространство? | | — Навигация: 7 методов руководства + 9 инструментов | | — Боевые пространства: Combat Fronts корректны? | | — Темп: матрица 3×3 + чередование интенсивности | | — Beat Chart: арка механик соответствует арке уровня | | | УРОВЕНЬ 8: ВАЛИДАЦИЯ ОПЫТА | | — Поток: баланс вызова и навыка? | | — Кривая интереса: крючок→нарастание→кульминация? | | — Убийцы удовольствия: ни одного из 5? | | — Цели проектирования: уважение + магический круг + | | | поток + интересные решения | | — Мотивация: внутренняя + внешняя поддерживаются? | | | УРОВЕНЬ 9: ВАЛИДАЦИЯ ИНТЕРФЕЙСА | | — 6 принципов UI (Фуллerton) | | — Сочность: фидбэк на каждое действие? | | — Время отклика: < 1/10 секунды? | | — Доступность: цветовая слепота, слух, моторика, | | | когнитивная доступность | | — Типы решений: нетривиальные / не безразличные | | | УРОВЕНЬ 10: ВАЛИДАЦИЯ

ДОКУМЕНТАЦИИ | | — Полнота: все 38 секций GDD (или эквивалент)? | | —  
Актуальность: обновлена после последнего плейтеста? | | — Обоснование:  
каждое решение с «почему»? | | — Трассировка: кто изменил + когда +  
почему? | | — Аудитория: каждый документ адресован конкретной роли? | |  
| \_\_\_\_\_ | ...

### 11.6.2. Адаптивное применение валидатора

Не все 10 уровней валидации одинаково важны для каждой игры. Валидатор Gidede применяет **адаптивную приоритизацию** на основе жанра и типа Core Loop:

| Жанр / Тип           | Критичные уровни                         | Важные уровни                       | Второстепенные                |
|----------------------|------------------------------------------|-------------------------------------|-------------------------------|
| PvP-экшн (FPS, MOBA) | 3 (Core Loop), 4 (Баланс), 9 (Интерфейс) | 2 (Механики), 7 (LD), 8 (Опыт)      | 6 (Нарратив), 5 (Экономика)   |
| Single-player RPG    | 6 (Нарратив), 5 (Прогрессия), 8 (Опыт)   | 2 (Механики), 3 (Core Loop), 7 (LD) | 4 (Баланс PvP), 9 (Интерфейс) |
| Стратегия            | 4 (Баланс), 5 (Экономика), 2 (Механики)  | 3 (Core Loop), 8 (Опыт)             | 6 (Нарратив), 7 (LD)          |
| Нарративная игра     | 6 (Нарратив), 8 (Опыт), 1 (Концепция)    | 7 (LD), 2 (Механики)                | 4 (Баланс), 5 (Экономика)     |
| Мобильная F2P        | 5 (Экономика), 3 (Core Loop), 8 (Опыт)   | 1 (Концепция), 9 (Интерфейс)        | 6 (Нарратив), 7 (LD)          |

### 11.6.3. Формат отчёта валидации

Результат валидации — структурированный отчёт с тремя уровнями серьёзности:

| Уровень        | Обозначение | Описание                                  | Действие                                    |
|----------------|-------------|-------------------------------------------|---------------------------------------------|
| Критический    | ●           | Проблема, делающая игру неработоспособной | Обязательное исправление до следующего шага |
| Предупреждение | ●           | Проблема, снижающая качество опыта        | Рекомендуется исправить в текущей итерации  |
| Информация     | ●           | Потенциальное улучшение                   | Можно отложить на следующую итерацию        |

Пример отчёта валидации концепции RPG:

``` Universal Design Validator — Отчёт #001

УРОВЕНЬ 1: ВАЛИДАЦИЯ КОНЦЕПЦИИ  Логлайн: «Одинокый рыцарь путешествует по проклятому королевству...» — конкретно  Целевая эстетика: Challenge + Narrative + Discovery — согласовано  Профиль аудитории: Мотивация «Вызов» + «Фантазия» — типично для RPG, но не дифференцирует от конкурентов  Уникальность: «Боевая система с комбо» — не уникально; 23 из 127 механик MechanicsDB в категории «Бой»

используют комбо. Рекомендация: добавить уникальную механику из MechanicsDB (например, «Управление временем» или «Система душ»)

УРОВЕНЬ 2: ВАЛИДАЦИЯ МЕХАНИК  Выбор: 3 типа атаки + блок + уклонение = осмысленный выбор  Последствия: каждая атака имеет уникальный урон/скорость/стан  Разнообразие: текущий набор — 5 действий; для RPG рекомендуется 8-12  Предсказуемость: формула урона скрыта — для RPG это нормально

УРОВЕНЬ 6: ВАЛИДАЦИЯ НАРРАТИВА  Драматическая арка: завязка (проклятие) → нарастание (путешествие) → кульминация (битва с источником проклятия)  Лудонарративный диссонанс: «Одинокый рыцарь» (нарратив) + «Комбо-система с 5 типами атаки» (механика) — одинокый рыцарь не должен иметь богатый арсенал; либо изменить нарратив на «мастер боевых искусств», либо упростить боевую систему  Агентивность: линейный сюжет с выборами-иллюзиями — для RPG рекомендуется минимум «ветвление с слиянием»

ИТОГО: 2 критических, 3 предупреждения, 1 информация Следующий шаг: исправить критические проблемы перед переходом к эскизному проекту ```

11.7. Шаблоны Gidede: конкретные форматы для генерации

11.7.1. Шаблон технического предложения (Gameplay Treatment)

```markdown

## [НАЗВАНИЕ ИГРЫ] — Техническое предложение

---

### 1. Тип игры

---

**Жанр:** [жанр из MechanicsDB] **Платформа:** [PC / Console / Mobile / Web]  
**Аудитория:** [мотивационный профиль по Йи] **Длительность:** [средняя сессия / общая кампания] **Количество игроков:** [1 / 2-4 / ММО]

## 2. Логлайн

---

[Один абзац: кто + где + что + почему]

## 3. Суть геймплея

---

[2-3 абзаца, описывающие Core Loop и ключевой опыт]

## 4. Уникальность

---

[Что отличает эту игру от аналогов? Конкретные механики, опыт, сеттинг]

## 5. Целевая эстетика (MDA)

---

[Какие из 8 эстетик MDA — целевые? Почему?]

## 6. Реализуемость

---

[Технические требования, команда, бюджет, сроки] ```

### 11.7.2. Шаблон эскизного проекта (Sketch Design)

```markdown

[НАЗВАНИЕ ИГРЫ] — Эскизный проект

1. Концепция (из Технического предложения)

[Краткое повторение]

2. Core Loop

[Визуальная диаграмма + описание каждой петли]

3. Ключевые механики

3.1. [Механика 1: название]

- Что делает игрок: [действие] - Что происходит: [реакция системы] -
Параметры: [настраиваемые значения] - Взаимодействие: [с какими
механиками связана]

3.2. [Механика 2: название]

[...]

4. Прогрессия

[Кривая опыта, дерево навыков, открытие контента]

5. Структура уровней

[Количество, последовательность, арка сложности]

6. Интерфейс

[Схема управления, HUD, меню]

7. Сеттинг и мир

[Описание вселенной, ключевых локаций, атмосферы] ```

11.7.3. Шаблон секции баланса (Balance Spec)

```markdown

## [НАЗВАНИЕ ИГРЫ] — Спецификация баланса

---

## 1. Тип баланса

[PvP / игрок-геймдизайнер / геймдизайнер-игрок]

## 2. Q-матрица компонентов

|           |           |           |           |   |
|-----------|-----------|-----------|-----------|---|
| Компонент | Атрибут 1 | Атрибут 2 | Атрибут 3 |   |
| -----     | -----     | -----     | -----     |   |
| A         | Q=8       | Q=5       | Q=7       | B |
| C         | Q=9       | Q=4       | Q=8       |   |

## 3. SPS-циклы

[A → B → C → A]

## 4. Cost-power curves

[Формула: стоимость → мощность для каждого элемента]

## 5. Золотое правило

[Валидация: случайный выбор ≠ оптимальная стратегия]

## 6. Масштабируемость

[Тест: добавление нового элемента не нарушает баланс] ````

### 11.7.4. Шаблон нарративной Библии

````markdown

[НАЗВАНИЕ ИГРЫ] — Нарративная Библия

1. Логлайн истории

[1 предложение]

2. Тип нарратива

[Embedded / Emergent / Гибрид]

3. Структура сюжета

[Линейная / Ветвящаяся / Хабовая / Открытый мир / Симуляция]

4. Трёхактная арка

Акт I: Завязка

[Экспозиция, главный конфликт, мотивация героя]

Акт II: Развитие

[Нарастание, препятствия, повороты]

Акт III: Кульминация

[Финальный конфликт, разрешение, трансформация]

5. Библия персонажа: [Имя]

- Мотивация: [внутренняя + внешняя] - Арка: [начальное состояние → конечное состояние] - Голос: [как говорит] - Отношения: [с кем и какие] - Противоречие: [что делает интересным]

6. Матрица квестов

[Тип × Цель = 32 комбинации; отмечены используемые]

7. Валидатор лудонарративного диссонанса

[Проверка: механики ↔ история на каждом уровне] ```

11.8. Процесс генерации документации в Gidede

Генерация документации — не одноразовое действие, а **итеративный процесс**, связанный с циклом проектирования. Каждый шаг генерации сопровождается валидацией, и каждый плейтест обновляет документацию.

11.8.1. Pipeline генерации

``` ШАГ 1: Ввод данных пользователем |— Жанр, платформа, аудитория |—  
Целевая эстетика (MDA) |— Ключевые механики (из MechanicsDB) |—  
Описание концепции (свободный текст)

ШАГ 2: Генерация Concept Card (One-Sheet) |— AI формулирует логлайн |— AI  
определяет уникальность на основе MechanicsDB |— AI генерирует  
визуальный крючок (промпт) |— Валидация: 8 фильтров Шелла

ШАГ 3: Генерация Core Loop Diagram |— AI проектирует Core Loop (обратный  
MDA) |— AI создаёт концентрические схемы циклов |— Валидация: 5 вопросов  
кор-геймплея (Гэри)

ШАГ 4: Генерация эскизного проекта |— Автозаполнение из MechanicsDB +  
Core Loop + Progression |— Визуальные диаграммы (flowcharts, Machinations)  
|— Валидация: Universal Design Validator (уровни 1-3)

ШАГ 5: Генерация спецификаций |— Balance Spec (из формул и моделей) |—  
Economy Spec (из Machinations-модели) |— Progression Spec (из кривых) |—  
Narrative Bible (из нарративного движка) |— Валидация: Universal Design  
Validator (уровни 4-9)

ШАГ 6: Генерация полного GDD |— Компиляция всех модулей в единый  
документ |— Валидация полноты (38 секций) |— Генерация оглавления и  
перекрёстных ссылок

ШАГ 7: Итерация |— Плейтест → обновление модели → обновление  
документации |— RITE-цикл: тест → фикс → тест → фикс → ... ```

### 11.8.2. AI vs алгоритмы в генерации документации

В духе модели AI/Algorithm Split (Пробел 7 из синтеза), генерация документации распределяется между AI и алгоритмами:

| Задача | Исполнитель | Обоснование | |-----|-----|-----| |  
Формулировка логлайна | AI | Требуется креативности и языкового чутья | |  
Визуальные диаграммы | Алгоритм | Формализованные структуры (Core Loop,

Machinations) | | Формулы баланса | Алгоритм | Математика, не требующая креативности | | Описания механик | AI + данные | AI формулирует текст, данные из MechanicsDB | | Нарратив | AI | Требуется креативности и понимания контекста | | Валидация | Алгоритм | Формализованные правила (чек-листы, формулы) | | Рекомендации | AI | Требуется понимания контекста и приоритизации | | Форматирование | Алгоритм | Механическая задача — шаблоны и компоновка |

---

## 11.9. Культура документации: от формальности к коммуникации

---

Роллингс и Моррис (Кн. 12) предупреждают об опасности **избыточной формализации**: «Сотрудники тратят чуть ли не больше времени на заполнение и обработку форм отчётности, чем на основную работу». Документация ради документации — не помощь, а помеха. Каждый документ должен отвечать на вопрос: «Кто это будет читать и зачем?» Если документ не имеет чёткой аудитории и цели — он не нужен.

Шелл (Кн. 1) формулирует принцип: **призма документации (102)** — «Что нужно помнить во время работы? Какая информация нужна команде?» Это смещает фокус с «как выглядит документ» на «какую функцию он выполняет». Красиво оформленный документ, который никто не читает — хуже, чем черновик, который все используют.

Гэри (Кн. 10) добавляет практическое правило: **дизайнерская документация — живой документ**, который обновляется с каждой итерацией. Если документ не обновлён после последнего плейтеста — он мёртвый и приносит вред, описывая игру, которая больше не существует. Для Gidede это означает: модуль документации отслеживает время последнего обновления каждого раздела и помечает устаревшие секции предупреждением.

Зубек (Кн. 6) резюмирует: документация не заменяет прототип, но дополняет его. Прототип показывает, как игра работает; документация объясняет, почему она работает именно так. Лучший процесс — симбиоз: прототип → документация → плейтест → обновление документации → новая итерация прототипа. И документация, и прототип — это проекции единой модели игры, но с разных ракурсов.

Для Gidede культура документации формализуется в три принципа:

1. **Документация = проекция модели** — не отдельный артефакт, а представление текущего состояния модели в человекочитаемом формате. Изменение модели → автоматическое обновление документации.

2. **Каждый документ имеет владельца и аудиторию** — владельцы отвечают за актуальность, аудитория определяет формат и уровень детализации. Документ без владельца = сирота, который устареет через неделю.

3. **Живая документация** — система отслеживает «возраст» каждого раздела и предупреждает о секциях, не обновлённых после последнего плейтеста или изменения модели. Устаревшая документация помечается как «требуется обновления» и не считается достоверной.

---

## 11.10. Итоговые принципы GDD и чек-листов

---

1. **Волшебного шаблона не существует** — но есть адаптивные шаблоны, которые настраиваются под конкретную игру, команду и этап разработки. Gidede генерирует документацию из модели, а не заполняет формы.

2. **Документация — ось производственного процесса** — вокруг неё вращается всё: решения, коммуникация, реализация. Без документации — хаос; с документацией — порядок.

3. **Три уровня: Техническое предложение → Эскизный проект → Полная документация** — каждый уровень имеет свою функцию, аудиторию и объём. Нельзя перепрыгнуть с One-Sheet прямо в полный GDD без промежуточного звена.

4. **Модульность** 优于 **монолитность** — несколько связанных модулей лучше одного гигантского документа. Модули связаны через единую модель данных, что гарантирует консистентность без ручной синхронизации.

5. **Чек-листы предотвращают слепые зоны** — 113 линз Шелла, 8 фильтров, эвристики Аптона, чек-листы баланса, навигации, нарратива — всё это инструменты систематической проверки. Без чек-листа дизайнер проверяет то, что помнит, и пропускает то, что забыл.

6. **Universal Design Validator = интеграция всех чек-листов** — 10 уровней валидации с адаптивной приоритизацией по жанру. Не все проверки одинаково важны для каждой игры — валидатор адаптируется.

7. **Документация — живой документ** — обновляется с каждой итерацией. Устаревшая документация вреднее отсутствующей. Gidede автоматически отслеживает актуальность.

8. **AI генерирует, алгоритмы валидируют** — разделение труда: AI формулирует текст, описания и рекомендации; алгоритмы вычисляют баланс, строят диаграммы и проверяют формальные критерии. Вместе они создают полную, валидированную и актуальную документацию.

---

## Библия геймдизайна Gidede

---

> **Проект:** Gidede — Game Design AI System > **Версия:** 1.0 > **Дата:** 2026-05-18  
> **Авторы:** Команда проекта Gidede > **Репозиторий:** <https://github.com/Angelionix/Gidede>

---

### Предисловие

---

Эта книга — результат беспрецедентного по масштабу исследования: 17 книг по геймдизайну, от фундаментальных трудов Джесси Шелла и Трейси Фуллертон до специализированных работ по экономике игр, балансу, левел-дизайну и нарративу. Каждая книга была проанализирована, ключевые концепции извлечены, формализованы и кросс-референцированы. Результат — не просто компиляция чужих идей, а интегрированная модель геймдизайна, в которой концепции из разных источников дополняют, подтверждают и углубляют друг друга.

Цель этой книги — стать теоретическим фундаментом для AI-системы Gidede, которая будет помогать геймдизайнерам создавать игры. Каждая концепция в книге сформулирована так, чтобы быть операциональной: не просто «важно учитывать мотивацию игрока», а «мотивация определяется через 12 типов Йи в 3 кластерах, и AI использует этот профиль для подбора механик из MechanicsDB». Каждое утверждение проверяемо. Каждая модель формализуема. Каждая рекомендация алгоритмизуема.

Книга не является учебником для начинающих — хотя может служить справочником. Она предназначена для двух аудиторий: (1) разработчиков AI-системы Gidede, которым нужна формальная спецификация знаний о

геймдизайне; (2) опытных геймдизайнеров, которые хотят систематизировать свои знания и понять, как разные подходы соотносятся друг с другом.

---

## Оглавление

---

### Глава 1. Фундамент геймдизайна

**Файл:** `docs/bible/bible\_2\_1\_fundament.md` (482 строки, 6263 слова)

| Раздел | Содержание | |-----|-----| | 1.1. Что такое игра: спектр определений | Пять определений от концептуальных до формальных; рабочее определение Gidede; магический круг; принцип перехода собственности | | 1.2. Принципы геймдизайна | Пять универсальных принципов (итеративный цикл, баланс, Core Loop, поток, эмерджентность); четыре атрибута веселья (Селлерс); принцип приоритета исключений; убийцы удовольствия (Фуллerton) | | 1.3. Элементная модель игры | Базовая тетрада Шелла; расширенная матрица 4×3 Бонда; восемь формальных элементов Фуллerton; трёхуровневая модель Зубека; интеграция моделей в систему Gidede | | 1.4. Типологии игроков | Модель Бартла (4 типа); восемь типов удовольствия ЛеБланка; модель мотивации Йи (12 типов в 3 кластерах); интеграция моделей для AI-системы | | 1.5. Типологии механик | Семь базовых механик Шелла; пять структурных типов Адамса/Дорманса; четыре семейства Зубека; MechanicsDB — пятиуровневая таксономия (7→5→16→127→жанры) |

**Ключевые модели:** Рабочее определение Gidede, Матрица 4×3 Бонда, 8 формальных элементов Фуллerton, MechanicsDB (5 уровней), Модель Йи (12 типов мотивации)

---

### Глава 2. Элементы игры

**Файл:** `docs/bible/bible\_2\_2\_elements.md` (355 строк, 5069 слов)

| Раздел | Содержание | |-----|-----| | 2.1. От фундамента к элементам | Архитектура игры: механика, динамика, эстетика, история как макроэлементы | | 2.2. Механика: правила, ресурсы и действия | Три уровня механики (операционный, фундаментальный, поведенческий); семь базовых механик Шелла; пять структурных типов; Engines/Economies/Ecologies (Селлерс); MechanicsDB | | 2.3. Динамика: петли, обратная связь и эмерджентность | Восемь характеристик петли ОС; шесть комбинаций (Шрайбер); четыре главные петли (Селлерс); эмерджентность и дихотомия Юла;

второпорядочный дизайн | | 2.4. Эстетика: ощущение, восприятие, впечатление | Восемь типов эстетики (ЛеБланк); эстетика как целевая функция генерации; сочность (juiciness) и фидбэк | | 2.5. История: нарратив, конфликт и агентивность | Пять проблем интерактивного повествования (Шелл); Story Machine; жемчужная нить; агентивность; лудонарративный диссонанс |

**Ключевые модели:** 8 характеристик петли ОС (Адамс/Дорманс), 6 комбинаций (Шрайбер), 4 главные петли (Селлерс), 8 типов эстетики (ЛеБланк), Engines/Economies/Ecologies (Селлерс)

---

## Глава 3. MDA Framework и модели анализа

**Файл:** `docs/bible/bible\_2\_3\_mda\_framework.md` (463 строки, 5576 слов)

| Раздел | Содержание | |-----|-----| | 3.1. Классический MDA: от механик к опыту | MDA-фреймворк (Hunicke, LeBlanc, Zubek); три уровня анализа; 8 типов эстетики; отображение «механика → динамика → эстетика» | | 3.2. Обратный MDA: от опыта к механикам | Reverse MDA (Бонд); генеративный процесс от целевой эстетики к механикам; интеграция с MechanicsDB | | 3.3. Двойной MDA-процесс Gidede | Аналитический MDA (проверка) + Генеративный MDA (создание); формализация обоих направлений | | 3.4. Линзы Шелла как инструмент MDA-анализа | 113 линз в 16 категориях; маппинг линз на уровни MDA; адаптивное применение | | 3.5. Модели анализа: от интуиции к формализации | Матрица 4×3 Бонда; цепочки навыков (Cook); геймплейные циклы (Зубек); концентрические схемы |

**Ключевые модели:** Классический MDA, Обратный MDA, Двойной MDA-процесс Gidede, 113 линз Шелла, Матрица 4×3 Бонда

---

## Глава 4. Core Loop — ядро игрового опыта

**Файл:** `docs/bible/bible\_2\_4\_core\_loop.md` (456 строк, 4557 слов)

| Раздел | Содержание | |-----|-----| | 4.1. Что такое Core Loop | Петля взаимодействия (Cook); «30 секунд веселья» (Halo); базовый цикл как минимальная FUN-единица | | 4.2. Структурные типы Core Loop | Engines / Economies / Ecologies (Селлерс); 6 временных масштабов; диагностика патологий (runaway, deadlock, stall) | | 4.3. Геймплейные циклы | Концентрические схемы (Зубек): микро → малый → средний → крупный →

макро; связь между уровнями | | 4.4. Пять вопросов кор-геймплея | Гэри (Кн. 10): главный конфликт, основное действие, препятствия, прогресс, завершение | | 4.5. Проектирование Core Loop для AI-системы | Pipeline генерации; от целевой эстетики к структуре цикла; валидация Core Loop |

**Ключевые модели:** Core Loop (Cook), Engines/Economies/Ecologies (Селлерс), Концентрические схемы (Зубек), 5 вопросов кор-геймплея (Гэри), 6 временных масштабов (Селлерс)

---

## Глава 5. Баланс и математические модели

**Файл:** `docs/bible/bible\_2\_5\_balance.md` (503 строки, 4165 слов)

| Раздел | Содержание | |-----|-----| | 5.1. Философия баланса | 12 типов баланса (Шелл); 7 типов (Шрайбер); 3 типа (Роллингс/Моррис); идеальный дисбаланс (Портнов) | | 5.2. Транзитивные модели | Cost-power curves (Шрайбер); формула балансировки диапазона (Гэззэуэй); опорная точка (Fulcrum); DPS-формулы | | 5.3. Нетранзитивные модели | Камень-ножницы-бумага; SPS-правило (Роллингс/Моррис); нетранзитивные матрицы выплат; йоми-слои (Сирлин) | | 5.4. Ситуационный баланс | Зависимость от контекста; жанровая специфика; правило двух (Сид Мейер) | | 5.5. Петли обратной связи и стабильность | 6 комбинаций (Шрайбер); Q-фактор (Роллингс/Моррис); золотое правило баланса; диагностика дисбаланса | | 5.6. Вероятностные модели | Ожидаемое значение; цепи Маркова; рейтинговые системы (Эло, Глико, TrueSkill) | | 5.7. Формула воспринимаемой сложности | Шрайбер: Perceived Difficulty =  $f(\text{Complexity}, \text{Familiarity}, \text{Randomness})$  |

**Ключевые модели:** 12+7+3 типов баланса, Cost-power curves, SPS-правило, Q-фактор, Золотое правило, Нетранзитивные матрицы, Цепи Маркова, Рейтинговые системы

---

## Глава 6. Экономика игры и системы прогрессии

**Файл:** `docs/bible/bible\_2\_6\_economy\_progression.md` (640 строк, 6484 слов)

| Раздел | Содержание | |-----|-----| | 6.1. Внутренняя экономика игры | Пять элементов (Источники, Стоки, Пулы, Конвертеры, Торговцы); пять структур (Адамс/Дорманс) | | 6.2. Machinations Framework | Визуальная нотация; 16+ паттернов дизайна; искусственные игроки; симуляция экономики | | 6.3. Классы ресурсов | Пять классов (Шрайбер): катализаторы, расходуемые,



возобновляемые, условные, мета-ресурсы | | 6.4. Системы прогрессии | 7 типов кривых (Шрайбер); 6 типов (Селлерс); эмерджентная прогрессия; макро-модель RPG | | 6.5. Фаусеты и дрейны (Faucets & Drains) | Баланс производства и потребления; конверсионные цепочки (Зубек); динамическая настройка | | 6.6. Экономические патологии | Инфляция, стагнация, арбитраж; диагностика и лечение; кейс Lunar Colony | | 6.7. Пять экономических структур | Статическая экономика, динамическая, мета-экономика, Player-driven, гибридная (Адамс/Дорманс + Фуллертон) |

**Ключевые модели:** Machinations (16 паттернов), 5 элементов экономики, 5 классов ресурсов, 7+6 кривых прогрессии, Конверсионные цепочки (Зубек)

---

## Глава 7. Level Design — принципы и паттерны

**Файл:** `docs/bible/bible\_2\_7\_level\_design.md` (510 строк, 4798 слов)

| Раздел | Содержание | |-----|-----| | 7.1. Философия Level Design | «Основа + Изюминка» (Кадиков); 6 типов изюминок; 5 категорий вау-моментов | | 7.2. Навигация и руководство игроком | 9 навигационных инструментов (Кадиков); 7 методов косвенного руководства (Бонд); навигация без UI | | 7.3. Боевые пространства | Combat Focus (Кадиков); Combat Fronts (Кн. 14); 4 стиля боевой игры; система укрытий | | 7.4. Мультиплеерный Level Design | 5 структур MP-карт (Кн. 14); PPP-анализ; читаемость (Readability); симметрия vs асимметрия | | 7.5. Темп и арка уровня | Матрица темпа 3×3; Gates & Valves; Beat Chart (Роджерс); «Музыка механик» | | 7.6. Хоррор-дизайн | 12 приёмов хоррора (Кадиков); управление напряжением; атмосфера страха |

**Ключевые модели:** «Основа + Изюминка», 9 навигационных инструментов, 7 методов руководства, Combat Fronts, Матрица темпа 3×3, Beat Chart

---

## Глава 8. Нарратив и эмоциональный дизайн

**Файл:** `docs/bible/bible\_2\_8\_narrative\_emotional\_design.md` (435 строк, 6441 слово)

| Раздел | Содержание | |-----|-----| | 8.1. Три термина: Сюжет, Повествование, Нарратив | Андрианова/Яковлева; различение формы, процесса и восприятия | | 8.2. Включённый vs эмергентный нарратив | Два фундаментальных подхода; спектр от полностью скриптового до полностью эмергентного | | 8.3. Агентивность | Три уровня: выбор, влияние, смысл; спектр

СЮОА ↔ песочница | | 8.4. Лудонарративный диссонанс | Определение; три типа согласованности (Гармония, Ирония, Диссонанс); валидатор | | 8.5. Структуры сюжета | 4 типа структуры; Кисётэнкэцу; трёхактная модель; схемы повествования | | 8.6. Персонажи | 6 типов персонажей; Библия персонажа; Циркумплекс отношений; трансформация | | 8.7. Квесты и конфликты | Матрица квестов  $8 \times 4 = 32$  комбинации; 8 типов конфликтов | | 8.8. Диалоги | 7 приёмов написания диалогов; голос персонажа; подача информации через диалог | | 8.9. Эмоциональный дизайн | Нейрохимия вовлечения (Селлерс); кривая интереса (Шелл); драматическая арка (Фуллертон) |

**Ключевые модели:** Матрица квестов  $8 \times 4$ , Валидатор лудонарративного диссонанса, 6 типов персонажей, Циркумплекс, Кисётэнкэцу, Нейрохимия вовлечения

---

## Глава 9. Монетизация и Retention

**Файл:** `docs/bible/bible\_2\_9\_monetization\_retention.md` (537 строк, 6660 слов)

| Раздел | Содержание | |-----|-----| | 9.1. Модели монетизации | Premium, F2P, Подписка, IAP, Ад-supported, Гибридные; критерии выбора модели | | 9.2. Экономика Free-to-Play | LTV, CAC, ROAS; воронка монетизации; формулы расчёта | | 9.3. Retention: наука удержания | D1/D7/D30 метрики; факторы Retention; кривые удержания | | 9.4. Системы LiveOps | Battle Pass; Seasons; Events; Daily/Weekly задания; A/B-тестирование | | 9.5. Аналитика геймдизайна | Acquisition, Retention, Conversion, Usage, Community (Селлерс); поведение → данные → решение | | 9.6. Этика монетизации | Тёмные паттерны; ответственный дизайн; регулирование; принцип уважения игрока |

**Ключевые модели:** LTV/CAC/ROAS формулы, D1/D7/D30 метрики, Воронка монетизации, Battle Pass экономика, 5 метрик аналитики (Селлерс)

---

## Глава 10. Плейтестинг и итерация

**Файл:** `docs/bible/bible\_2\_10\_playtesting\_iteration.md` (497 строк, 5715 слов)

| Раздел | Содержание | |-----|-----| | 10.1. Философия плейтестинга | Правило цикла (Шелл); Playcentric Design Process (Фуллертон); проблема второго порядка (Зубек) | | 10.2. Прототипирование | Бумажное (Фуллертон, Бонд); цифровое (4 типа); прототип как минимальная проверяемая единица | | 10.3. Методы плейтестирования | RITE-метод; процесс плейтест-сессии;

наблюдение и сбор данных; научный метод (Гэззэуэй) | | 10.4. Итеративный процесс | 4 уровня тестирования (Фуллerton); принцип «одно изменение за итерацию»; три вывода после теста | | 10.5. Пять стадий дизайна | Гэри: кор-геймплей → доработка → компоненты → балансировка → полировка | | 10.6. Аналитические методы валидации | Симуляция с искусственными игроками; математический анализ баланса; аналитика поведения |

**Ключевые модели:** RITE-метод, 4 уровня тестирования, 5 стадий дизайна (Гэри), Научный метод (Гэззэуэй), Искусственные игроки (Machinations)

---

## Глава 11. Шаблоны GDD и чек-листы

**Файл:** `docs/bible/bible\_2\_11\_gdd\_templates\_checklists.md` (781 строка, 6524 слова)

| Раздел | Содержание | |-----|-----| | 11.1. Философия документации | «Волшебного шаблона не существует» (Шелл); две цели (хранение + передача); документация — ось процесса | | 11.2. Три уровня документации | Техническое предложение (5-6 стр.); Эскизный проект (15-30 стр.); Полная документация (50-200+ стр.) — Роллингс/Моррис | | 11.3. Форматы документации | One-Sheet, Ten-Pager, Полный GDD (38 секций Роджерса), Модульная документация Gidede (13 модулей) | | 11.4. Специализированные типы | 11 типов нарративной документации; дизайн-документы Фуллертон; концепт-документ Селлерса | | 11.5. Система чек-листов | 113 линз Шелла; 8 фильтров идеи; эвристики Аптона; чек-листы баланса, навигации, нарратива | | 11.6. Universal Design Validator | 10 уровней валидации с адаптивной приоритизацией; формат отчёта | | 11.7. Шаблоны Gidede | Markdown-шаблоны: Gameplay Treatment, Sketch Design, Balance Spec, Narrative Bible | | 11.8. Процесс генерации документации | Pipeline из 7 шагов; распределение AI vs алгоритмы | | 11.9. Культура документации | Принципы живой документации; опасность избыточной формализации |

**Ключевые модели:** 38 секций GDD (Роджерс), 3 уровня документации (Роллингс/Моррис), Universal Design Validator (10 уровней), 113 линз Шелла, Модульная документация (13 модулей)

---

## Карта кросс-референций

Каждая глава Библии связана с остальными через общие модели и концепции. Ниже представлена карта ключевых кросс-референций:

...

|                                                          |                                                            |                                                           |
|----------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------|
| КАРТА КРОСС-РЕФЕРЕНЦИЙ БИБЛИИ                            |                                                            |                                                           |
| Глава 1                                                  | → Глава 2 (элементная модель → механика/динамика/эстетика) | Фундамент                                                 |
| → Глава 3 (определения → MDA)                            | → Глава 4 (Core Loop → структурный тип)                    | → Глава 5 (принципы → 12 типов баланса)                   |
| → Глава 10 (итеративный цикл → плейтестинг)              |                                                            |                                                           |
| Глава 2                                                  | → Глава 3 (эстетика → MDA)                                 | Элементы                                                  |
| → Глава 4 (динамика → Core Loop)                         | → Глава 5 (петли ОС → стабильность баланса)                | → Глава 6 (экономика → внутренняя экономика)              |
| → Глава 7 (пространство → Level Design)                  | → Глава 8 (история → нарратив)                             |                                                           |
| Глава 3                                                  | → Глава 1 (обратный MDA → целевая эстетика)                | MDA                                                       |
| → Глава 4 (MDA → Core Loop дизайн)                       | → Глава 5 (динамика → модели баланса)                      | → Глава 10 (аналитический MDA → валидация)                |
| → Глава 11 (MDA → чек-листы валидации)                   |                                                            |                                                           |
| Глава 4                                                  | → Глава 2 (Engines/Economies/Ecologies → типология систем) | Core Loop                                                 |
| → Глава 5 (патологии Core Loop → баланс)                 | → Глава 6 (Core Loop → экономическая структура)            | → Глава 9 (Retention → Core Loop как фактор удержания)    |
| → Глава 10 (Core Loop → прототипирование и тестирование) |                                                            |                                                           |
| Глава 5                                                  | → Глава 4 (баланс → стабильность петель Core Loop)         | Баланс                                                    |
| → Глава 6 (cost curves → экономика, SPS → конверсии)     | → Глава 9 (баланс → монетизация, LTV)                      | → Глава 11 (Q-фактор, SPS → чек-листы)                    |
| Глава 6                                                  | → Глава 2 (Machinations → петли ОС)                        | Экономика                                                 |
| → Глава 4 (экономика → тип Core Loop)                    | → Глава 5 (экономический баланс → формулы)                 | → Глава 9 (экономика → монетизация, фаусеты/дрейны → LTV) |
| Глава 7                                                  | → Глава 2 (пространство → механика пространства)           | LD                                                        |
| → Глава 5 (баланс боевых пространств)                    | → Глава 8 (нарративный LD → квесты и сюжет)                | → Глава 10 (LD → плейтест уровней)                        |

|                                                                                                                                                                                                                                        |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Глава 8  → Глава 2 (история → элемент тетрады)     Нарратив  → Глава 3 (эстетика Narrative → MDA)      → Глава 7 (нарративный LD → хоррор, атмосфера)      → Глава 11 (нарративная документация → 11 типов)                            |  |
| Глава 9  → Глава 4 (Retention → качество Core Loop)     Монетиз.  → Глава 6 (экономика → монетизационные модели)      → Глава 10 (аналитика → метрики плейтеста)                                                                       |  |
| Глава 10  → Глава 1 (итеративный цикл → фундаментальный принцип)     Плейтест  → Глава 3 (аналитический MDA → валидация опыта)      → Глава 5 (баланс → математическая валидация)      → Глава 11 (плейтест → обновление документации) |  |
| Глава 11  → Все главы (чек-листы → валидация каждого аспекта)     GDD  → Глава 1 (фундамент → структура GDD)      → Глава 10 (живая документация → итерация)                                                                           |  |
| ...                                                                                                                                                                                                                                    |  |

## Карта модулей базы знаний (KB)

Каждая глава Библии основана на одном или нескольких модулях базы знаний, выявленных в ходе синтеза (Фаза 1.3). Ниже представлено отображение глав на модули KB:

| Глава | Модули KB | Источники (книги) | |-----|-----|-----| | 1. Фундамент | KB-01 (Определения), KB-02 (Элементы), KB-13 (Опыт и мотивация) | Шелл (1,2), Фуллертон (5), Зубек (6), Селлерс (13), Бонд (17) | | 2. Элементы | KB-04 (MechanicsDB), KB-05 (Типология систем), KB-06 (Петли ОС) | Шелл (1,2), Адамс/Дорманс (4), Селлерс (13), SW.BAND (15) | | 3. MDA | KB-03 (MDA Framework), KB-04 (MechanicsDB) | Hunicke/LeBlanc/Zubek, Бонд (17), Шелл (1) | | 4. Core Loop | KB-07 (Core Loop), KB-05 (Системы), KB-06 (Петли) | Селлерс (13), Зубек (6), Гэри (10), Cook | | 5. Баланс | KB-08 (Баланс) | Шелл (1), Шрайбер (7), Роллингс/Моррис (12), Гэзэуэй (9) | | 6. Экономика | KB-09 (Прогрессия), KB-10 (Экономика) | Адамс/Дорманс (4), Шрайбер (7), Фуллертон (5), Зубек (6) | | 7. Level Design | KB-12 (Level Design) | Кадиков (11), Кн. 14 (LD), Бонд (17), Роджерс (8) | | 8. Нарратив | KB-11 (Нарратив), KB-13 (Опыт) | Андрианова/Яковлева (3), Адамс/Дорманс (4), Шелл (1), Селлерс (13) | | 9. Монетизация | KB-10 (Экономика), KB-13 (Опыт) | Селлерс (13), Фуллертон (5), веб-исследования | | 10. Плейтестинг | KB-14 (Валидация), KB-16 (Процесс) |

Фуллerton (5), Шелл (1), Гэри (10), Бонд (17), Гэзэуэй (9) | | 11. GDD | KB-15 (Документация), KB-14 (Валидация) | Шелл (1), Роджерс (8), Роллингс/Моррис (12), Фуллerton (5), Андрианова/Яковлева (3) |

---

## Статистика Библии

---

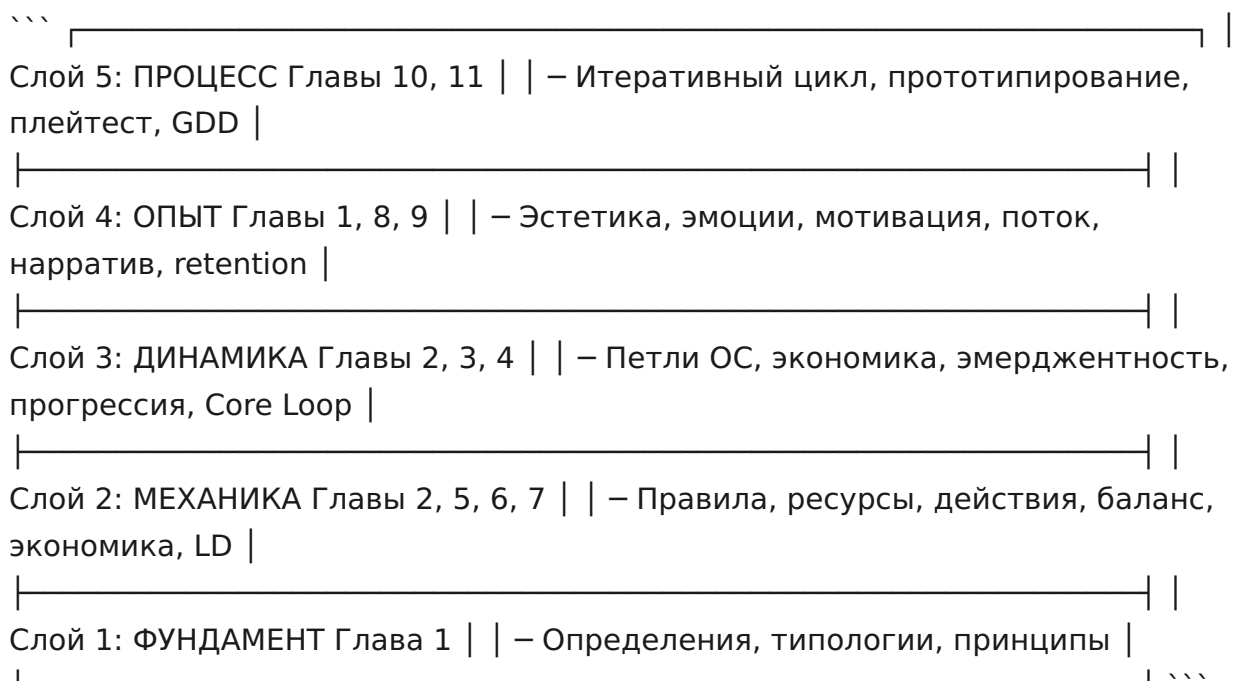
| Метрика | Значение | |-----|-----| | Количество глав | 11 | | Общий объём (строки) | 5 659 | | Общий объём (слова) | ~62 000 | | Общий объём (знаки) | ~420 000 | | Моделей и фреймворков | 80+ | | Формализованных моделей | 35+ | | Чек-листов | 15+ | | Книг-источников | 17 | | Модулей базы знаний | 17 | | Кросс-референсов | 50+ |

---

## Пятислойная модель Gidede и её отображение на главы Библии

---

В ходе синтеза исследований (Фаза 1.3) была выявлена пятислойная модель геймдизайна, в которой каждый слой соответствует уровню абстракции. Главы Библии покрывают все пять слоёв:



## Десять пробелов и их покрытие в Библии

---

В ходе синтеза (Фаза 1.3) были выявлены 10 пробелов в существующей литературе — аспектов геймдизайна, не покрытых ни одной книгой в отдельности. Библия заполняет эти пробелы через интеграцию концепций из разных источников:

| # | Пробел | Покрытие в Библии | |---|-----|-----| | 1 | Нет модели «Механика → Опыт» | Глава 3: Двойной MDA-процесс (Аналитический + Генеративный) | | 2 | Нет модели «Баланс от концепции до валидации» | Глава 5: Интеграция 12+7+3 типов баланса + cost curves + Q-фактор + золотое правило | | 3 | Нет модели «Генерация уровня» | Глава 7: «Основа + Изюминка» + Combat Fronts + 7 методов руководства + матрица темпа | | 4 | Нет модели «Генерация нарратива от механик» | Глава 8: Матрица квестов 8x4 + валидатор диссонанса + 6 типов персонажей | | 5 | Нет единой модели валидации | Глава 11: Universal Design Validator (10 уровней) | | 6 | Нет модели генерации документации | Глава 11: Модульная документация (13 модулей) + Pipeline генерации | | 7 | Нет модели AI-интеграции | Все главы: чёткое разделение AI vs алгоритмы в каждой модели | | 8 | Нет модели монетизации и Retention | Глава 9: LTV/CAC/ROAS + Battle Pass + D1/D7/D30 + этика | | 9 | Нет UX/UI-паттернов | Глава 7 (навигация) + Глава 11 (UI-чек-листы) → детализация в Фазе 2.5 | | 10 | Нет практических кейсов | Примеры贯穿 все главы → Case Library планируется в Фазе 3 |

---

## От Библии к AI-системе: ключевые алгоритмы

---

Каждая глава Библии содержит модели, которые могут быть напрямую формализованы как алгоритмы для AI-системы. Ниже перечислены ключевые алгоритмы, которые будут разработаны в Фазе 3:

### Алгоритмы генерации

| Алгоритм | Источник (глава) | Вход | Выход | |-----|-----|-----|-----| | Генерация концепции | 1, 3 | Жанр + эстетика + аудитория | Концепция игры (логлайн, уникальность, Core Loop) | | Reverse MDA Generator | 3 | Целевая эстетика | Динамики → механики → MechanicsDB выбор | | Core Loop Designer | 4 | Тип (Engine/Economy/Ecology) + эстетика | Структура Core Loop + концентрические схемы | | Mechanics Selector | 2 | Жанр + эстетика + Core Loop | Набор механик из MechanicsDB + проверка совместимости | | Economy Builder | 6 | Core Loop + тип экономики | Machinations-модель + faucets/drains +

конверсии | | Balance Validator | 5 | Набор компонентов + атрибуты | Q-матрица + SPS-циклы + golden rule check | | Level Structure Generator | 7 | Механики + целевой опыт | «Основа + Изюминка» + навигация + темп | | Narrative Generator | 8 | Механики + тип нарратива | Структура сюжета + персонажи + квесты | | GDD Generator | 11 | Все данные проекта | Документация нужного формата |

## Алгоритмы валидации

| Алгоритм | Источник (глава) | Проверяемый аспект |  
|-----|-----|-----| | Universal Design Validator | 11 | 10 уровней валидации с адаптивной приоритизацией | | Ludonarrative Consistency Check | 8 | Согласованность механик и нарратива | | Balance Matrix Analyzer | 5 | Q-фактор, SPS, золотое правило, cost-power curves | | Core Loop Pathology Detector | 4 | Runaway, deadlock, stall диагностика | | Economic Pathology Detector | 6 | Инфляция, стагнация, арбитраж | | Fun Killers Detector | 1 | 5 убийц удовольствия (Фуллerton) | | Apton Heuristics Validator | 2, 11 | 6 эвристик: выбор, разнообразие, последствия, предсказуемость, неопределённость, удовлетворение | | Retention Predictor | 9 | Прогноз D1/D7/D30 на основе дизайна |

---

## Источники

Библия основана на анализе следующих 17 книг:

| # | Автор | Название | Приоритет | |---|-----|-----|-----| | 1 | Джесси Шелл | Геймдизайн. Как создать игры, которые нельзя оторвать | ● Критический | | 2 | — | Искусство геймдизайна | ● Критический | | 3 | Андрианова, Яковлева | Как создавать истории (2023) | ● Высокий | | 4 | Адамс, Дорманс | Игровая механика. Продвинутый игровой дизайн | ● Критический | | 5 | Tracy Fullerton | Game Design Workshop (5th Edition, 2024) | ● Критический | | 6 | Роберт Зубек | Элементы гейм-дизайна (2022) | ● Высокий | | 7 | — | Игровой баланс. Геймдев, наука, точная наука | ● Критический | | 8 | Scott Rogers | Level Up! | ● Высокий | | 9 | Дакс Гэззэуэй | Введение в дизайн игровых систем | ● Высокий | | 10 | Джастин Гэри | Думай как гейм-дизайнер | ● Средний | | 11 | М. Кадиков | Проектирование виртуальных миров | ● Высокий | | 12 | Роллингс, Моррис | Проектирование и архитектура игр | ● Средний | | 13 | Michael Sellers | Advanced Game Design | ● Критический | | 14 | — | LD — In pursuit of better levels | ● Высокий | | 15 | SW.BAND | Колоды карт геймдизайнера | ● Высокий |



## Следующие шаги

---

Библия геймдизайна — теоретический фундамент. Следующие фазы проекта Gidede:

**Фаза 2.5: Концепция программы** — определение целевой аудитории, функциональных блоков, AI-интеграции, архитектуры, UX/UI концепции, технологического стека, MVP vs Full Version.

**Фаза 3: Формализация алгоритмов** — превращение моделей Библии в конкретные алгоритмы: генератор концепции, Core Loop Designer, Mechanics Selector, Balance Validator, Economy Builder, Narrative Generator, GDD Generator, AI-prompts.

**Фаза 4: Разработка веб-приложения** — реализация AI-системы Gidede как интерактивного веб-приложения: модули генерации, валидации, документации и AI-ассистент.

---